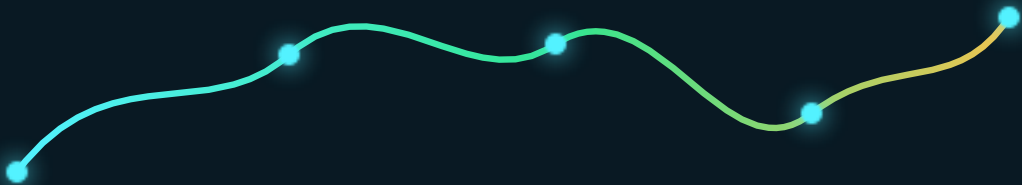


AI가 만든 자동매매, 검증은 누가 할까?

백테스트 결과를 의심하고
실전 전에 안전장치를 세우는 법



과거 성과 → 새 기간 검증 → 모의투자 → 제한된 실행

Ted Park

데이터 | 검증 | 안전장치 | 실행

AI가 만든 자동매매, 검증은 누가 할까?

백테스트 결과를 의심하고 실전 전에 안전장치를 세우는 법

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

차례

머리말	1
교육용 자료와 제공 범위	1
이 장의 흐름	2
학습 목표	2
나를 속인 차트	2
기술적 분석의 현실	3
르네상스 테크놀로지스 사례에서 얻은 관점	3
시장을 바라보는 다른 방식	4
AI 코딩 어시스턴트가 가능하게 한 것	4
이 책을 마치면 만들 수 있는 것	5
이 책을 머신러닝 트레이딩 학습서로 읽는 법	5
읽다가 길을 잃었을 때 보는 지도	10
하루를 따라가 보는 전체 예시	11
모델을 더 쉽게 쪼개면	13
르네상스 사례에서 얻은 원칙	13
1. 인과성 원칙	13
2. 앙상블 원칙	14
3. Walk-forward 검증 원칙	14
앞으로의 여정	14
참고: 백테스트 성과를 읽는 법	15
전체 챕터 로드맵	15
회독별로 다르게 읽기	16
Part별 최소 이해 목표	17
AI를 페어 프로그래머로 쓰기	18

이 장에서 반드시 기억할 것	19
사전 요구사항	19
0장. 바이브코딩 퀵스타트—먼저 한 번 검증해보기	20
먼저 답을 정한다	20
본문만으로 실행하는 확인 코드	21
AI에게 요청하는 방법	22
시간 순서를 확인한다	22
실패를 좁히는 순서	22
다음 장으로 넘어갈 기준	23
1장. 시스템 아키텍처—AI 퀀트 트레이딩의 설계 철학	24
이 장의 흐름	25
학습 목표	25
1.1 시스템 구조	26
왜 레이어로 나누는가	26
머신러닝 관점으로 다시 보기	28
모든 장을 같은 방식으로 읽기	30
1.2 디렉터리 구조	31
1.3 핵심 구성요소	32
SignalGenerator —공통 약속	32
SIGNAL_REGISTRY —등록된 전략 목록	32
BacktestResult —표준 결과 형식	33
SAC —직접 구현한 강화학습 에이전트	33
1.4 인과성 원칙	34
1.5 데이터 흐름—백테스트는 어떤 순서로 도는가	35
1.6 개발 환경 설정	35
Python 3.12 환경	35
핵심 패키지	36
설정 파일	36
1.7 데이터 캐시 초기화	37
1.8 첫 번째 백테스트 실행	38
연습문제	39

정리	39
더 읽을거리	40
2장. 데이터 파이프라인—DuckDB와 실시간 피드	41
이 장의 흐름	42
학습 목표	42
2.1 큰 그림—데이터 흐름	43
원자료가 피처가 되는 모습	44
2.2 <code>download_bars()</code> —단일 다운로드 창구	45
함수가 지원하는 인터벌	45
핵심 흐름—2단계 폴백	46
폴백이 실제로 일어났는지 확인하기	46
첫 줄은 30% 기준에 걸려 <code>yfinance</code> 로 넘어간 경우이고, 둘째 줄은 모든 소스가 실패해 빈 <code>DataFrame</code> 이 돌아온 경우다. 이렇게 받아온 데이터를 어디에 어떤 형태로 쌓아 두는지가 다음 절의 주제다.	47
2.3 <code>DuckDBCache</code> — <code>Parquet</code> + 메모리 <code>DuckDB</code>	47
왜 <code>DuckDB</code> 인가	47
저장 구조—월별 <code>Parquet</code>	48
동시 접근을 어떻게 다루는가	48
저장 함수의 4단계	49
2.4 기준 시간대로 변환한 뒤 타임존 제거하기	49
올바른 패턴	50
2.5 피처 엔지니어링—4차원 일봉 피처	50
4가지 피처가 보는 것	51
핵심 계산—네 가지 일별 피처	51
결측치—모델이 읽으면 안 되는 빈칸	52
2.6 칼만 평활화—노이즈를 줄이는 보조 도구	53
2.8 Z-score —평균 회귀 신호	54
2.9 모델 입력으로 넘기기—실전 패턴	55
연습문제	56
정리	56
더 읽을거리	57
3장. 백테스트 엔진—신뢰할 수 있는 시뮬레이션	58

이 장의 흐름	59
학습 목표	59
3.1 전체 그림—신호·체결·손익의 시간 순서	60
3.2 실행 가능한 손익 회계 실습	62
3.3 비용과 최종 청산을 한 번만 반영하기	64
3.4 샤프지수— <code>_build_result()</code>	65
변동성 보정—샤프지수	65
MDD와 Win Rate	66
더 정교한 지표 (카탈로그)	67
3.5 PnL Gini —집중도 지표	67
3.6 디플레이션된 샤프지수(DSR) —다중 검정 보정	67
3.7 PortfolioConfig —전략별 권장 설정	68
3.8 성능 지표 해석	69
연습문제	70
정리	70
더 읽을거리	71
4장. HMM 기초—시장 레짐을 확률로 추론하기	72
HMM을 머신러닝 모델로 보면	73
같은 신호도 레짐이 다르면 뜻이 달라진다	74
이 장의 흐름	75
학습 목표	76
첫 회독에서 볼 것	76
4.0 전체 그림— <code>hmm_daily_mr.py</code> 파이프라인	77
4.1 HMM 3요소— A , B , π	78
전이 행렬 A	78
방출 분포 B	78
초기 상태 분포 π	79
4.2 Baum-Welch —학습 직관	79
4.3 <code>_DailyHMM.fit()</code> —실제 구현	80
<code>min_covar=1e-3</code> 의 역할	82
4.4 분산 기반 레짐 정렬	83

4.5 <code>_DailyHMM.predict_proba()</code> —레짐 확률 계산	85
4.6 <code>_update_hmm()</code> —10봉 주기 재학습	86
4.7 <code>_check_entry()</code> —Z-score + HMM 게이트	87
4.8 <code>_check_exit()</code> —청산 조건 6가지	88
4.9 <code>PureZscoreDailySignal</code> —기준선	89
4.10 전체 흐름— <code>on_bar()</code> 호출 순서	90
연습문제	90
이 장에서 반드시 기억할 것	91
더 읽을거리	92
5장. HMM Forward-Only —미래 정보 없는 실시간 추론	93
Forward-only를 한 장면으로 보기	96
이 장의 흐름	97
학습 목표	98
5.0 전체 그림— <code>hmm_core.py</code> 파이프라인	99
5.1 <code>predict_proba</code> 가 미래 정보를 섞는 경우	100
5.2 Forward 알고리즘 재귀식	101
5.3 <code>_BWHMM.forward_proba_causal()</code> —학습 후 α 초기화	102
5.4 <code>_BWHMM.step_forward()</code> —한 스텝 온라인 추론	103
훈련과 OOS 구간을 잇는 흐름	104
5.5 <code>_precompute_cov_fast()</code> —속도 최적화	105
5.6 7-State Regime Structure — <code>_REGIME_A_SCALE</code>	105
5.7 <code>GlobalRegimeLayer.step()</code> —단일 승수로 줄이기	106
5.8 <code>dominant_state</code> —3봉 확인 안정화	107
5.9 Walk-Forward 사용 예시	107
5.10 <code>transition_risk</code> —레짐 불확실성 측정	108
연습문제	109
이 장에서 반드시 기억할 것	109
더 읽을거리	110
6장. 칼만 필터—노이즈 제거와 혁신 Z-스코어	112
이 장의 흐름	114

학습 목표	115
6.0 전체 그림—칼만 필터의 두 가지 역할	115
6.1 로컬 레벨 모델—직관적인 수학	116
6.2 KalmanFilter1D —HMM 피쳐 평활화	118
6.3 4차원 피쳐별 칼만 파라미터	119
6.4 <code>_kalman_innovations()</code> —가격에서 Innovation 뽑기	120
6.5 <code>_build_z_series()</code> —인과적 Z-score	121
6.6 <code>_calc_half-life()</code> —AR(1) 반감기 추정	122
6.7 KalmanSelfMRSigal —진입과 청산 로직	122
진입 4중 필터	123
청산 6중 체크	124
6.8 <code>kalman_self_mr</code> vs <code>hmm_daily_mr</code> 비교	124
6.9 백테스트 실행 예시	125
연습문제	126
이 장에서 반드시 기억할 것	126
더 읽을거리	127
7장. 크로스애셋 Joint HMM —여러 시장을 함께 보는 레짐 탐지	128
이 장의 흐름	130
학습 목표	130
7.0 전체 파이프라인—6단계	131
7.1 크로스애셋이 필요한 이유	132
7.2 bar 단위 노이즈 제거—Kalman 필터	133
7.3 20차원 세션 관측 벡터	134
7.4 Layer 1—Macro HMM (일봉)	135
7.5 Layer 2—Micro HMM (세션)	136
7.6 Walk-forward 구조	137
7.7 <code>macro_bias</code> -> Kelly 변조	137
7.8 P(up) 계산—상승 확률은 어떻게 만들어지나	138
7.9 결과를 만든 구조적 요인	138
코드로 보기—합성 데이터로 Macro HMM 학습 흐름 확인	138

연습문제	140
정리	140
더 읽을거리	141
8장. 2계층 HMM 앙상블—Layer A × Layer B	142
이 장의 흐름	144
학습 목표	144
8.1 왜 두 레이어가 필요한가	145
8.2 왜 두 레이어 + W2 투표인가	146
8.3 Layer A: GlobalRegimeLayer	147
8.3.1 레짐 안정화—3바 연속 확인	148
8.4 Layer B: 8개 서브모델 개요	148
8.4.1 HMM 서브모델 (대표 2개)	149
8.4.2 비-HMM 서브모델 (대표 2개)	149
8.5 SubModelSpec —서브모델을 데이터로 선언하기	150
8.6 HMMForwardPredictor: Layer B HMM Predictor	151
8.6.1 fit() —학습 단계가 하는 일	151
8.6.2 step() —새 봉마다 기대 수익률 갱신	151
8.7 W2 Wasserstein Voting —아웃라이어 억제	152
8.8 RenaissanceEnsemble5m.step() —두 레이어 결합	153
8.9 설계 원칙 요약	154
코드로 보기—2단계 HMM 최소 구현	154
연습문제	155
8.10 이 장에서 반드시 기억할 것	156
더 읽을거리	157
9장. 켈리 절반 포지션 사이징	158
이 장의 흐름	160
학습 목표	160
9.1 켈리 기준의 기초	161
9.2 calculate_kelly_size() 구조	162
9.3 vol_mult —변동성이 커질 때 포지션 줄이기	164

9.4 macro_gate —시장 상태가 헛갈리면 줄이기	164
9.5 novelty_gate —본 적 없는 시장이면 줄이기	165
9.6 ER Amplifier —신호가 일치할 때 키우기	166
9.7 자산 클래스별 매크로 편향 조정	166
9.8 fractional_kelly() —5분봉용 단순 버전	167
9.9 두 가지 Kelly 구현 비교	168
9.10 KELLY_MIN = 0.10으로 설정한 이유	169
시나리오로 보기	169
연습문제	170
정리	170
더 읽을거리	171
10장. HMM 포워드 양상블—V1 -> V3 진화	172
왜 양상블인가	173
이 장의 흐름	174
학습 목표	174
첫 회독에서 볼 것	174
10.1 아키텍처 비교—V1, V2, V3	175
10.2 V1/V2 공통 코어—Forward α 로 $E[r]$ 계산하기	176
10.2.1 인과성(causality) 함정 피하기	177
10.3 버전마다 어떤 정보를 더했는가	178
V2에서 추가—섹터·주문 흐름	178
V3에서 추가—멀티 타임프레임 & 비-HMM	178
V3 동적 가중치—성과 좋은 서브모델에 더 큰 비중	179
10.4 V2 핵심 아이디어—Optuna 임계값 재최적화	180
10.5 V2/V3 성과 비교 요약	180
10.6 적응형 임계값— $\sigma \times \text{scale}$	181
따라 해보기—Optuna로 임계값 최적화 (선택)	181
연습 문제	182
정리	183
더 읽을거리	183

11장. 통계적 차익거래—페어 선정	184
이 장의 흐름	186
학습 목표	186
11.1 왜 섹터 안에서 짝을 고를까?	187
11.2 페어 선정 파이프라인	188
공적분 검정—한 페어를 판정하기	189
전체 파이프라인—섹터별로 모든 페어 검정	190
11.3 반감기(halflife) 계산	191
11.4 Kalman 필터로 동적 헤지 비율 구하기	192
11.5 페어 품질 지표	193
11.6 페어 선정 파라미터 한눈에 보기	193
11.7 페어 선별 기준 요약	194
따라 해 보기—합성 페어로 파이프라인 돌려 보기	195
연습 문제	196
정리	197
더 읽을거리	197
12장. 통계적 차익거래—신호 생성	198
진입보다 청산이 먼저다	199
이 장의 흐름	200
학습 목표	200
12.1 진입 자격 (Entry Prerequisites)	201
12.2 진입 점수 (entry_score)	202
왜 곱셈인가	203
숫자 예시 한 줄	203
수렴 모멘텀(conv_boost)	203
12.3 청산 조건 (우선순위 순서)	204
어느 조건이 먼저 걸리는지—두 시나리오	205
12.4 BW-HMM 스프레드 레짐 보정	205
12.5 진입·청산 흐름 요약	206
12.6 코드로 보기—진입 점수 계산과 진입 자격 판정	207
12.7 RL과 결합하는 방법 (다음 장 예고)	208

정리	209
연습문제	210
더 읽을거리	210

머리말

이 책은 **본문의 설명·계산 예·상태 흐름으로 설계와 검증 원리를 이해하도록** 구성했다. 교육용 ZIP은 반복 실험을 위한 선택 자료다. `app.trading`이나 상용 앱 클래스 이름이 등장하는 코드는 구조를 설명하는 발췌일 수 있으며, 별도의 전체 제품 소스를 복원하라는 뜻은 아니다. 독립 실행 예제는 실행 조건과 기대 결과를 함께 표시한다.

교육용 자료와 제공 범위

교육용 파일 묶음 이름은 `stock-trading-ai-examples.zip`이다. 포함 범위는 `accounting-lab/`의 표준 라이브러리 백테스트와 수작업 사례 테스트다. 이 표준 라이브러리 예제에는 별도 패키지 잠금 파일이 필요 없다. 0·3장에 본문 실행 예와 기대값을 적었고 README에는 반복 시험 명령을 모았다. 전체 상용 앱이나 자동매매 운영 저장소의 접근권한을 제공하는 상품은 아니다.

자료는 위키독스의 책 사용법·전체 목차 안내에 기재된 제공 경로로 받는다. 해당 경로가 아직 표시되지 않은 판매 준비 단계라면 자료 배포가 완료된 것으로 해석하지 않는다. 링크나 첨부이 없을 때는 저자의 공개 안내 페이지에 문의한다. PDF의 코드 설명과 실행 가능한 교육용 묶음의 범위를 구분한다.

이 장의 흐름

이 머리말은 책의 방향을 먼저 잡는다. 차트 중심의 판단이 왜 한계에 부딪히는지 보고, 이 책이 어떤 방식으로 퀀트 시스템을 만드는지 확인한다.

TIP

가장 빠른 시작: 앞의 3쪽에서 이 책이 자신에게 맞는지 확인했다면, 이 머리말을 한 번에 완독하지 않아도 된다. 0장으로 이동해 첫 실행을 마친 뒤 돌아와도 흐름이 끊기지 않는다. 뒤쪽 원칙·로드맵·AI 활용법은 막힐 때 찾아보는 안내서로 써도 된다.

- 차트가 나를 속였던 경험
- 기술적 분석이 지금 시장에서 흔들리는 이유
- 르네상스 테크놀로지스 사례에서 가져온 핵심 관점
- 이 책의 모델들이 하나의 시스템으로 연결되는 방식
- 앞으로 만들 시스템과 필요한 준비

학습 목표

이 머리말을 읽고 나면 다음을 설명할 수 있어야 한다.

- 왜 이 책이 차트 패턴이 아니라 데이터 파이프라인과 검증에서 시작하는지
- “미래 데이터를 몰래 참고하는 오류(Look-ahead Bias)”가 왜 퀀트 시스템에서 치명적인지
- HMM, 강화학습, 통계적 차익거래가 이 책 안에서 어떤 역할을 맡는지
- 학습, 추론, 피쳐, 결측치 같은 머신러닝 기본 용어가 트레이딩 시스템에서 무엇을 뜻하는지
- 이 책을 읽기 전에 어느 정도의 Python과 통계 지식이 필요한지

나를 속인 차트

캔들스틱 차트를 한참 들여다보다가 매수 타이밍이라고 판단한 적이 있다. 같은 지지선에서 세 번이나 반등했고, RSI는 과매도권에 있었으며, MACD 골든크로스도 교과서에서 보던 모양 그대로였다. 여러 신호가 한 방향을 가리키는 것처럼 보였고, 나는 그 신호를 믿고 매수했다.

다음 날 가격은 그 “완벽한” 지지선을 그대로 뚫고 내려갔다. 처음 겪은 일도 아니었고, 마지막도 아니었다. 다만 그날 이후로 질문이 조금 달라졌다. 단순히 내가 차트를 못 읽은 것인지, 아니면 애초에 내가 의지하던 도구들이 지금의 시장을 설명하기에 부족한 것인지 따져봐야 했다.

기술적 분석의 현실

그 후 몇 달 동안 논문, 트레이딩 포럼, 성과 데이터를 찾아보며 같은 질문을 반복했다. 결론부터 말하면, 내가 의지하던 도구들은 지금의 시장을 설명하기에 부족했다.

전통적인 기술적 분석이란 캔들스틱 패턴, 이동평균 크로스오버, 볼린저밴드, RSI 같은 도구들을 말한다. 이런 도구가 쓸모없다는 뜻은 아니다. 다만 패턴 하나만으로 안정적인 우위를 주장하려면 시장 국면, 거래 비용, 표본 밖 성과까지 함께 검증해야 한다. 내가 쓰던 방식에는 그 검증이 빠져 있었다.

오늘날 시장에서는 알고리즘이 사람보다 훨씬 빠르게 주문을 처리하고, 거래량의 큰 비중을 알고리즘이 차지한다. 과거에는 군중 심리를 반영하던 패턴이, 이제는 기계가 먼저 감지하고 먼저 거래하는 신호가 됐다.

차트에서 지지선이 보이면 사람만 아는 게 아니다. 자동화된 전략도 같은 가격대의 주문 흐름과 유동성을 관찰한다. 스톱로스 주문이 몰린 구간에서는 체결과 변동성이 커질 수 있다. 따라서 과거에 관찰한 패턴이 지금도 같은 방식으로 작동한다고 가정해서는 안 된다.

비관적인 얘기를 하려는 것이 아니다. 개인 투자자가 흔히 쓰는 도구 중 상당수가 지금과는 다른 시장을 전제로 만들어졌다는 사실을 짚는 것뿐이다.

그래서 이 책은 “차트를 더 잘 읽는 법”에서 시작하지 않는다. 대신 시장을 데이터를 만들어내는 하나의 과정으로 본다. 그 안에 숨은 상태를 추정하고, 그 추정이 미래 데이터를 몰래 참조하지 않았는지 검증하는 방향으로 나아간다.

르네상스 테크놀로지스 사례에서 얻은 관점

이런 접근을 가장 먼저, 그리고 가장 철저하게 실천한 곳이 르네상스 테크놀로지스다.

짐 사이먼스는 1978년 모네펜트릭스(Monometrics)를 설립했고, 회사는 1982년 르네상스 테크놀로지스로 이름을 바꿨다. 메달리온 펀드는 1988년에 출범했다. 르네상스는 전통적인 재량 트레이더보다

수학자, 물리학자, 컴퓨터 과학자를 적극적으로 채용해 가격 데이터의 통계적 패턴을 연구한 것으로 알려져 있다. 메달리온은 이후 이례적으로 높은 장기 성과를 기록했다.

시장을 바라보는 다른 방식

사이먼스가 특별한 직관을 타고난 것은 아니다. 차이는 시장을 바라보는 방식에 있었다.

이 책은 르네상스의 공개된 정량적 접근에서 한 가지 관점을 가져온다. 시장을 하나의 고정된 법칙으로 설명하기보다, 성격이 다른 여러 국면이 바뀌는 과정으로 보는 것이다. 이것이 르네상스가 HMM을 썼다는 주장은 아니다. 구체적인 내부 모델은 공개되지 않았다. 이 책에서는 그 관점을 구현할 도구로 “숨겨진 상태(hidden state)”를 추정하는 HMM을 선택한다.

이 상태들은 눈에 보이지 않지만 데이터로 추정할 수 있다. 현재 시장 상태를 파악하면 그에 맞는 전략도 고를 수 있다.

HMM(은닉 마르코프 모델, Hidden Markov Model)은 바로 이 “보이지 않는 상태를 데이터로 추정하는” 문제를 다루는 도구다. 이 책 중반부에서 자세히 다룬다.

나는 르네상스 테크놀로지스 내부자가 아니고, 그 회사의 정확한 방법론을 알지 못한다. 다만 공개된 인터뷰와 학술 연구를 참고해 그 철학의 일부를 코드로 구현해볼 수 있었다. 이 책은 그 과정을 따라가는 기록이다.

AI 코딩 어시스턴트가 가능하게 한 것

2023년 이후 AI 코딩 어시스턴트가 실용적인 수준에 도달했다. 그 덕분에 이 책을 쓸 수 있었다. GitHub Copilot, Claude, ChatGPT 같은 도구는 복잡한 알고리즘을 설명하고, 미묘한 구현 오류를 찾고, 수식이 코드에서 어떤 의미인지 풀어주는 데 도움이 된다. 이해를 대신해주는지는 않는다. 하지만 이해에 도달하는 시간과 시행착오는 크게 줄여준다.

나는 이 책의 시스템을 AI와 함께 만들었다. 어려운 수학에서 막힐 때, 미묘한 버그를 찾을 때, 하이퍼파라미터(모델의 성능을 좌우하는 설정값)를 정할 때 모두 AI의 도움을 받았다. 이 책은 그 과정을 처음부터 다시 밟는다. 이론 설명에서 멈추지 않고, 지금 쓸 수 있는 도구로 실제 시스템을 어떻게 만드는지 보여주는 것이 목표다.

이 책을 마치면 만들 수 있는 것

시스템	한 줄 설명
HMM 시장 국면 감지기	미래 데이터 없이, 지금까지의 데이터만으로 시장이 어떤 국면(상승·횡보·하락)에 있는지 추정한다.
여러 자산을 함께 보는 HMM	주식, 채권 등 여러 자산의 정보를 함께 읽고 단일 자산 HMM에 보완 정보가 되는지 검증한다.
2단계 거시/미시 HMM	거시 경제와 개별 자산을 두 단계로 보고 시장을 Risk-On(위험 선호), Neutral(중립), Risk-Off(안전 선호)로 분류한다.
강화학습 트레이딩 에이전트	HMM이 감지한 국면을 입력받아 포지션을 스스로 결정한다.
통계적 차익거래 시스템	함께 움직이는 자산 쌍을 찾아 그 가격 차이를 노린다.
실전 배포 시스템	IBKR(Interactive Brokers)에 연결해 주문을 내고 체결 상태를 확인한다.

각 부품이 왜 필요한지, 잘못 만들면 어떻게 무너지는지를 이해하는 것이 이 책의 진짜 목표다.

이 책을 머신러닝 트레이딩 학습서로 읽는 법

이 책은 HMM과 강화학습을 다루지만, 더 넓게 보면 **머신러닝으로 주식 트레이딩 시스템을 만드는 학습서**다. 그래서 처음부터 몇 가지 말을 분명히 해두는 편이 좋다.

조금 더 정확히 말하면, 이 책은 **머신러닝과 딥러닝으로 주식시장을 파헤치는 책**이다. 차트를 눈으로 보며 “오를 것 같다”를 말하는 대신, 시장을 숫자로 쪼개고, 그 숫자에서 반복되는 구조를 찾고, 그 구조가 실제 매매에서 통하는지 검증한다.

전체 흐름은 다음 한 줄이다.

주식시장 → 데이터 → 피쳐 → 머신러닝 모델 → 딥러닝 에이전트 → 검증 → 주문

처음 읽을 때는 이 한 줄만 기억해도 충분하다.

단계	쉬운 말	이 책에서 하는 일
주식시장	복잡하게 움직이는 대상	가격, 거래량, 변동성, 금리, VIX가 계속 바뀐다
데이터	시장을 기록한 숫자	OHLCV, 스프레드, 매크로 지표를 모은다
피쳐	모델이 읽기 쉬운 요약	수익률, 변동성, 모멘텀, Z-score로 바꾼다
머신러닝	시장 상태를 읽는 도구	HMM으로 레짐을 추정한다
딥러닝/RL	행동을 배우는 도구	SAC가 포지션 크기를 정한다
검증	속았는지 확인하는 과정	OOS와 Walk-forward로 성과를 본다
주문	실제 돈이 움직이는 단계	리스크 조건을 통과한 신호만 보낸다

중요한 점은 “예측”보다 “분해”다. 이 책은 내일 종가 하나를 맞히려는 책이 아니다. 시장을 여러 조각으로 나누어 본다. 지금이 평균회귀(가격이 평균 수준으로 되돌아오는 성질)가 먹히는 장인지, 추세가 강한 장인지, 위험해서 쉬어야 하는 장인지를 차례로 묻고, 그에 맞춰 포지션을 얼마나 가져갈지 정한다.

질문 1: 지금 시장은 어떤 상태인가? → HMM
 질문 2: 이 페어는 벌어졌는가? → Z-score / StatArb
 질문 3: 지금 얼마나 크게 들어갈까? → SAC 강화학습
 질문 4: 이 결과를 믿어도 되는가? → OOS / Walk-forward 검증

이렇게 보면 뒤에서 나오는 HMM, 칼만 필터, SAC, MLflow가 따로 노는 기술이 아니다. 모두 하나의 목적을 위해 이어진다. **시장을 숫자로 읽고, 모델로 판단하고, 검증으로 걸러낸 뒤, 안전하게 실행하는 것이다.**

그렇다면 이 흐름의 한가운데에서 판단을 맡는 모델이란 무엇일까. 머신러닝 모델은 마법처럼 가격을 맞히는 장치가 아니다. 과거 데이터에서 반복되는 구조를 찾아, 새 데이터가 들어왔을 때 현재 상황을 판단하는 도구다.

여기서 **모델(model)**이라는 말이 처음에는 헛갈릴 수 있다. 직관적으로는 이렇게 생각하면 된다.

모델 = 숫자를 넣으면 판단을 돌려주는 계산기

일반 계산기는 $2 + 3$ 을 넣으면 5를 돌려준다. 머신러닝 모델은 오늘의 수익률, 변동성, 거래량 같은 숫자를 넣으면 “평균회귀 70%, 추세 20%, 위험 10%”같은 판단을 돌려준다.

오늘 시장 숫자들 수익률, 변동성	---->	학습된 모델 HMM	---->	현재 시장 판단 MR 70%, Trend 20%, Volatile 10%
-----------------------	-------	---------------	-------	---

Python 함수에 비유하면 아래와 같다.

```
def model(today_features):
    return prediction
```

함수처럼 입력을 넣으면 출력이 나온다. 차이는 함수 안의 규칙을 사람이 전부 직접 쓰지 않는다는 점이다. 머신러닝에서는 알고리즘이 과거 데이터를 보고 함수 안의 숫자들을 맞춘다. 학습이 끝난 뒤에는 새 피처를 넣어 판단을 얻는다.

이 관계를 가장 짧게 쓰면 다음 세 줄이다.

데이터로 학습한다	-> 모델 안의 숫자가 정해진다
새 피처를 넣는다	-> 모델이 판단을 낸다
그 판단을 검증한다	-> 실전에 쓸지 말지 결정한다

실제 시스템에서는 학습이 끝난 모델을 파일로 저장해 둔다. 그래야 매번 처음부터 다시 학습하지 않고, 필요할 때 파일을 불러와 바로 추론할 수 있다.

학습 단계
5년치 데이터 -> 피처 생성 -> `model.fit(X_train)` -> 모델 파일 저장
예: `hmm_model.pkl`, `sac_actor.pt`

추론/서빙 단계
오늘 데이터 -> 피처 생성 -> 모델 파일 로드 -> `model.predict(today_features)` -> 판단 결과

여기서 **서빙(serving)**은 저장된 모델 파일을 메모리에 올려두고, 다른 코드가 “오늘 피처를 줄 테니 판단 결과를 달라”고 요청할 수 있게 열어두는 일이다. 작은 프로젝트에서는 Python 코드가 그 파일을 직접 불러와도 된다. 운영 시스템에서는 FastAPI 같은 서버가 이를 미리 올려 두고 HTTP 요청으로 결과를 돌려준다.

헛갈리는 말	쉬운 설명	이 책의 예
데이터	모델이 공부하는 재료	5년치 OHLCV, 거래량, 금리
피처	데이터를 모델이 읽기 좋게 바꾼 숫자	수익률, 변동성, 모멘텀
알고리즘	모델을 만드는 방법	HMM의 Baum-Welch, SAC 학습 알고리즘
파라미터	학습으로 정해지는 함수 안의 숫자	HMM의 A, B, π / 신경망 가중치
모델	학습된 파라미터를 가진 판단 함수	오늘 피처 -> 시장 상태 확률

요리로 비유하면 데이터는 재료, 알고리즘은 조리법, 파라미터는 불 세기와 양념 비율, 모델은 그 조리법과 비율이 정해진 완성된 조리 방식이다. 같은 조리법이라도 어떤 재료로 연습했는지에 따라 결과가 달라진다. 머신러닝도 다르지 않다. 같은 HMM 알고리즘이라도 어떤 기간의 데이터를 학습했는지에 따라 만들어지는 모델은 달라진다.

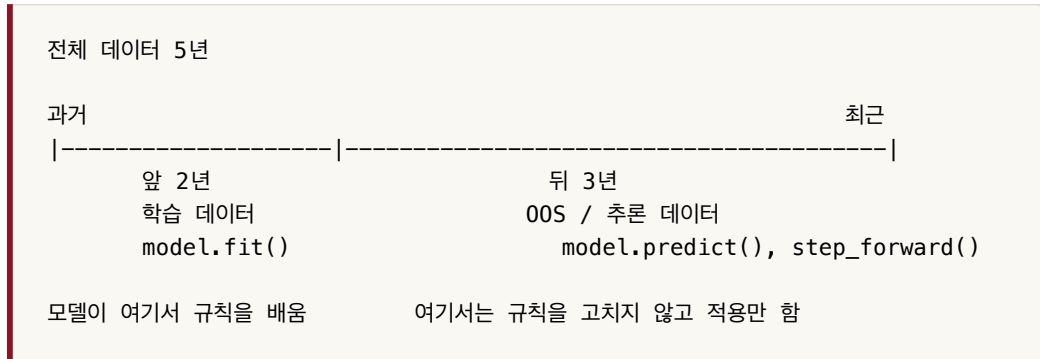
이 책의 HMM도 마찬가지다. HMM은 내일 종가를 직접 예측하지 않는다. 과거의 수익률, 변동성, 거래량 패턴을 보고 “지금 시장이 평균회귀에 가까운가, 추세에 가까운가, 위험 구간에 가까운가”를 확률로 추론한다. 이렇게 구분한 시장 상태를 앞으로 **레짐(regime)**이라고 부른다.

앞으로 반복해서 나올 머신러닝 용어를 트레이딩 언어로 바꾸면 다음과 같다.

용어	일반적인 뜻	이 책에서의 뜻
데이터	모델이 읽는 원재료	OHLCV, 스프레드, 금리, VIX 같은 시장 기록
피처(feature)	원자료를 모델이 읽기 쉽게 바꾼 숫자	수익률, 변동성, 모멘텀, 거래량 비율, Z-score
학습(training / fit)	과거 데이터로 모델 파라미터를 맞추는 일	HMM의 레짐 분포, SAC의 정책 네트워크를 과거 구간에서 조정
추론(inference / predict)	학습된 모델로 새 입력을 판단하는 일	오늘까지의 피처로 현재 레짐 확률이나 포지션 크기를 계산
결측치(missing value / NaN)	값이 비어 있거나 계산할 수 없는 칸	상장 전 데이터, 거래 정지, rolling window 초기 구간

용어	일반적인 뜻	이 책에서의 뜻
검증(validation / OOS)	처음 보는 데이터에서도 통하는지 확인	훈련에 쓰지 않은 뒤쪽 기간으로 백테스트

이 구분이 중요한 이유는 간단하다. **학습과 추론을 섞으면 백테스트가 거짓말을 한다.** 학습은 과거 훈련 구간에서만 해야 하고, 추론은 그 시점까지 이미 관측된 데이터만 써야 한다. 결측치를 아무렇게나 0으로 채워셔도 안 된다. 0은 “값이 없다”가 아니라 “정말 0이다”라는 뜻으로 모델에 전달되기 때문이다. 가장 쉬운 예로 5년치 데이터가 있다고 하자. 앞 2년은 모델이 공부하는 구간이고, 뒤 3년은 모델이 처음 보는 시험 구간이다. 실제 비율은 전략마다 달라질 수 있지만, 개념은 항상 같다.



여기서 뒤 3년을 **OOS(Out-of-Sample)** 라고 부른다. 말 그대로 “샘플 밖”, 즉 학습에 쓰지 않은 구간이다. OOS는 과거 데이터 안에 있지만, 모델 입장에서는 미래처럼 남겨둔 시험지다. 앞 2년으로 학습한 모델이 뒤 3년에서도 버티면, 그제야 “과거를 외운 것만은 아닐 수 있다”고 말할 수 있다.

반대로 5년 전체로 먼저 학습한 뒤 같은 5년에서 성과를 재는 것은, 시험지를 미리 보고 시험을 치는 일과 같다. 수익률은 좋아 보이지만 실전에서는 재현되지 않는다.

따라서 이 책을 읽을 때는 매 장에서 다음 네 질문을 스스로 던져 보면 된다.

1. 지금 모델이 읽는 피처는 무엇인가?
2. 이 피처에 결측치나 미래 정보가 섞이지 않았는가?
3. 학습은 어느 기간으로 했는가?
4. 추론은 실제 거래 시점에도 같은 방식으로 재현 가능한가?

HMM, 칼만 필터, SAC, MLflow 같은 이름은 뒤에서 하나씩 익히면 된다. 먼저 잡아야 할 뼈대는 **데이터 -> 피처 -> 학습 -> 추론 -> 검증 -> 주문**이라는 머신러닝 트레이딩의 기본 흐름이다.

읽다가 길을 잃었을 때 보는 지도

이 책에는 HMM, Kalman, Kelly, SAC, MLflow처럼 이름이 다른 도구가 많이 나온다. 처음 읽을 때는 도구 이름보다 **각 도구가 답하는 질문**을 먼저 보면 된다.

질문	도구	쉬운 역할
시장 기록이 믿을 만한가?	데이터 파이프라인	깨끗한 자료 준비
모델이 읽을 숫자는 무엇인가?	피쳐 엔지니어링	원자료를 질문으로 바꾸기
숫자가 너무 흔들리지는 않는가?	Kalman	노이즈 줄이기
지금 시장은 어떤 장인가?	HMM	시장 날씨 읽기
여러 모델 의견이 일치하는가?	Ensemble	회의실에서 의견 합치기
어떤 페어를 거래할까?	StatArb / Pairs	거래 후보 고르기
지금 얼마나 벌었나?	Z-score	고무줄 늘어남 재기
얼마나 크게 들어갈까?	Kelly / SAC	포지션 크기 정하기
이 거래를 버틸 수 있는가?	Risk	손실 한도 확인
주문이 실제로 처리됐는가?	Broker	체결과 포지션 확인
운영 중 계속 정상인가?	Monitoring / MLflow	성과와 모델 이력 감시

이 표를 계속 옆에 두고 읽으면 된다. 어려운 장이 나와도 “지금 새 알고리즘을 외우는 중”이라고 생각하지 말고, “이 장은 위 질문 중 어느 것을 더 잘 답하려고 하는가”를 찾으면 된다.

또 하나의 기준은 **입력과 출력**이다. 어떤 모델이나 모듈이 나오면 먼저 아래 세 줄을 채워보자.

입력: 무엇을 받는가?
 출력: 무엇을 돌려주는가?
 다음 단계: 그 출력이 어디로 가는가?

예를 들어 HMM은 이렇게 정리된다.

입력: 수익률, 변동성, 모멘텀, 거래량 같은 피쳐
 출력: MR / Trend / Volatile 레짐 확률
 다음 단계: Z-score 해석, SAC 상태 벡터, 리스크 판단에 사용

SAC는 이렇게 정리된다.

입력: HMM 레짐 확률, Z-score, 현재 포지션, 손익, 변동성
 출력: 환경 계약에 따라 ENTER/HOLD/EXIT 후보 또는 -1~+1 목표 포지션
 다음 단계: 행동 후처리와 Risk 레이어가 의미·한도를 확인한 뒤 주문 여부 결정

이렇게 입력과 출력을 잡으면, 수식이나 코드가 조금 어려워도 전체 흐름을 잃지 않는다.

하루를 따라가 보는 전체 예시

개념이 추상적으로 느껴지면 하루짜리 장면으로 생각하면 쉽다. 예를 들어 2026년 5월 20일 장 마감 후 시스템이 다음 날 판단을 준비한다고 하자.

1. 데이터 수집
오늘까지의 가격, 거래량, 금리, VIX를 가져온다.
2. 피쳐 생성
원자료를 수익률, 변동성, 모멘텀, Z-score 같은 숫자로 바꾼다.
3. HMM 추론
학습된 HMM 모델 파일을 불러와 오늘 시장 레짐 확률을 계산한다.
예: MR 65%, Trend 20%, Volatile 15%
4. 페어 신호 확인
두 종목의 가격 차이가 얼마나 벌어졌는지 Z-score로 본다.
예: z-score = 2.4
5. RL/SAC 추론
HMM 확률, Z-score, 현재 포지션, 손익 상태를 함께 넣는다.
SAC가 "0.35만큼 진입" 같은 포지션 크기를 돌려준다.
6. 리스크 점검
포지션 한도, 손실 한도, 거래 비용, 유동성을 확인한다.

7. 주문 또는 대기

조건을 통과하면 주문을 내고, 아니면 HOLD로 끝낸다.

이 예시에서 모델이 새로 공부하는 단계는 없다. 장중이나 실시간 추론 단계에서는 이미 학습된 모델 파일을 불러와 판단만 한다. 다시 말해 운영 중인 모델은 매 순간“학습하는 학생”이 아니라“시험장에서 답안을 내는 계산기”에 가깝다.

반대로 학습은 별도의 시간에 한다. 예를 들어 주말이나 장 마감 후에 과거 데이터를 모아 HMM과 SAC를 다시 학습하고, 성능이 OOS에서 좋아졌을 때만 새 모델 파일을 등록한다. 그래서 실전 시스템은 아래처럼 두 개의 시간표로 나뉜다.

느린 시간표: 학습 / 검증 / 모델 등록

과거 데이터 -> 학습 -> OOS 검증 -> 모델 파일 저장 -> 승인

빠른 시간표: 실시간 추론 / 주문

오늘 데이터 -> 피쳐 생성 -> 모델 파일 로드 -> 추론 -> 리스크 체크 -> 주문 -> 체결 확인
-> 모니터링

이 둘을 섞지 않는 것이 중요하다. 느린 시간표는 모델을 만드는 공장이고, 빠른 시간표는 만들어진 모델을 사용하는 매장이다. 공장에서 레시피를 계속 바꾸는 것은 가능하지만, 손님에게 음식을 내는 순간에는 이미 정해진 레시피로 빠르고 일관되게 만들어야 한다.

여기에 하나를 더 붙이면 운영 시스템의 그림이 완성된다. 주문은 끝이 아니다. 주문이 브로커에 접수됐는지, 실제로 체결됐는지, 내부 포지션과 브로커 포지션이 일치하는지, 이후 성과가 기대 범위 안에 있는지 계속 확인해야 한다.

모델 관점:

오늘 상태 -> 판단 결과

운영 관점:

판단 결과 -> 주문 -> 체결 -> 포지션 -> 손익 -> 알림

그래서 이 책의 후반부에서 리스크 관리, IBKR 연결, 모니터링을 다룬다. 머신러닝 모델이 좋은 판단을 해도, 주문이 실패하거나 체결 확인이 누락되거나 성과 저하를 늦게 발견하면 실전 시스템은 무너진다. **주식시장을 파헤치는 일은 모델을 만드는 데서 끝나지 않고, 그 모델이 실제 돈 앞에서 계속 정상 작동하는지 확인하는 데까지 이어진다.**

모델을 더 쉽게 쪼개면

모델이라는 단어가 아직 모호하면 세 부분으로 나누어 보면 된다.

모델 = 입력 형식 + 학습된 숫자 + 출력 의미

HMM 모델을 예로 들면 입력 형식은 [수익률, 변동성, 모멘텀, 거래량 비율]이다. 학습된 숫자는 전이행렬 A, 상태별 평균과 분산, 초기 확률이다. 출력 의미는 [MR 확률, Trend 확률, Volatile 확률]이다.

SAC 모델을 예로 들면 입력 형식은 28차원 상태 벡터다. 학습된 숫자는 신경망 가중치다. 출력 의미는 포지션 크기다.

모델	입력	학습된 숫자	출력
HMM	시장 피쳐	A, 상태별 분포, π	레짐 확률
SAC Actor	RL 상태 벡터	신경망 가중치	포지션 크기
Z-score 규칙	스프레드 통계	평균과 표준편차	진입 후보

따라서“모델 파일을 저장한다”는 말은 단순히 파일 하나를 저장한다는 뜻이 아니다. 나중에 같은 입력을 넣었을 때 같은 의미의 출력을 얻기 위해, 입력 순서와 학습된 숫자와 출력 해석을 함께 고정한다는 뜻이다.

르네상스 사례에서 얻은 원칙

이 책 전체를 관통하는 원칙이 세 가지 있다. 르네상스 테크놀로지스의 트레이딩 방식을 분석하면서 뽑아낸 것들이다.

1. 인과성 원칙

미래 데이터는 현재 결정에 쓰면 안 된다.

당연하게 들리지만, 코드로 구현하다 보면 생각보다 쉽게 어긴다. 흔히 쓰는 라이브러리 함수 중에도 내부에서 미래 정보를 살짝 섞어 쓰는 것들이 있다. 이 책에서는 그런 함수 대신, “지금까지의 데이터만 본다”는 약속을 지키는 방식(Forward Algorithm)을 직접 구현한다. 자세한 이유는 5장에서 설명한다. 한 번 이해하고 나면 이후 코드에서 같은 문제가 바로 눈에 들어온다.

2. 앙상블 원칙

모델 하나에만 의존하지 않는다.

이 시스템은 일봉 HMM, 시간봉 HMM, 강화학습 에이전트, 통계적 차익거래 신호를 함께 사용한다. 모델의 판단이 엇갈리면 전략에 따라 관망하거나 포지션을 줄인다. 모든 모델의 만장일치를 항상 요구하는 것은 아니다. 어떤 출력은 진입 게이트로, 어떤 출력은 가중치나 포지션 배율로 쓰이므로 결합 규칙을 먼저 확인해야 한다.

3. Walk-forward 검증 원칙

전략이 실제로 작동하는지 확인하는 방법은 하나뿐이다.

모델이 한 번도 보지 못한 데이터로, 시간 순서대로 반복 테스트하는 것이다. 데이터를 한 번 나눠 훈련하고 테스트하는 방식만으로는 부족하다. Walk-forward 검증은 이 책 모든 전략에 적용하는 기본 기준이다.

앞으로의 여정

전체 시스템은 데이터 -> 노이즈 제거 -> 국면 감지 -> 포지션 결정 -> 검증과 실전 운영의 순서로 쌓아 올린다. 각 단계가 하나의 챕터 묶음으로 이어진다.

단계	하는 일
1	시장 데이터를 수집하고 저장한다
2	가격에 섞인 잡음을 줄인다

단계	하는 일
3	여러 자산을 함께 보는 HMM으로 시장 국면을 추정한다
4	거시/미시 두 단계로 국면을 Risk-On / Neutral / Risk-Off로 정리한다
5	국면에 맞춰 얼마나 큰 포지션을 잡을지 정한다
6	통계적 차익거래로 추가 수익 기회를 찾는다
7	강화학습 에이전트가 포지션 판단을 보조한다
8	리스크 관리, 브로커 연결, 모니터링으로 운영 조건을 갖춘다

처음 읽을 때는 0장에서 실행 경험을 만든 뒤 1~18장을 **기본 시스템을 한 번 완성하는 흐름**으로 보면 된다. 19~27장은 같은 재료를 더 깊게 다루는 심화부다. 페어 이론, 완전한 HMM 파이프라인, 운영 검증, 분포형 RL, MLflow 서빙처럼 실제 운용 단계에서 다시 필요해지는 내용을 모아 둔 구간이다.

참고: 백테스트 성과를 읽는 법

이 책에는 여러 백테스트 결과가 등장하지만, 특정 숫자를 성과 약속처럼 읽으면 안 된다. 같은 전략도 데이터 기간, 거래 비용, 종목군(universe), 재학습 주기, 슬리피지 가정에 따라 결과가 크게 달라진다. 첫 회독에서는 숫자보다 다음 세 가지를 본다.

- 미래 데이터를 보지 않았는가
- 최대 낙폭과 거래 비용을 함께 봤는가
- 표본 밖 구간에서도 비슷한 패턴이 유지되는가

전체 챗터 로드맵

Part	챕터	주제
시작	머리말·0장	독자 수준 확인, AI와 첫 백테스트 실행
I	1-3장	시스템 전체 그림, Look-ahead Bias, 데이터 파이프라인, 백테스트 엔진
II	4-5장	HMM 기본 구조와 학습 알고리즘, 미래 정보 없이 추론하는 법
III	6-8장	칼만 필터(노이즈 제거), 여러 자산을 함께 보는 HMM, 2단계 거시/미시 HMM (시스템의 핵심)
IV	9-10장	포지션 크기 정하기, HMM 앙상블로 방향 신호 만들기
V	11-12장	함께 움직이는 페어 찾기와 통계적 차익거래 신호 만들기
VI	13-15장	강화학습 환경, SAC 에이전트, 하이퍼파라미터 튜닝
VII	16-18장	리스크 관리, 브로커 연동, 실시간 모니터링
VIII	19-27장	심화: 페어 이론, 레짐 기반 Z-score, 완전 파이프라인, 운영 검증, 분포형 RL, MLflow 서빙

표에 등장한 용어 대부분은 처음 보면 낯설다. 지금 외울 필요는 없다. 첫 회독에서는 Part I~VII로 “데이터에서 주문과 모니터링까지”한 바퀴를 돈다. Part VIII은 두 번째 회독 또는 실제 시스템을 다듬을 때 필요한 심화 구간으로 보면 된다.

회독별로 다르게 읽기

이 책은 한 번에 모든 수식과 코드를 이해하도록 쓴 책이 아니다. 처음부터 모든 파라미터와 구현 세부를 잡으려고 하면 흐름을 놓치기 쉽다. 회독마다 목표를 다르게 잡는 편이 낫다.

회독	목표	읽는 기준
1회독	큰 흐름 잡기	데이터가 어떻게 모델 판단과 주문으로 이어지는지 본다
2회독	코드와 개념 연결	각 장의 입력, 출력, 위험 지점을 코드에서 확인한다
3회독	실험과 수정	파라미터를 바꾸고 OOS 결과가 어떻게 달라지는지 본다

바이브코더라면 회독보다 **작동 경로**를 먼저 잡아도 된다. 0장 → 1~3장 → 11~12장 → 16~18장 순서로 읽으면 데이터, 백테스트, 단순한 페어 신호, 리스크, 주문, 모니터링까지 한 바퀴를 먼저 볼 수 있다. 4~10장의 HMM과 13~15장의 강화학습 내부는 그다음 회독에서 돌아온다. 건너뛴 장이 덜 중요하다라는 뜻이 아니라, 추상적인 모델을 보기 전에 시스템의 입출력을 먼저 경험하라는 뜻이다.

1회독에서는 수식을 완전히 이해하지 못해도 괜찮다. 대신 각 장이 아래 질문 중 무엇을 다루는지 잡으면 된다.

이 장은 데이터를 준비하는 장인가?
 시장을 읽는 장인가?
 기회를 찾는 장인가?
 크기를 정하는 장인가?
 위험을 막는 장인가?
 운영 상태를 확인하는 장인가?

2회독에서는 코드로 내려간다. `fit()`은 어디서 호출되는지, `predict()`나 `step_forward()`는 어느 구간에서만 실행되는지, OOS(out-of-sample, 표본 외) 구간이 학습 구간과 섞이지 않았는지 확인한다. 이때부터 수식은 “외울 대상”이 아니라 “코드가 왜 이렇게 생겼는지 설명하는 지도”가 된다.

3회독에서는 실험한다. 윈도우 길이, 진입 임계값, 레짐 수, 거래 비용, 리스크 한도 같은 값을 조금씩 바꿔본다. 단, 같은 OOS 결과를 보면서 계속 튜닝하면 안 된다. 값을 바꿨다면 어떤 가설로 바꿨는지 기록하고, 검증 구간과 최종 테스트 구간을 분리해야 한다.

Part 별 최소 이해 목표

각 Part에서 처음 읽을 때 반드시 잡아야 할 한 문장은 다음과 같다.

Part	최소 이해 목표
I	데이터와 백테스트가 거짓말하지 않게 기반을 만든다
II	HMM은 가격이 아니라 시장 상태를 확률로 읽는다
III	노이즈를 줄이고 여러 시장을 함께 봐야 레짐 판단이 안정된다
IV	신호는 방향이고, Kelly는 그 방향을 돈의 크기로 바꾼다
V	페어 트레이딩은 종목 방향이 아니라 두 가격의 관계를 거래한다
VI	RL(강화학습)은 새 신호를 만드는 만능 AI가 아니라 포지션 크기를 조절하는 총이다
VII	모델이 맞아도 리스크, 주문, 모니터링이 틀리면 실전 시스템은 실패한다
VIII	같은 재료를 더 엄격하게 검증하고 운영 가능한 형태로 다듬는다

첫 회독의 기준은 이 표 하나면 충분하다. 세부 수식이 어렵더라도 각 Part의 최소 목표를 자기 말로 설명할 수 있다면 그 회독은 성공이다.

AI를 페어 프로그래머로 쓰기

이 책은 막히는 부분마다 AI 코딩 어시스턴트를 어떻게 활용했는지 직접 보여준다. 알고리즘을 이해할 때 쓴 프롬프트, 어시스턴트가 찾아낸 미묘한 버그, 그 대화 덕분에 더 나은 방향으로 바꾼 순간들이 본문 곳곳에 담겨 있다.

목표는 생각을 기계에 넘기는 게 아니다. **좋은 시니어 동료를 대하듯** 쓰면 된다. 모르는 건 묻고, 이해한 건 확인하고, 아는 부분은 빠르게 넘어가고, 막힌 부분은 함께 풀어간다.

각 장의 프롬프트는 네 요소를 반복한다.

목표: 이번에 얻을 한 가지 결과
범위: 읽거나 수정해도 되는 파일
금지: 실주문, 미래 데이터 사용, 근거 없는 파일명 추측
검증: 어떤 명령과 출력으로 완료를 확인할지

AI가 만든 코드는 실행 결과와 테스트로 확인한다. 특히 트레이딩에서는 “코드가 실행된다”와 “검증이 정직하다”가 전혀 다른 조건이다. 미래 데이터가 섞이지 않았는지, 거래 비용이 반영됐는지, 실주문으로 연결되지 않았는지를 사람이 마지막에 확인해야 한다.

퀀트 트레이딩이 박사들만의 영역은 아니다. 도구와 방향을 제대로 잡으면 실용적인 트레이딩 시스템을 직접 만들 수 있다. 이 책은 그 과정을 기록한 것이다.

이 장에서 반드시 기억할 것

- 이 책은 차트 패턴을 더 잘 외우는 책이 아니다. 시장을 “데이터로 보는 시스템”을 만드는 책이다.
- 가장 중요한 약속은 단 하나, “**미래 데이터를 현재 결정에 쓰지 않는다**”이다. 이 약속이 깨지면 좋아 보이는 수치도 정직한 검증 결과로 믿을 수 없다.
- 모델 하나에 모든 걸 걸지 않는다. 모델 합의도, 가중 평균도, 위험할 때의 관망도 사전에 정한 결합 규칙과 OOS 검증을 거쳐 사용한다.
- 처음부터 모든 수학을 완벽히 알 필요는 없다. 각 장에서 “왜 이 개념이 필요한가”, “잘못 만들면 어떤 문제가 생기는가”를 순서대로 따라가면 된다.

사전 요구사항

본격적인 내용에 들어가기 전, 다음 정도는 준비되어 있어야 한다.

- **Python 3.12** —이 책의 예제 저장소는 $\geq 3.12, < 3.13$ 으로 버전을 고정한다.
- **NumPy와 Pandas 기본 사용 경험** —배열을 다루고 데이터프레임을 조작할 수 있는 정도면 된다.
- **기초 통계 개념** —평균, 분산, 정규분포가 무엇인지 아는 수준이면 충분하다. 더 어려운 개념은 필요할 때 책 안에서 설명한다.
- **호기심** —시장이 왜 그렇게 움직이는지 알고 싶은 마음이 있으면 된다.

준비됐다면, 다음 장에서 전체 시스템의 설계도를 펼쳐 보자.

0장. 바이코딩 퀵스타트

—먼저 한 번 검증해보기

이 장의 목표는 큰 자동매매 시스템을 설치하는 것이 아니다. 본문의 작은 숫자 예를 따라가며 **AI가 설
명한 손익이 맞는지 직접 확인하는 것**이다. 운영 저장소·브로커 계정·시장 데이터 없이 읽을 수 있다.
Python을 실행할 수 있다면 아래 코드까지 확인하고, 실행 환경이 없으면 표를 손으로 계산한다.

먼저 답을 정한다

초기 현금 1,000으로 가격 100인 자산 10개를 보유한다고 하자. 이후 가격은 80으로 떨어졌다가 100으로 돌아온다. 비용은 이 첫 계산에서 0으로 둔다.

시점	현금	보유 수량	가격	총자산
거래 전	1,000	0	100	1,000
매수 후	0	10	100	1,000

시점	현금	보유 수량	가격	총자산
가격 하락	0	10	80	800
가격 회복	0	10	100	1,000
청산 후	1,000	0	100	1,000

총자산은 현금 + 수량 × 가격이다. 최종 손익은 0이지만 가장 큰 낙폭은 $(1000-800)/1000=0.2$, 즉 20%다. 최종 잔고만 보면 중간에 감수한 위험을 놓친다.

본문만으로 실행하는 확인 코드

다음은 `first_check.py`라는 새 파일에 붙인다. Python 표준 라이브러리만 사용하며 파일 다운로드가 필요 없다.

```
prices = [100.0, 80.0, 100.0]
initial = 1000.0
quantity = initial / prices[0]
cash = initial - quantity * prices[0]
equity = [initial] + [cash + quantity * p for p in prices]
peak = initial
mdd = 0.0
for value in equity:
    peak = max(peak, value)
    mdd = max(mdd, (peak - value) / peak)
final_cash = cash + quantity * prices[-1]
assert final_cash == 1000.0
assert mdd == 0.2
print(final_cash, mdd)
```

터미널에서 `python3 first_check.py`를 실행하면 `1000.0 0.2`가 출력된다. Windows에서 Python 명령이 `py`라면 `py first_check.py`를 쓴다. 이 코드는 고정 수량의 손익 계산이다. 신호 생성·실제 주문·HMM 학습을 포함하지 않는다. 처음부터 끝까지 실행 가능한 비용 포함 엔진은 3장 본문에 있다.

시에게 요청하는 방법

위 표와 Python 코드를 함께 검토해줘.

1. 총자산과 최대 낙폭을 손으로 계산해줘.
2. 현금만 기록하면 어떤 위험을 놓치는지 설명해줘.
3. 매수·매도에 각각 10bp 수수료가 붙으면 최종 현금을 계산해줘.
4. 브로커·외부 데이터·새 패키지를 추가하지 마.

답을 낸 뒤 표의 어느 값으로 검증했는지 표시해줘.

10bp는 거래 금액의 0.1%다. 수량 10을 100에 매수·매도하면 각 비용은 1, 합계는 2, 최종 현금은 998이다. 이 비용 예에서는 초기 전액을 수량으로 정해 수수료 1만큼 현금이 잠식 음수가 된다. 차입을 허용하지 않는 실제 계좌라면 수수료를 남기도록 매수 수량을 줄여야 한다. 3장의 고정 수량 실습과 실제 계좌 제약을 구분한다.

시간 순서를 확인한다

신호가 오늘 종가를 보고 만들어졌다면 그보다 앞선 오늘 시가로 체결했다고 가정할 수 없다. 이 책의 기본 실습은 **t봉 종가 뒤에 신호 확정 -> t+1봉 시가 체결 -> t+1봉 종가 평가**다. 앞서 확정된 신호로 시가에 진입한 포지션의 당일 손익은 정상적으로 포함한다.

실패를 좁히는 순서

증상	먼저 확인할 것
Python 명령을 찾지 못함	설치된 명령이 python3 또는 py인지 확인
들여쓰기 오류	코드 블록만 복사했는지와 들여쓰기 확인
낙폭이 0으로 나옴	가격 80의 자산 800을 기록했는지 확인
최종 현금과 손익 합계 불일치	누적 손익을 다시 더했는지·비용이 중복됐는지 확인

새 패키지를 설치해 오류를 덮지 말고 위 코드와 다른 줄부터 확인한다. 좋은 답을 생성했는지보다 기대값과 실제 출력이 맞는지를 우선한다.

다음 장으로 넘어갈 기준

- 최종 손익 0과 최대 낙폭 20%가 동시에 성립하는 이유를 설명한다.
- 수수료를 실제 체결마다 한 번씩 계산한다.
- 신호 생성 시각과 체결 시각을 구별한다.

이 세 가지를 확인하면 1장의 구조와 3장의 확장 실습으로 넘어간다. `accounting-lab` ZIP은 테스트와 CSV를 더 제공하는 선택 자료다. 책 속 `app.trading` 같은 경로는 저자의 시스템 구조를 설명하는 사례 이름이며, 그 비공개 패키지가 있어야 본문의 개념을 이해하는 전제는 두지 않는다.

1장. 시스템 아키텍처—AI 퀀트 트레이딩의 설계 철학

Part I —기본 뼈대 (1~3장) 이 파트에서는 수익을 내는 전략보다 먼저, 전략을 속이지 않는 기반을 만든다. 1장은 전체 설계도, 2장은 데이터 파이프라인, 3장은 백테스트 엔진이다. 첫 회독에서는 “데이터가 들어와서, 검증 가능한 백테스트 결과로 나오는 순서”만 잡으면 충분하다.

봉 차트를 처음 배웠을 때, 이걸로 시장을 읽을 수 있다고 생각했다. 지지선과 저항선을 그리고 캔들 패턴을 외웠다. 도지, 해머, 장악형 패턴까지 공부할수록 더 많은 모양이 눈에 들어왔다. 처음으로 차트가 읽히는 것 같던 그 순간이 아직도 기억난다.

막상 거래해보니 달랐다. 지지선이라 생각한 자리에서 가격이 반등하는 날도 있었다. 하지만 다음에는 그 자리를 그대로 뚫고 내려갔다. 같은 캔들 패턴이 어떤 날은 맞고, 어떤 날은 반대로 움직였다. 패턴이 실제로 있는 것인지, 내가 사후에 만들어내는 것인지 구분하기 어려웠다.

나중에는 질문이 잘못됐다는 사실을 알았다. 차트 패턴이 보인다는 것과 그 패턴이 비용 반영 후에도 반복되는 예측력을 가진다는 것은 다르다. 오늘날 시장에서는 자동화된 주문이 같은 가격대의 유동성과 주문 흐름에 빠르게 반응한다. 따라서 지지선·저항선만 보고 내린 판단은 시장 국면과 거래 비용이 달라지면 쉽게 무너질 수 있다.

이 사실을 받아들이면 선택은 두 가지다. 계속 차트를 보면서 알고리즘에 당하지 않기를 바라거나, 시장을 보는 방식 자체를 바꾸거나.

나는 두 번째를 선택했고, 그 결과가 이 책이다. 이 책의 핵심 관점은 하나다—**가격 자체를 예측하려 하지 않는다. 대신 가격 뒤에 숨어 있는 시장 국면(레짐, regime)을 추론한다.** 그 도구가 HMM(은닉 마르코프 모델, Hidden Markov Model)이다. 자세한 내용은 뒤 장에서 다룬다.

AI가 나오기 전까지 이런 시스템을 개인이 만드는 것은 사실상 불가능에 가까웠다. 통계 지식, 데이터 파이프라인, 백테스트 엔진, 실행 시스템, 모니터링—이 모든 것을 혼자 구축해야 했기 때문이다. 지금은 AI와 함께 이 아키텍처를 혼자 구축할 수 있다. 이 책은 그 과정을 처음부터 끝까지 다룬다.

1장은 시스템 전체의 설계 철학을 다룬다. 각 구성 요소가 왜 그 위치에 있는지, 서로 어떻게 연결되는지 먼저 파악해두면 이후 내용을 따라가기 훨씬 쉽다.

이 장의 흐름

먼저 전체 그림을 잡고, 그다음 환경 설정과 첫 실행으로 내려간다.

- 시스템 구조—5개 레이어가 어떻게 쌓이는지
- 디렉터리 구조—어떤 폴더가 어떤 역할을 하는지
- 핵심 구성요소—모든 전략이 따르는 공통 약속
- 인과성 원칙—미래 데이터를 실수로 쓰지 않는 규칙
- 데이터 흐름—백테스트가 돌아가는 순서
- 개발 환경 설정과 첫 백테스트 실행

학습 목표

이 장을 마치면:

- 5개 레이어로 구성된 아키텍처에서 데이터가 어떤 경로로 흐르는지 그릴 수 있다
- HMM Signal, SAC RL Agent, StatArb 신호가 각각 어떤 역할을 하는지 한 문장으로 말할 수 있다
- Look-ahead Bias(미래 데이터를 실수로 사용하는 버그)가 무엇이고 왜 위험한지 설명할 수 있다
- 머신러닝 관점에서 데이터, 피처, 학습, 추론, 주문 실행이 어떤 레이어에 놓이는지 설명할 수 있다
- 등록된 전략 목록을 확인하고 첫 백테스트를 실행할 수 있다

이 장에서 만드는 것: 전략 코드를 한 줄도 짜지 않았지만, 이 장 마지막에서 아래 명령 한 줄이 실행된다.

```
python -m app.trading.backtest.runner --signal hmm_daily_mr --days 365
```

실행하면 score(변동성 대비 수익), Sortino(하방 변동성 대비 수익), MDD(최대 낙폭) 같은 수치가 출력된다. 작동 원리는 이 책 전체에서 설명한다. 지금은 전체 구조를 파악하면 된다.

TIP

바이브코딩 미션

- **얻을 결과:** 등록된 전략 목록과 기본 백테스트 결과를 출력한다.
- **첫 회독:** 클래스 이름보다 데이터 → 신호 → 리스크 → 결과 순서만 본다.
- **용어 발판:** 전략은 매매 판단 규칙, 백테스트는 과거 데이터 모의실험, MDD는 운용 중 가장 깊었던 낙폭이다.
- **AI 요청:** “이 장의 데이터·신호·체결·회계 흐름을 표로 정리해줘. 각 단계의 입력과 출력, 미래 정보가 들어올 수 있는 지점을 하나씩 짚어줘.”
- **완료 확인:** 전략 이름, 거래 수, MDD가 출력되면 성공이다.

1.1 시스템 구조

왜 레이어로 나누는가

전략을 하나의 큰 스크립트로 짜면 데이터 수집, 신호 생성, 주문 실행이 한 파일에 뒤섞인다. 이렇게 되면 “데이터를 바꿔서 다시 돌려보자”는 간단한 요구조차 코드 전체를 건드려야 한다. 책임을 층으로 나누면 한 층을 바꿔도 다른 층은 그대로 둘 수 있다.

이 시스템은 다섯 개의 레이어로 나뉜다. 각 레이어는 따로 테스트할 수 있고, 새 전략을 추가할 때 아래 레이어를 수정할 필요가 없다.

레이어	역할
Layer 5 —모니터링	대시보드로 결과를 본다
Layer 4 —실행 & 리스크	주문을 내고 위험을 관리한다
Layer 3 —Ensemble	여러 신호를 합쳐 최종 포지션을 정한다
Layer 2 —Signal 생성	HMM, 강화학습, 통계 차익거래로 매수·매도 신호를 만든다
Layer 1 —데이터	가격 데이터를 받아 피처로 변환한다

데이터는 아래에서 위로 흐른다. Layer 1에서 원시 가격 데이터를 받아 피처로 변환한다. Layer 2의 Signal 생성기들이 그 피처를 보고 매수·매도 신호를 만든다. Layer 3의 Ensemble이 신호들을 합쳐 최종 포지션을 결정한다. Layer 4가 주문을 내고, Layer 5가 결과를 보여준다.

이후 장에서 HMM, SAC, StatArb가 차례로 등장하는데, 서로 경쟁하는 기법이 아니라 **각자 다른 레이어에서 다른 역할을 맡은 부품**이다.

레이어를 나누는 또 다른 이유는 실패 지점을 빨리 찾기 위해서다. 수익률이 나빠졌을 때 “모델이 틀렸다”라고 바로 결론 내리면 안 된다. 데이터가 깨졌을 수도 있고, 피처 순서가 바뀌었을 수도 있고, 주문이 일부만 체결됐을 수도 있다.

레이어	맡는 일	막아야 할 실패
Layer 1 데이터	시장 기록을 피처로 변환	결측치, 타임존 오류, stale 데이터
Layer 2 Signal	HMM/Pairs/RL 판단 생성	미래 누출, 잘못된 레짐, 과도한 신호
Layer 3 Ensemble	여러 판단을 결합	한 모델의 착각에 과하게 의존
Layer 4 실행 & 리스크	주문 전 최종 안전 확인	과대 포지션, 손절 실패, 체결 불일치
Layer 5 모니터링	운영 상태 감시	연결 끊김, 점수 하락, 알림 누락

이 표를 보면 아키텍처가 단순히 코드를 예쁘게 나눈 것이 아니라는 점이 보인다. 각 레이어는 특정 실패를 막기 위한 방어선이다.

초보자는 이 구조를 “시장 해부 순서”로 봐도 된다.

- | | |
|------------------|-----------------|
| 1. 시장을 기록한다 | 데이터 |
| 2. 읽기 좋은 숫자로 바꾼다 | 피쳐 |
| 3. 상태를 알아낸다 | HMM |
| 4. 기회를 찾는다 | Pairs / Z-score |
| 5. 크기를 정한다 | SAC / RL |
| 6. 위험을 막는다 | Risk |
| 7. 실제로 보낸다 | Broker |
| 8. 계속 감시한다 | Monitoring |

머신러닝은 3번에서 시장 상태를 읽는 데 쓰이고, 딥러닝/RL은 5번에서 행동의 크기를 정하는 데 쓰인다. 둘을 한꺼번에 이해하려고 하면 어렵다. 먼저“HMM은 시장을 읽고, SAC는 행동을 고른다”정도로 나눠 잡으면 된다.

머신러닝 관점으로 다시 보기

같은 구조를 머신러닝 용어로 다시 쓰면 더 선명해진다. Layer 1은 원시 데이터를 모델 입력으로 바꾸는 **피쳐 엔지니어링** 계층이다. Layer 2는 피쳐를 받아 시장 상태나 행동을 계산하는 **모델 추론** 계층이다. Layer 3은 여러 모델의 출력을 합쳐 최종 결정을 만드는 **의사결정** 계층이다. Layer 4부터는 머신러닝이 아니라 실제 돈이 움직이는 **운영·리스크** 계층이다.

여기서 모델은“파일 하나”나“수식 하나”만을 뜻하지 않는다. 직관적으로는 **판단 계산기**다. 시장 숫자를 넣으면 현재 시장 상태나 포지션 크기를 돌려준다.

사람이 만든 규칙:
price > moving_average 이면 BUY

머신러닝 모델:
today_features -> trained_model -> regime_probability
오늘 시장 숫자들 -> 학습된 판단 계산기 -> 레짐 확률

일반 규칙은 사람이 임계값과 조건을 직접 정한다. 머신러닝 모델은 과거 데이터로 그 판단 기준 안의 숫자들을 맞춘다. 그래서 모델을 이해할 때는“어떤 파일인가”보다“무엇을 입력으로 받고, 무엇을 출력하는가”를 먼저 보면 된다.

머신러닝 단계	이 시스템의 위치	예시
데이터 수집	Layer 1	OHLCV 다운로드, DuckDB 캐시

머신러닝 단계	이 시스템의 위치	예시
피쳐 생성	Layer 1	수익률, 변동성, 모멘텀, Z-score
학습	Layer 2 내부	HMM fit(), SAC train()
추론	Layer 2 내부	HMM 레짐 확률, SAC 포지션 크기
결정	Layer 3	HMM+Pairs+RL 신호 결합
실행	Layer 4	주문, 손절, 포지션 한도
감시	Layer 5	체결, 연결, 점수 하락 알림

초보자가 가장 많이 헷갈리는 지점은 **학습**과 **추론**이다. 학습은 과거 데이터를 보고 모델 내부 값을 맞추는 과정이다. HMM이라면 전이확률과 상태별 분포를 찾는 일이고, SAC라면 신경망 가중치를 조정하는 일이다. 추론은 이미 학습된 모델에 오늘의 피쳐를 넣어 판단을 얻는 과정이다. 오늘 장중에는 미래 데이터를 모르므로, 추론은 반드시 현재 시점까지의 데이터만 써야 한다.

이 책에서 계속 강조하는 Look-ahead Bias는 결국 이 경계가 무너지는 때 생긴다. 학습에 써야 할 미래 구간이 추론에 섞이거나, 검증 구간까지 포함해 스케일러를 fit하면 모델은 실제 거래에서 알 수 없는 정보를 배운다. 구조를 레이어로 나누는 이유도 이 경계를 코드에서 지키기 위해서다.

데이터 관점에서는 아래 그림 하나로 이해하면 된다.

1) 학습 단계

과거 데이터 -----> 모델 내부 규칙
 수익률, 변동성, 거래량비율 A, B, π / 신경망 가중치
`model.fit(X_train)`

2) 추론 단계

새 데이터 한 줄 -----> 현재 판단
 오늘의 수익률, 변동성 레짐 확률 / 포지션 크기
`model.predict(x_today)`

3) OOS 검증

학습에 쓰지 않은 뒤쪽 구간을 하루씩 흘려보내며
 "그때그때 알 수 있었던 정보만으로 판단했는가"를 확인한다.

OOS 구간에서 모델 파라미터를 계속 고치면 검증이 아니다. OOS에서는 학습한 규칙을 고정하고, 새 봉이 들어올 때마다 추론만 해야 한다. 이 원칙이 지켜져야 백테스트가 실제 운영과 같은 조건에 가까워진다.

운영까지 포함하면 모델의 생애는 네 단계다.

1. 학습
과거 데이터로 모델 안의 숫자를 맞춘다.
2. 저장
학습된 숫자와 설정을 모델 파일로 저장한다.
예: `hmm_model.pkl`, `sac_actor.pt`, `scaler.pkl`
3. 로드/서빙
트레이딩 시스템이 시작될 때 모델 파일을 메모리에 올린다.
또는 FastAPI 서버가 모델을 올려두고 요청을 기다린다.
4. 추론
새 봉이 들어올 때마다 피처를 만들고 모델에 넣어 결과를 받는다.
예: 레짐 확률, 포지션 크기, 매수/매도 허용 여부

이 흐름에서 조심할 점은 모델 파일만 저장하면 끝이 아니라는 것이다. 모델이 학습 때 사용한 스케일러, 피처 순서, 레짐 번호 매핑도 함께 저장되어야 한다. 학습 때는 [수익률, 변동성, 모멘텀, 거래량] 순서였는데 추론 때 순서가 바뀌면, 모델은 전혀 다른 의미의 숫자를 읽게 된다.

실전 운영까지 포함하면 한 가지 단계가 더 붙는다. 모델이 결과를 낸 뒤에도 시스템은 계속 확인해야 한다. 주문이 브로커에 접수됐는지, 체결 수량이 맞는지, 포지션이 내부 기록과 일치하는지, 최근 점수가 나빠지지 않았는지를 감시한다. 그래서 Layer 5 모니터링은 장식이 아니라 운영 중 검증이다.

학습/추론은 모델이 맞는지 확인한다.
리스크/실행은 주문이 안전한지 확인한다.
모니터링은 운영 중에도 계속 정상인지 확인한다.

모든 장을 같은 방식으로 읽기

이 책의 각 장은 이름은 다르지만 같은 구조를 가진다. 초보자는 매 장을 읽기 전에 다음 네 칸을 먼저 채우면 된다.

확인할 것	질문
입력	이 장의 코드가 어떤 데이터를 받는가?
변환	그 데이터를 어떤 판단용 숫자로 바꾸는가?
출력	결과로 어떤 신호, 확률, 크기, 상태를 내는가?
위험	여기서 틀리면 어떤 방식으로 백테스트나 실전이 망가지는가?

예를 들어 6장 칼만 필터는 다음과 같다.

항목	답
입력	노이즈가 섞인 가격 또는 피쳐
변환	관측값을 얼마나 믿을지 계산해 평활화
출력	덜 흔들리는 피쳐, Innovation Z-score
위험	노이즈를 레짐 변화로 착각

17장 IBKR 연결도 같은 방식으로 읽을 수 있다.

항목	답
입력	리스크를 통과한 주문 의도
변환	브로커 계약, 주문 객체, 체결 상태로 변환
출력	접수/체결/포지션 동기화 결과
위험	내부 포지션과 실제 브로커 포지션 불일치

이렇게 보면 머신러닝 장과 운영 장이 따로 떨어져 있지 않다. 모두“입력 -> 변환 -> 출력 -> 검증”이라는 같은 구조를 가진다.

1.2 디렉터리 구조

파일 구조를 외울 필요는 없다. 초보자는 아래 네 묶음의 역할만 기억하면 된다.

```

stock_trading_ai_py/
+-- app/trading/
|   +-- backtest/          # 전략을 검증하는 엔진
|   +-- backtest/signals/  # HMM, StatArb, RL 신호 목록
|   +-- rl/                # SAC 강화학습 구현
|   +-- cache/             # 가격 데이터 캐시
+-- refresh_cache.py       # 데이터 캐시 갱신
+-- pyproject.toml         # Python 의존성

```

backtest/는 전략을 돌리는 엔진, signals/는 전략의 본체, rl/은 강화학습 모델, cache/는 가격 데이터 저장소다. 이 디렉터리 이름은 책임을 나누는 사례다. 데이터 읽기는 cache, 미래 정보를 보지 않는 신호 판단은 signals, 현금·수량·비용 평가는 backtest의 책임으로 구별한다. 파일 트리를 복원하지 않아도 각 입력이 어느 경계를 통과하는지 따라갈 수 있다.

1.3 핵심 구성요소

SignalGenerator —공통 약속

전략마다 입출력 형식이 다르면 백테스트 엔진이 전략을 일관되게 다룰 수 없다. 그래서 모든 전략은 같은 약속(interface)을 따른다. 어떤 전략이든 두 단계로 돌아간다.

1. initialize() —전략이 시작될 때 한 번 호출되어 데이터를 준비한다.
2. on_bar() —새 봉(가격 데이터 한 칸)이 들어올 때마다 호출되어 매수·매도 신호를 돌려준다.

전략 인터페이스의 최소 계약은“과거 자료로 준비하기”와“현재까지의 자료로 목표 신호 반환하기”다. 입력에는 봉 시각·확정 관측·현재 보유 상태를, 출력에는 방향·목표 크기·유효 시각·거부 사유를 둔다. 신호 함수가 잔고를 직접 바꾸지 않도록 체결과 회계는 별도 엔진이 담당한다.

SIGNAL_REGISTRY —등록된 전략 목록

SIGNAL_REGISTRY는 전략 이름과 실제 구현 클래스를 연결하는 목록이다. 여기서 외울 것은 개별 이름이 아니라“**전략을 실행할 때 문자열 하나로 구현을 선택한다**”는 구조다.

대표 신호는 세 묶음으로 나뉜다.

목록	예시	역할
HMM 계열	hmm_daily_mr 등	시장 레짐을 추정한다
StatArb 계열	stat_pair_rl 등	페어와 평균회귀 신호를 만든다
통합 계열	regime_orchestrator 등	여러 신호를 결합한다

등록된 전체 목록은 다음 명령으로 본다.

```
python -m app.trading.backtest.runner --list
```

BacktestResult —표준 결과 형식

전략마다 결과 형식이 다르다면 비교가 어렵다. BacktestResult는 모든 전략의 성과 지표를 같은 형식으로 담은 결과 객체다. 처음에는 아래 네 지표만 보면 된다.

지표	의미	먼저 볼 이유
score	변동성 대비 수익	전략 품질의 1차 기준
mdd	최대 낙폭	계좌가 얼마나 흔들렸는지
win_rate	이긴 거래 비율	신호가 너무 무작위인지 확인
annual_return	연환산 수익률	실제 기대 수익 규모

sortino, cvar_95, calmar 같은 지표는 뒤에서 다시 나온다. 1장에서는“모든 전략 결과가 같은 표준 형식으로 모인다”는 점만 이해하면 된다.

최소 자본 요구: capital = \$100,000이 기본값이지만, SPY 500주를 사면 \$300k 이상이 필요하다. 계좌 자금이 부족하면 포지션 계산이 제대로 동작하지 않는다. 실제 계좌 규모에 맞게 capital을 수정하거나, max_position_usd 파라미터로 종목당 상한을 따로 지정하라.

SAC —직접 구현한 강화학습 에이전트

이 책의 강화학습 부분은 범용 라이브러리(stable-baselines3)를 쓰지 않고, **SAC(Soft Actor-Critic)**를 PyTorch로 직접 구현했다. 트레이딩에 맞춰 세부 동작을 제어하기 위해서다.

자세한 구조는 13장과 14장에서 다룬다. 지금은 입출력 모양만 기억하면 된다.

- **입력:** 시장 피쳐 28개
- **출력:** 포지션 크기 1개

전체 학습 설정은 `app/trading/rl/config.py`에 있다.

1.4 인과성 원칙

인과성 원칙: 미래 데이터를 현재 의사결정에 사용하면 백테스트 결과는 좋아 보이지만 실전에서 는 재현되지 않는다. 학습 데이터에만 `fit`을 적용하고, 추론은 시점 t 까지의 데이터만 사용해야 한다.

Look-ahead Bias는 백테스트 수익률을 가짜로 부풀리는 가장 흔한 버그다. 시점 t 에서 의사결정을 내릴 때, $t + 1$ 이후의 데이터가 새어 들어가는 상황을 말한다. 백테스트에서는 미래 데이터를 알기 때문에 실수로 끼어들기 쉽다.

초보자가 가장 자주 만나는 두 가지 함정은 다음과 같다.

함정 1 —HMM에서 미래 관측치까지 참조

`hmm.predict_proba(X_all)`로 전체 구간의 사후확률을 구한 뒤 과거 각 시점의 신호로 쓰면 위험하다. 중간 시점 t 의 사후확률에는 $t + 1$ 이후 관측치가 반영되기 때문이다. 시점 t 의 결과는 $x_1, \dots, x_t$ 만 보고 계산해야 한다. 이 책에서는 5장의 `forward_proba_causal()`과 `step_forward()`로 이 규칙을 지킨다. 단, 현재까지 자른 창에서 `predict_proba(...)[-1]`만 읽는 경우에는 마지막 시점 뒤의 데이터가 없으므로 미래 참조가 아니다. 두 사용법을 구분해야 한다.

함정 2 —스케일러를 전체 기간으로 fit

```
# 잘못된 코드
scaler = StandardScaler()
scaler.fit(X_all) # 미래 분포가 과거 스케일에 영향

# 올바른 코드
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

`fit`이 평균과 표준편차를 계산할 때 미래 데이터를 보면, 백테스트는 좋게 나오지만 실제 운용에서는 그 평균을 알 수 없다. 규칙은 간단하다—`fit`은 훈련 구간에만, 테스트 구간에는 `transform`만 호출한다.

Walk-forward 검증: 한 번의 분리로 끝내지 않고 시간 순서대로 반복 분리한다. 이 프로젝트의 `OOS_RATIO=0.40`은 전체 기간의 마지막 40%를 OOS(Out-of-Sample, 학습에 쓰지 않은 검증 구간)로 쓴다. 3장에서 상세히 다룬다.

1.5 데이터 흐름—백테스트는 어떤 순서로 도는가

백테스트 한 번이 도는 흐름은 네 단계다.

1. 실행할 Signal들이 어떤 시간 간격(1d, 1h, 5m)을 요구하는지 모은다.
2. 필요한 가격 데이터를 캐시에서 읽거나 새로 다운로드한다.
3. 각 Signal을 같은 데이터와 같은 조건으로 실행한다.
4. 결과를 `BacktestResult` 목록으로 돌려준다.

비교 실행기는 같은 데이터 스냅샷·평가 기간·초기자산·비용·포지션 한도를 각 전략에 전달하고 결과를 전략 ID로 모은다. 병렬 실행하더라도 전략마다 독립 계좌 상태를 사용해야 한다. 한 전략이 사용한 캐시나 난수 상태를 다른 전략이 바꾸지 않도록 입력을 고정한다.

1.6 개발 환경 설정

Python 3.12 환경

가상 환경은 한 번만 만들면 된다. 아래 네 줄을 차례로 입력한다.

```
cd stock_trading_ai_py
uv venv .venv --python 3.12
source .venv/bin/activate
uv sync
```

`uv sync`는 OS에 맞춰 CUDA(Linux) 또는 MPS(macOS) 빌드를 자동으로 고른다. 실패하면 가장 먼저 Python 버전을 확인한다—`pyproject.toml`에 `requires-python = ">=3.12,<3.13"`이 고정되어 있다.

핵심 패키지

이름과 역할만 한 번 훑어두자. 세부 사용법은 각 기능을 다루는 장에서 다시 나온다.

패키지	역할
<code>hmmlearn</code>	HMM (레짐 추정)
<code>torch</code>	SAC 신경망
<code>optuna</code>	하이퍼파라미터 최적화
<code>duckdb</code>	로컬 데이터 캐시
<code>yfinance</code>	주가 수집
<code>pykalman</code>	Kalman Filter
<code>statsmodels</code>	시계열 통계 검정
<code>ib-async</code>	IBKR 주문 API

정확한 버전은 `pyproject.toml`에 고정되어 있다.

버전을 정확히 고정하라: `torch>=2.0`처럼 범위만 지정하면 몇 달 뒤 패키지가 업데이트될 때 코드가 맞지 않을 수 있다. 프로덕션 배포 전에 `uv lock` 또는 `pip freeze > requirements.txt`로 정확한 버전을 기록해 두라.

설정 파일

API 키, 파일 경로, 상수는 `.env` 파일 하나에 모아서 관리한다. 코드에 하드코딩하지 않으면 키를 git 저장소에 실수로 올릴 위험이 줄어든다.

```
cp .env.example .env
```

```
# .env
IBKR_HOST=127.0.0.1
IBKR_PORT=7497
IBKR_CLIENT_ID=1
DASHBOARD_TOKEN=<CHANGE_ME>
DASHBOARD_SERVER_URL=http://127.0.0.1:8765
```

DASHBOARD_TOKEN은 반드시 본인만 아는 값으로 바꾼다. 기본값을 그대로 두면 대시보드 인증이 사실상 무력화된다.

1.7 데이터 캐시 초기화

환경이 준비됐으면 다음은 데이터다. S&P500 일봉·시간봉·5분봉 데이터를 한 번에 받는다. 예제 환경에서는 첫 다운로드에 약 30분이 걸렸지만, 실제 시간은 네트워크와 데이터 제공자의 응답 속도에 따라 달라진다.

```
python refresh_cache.py
python _check_duckdb_cache.py
```

_check_duckdb_cache.py 출력은 다음과 같은 형태다.

인터벌	커버리지	기간
daily_730d	498/500 종목	2023-04 ~ 2025-04
hourly_730d	493/500 종목	2023-04 ~ 2025-04
5m	312/500 종목	2025-02 ~ 2025-04 (60일)

예제 실행에서 캐시 크기는 약 3.2 GB였다. 기간과 수집 주기, 압축 방식에 따라 크기는 달라진다. 500 종목 중 일부가 빠졌다면 곧바로 정상으로 간주하지 말고, 상장 이력·티커 변경·데이터 제공자 응답 누락을 확인해 제외 이유를 기록한다. 다운로드 자체가 실패하면 먼저 디스크 용량과 네트워크 연결을 확인한다.

1.8 첫 번째 백테스트 실행

이 절에서 1장 첫머리의 약속을 지킨다. 명령 한 줄로 실제 백테스트가 돌아간다.

```
# 단일 전략
python -m app.trading.backtest.runner --signal hmm_daily_mr --days 365

# 두 전략 비교
python -m app.trading.backtest.runner \
    --signal hmm_daily_mr hmm_daily_pure_z \
    --days 730

# 등록된 전략 목록
python -m app.trading.backtest.runner --list
```

비교 명령을 돌리면 두 전략의 성과 지표가 나란히 출력된다. 표를 읽을 때 초점은 다음과 같다.

Strategy	핵심 차이	확인할 것
HMM Daily MR	레짐 필터 사용	손실 구간이 줄었는가
Pure Z-score (HMM 없음)	고정 Z-score만 사용	시장 국면 변화에 취약한가

여기서 익혀야 할 것은 코드가 아니라 결과 표를 읽는 방식이다. 같은 Z-score 신호라도 HMM 레짐 필터를 추가하면 손실 구간과 거래 빈도가 달라진다. 이 책에서 반복해서 확인하게 될 핵심 패턴이다.

결과 해석 기준: 처음 실행했을 때 어떤 숫자를 기준으로 볼까.

지표	먼저 볼 질문
손익 곡선	특정 구간에만 수익이 몰려 있지 않은가
MDD(최대 낙폭)	계좌가 감당할 수 있는 낙폭인가
승률	신호가 무작위처럼 보이지 않는가

첫 실행에서는 숫자 자체보다 결과가 재현되는지, 거래 비용(수수료·슬리피지)을 넣었을 때도 방향성이 유지되는지 확인한다.

실패하면 볼 것: 출력에 “정상”이 아니라 “오류”가 표시되면, (1)캐시가 비어 있는지(`python _check_duckdb_cache.py`), (2)초기 자본 설정이 너무 작은지, (3)`--days`가 캐시 기간을 벗어나는지 순서로 점검한다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. 5-레이어 아키텍처에서 데이터가 Layer 1에서 Layer 5까지 흐르는 과정을 자신의 말로 설명하라. 각 레이어의 책임 한 줄이면 충분하다.
2. 등록된 전략 목록을 출력하는 명령은 무엇인가? 출력에서 HMM 계열 전략을 골라보라.
3. 다음 코드가 왜 Look-ahead Bias(미래 정보를 미리 끌어다 쓰는 오류)를 일으키는지 한두 문장으로 설명하라.

```
scaler = StandardScaler().fit(X_all)
X_scaled = scaler.transform(X_all)
```

정리

이 장에서 반드시 기억할 것은 다음 다섯 가지다.

- **5개 레이어** —데이터·피쳐 -> Signal -> Ensemble -> 실행·리스크 -> 모니터링 순으로 데이터와 결과가 흐른다.
- **Signal은 공통 약속을 따른다** —`initialize()`로 준비하고 `on_bar()`로 신호를 낸다.
- **모든 전략은 같은 형식으로 비교된다** —`BacktestResult`에 표준화되어 모인다.
- **미래 데이터를 쓰면 안 된다** —`fit`은 훈련 구간에만, 추론은 시점 t 까지만.
- **첫 백테스트를 직접 돌릴 수 있다** —명령 한 줄로 결과 표가 나온다.

다음 장에서는 `download_bars()`가 DuckDB에 데이터를 캐시하는 방식과 피쳐 엔지니어링 구성을 코드로 직접 살펴본다.

더 읽을거리

- **책:** Marcos Lopez de Prado, *Advances in Financial Machine Learning* (Wiley, 2018) —인과 성과 Look-ahead Bias를 꼼꼼히 다룬다.
- **논문:** Bailey et al., *The Probability of Backtest Overfitting* —반복 실험에서 우연히 좋은 전략을 고르는 위험을 분석한다.
- **코드:** `mlfinlab` 라이브러리—시계열 특화 피처와 검증 도구가 들어 있다.

2장. 데이터 파이프라인— DuckDB와 실시간 피드

퀀트 시스템의 품질은 데이터 품질을 넘어설 수 없다. 당연한 말처럼 들리지만, 파이프라인 없이 전략 코드부터 짜기 시작하면 이 말이 왜 중요한지 직접 확인하게 된다. 모델이 아무리 정교해도 입력 데이터가 잘못되면, 백테스트의 좋은 성과는 실제 수익 가능성이 아니라 데이터 오류를 학습한 결과일 수 있다.

처음 백테스팅을 시작할 때는 Yahoo Finance에서 내려받은 CSV 파일이 전부였다. OHLCV(시가·고가·저가·종가·거래량) 데이터로 검증하면 성과는 좋게 나왔는데, 실거래에 적용하면 결과가 달랐다. 전략 로직이 망가진 게 아니었다. 애초에 백테스트가 실제 거래 조건과 다른 데이터를 보고 있었던 것이다.

원인은 세 가지였다.

첫째, **생존 편향**이다. 백테스트 기간 중 상장폐지된 종목은 현재 데이터에 남아 있지 않을 수 있다. 그 종목들을 제외하면 “살아남은 종목”만 보는 셈이라 성과가 실제보다 좋게 나온다.

둘째, **미래 데이터 참조**다. 종가로 계산한 지표를 당일 매매 신호로 쓰면, 장 마감 후에야 알 수 있는 정보를 미리 사용한 셈이 된다. 현실에서는 불가능한 거래다.

셋째, **배당·분할 미반영**이다. 주가 분할을 조정하지 않으면 해당 날짜 전후로 가격이 갑자기 절반으로 떨어진 것처럼 보인다. 전략이 이 급변을 진짜 신호로 받아들인다.

셋 중 하나만 있어도 백테스트 결과는 믿기 어렵다.

퀀트 데이터 파이프라인은 이 문제들을 코드 수준에서 해결한다. 중심 도구는 DuckDB(Parquet 파일에 SQL 쿼리를 바로 던질 수 있는 가벼운 데이터베이스)다. Pandas로는 버거운 규모도 안정적으로 다룰 수 있고, 전체를 한꺼번에 메모리에 올리지 않아도 처리할 수 있다. 실시간 피드 연동, 품질 검증, 생존 편향 방지를 파이프라인 안에서 처리하면 전략 코드가 훨씬 단순해진다.

이 장의 흐름

먼저 전체 그림을 잡고, 그다음 구현과 검증으로 내려간다.

1. 전체 데이터 흐름—API에서 모델 입력까지
2. `download_bars()` —단일 다운로드 창구와 폴백 구조
3. `DuckDBCache` —Parquet 파일과 메모리 DuckDB의 조합
4. 기준 시간대로 변환한 뒤 타임존 제거하기
5. 4차원 일봉 피처와 결측치 처리
6. 칼만 평활화, Z-score, 모델 입력 형태

학습 목표

이 장을 마치면 다음을 할 수 있다.

- `download_bars()` 가 FMP(Financial Modeling Prep)에 먼저 데이터를 요청하고, 실패하면 yfinance나 DuckDB 캐시로 전환하는 **폴백(fallback, 대체 경로)** 구조를 설명한다.
- `DuckDBCache`가 Parquet 파일과 메모리 DuckDB를 함께 쓰는 이유와, 여러 프로세스가 동시에 접근해도 충돌이 나지 않는 원리를 설명한다.
- 일봉 데이터에서 수익률·실현변동성·모멘텀·거래량비율 4가지 피처를 계산하고, 칼만 필터(관측값에서 흔들림을 걸러 내 추세만 추정하는 기법)로 노이즈를 줄이는 의미를 이해한다.
- rolling window 때문에 생기는 결측치(NaN)를 왜 제거하거나 별도로 처리해야 하는지 설명한다.
- `DataFrame`에 timezone 정보가 섞이면 어떤 오류가 생기는지, 저장 기준 시간대로 먼저 변환한 뒤 타임존을 제거해야 하는 이유를 설명한다.

이 장에서 만드는 것: `download_bars()` 로 S&P500 일봉 데이터를 내려받아 DuckDB Parquet 캐시에 저장하고, 4가지 피쳐 배열로 변환하는 파이프라인을 만든다.

TIP

바이브코딩 미션

- **연을 결과:** 한 종목의 데이터를 내려받아 피쳐 배열의 행·열 수를 확인한다.
- **첫 회독:** DuckDB 내부 구현보다 원자료가 피쳐로 바뀌는 경계를 본다.
- **용어 발판:** 피쳐는 모델용 가공 숫자, `shape`는 데이터의 행·열 크기, DuckDB는 파일처럼 다룰 수 있는 분석용 데이터베이스다.
- **AI 요청:** “`download_bars()` 의 실제 호출 예를 찾아 한 종목만 읽고 `shape`, 날짜 범위, 결측치 수를 출력하는 최소 점검 코드를 만들어줘. 원본 파이프라인은 수정하지 마.”
- **완료 확인:** 날짜가 과거→최근 순서이고 결측치 처리 결과가 설명되면 성공이다.

2.1 큰 그림—데이터 흐름

먼저 데이터가 어떻게 흘러가는지부터 보자. API 요청에서 시작해 모델에 들어가는 피쳐 행렬로 끝나는 한 줄짜리 경로다.

```
yfinance / FMP API
|
v (최초 다운로드)
download_bars()
|
└─ FMP 우선
    |   | 실패 또는 30% 미만
    |   v
    └─ yfinance / DuckDB 폴백
        |
        v
DuckDBCache.save_ohlcv_data()
|
v
```

```

cache/.../ohlcv/{symbol}/{interval}/YYYY-MM.parquet
|   (월 별 파 티 션, Append-only)
v
compute_daily_features()
|
v
np.ndarray shape=(T, 4) -> HMM.fit()

```

마지막 단계의 HMM(은닉 마르코프 모델)은 시장 국면을 추정하는 모델로, 4장에서 본격적으로 다룬다. 여기서는“피쳐 행렬을 받아 가는 소비자”로만 봐 두면 된다.

캐시가 이미 있으면 네트워크 요청 없이 Parquet 파일에서 바로 읽는다. 이 구조가 중요한 이유는 속도만이 아니다. 같은 입력 데이터로 같은 백테스트를 반복할 수 있어야 결과를 비교할 수 있다. 매번 API에서 조금씩 다른 값을 받으면, 전략 변경의 효과와 데이터 변경의 효과가 섞인다.

원자료가 피처가 되는 모습

초보자가 가장 먼저 헛갈리는 부분은“가격 데이터가 어떻게 모델 입력이 되는가”다. 모델은 차트 그림을 보는 것이 아니라 숫자 표를 읽는다. 그래서 원자료를 그대로 넣기보다, 모델이 읽기 쉬운 요약 숫자로 바꾼다.

원자료 OHLCV

날짜	open	high	low	close	volume
2026-05-18	100	103	99	102	1,200,000
2026-05-19	102	104	101	101	1,450,000
2026-05-20	101	105	100	104	1,800,000

피쳐 행렬 X

날짜	return	volatility	momentum	volume_ratio
2026-05-18	...	...	...	...
2026-05-19	-0.0098	...	...	1.12
2026-05-20	0.0297	...	...	1.31

OHLCV는 사람이 보기에는 익숙하지만 모델 입장에서 의미는 덜 정리된 원자료다. 피쳐 행렬 X는“오늘 수익률은 얼마인가”, “최근 변동성은 커졌는가”, “거래량은 평소보다 많은가”처럼 질문을 바꿔 놓은 표다.

이 표의 한 행은 하루를 뜻하고, 한 열은 모델이 읽는 관점 하나를 뜻한다.

```

행(row)      = 특정 날짜 하나
열(column)    = 모델이 읽을 질문 하나
X[t]          = t일에 모델에게 넣는 숫자 묶음

```

그래서 학습과 추론도 결국 이 피쳐 행렬을 시간축으로 나누는 일이다.

```

X 전체
|-----|-----|
      X_train      X_oos
      학습용       검증/추론용

model.fit(X_train)
for x_today in X_oos:
    model.predict(x_today)

```

이때 `X_oos`는 이미 과거 데이터 안에 있는 구간이지만, 모델에게는 미래처럼 숨겨 둔 구간이다. 코드는 하루씩 앞으로 이동하면서 그날까지 알 수 있는 `x_today`만 모델에 넣어야 한다. 이것이 데이터 관점에서 보는 OOS(Out-of-Sample, 표본 외) 검증이다.

2.2 download_bars() —단일 다운로드 창구

`download_bars()`는 이 책 전체에서 데이터를 내려받는 단일 창구다. Yahoo Finance든 FMP든 모두 이 함수 하나로 가져온다.

내려받는 코드가 여러 파일에 흩어져 있으면, 폴백 정책·시간대 처리·캐시 저장 방식이 파일마다 제각각이 된다. 함수를 하나로 모으면 이런 규칙을 한곳에서 관리할 수 있다.

함수가 지원하는 인터벌

인터벌	최대 기간	1차 소스	폴백
1d	무제한	FMP	yfinance

인터벌	최대 기간	1차 소스	폴백
1h	730일	FMP	DuckDB
5m~30m	60일	FMP	yfinance

4h와 1wk는 기저 인터벌(1h, 1d)을 묶어 만든다. 같은 거래소 시간대와 세션 경계 안에서 open은 첫 값, high는 최댓값, low는 최솟값, close는 마지막 값, volume은 합계로 집계한다. 아직 닫히지 않은 묶음은 확정 봉과 구분하고, 주봉은 휴장일을 포함한 해당 거래 주의 마지막 실제 거래 시각을 기록한다. 단순히 UTC 자정부터 네 행씩 묶으면 세션이 어긋날 수 있다.

핵심 흐름—2단계 폴백

데이터 로더의 계약은 (종목, 주기, 시작, 끝) -> 시각순으로 정렬된 OHLCV다. 중복 시각은 출처와 수정 시각을 기준으로 결정하고, 가격이 양수인지·고가가 저가보다 낮지 않은지·거래량이 음수가 아닌지 검사한다. 실패한 다운로드를 빈 정상 데이터처럼 캐시에 저장하지 않는다. 동작 순서는 다음과 같다.

1. **1단계**: FMP에 요청한다.
2. **2단계**: 받아온 종목 수가 요청의 30% 미만이면 실패로 보고, 일봉은 yfinance로, 시간봉은 DuckDB 캐시로 전환한다.

결과는 `{symbol: DataFrame}` 형태의 딕셔너리다. 어떤 종목도 받지 못하면 빈 딕셔너리 `{}`를 반환한다.

폴백 기준이 왜 30%인가: FMP에서 500개 종목을 요청했을 때 150개 미만이 돌아오면 실패로 본다. 시장 휴일, API 한도 초과, 네트워크 문제로 일부 종목이 빠질 수 있기 때문이다.

재시도 없음—운영 시 주의: 현재 구현은 API 요청이 실패하면 곧바로 폴백으로 넘어간다. 네트워크가 1초 끊겼다가 복구돼도 마찬가지다. 실시간 운영 환경이라면 `tenacity` 같은 라이브러리로 재시도를 덧붙이는 편이 안전하다.

폴백이 실제로 일어났는지 확인하기

폴백은 조용히 일어나기 때문에, 디버그 로그를 켜서 어디서 전환되었는지 확인해야 한다.

첫 줄은 30% 기준에 걸려 yfinance로 넘어간 경우이고, 둘째 줄은 모든 소스가 실패해 빈 DataFrame이 돌아온 경우다. 이렇게 받아온 데이터를 어디에 어떤 형태로 쌓아 두는지가 다음 절의 주제다. |

47

```
import logging
logging.getLogger("quant.data").setLevel(logging.DEBUG)
```

출력 예시는 다음과 같다.

DEBUG: FMP coverage 18% < 30% for AAPL ... — falling back to yfinance

WARNING: All sources failed for TSLA 2008-01-01 — returning empty DataFrame

첫 줄은 30% 기준에 걸려 yfinance로 넘어간 경우이고, 둘째 줄은 모든 소스가 실패해 빈 DataFrame이 돌아온 경우다. 이렇게 받아온 데이터를 어디에 어떤 형태로 쌓아 두는지가 다음 절의 주제다.

실패하면 볼 것: 마지막 returning empty DataFrame 경고가 보이면 그 구간의 신호는 0으로 처리된다. 백테스트 결과가 이상하면 이 로그부터 확인한다.

2.3 DuckDBCache —Parquet + 메모리 DuckDB

캐시 계층이 답해야 할 질문은 두 가지다. **어디에** 저장할지(Parquet 파일)와 **어떻게** 읽을지(메모리 DuckDB)다.

파티셔닝(Partitioning)이란? 데이터를 여러 파일로 나눠 저장하는 방식이다. DuckDB는 WHERE symbol='AAPL' AND ym='2024-01' 같은 쿼리를 실행할 때, 조건에 해당하는 파일만 읽고 나머지는 건너뛴다.

왜 DuckDB인가

CSV나 SQLite와 비교한 장단점을 표로 정리한다.

	SQLite	CSV	Parquet + DuckDB
쿼리 속도	보통	느림	빠름
멀티프로세스	단일 writer	O	O (Append-only)
압축	선택	없음	자동
파일 크기	보통	큼	작음

특히 백테스트는 “특정 종목, 특정 기간”만 반복해서 읽는 작업이라, 조건에 맞는 파일만 골라 읽는 방식이 효과적이다.

저장 구조—월별 Parquet

종목·인터벌의 2단 디렉터리 아래에, 연월 단위로 나눈 파일을 둔다.

```
cache/historical_data/ohlcv/
├─ AAPL/
│  └─ 1d/
│     └─ 2023-01.parquet
│     └─ 2023-02.parquet
│     └─ ...
│  └─ 1h/
│     └─ 2024-11.parquet
└─ MSFT/...
```

월별로 잘게 나누는 이유는 단순하다. 백테스트가 1년치 데이터를 본다면 12개 파일만 열면 된다. 종목당 한 덩어리로 저장했다면 10년치 전체를 읽어야 한다.

동시 접근을 어떻게 다루는가

여러 프로세스가 같은 캐시를 갱신할 때는 파일 잠금과 원자적 교체를 구분한다. 잠금은 작성자가 돌이 되는 것을 막고, 교체는 독자가 반쯤 쓴 파일을 읽는 것을 막는다. 프로세스 내부의 스레드 락만으로 다른 프로세스를 막을 수는 없다. 아래 두 패턴을 함께 적용한다.

- **Singleton:** 프로세스 안에서 DuckDBCache 객체를 하나만 만든다. 캐시 경로 설정이 여러 곳에서 달라지는 사고를 막는다.

- **ThreadLocal 연결:** 스레드마다 자기만의 DuckDB 연결을 갖는다. 스레드끼리 하나를 공유하면 쿼리가 꼬일 수 있어서다.

Parquet 파일은 여러 프로세스가 동시에 읽을 수 있다. 쓰기는 월별 파티션 파일 단위로만 일어나고, 새 데이터가 들어오면 해당 월의 파일 하나만 쓴다. 서로 다른 종목·다른 달의 파일은 건드리지 않으므로 잠금 충돌이 없다.

저장 함수의 4단계

저장 계약은 검증한 테이블을 임시 경로에 쓰고 성공했을 때 최종 경로로 바꾸는 것이다. 예외가 나면 임시 파일을 정리하고 기존 정상 캐시는 유지한다. 데이터 파일과 메타데이터를 별개로 바꾼다면 세대 ID를 넣어 둘이 같은 버전인지 확인해야 한다.

1. 판다스의 날짜 인덱스(DatetimeIndex)를 date 컬럼으로 옮긴다.
2. 타임존이 있으면 기준 시간대인 UTC로 변환한 뒤 타임존 정보를 제거한다.
3. 컬럼명을 소문자로 통일한다 (Open -> open).
4. 월별로 그룹화해 YYYY-MM.parquet 파일에 저장한다.

해당 월의 파일이 이미 있으면, 새 데이터를 기존 데이터에 합치고 날짜 중복을 제거한 뒤 다시 쓴다. 같은 다운로드를 여러 번 돌려도 중복이 쌓이지 않는다.

2단계에서 타임존을 없애는 이유는 다음 절에서 따로 다룬다. 이 프로젝트에서 가장 자주 사고가 나는 지점이기 때문이다.

2.4 기준 시간대로 변환한 뒤 타임존 제거하기

yfinance와 일부 API는 거래소 현지 시간이나 UTC 타임존이 붙은 DatetimeIndex를 반환한다. 서로 다른 기준이 섞이면 두 가지 문제가 생긴다.

문제 1 —pandas concat 오류: 타임존이 있는 DataFrame과 없는 DataFrame을 합치려고 하면 `TypeError: Cannot join tz-naive and tz-aware DatetimeIndex`가 난다.

문제 2 —프로젝트 스키마 불일치: DuckDB 자체는 타임존이 있는 `TIMESTAMPTZ`도 지원한다. 다만 이 프로젝트의 저장 스키마는 UTC 기준의 `timestamp-naive` `TIMESTAMP`로 통일하므로, `tz-aware` 값을 그대로 보내면 변환 오류나 조용한 시간 불일치가 생길 수 있다.

올바른 패턴

다운로드 직후에는 **먼저 UTC로 맞춘 다음** 타임존 표시만 제거한다. `tz_localize(None)` 만 바로 호출하면 시계에 표시된 시간은 그대로 둔 채 시간대 정보만 버리므로, 서로 다른 시간대의 데이터가 몇 시간 어긋난 채 합쳐질 수 있다.

```
def download_ohlcv(symbol, interval="1d"):
    df = yf.download(symbol, period="5y", interval=interval,
        auto_adjust=True)
    idx = pd.to_datetime(df.index)
    if idx.tz is not None:
        idx = idx.tz_convert("UTC").tz_localize(None)
    df.index = idx # UTC 기준 naive 저장
    df.columns = [c.lower() for c in df.columns]
    return df
```

확인할 것: `df.index.tz`는 `None`이고, 서로 다른 공급자의 같은 봉이 동일한 UTC 시각에 놓여야 한다. 일봉에서 거래소의 세션 날짜가 중요하다면 UTC 시각과 별도로 `session_date`를 거래소 현지 날짜로 보존한다.

2.5 피처 엔지니어링—4차원 일봉 피처

지금까지는 원본 가격을 모으고 저장하는 이야기였다. 이제 그 데이터를 모델이 읽을 수 있는 형태로 바꿀 차례다.

레짐 분류에 쓸 HMM(은닉 마르코프 모델, Hidden Markov Model)은 주가 자체를 입력으로 받지 않는다. 주가에서 계산한 파생 값, 즉 **피처(feature)**를 입력으로 받는다. 모델 자체의 원리는 뒤에서 따로 다루고, 여기서는 입력을 만드는 일에 집중한다.

피처는 머신러닝 모델이 읽을 수 있는 언어다. 사람은 차트를 보고 “최근 변동성이 커졌다”, “한 달 동안 상승 흐름이다”라고 말할 수 있지만, 모델은 그런 문장을 이해하지 못한다. 그래서 가격과 거래량을 수익률, 변동성, 모멘텀, 거래량 비율 같은 숫자 열로 바꿔야 한다. 이 변환이 피처 엔지니어링이다.

나쁜 입력은 좋은 모델도 망친다. 가격 수준을 그대로 넣으면 20달러 주식과 400달러 주식을 다른 성격의 시장으로 착각할 수 있다. 반대로 로그 수익률은 가격 수준이 달라도 “얼마나 움직였는가”를 같은

단위로 비교하게 해준다. 이 장에서 만드는 4차원 피쳐는 HMM이 레짐을 구분할 수 있도록 시장 움직임을 네 방향에서 요약한다.

4가지 피쳐가 보는 것

차원	의미	무엇을 보여주는가
0	1일 로그 수익률	어제 대비 오늘 가격이 얼마나 움직였나
1	5일 실현변동성	최근 가격이 얼마나 출렁이는가
2	20일 모멘텀	한 달 전 대비 추세가 어느 방향인가
3	거래량 비율	평소보다 거래가 얼마나 활발한가

차원 0·3처럼 빠른 변화가 필요한 피쳐는 Q/R을 상대적으로 크게, 차원 1·2처럼 노이즈를 억제할 피쳐는 작게 설정하는 설계를 비교할 수 있다. Q는 상태 변화 분산, R은 관측 잡음 분산이다. 두 값의 단위는 해당 피쳐의 제공 단위이므로 원시 가격용 숫자를 표준화 수익률에 그대로 쓰지 않는다. 6장의 예측·갱신 식과 혁신 잔차를 기준으로 학습 구간에서 설정을 정하고 OOS에서는 고정한다.

핵심 계산—네 가지 일별 피쳐

```
import numpy as np

def compute_daily_features(close, volume):
    close = np.asarray(close, dtype=float)
    volume = np.asarray(volume, dtype=float)
    if close.ndim != 1 or volume.shape != close.shape:
        raise ValueError("equal-length 1D arrays required")
    if not np.all(np.isfinite(close)) or np.any(close <= 0):
        raise ValueError("close must be finite and positive")
    if not np.all(np.isfinite(volume)) or np.any(volume < 0):
        raise ValueError("volume must be finite and nonnegative")
    n = len(close)
    feat = np.full((n, 4), np.nan)
    ret = np.full(n, np.nan)
    ret[1:] = np.diff(np.log(close))
```

```

feat[:, 0] = ret
for t in range(n):
    if t >= 5:
        feat[t, 1] = np.std(ret[t-4:t+1], ddof=1)
    if t >= 20:
        feat[t, 2] = close[t] / close[t-20] - 1
        base = np.mean(volume[t-20:t])
        if base > 0:
            feat[t, 3] = volume[t] / base
return feat

features = compute_daily_features([100.0] * 25, [10.0] * 25)
assert features.shape == (25, 4)
assert np.allclose(features[20], [0.0, 0.0, 0.0, 1.0])
assert np.isfinite(features).all(axis=1).sum() == 5

```

이 예는 NumPy가 필요한 독립 함수다. 5일 변동성은 표본 표준편차이며 연산하지 않는다. 20일 모멘텀에는 오늘과 20일 전 증가가 필요하므로 처음 20행은 네 피처가 모두 준비되지 않는다. 거래량 비율의 기준은 오늘을 제외한 직전 20봉 평균이다. 가격 0·음수는 작은 양수로 숨기지 않고 오류로 거부한다. 매일 증가까지의 관측으로 계산한 피처는 그 증가가 확정된 뒤에만 사용할 수 있다.

결측치—모델이 읽으면 안 되는 빈칸

피처를 만들면 거의 반드시 결측치가 생긴다. 20일 모멘텀은 적어도 과거 20일이 있어야 계산된다. 따라서 데이터 시작 후 처음 20행은 값이 비어 있는 것이 정상이다. 거래 정지, 상장폐지, API 실패, 휴장일 차이 때문에 중간에 빈칸이 생길 수도 있다.

이때 가장 위험한 습관은 무조건 0으로 채우는 것이다.

```

# 위험한 코드
X = np.nan_to_num(features, nan=0.0)

```

0은“값이 없다”가 아니다. 수익률 0은 가격이 움직이지 않았다는 뜻이고, 변동성 0은 거의 움직임이 없었다는 뜻이다. 결측치를 0으로 채우면 모델은 데이터가 없는 날을“매우 조용한 시장”으로 오해할 수 있다.

이 책의 기본 규칙은 세 가지다.

상황	처리
rolling window 초기 구간	해당 행 제거
일부 피쳐만 중간 결측	직전 값으로 제한적으로 보간하거나 해당 행 제거
가격·거래량 원본 결측	신호 계산 중단 또는 해당 종목 제외

처음에는 행 제거가 가장 안전하다. 데이터가 줄어드는 단점은 있지만, 모델에 거짓 신호를 넣지 않는다. 운영 환경에서 보간을 쓰더라도 “방금 값이 비었으니 임시로 채웠다”는 로그와 모니터링이 있어야 한다. 결측치는 단순한 전처리 문제가 아니라, 나중에 주문까지 이어질 수 있는 리스크다.

2.6 칼만 평활화—노이즈를 줄이는 보조 도구

앞 절에서 만든 피쳐는 노이즈가 많다. 특히 거래량 비율은 하루마다 크게 튜다. 이것 부드럽게 다듬어 모델이 추세를 더 잘 잡게 하는 도구가 **칼만 필터(Kalman Filter)**다.

직관만 기억하면 충분하다.

- 매일 새 관측값이 들어올 때마다, “지금까지의 추정값”을 조금씩 갱신한다.
- 관측값을 얼마나 믿을지(R), 신호 자체가 얼마나 천천히 변한다고 가정할지(Q)를 두 파라미터로 조절한다.
- R이 크면 -> 관측값을 덜 믿는다 -> 평활화가 강해진다 (선이 부드러워진다).
- R이 작으면 -> 관측값을 더 믿는다 -> 원본에 가까워진다.

1차원 칼만 갱신은 예측 분산 $P_{\text{pred}} = P + Q$, 이득 $K = P_{\text{pred}} / (P_{\text{pred}} + R)$, 추정값 $x = x + K * (z - x)$, 분산 $P = (1 - K) * P_{\text{pred}}$ 순서다. 첫 유효 관측으로 x 를 초기화하고 $P \cdot Q \cdot R$ 은 유한한 양수로 둔다. 관측이 NaN이면 관측 갱신을 건너뛰되 결측 상태를 별도로 기록한다. 관측이 오래 없으면 평활값이 존재하더라도 거래 입력으로 사용하지 않는다. 여기까지가 데이터 수집·저장·피쳐·평활화의 기본 흐름이다. ## 2.7 시간봉 피쳐—6차원 확장

시간봉 전략은 4차원만으로는 정보가 부족해서 6차원을 쓴다. 일봉에서 쓰던 4차원(수익률·실현변동성·모멘텀·거래량비율)에 아래 두 차원을 덧붙인 형태이므로, 추가되는 두 차원만 적는다.

추가 차원	의미
Dim 4	봉 체결 비율—그 시간 안에서 가격이 어느 방향으로 끝났는지
Dim 5	VWAP 편차—평균 체결가 대비 종가가 얼마나 위/아래에 있는지

두 차원도 시점까지 확정된 관측만 사용한다. 예를 들어 5분 수익률은 연속 증가 비율에서, 실현 변동성은 과거 수익률 창에서 계산한다. 창이 충분히 쌓이기 전에는 값을 0으로 채워 정상 입력처럼 쓰지 않고 위밍업으로 표시한다. 장중 봉수와 일봉수는 같은 창 길이 숫자라도 시간 길이가 다르다.

2.8 Z-score —평균 회귀 신호

평균 회귀(Mean Reversion) 전략의 진입 신호는 **Z-score**다. “지금 가격이 최근 평균에서 표준편차 몇 개만큼 떨어져 있는가?”를 본다.

```
def compute_zscore(close, window=20):
    T = len(close)
    z = np.full(T, np.nan, dtype=np.float64)
    for i in range(window, T):
        win = close[i - window: i]
        mu = float(np.mean(win))
        sigma = float(np.std(win))
        if sigma > 1e-10:
            z[i] = (close[i] - mu) / sigma
    return z
```

중요한 줄: 윈도우 인덱스는 $i - \text{window} : i$ 로, 현재 시점 i 를 포함하지 않는다. 미래 데이터를 보지 않기 위한 **인과성(causality) 보장**이다.

해석: $-z > +1.5 \rightarrow$ 과매수 상태 $\rightarrow$ 숏 진입 고려 $-z < -1.5 \rightarrow$ 과매도 상태 $\rightarrow$ 롱 진입 고려
윈도우 길이는 봉 주기에 맞춰 다르게 잡는다. 일봉은 $\text{window}=20$ (약 1개월), 시간봉은 $\text{window}=140$ 을 쓴다.

2.9 모델 입력으로 넘기기—실전 패턴

지금까지 만든 조각을 모아, HMM(은닉 마르코프 모델)에 데이터를 넘기는 전체 흐름을 보자.

```
from app.data.download import download_bars
from app.trading.backtest.features import compute_daily_features

# 1. 데이터 로드 (캐시 -> 폴백)
data = download_bars(["AAPL"], days=730, interval="1d")
df = data["AAPL"]

# 2. 4차원 피쳐 계산
feat = compute_daily_features(df["close"].values, df["volume"].values)

# 3. NaN 행 제거 (초기 rolling window 구간)
valid = ~np.isnan(feat).any(axis=1)
X = feat[valid] # shape: (T_valid, 4)

# 4. 훈련/검증 분할 (시간 순서 유지)
train_len = int(len(X) * 0.7)
X_train = X[:train_len]
X_oos = X[train_len:]
```

마지막 단계에서 나오는 `X_oos`는 out-of-sample, 즉 학습에 쓰지 않고 검증에만 쓰는 구간이다.

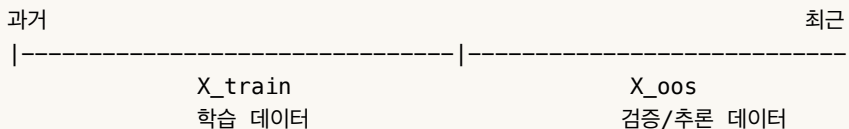
확인할 출력: - `X.shape[1] == 4` (피쳐 차원) - `X.shape[0] < len(df)` —NaN 행이 빠졌으므로 원본보다 짧다 - `len(X_train) + len(X_oos) == len(X)`

이때 시계열은 절대 섞어서 분할하면 안 된다. 앞에서 뒤로 시간 순서를 유지해야 한다.

실패하면 볼 것: HMM `fit()`에서 `ValueError: Input contains NaN`이 뜨면 3단계의 NaN 제거를 빠뜨린 것이다.

이 분할을 시간축으로 그리면 다음과 같다.

`X` = 모델이 읽는 피쳐 행렬



```

scaler.fit(X_train)
hmm.fit(X_train)

for x in X_oos:
    proba = hmm.step_forward(x)    # 모델을 새로 학습하지 않고 판단만 한다

```

핵심은 `X_oos`를 `fit()`에 넣지 않는 것이다. `X_oos`는 과거 파일 안에 들어 있지만, 검증할 때는“아직 도착하지 않은 미래 데이터”처럼 취급한다. 모델은 `X_train`에서 규칙을 배운 뒤, `X_oos`를 하루씩 받으며 추론만 한다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. `download_bars(['AAPL'], days=365)`을 처음 실행할 때와 두 번째 실행할 때 동작이 어떻게 다른지 설명하라. Parquet 파일이 디스크 어디에 생기는지 직접 확인한다.
2. 현재 파티션 기준은 `symbol / interval / 연월`이다. 이를 `symbol / interval / 분`기로 바꾸면 어떤 쿼리가 빨라지고 어떤 쿼리가 느려지는가? 각각 구체적인 예시를 든다.
3. 뉴욕 시간 인덱스와 UTC 인덱스를 각각 만든 뒤, (a) 곧바로 `tz_localize(None)`한 결과와 (b) UTC로 변환한 뒤 `tz_localize(None)`한 결과를 비교하라. 같은 순간이 어느 방식에서 정확히 정렬되는지 설명한다.

정리

이번 장에서 만든 파이프라인은 두 가지 일을 한다. API에서 받은 OHLCV(시가·고가·저가·종가·거래량) 데이터를 Parquet 파일로 저장하고, 그 데이터를 모델이 읽을 수 있는 피처 행렬로 변환한다. 결과물은 다음 장의 Backtest Engine에 바로 넘길 수 있다.

이 장에서 반드시 기억할 것

- `download_bars()` 는 데이터를 받는 유일한 창구다. FMP가 실패하면 yfinance나 DuckDB 캐시로 자동 전환된다.
- DuckDB 캐시는 종목·인터벌·연월 단위로 Parquet 파일을 나눠 저장한다. 필요한 파일만 골라 읽기 때문에 빠르다.
- 타임존이 있는 데이터는 프로젝트 기준인 UTC로 먼저 변환한 뒤 `tz_localize(None)` 으로 표지만 제거한다. 일봉 세션 날짜가 필요하면 별도 열로 보존한다.
- 일봉 모델 입력은 4차원 피쳐(수익률·실현변동성·모멘텀·거래량비율) 행렬이다. 칼만 필터(관측값의 잡음을 걸러 추세를 매끄럽게 만드는 도구)는 그 위에 얹어서 쓴다.
- Z-score는 현재 시점을 포함하지 않는 과거 윈도우로만 계산한다. 미래 값을 보면 백테스트가 거짓말을 한다.

다음 장에서는 Backtest Engine을 만든다. PortfolioSimulator의 `on_bar()` 루프, **Walk-forward 검증**(시간 순서대로 데이터를 나눠 과적합을 막는 방법), 샤프·소르티노·MDD(최대낙폭) 같은 성과 지표를 코드로 확인한다.

더 읽을거리

- 공식 문서: [DuckDB Python API](#)
- 라이브러리: yfinance GitHub —다운로드 실패 시 자동 재시도 설정 참조

3장. 백테스트 엔진—신뢰할 수 있는 시뮬레이션

“백테스트에서 잘 됐는데 실전에서 안 된다.”

퀀트 커뮤니티에서 농담처럼 쓰이지만, 가장 흔하고 비용이 큰 실수다. 백테스트는 과거 데이터로 전략을 검증하는 방법이다. 과거에 통했다는 것만 증명할 뿐, 미래에도 통한다는 보장은 아니다.

대부분의 백테스트는 현실을 너무 낙관적으로 가정한다. 슬리피지(주문이 의도한 가격과 실제 체결 가격의 차이)도 없고, 수수료도 실제보다 낮다. 시장 충격(대량 주문이 가격을 밀어내는 현상)도 없다. 이런 조건에서 나온 성과는 현실이 아니라 이상적인 가상 환경의 결과다.

신뢰할 수 있는 백테스트 엔진에는 세 가지가 필요하다.

첫째, **룩어헤드 바이어스(look-ahead bias) 방지**다. 신호를 만들 때, 그 시점에 실제로 알 수 있었던 데이터만 써야 한다. 미래 데이터가 섞이면 전략이 실제보다 좋아 보인다.

둘째, **거래 비용의 현실적 반영**이다. 스프레드, 슬리피지, 수수료를 빠짐없이 포함해야 한다.

셋째, **유동성 검증**이다. 주문 크기가 시장에서 실제로 소화할 수 있는 거래량을 초과하지 않는지 확인해야 한다.

하나라도 빠지면 백테스트 결과는 실제보다 좋게 나온다.

이 장에서는 신호·체결·평가의 시간 순서와 거래비용 회계를 작은 실행 예제로 검증한다. 유동성과 시장 충격은 실전 적용 전에 별도로 검증해야 한다.

이 장의 흐름

먼저 백테스트가 봉마다 무엇을 하는지 전체 그림을 본다. 그다음 핵심 코드를 읽고, 성과를 어떻게 숫자로 평가하는지 본다.

- 3.1 전체 그림—신호·체결·손익의 시간 순서
- 3.2 실행 가능한 손익 회계 실습
- 3.3 비용과 최종 청산을 한 번만 반영하기
- 3.4 샤프지수—`_build_result()`
- 3.5 PnL Gini —집중도 지표
- 3.6 디플레이션된 샤프지수(DSR) —다중 검정 보정
- 3.7 PortfolioConfig —전략별 권장 설정
- 3.8 성능 지표 해석

학습 목표

이 장이 끝나면 다음을 할 수 있다.

- `simulate()`가 이전 종가 신호를 다음 시가에 체결하고 종가로 평가하는 순서를 설명한다.
- OOS(Out-of-Sample) 분리의 목적과 교육용 회계 함수가 신호 모델 학습을 포함하지 않는다는 점을 설명한다.
- 샤프지수, MDD, Win Rate가 각각 무엇을 측정하는지 한 줄로 말한다. Sortino/Calmar/Omega/VaR은 카탈로그 수준으로 안다.
- 디플레이션된 샤프지수(DSR)가 왜 필요한지 설명하고 p-value 기준으로 해석한다.

이 장에서 만드는 것: 봉마다 정해진 순서로 신호 -> 체결 -> 평가를 처리하는 시뮬레이터를 만든다. “백테스트는 좋은데 실전에서는 왜 안 되는가?”—이 장이 그 질문에 구조적으로 답한다.

TIP**바이브코딩 미션**

- **얻을 결과:** 백테스트 한 번에서 진입·청산·거래비용이 처리되는 순서를 찾는다.
- **첫 회독:** 모든 지표 공식보다 미래 데이터를 보지 않는 봉 처리 순서를 우선한다.
- **용어 발판:** 봉(bar)은 일정 시간의 가격 묶음, 신호는 매매 후보 판단, 체결은 주문이 실제 거래로 성립한 상태다.
- **AI 요청:** “simulate()를 읽고 한 봉에서 일어나는 일을 순서대로 요약해줘. 각 단계가 사용하는 시점의 데이터도 함께 표시하고 코드는 수정하지 마.”
- **완료 확인:** 신호 계산 시점과 체결 시점을 구분해 설명할 수 있으면 성공이다.

3.1 전체 그림—신호·체결·손익의 시간 순서

WARNING

가장 흔한 Backtest 오류: 당일 종가(close)로 진입 신호를 만들고 당일 종가로 체결하는 패턴. 실전에서는 종가가 확정되는 순간 이미 거래 시간이 끝났다. 신호는 **이전 봉** 데이터로 만들고, **다음 봉 시가**로 체결해야 한다.

백테스트에서 가장 먼저 해야 할 일은 데이터를 시간 순서대로 나누는 것이다. 예를 들어 5년치 데이터가 있다면 앞 2년을 학습 구간으로 두고 뒤 3년을 OOS 구간으로 남겨둘 수 있다. 비율은 전략마다 다르다. 이 책의 기본 엔진은 조금 더 일반적인 설정으로, 앞 60%를 학습용, 뒤 40%를 OOS 평가용으로 쓴다.

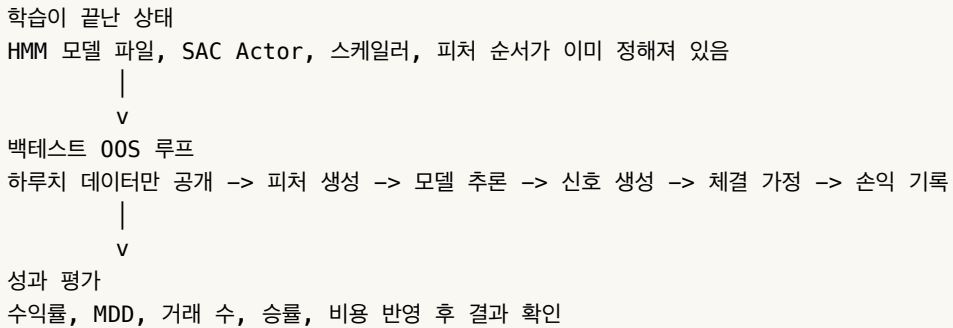
과거 구간으로 신호 모델 학습

- > 평가 시작 시 현금·수량·총자산 기준값 기록
- > 이전 봉 종가 뒤에 확정된 목표를 이번 봉 시가에 체결
- > 체결별 현금·수량·수수료 갱신
- > 마지막 봉이면 잔여 수량 청산 및 비용 반영
- > 총자산 = 현금 + 보유 수량 × 이번 봉 종가
- > 봉별 손익 = 이번 총자산 - 이전 총자산
- > 다음 봉에 쓸 목표를 이번 봉까지의 정보로 결정

OOS 구간은 “이미 가진 과거 데이터”지만, 검증할 때는 실제 미래처럼 다룬다. 모델이 그 구간을 미리 보고 학습하면 안 된다. 백테스트 엔진이 OOS 구간에서만 루프를 도는 이유가 여기에 있다.

한 가지 함정이 더 있다. OOS 결과를 본 뒤 파라미터를 바꾸고 같은 OOS를 반복해서 확인하면, 구간도 사실상 학습에 사용한 샘플이 된다. 고정 60/40 분리는 첫 검증 구조일 뿐 최종 보증이 아니다. 탐색 중에는 별도 검증 구간을 쓰고, 최종 홀드아웃은 가능한 한 적게 열어 보거나 15장과 24장의 Walk-forward 절차로 여러 시간 구간을 순서대로 평가한다.

초보자는 백테스트를 **과거 데이터로 만든 모의 실전장**이라고 보면 된다. 모델이 학습을 끝낸 뒤, 과거의 어느 시점으로 돌아가서 그날 알 수 있었던 정보만 보고 하루씩 앞으로 걸어가는 실험이다. 그래서 백테스트는 단순히 수익률을 계산하는 코드가 아니라, 머신러닝 모델이 실전처럼 작동하는지 확인하는 검증 장치다.



여기서 중요한 말은“하루치 데이터만 공개”다. OOS 루프의 2024년 3월 5일에서는 2024년 3월 6일 이후 데이터를 모른다고 가정한다. 컴퓨터는 전체 파일을 가지고 있지만, 엔진은 일부러 그 정보를 모델에 주지 않는다. 이 규칙을 지켜야 백테스트가 실전 리허설이 된다.

백테스트 엔진은 봉을 시간 순서대로 처리한다. **이전 봉 종가 뒤에 정한 신호 -> 이번 봉 시가 체결 -> 이번 봉 종가 평가**가 이 실습의 계약이다. 이전 신호로 이번 시가에 진입했다면 같은 봉의 시가에서 종가까지 발생한 손익을 기록하는 것이 맞다. 잘못된 것은 이번 종가를 보고 그보다 앞선 시가에 거래했다고 가정하는 것이다. 기존 포지션의 청산과 신규 진입도 실제 체결 시점의 현금·수량·비용에 반영한다.

OOS(Out-of-Sample)만 평가한다: oos_start 이전 구간은 훈련 데이터다. 모델은 훈련 데이터로 학습하지만, 수익률은 그 뒤 OOS 구간에서만 측정한다.

3.2 실행 가능한 손익 회계 실습

운영 엔진 전체 대신 회계 계약을 독립적으로 검증하는 작은 엔진을 사용한다. 교육용 코드 묶음의 `accounting-lab/backtest.py`와 동일한 함수이며 Python 표준 라이브러리만 사용한다. HMM이나 브로커가 없어도 실행할 수 있다.

`cash`는 통화 금액, `quantity`는 부호가 있는 보유 수량이다. 총자산은 `cash + quantity * close`이고, 봉별 손익은 이전 총자산과의 차이이다. 보유 중 평가손익을 빼거나 청산 시 누적 손익을 다시 더하면 지표가 왜곡된다.

`targets[t]`는 `t`봉 종가 이후 정한 목표 방향이며 `t+1`봉 시가에 체결한다. 과거 정보만 사용해 목표를 생성하는 책임은 신호 생성기에 있다. 마지막 봉에서는 청산과 비용 차감 이후 총자산을 기록한다.

```
import math

def simulate(
    bars, targets, initial=1000.0, fee_bps=0.0, slippage_bps=0.0
):
    """Fill targets[t], known after close[t], at open[t+1].

    Use fixed unit size.

    cash is currency; quantity is signed units; equity = cash +
    quantity*close.
    Omit financing, margin, taxes, dividends and market impact.
    """
    if len(bars) != len(targets) or not bars:
        raise ValueError("Provide one target per nonempty bar sequence")
    if not math.isfinite(initial) or initial <= 0:
        raise ValueError("initial must be positive and finite")
    if any(
        not math.isfinite(x) or x < 0 or x >= 10000
        for x in (fee_bps, slippage_bps)
    ):
        raise ValueError(
            "costs must be finite and between 0 and 10000 bps"
        )
    if any(t not in (-1, 0, 1) for t in targets):
        raise ValueError("target must be -1, 0 or 1")
    if any(
        not math.isfinite(b[k]) or b[k] <= 0
```

```

        for b in bars
        for k in ("open", "close")
    ):
        raise ValueError("prices must be positive and finite")
    cash, quantity = initial, 0.0
    unit = initial / bars[0]["open"]
    equity, pnl, fills = [initial], [], []

    def trade(delta, price, index, reason):
        nonlocal cash, quantity
        execution = price * (
            1 + math.copysign(slippage_bps / 10000, delta)
        )
        fee = abs(delta) * execution * fee_bps / 10000
        cash -= delta * execution + fee
        quantity += delta
        fills.append(
            {
                "bar": index,
                "quantity": delta,
                "price": execution,
                "fee": fee,
                "reason": reason,
            }
        )

    for i, bar in enumerate(bars):
        desired = targets[i - 1] * unit if i else 0.0
        delta = desired - quantity
        if delta:
            trade(delta, bar["open"], i, "previous_close_target")
            # Liquidate before recording the last equity sample.
            # Include the final fill cost.
            if i == len(bars) - 1 and quantity:
                trade(-quantity, bar["close"], i, "final_liquidation")
            current = cash + quantity * bar["close"]
            pnl.append(current - equity[-1])
            equity.append(current)
    peak, mdd = initial, 0.0
    for value in equity:
        peak = max(peak, value)
        mdd = max(mdd, (peak - value) / peak)

```

```

assert math.isclose(initial + sum(pnl), equity[-1], abs_tol=1e-8)
return {
    "equity": equity,
    "bar_pnl": pnl,
    "final_equity": equity[-1],
    "return": equity[-1] / initial - 1,
    "mdd": mdd,
    "fills": fills,
    "fees": sum(f["fee"] for f in fills),
}

```

다음 확인 코드까지 backtest.py 아래에 붙이면 CSV나 외부 자료 없이도 실행할 수 있다. ZIP의 테스트는 이 사례를 포함한 여러 경계를 자동 검사한다.

```

bars = [
    {"open": 100.0, "close": 100.0},
    {"open": 100.0, "close": 80.0},
    {"open": 80.0, "close": 100.0},
]
result = simulate(bars, [1, 1, 0])
assert result["equity"] == [1000.0, 1000.0, 800.0, 1000.0]
assert math.isclose(result["mdd"], 0.2)
print(result["final_equity"], result["mdd"]) # 1000.0 0.2

```

3.3 비용과 최종 청산을 한 번만 반영하기

가격이 100 → 80 → 100일 때 1,000을 투자했다면 최종 손익이 0이어도 보유 중 낙폭은 20%다. 기본 CSV 실습의 자산 곡선은 거래 전 기준값까지 포함해 [1000, 1000, 800, 1000]이다. 최종 현금만 기록하면 이 손실 구간이 사라진다.

수수료는 실제 체결 금액에 매번 한 번만 적용한다. 수량 10개를 100에 사고 100에 팔며 각 방향 10bp를 내면 총비용은 2이고 최종 자산은 998이다. 진입 비용을 이미 차감했다면 청산 시 왕복 비용을 다시 차감하지 않는다.

```

python3 backtest.py
python3 -m unittest -v

```

```
python3 backtest.py --fee-bps 10 --slippage-bps 5
```

교육용 코드의 accounting-lab 폴더에서 실행한다. bp는 0.01%포인트다. 비용을 0.5·10·25bp 처럼 바꿔 민감도를 확인하되, 이 값들을 모든 시장의 적정 비용으로 해석하지 않는다. 매수 체결가는 높이고 매도 체결가는 낮추는 고정 슬리피지를 적용한다.

실습의 한계: 최초 시가로 정한 고정 수량을 사용하며 실제 투자전략·증거금·차입비·세금·기업행동·시장 충격은 포함하지 않는다. 이 작은 엔진을 운영 포트폴리오 엔진과 혼동하거나 실거래에 연결하지 않는다.

3.4 샤프지수—_build_result()

백테스트가 끝나면 결과 빌드 단계에서 성과 지표를 계산해 결과 객체로 반환한다. 첫 회독에서 꼭 봐야 할 지표는 세 개다.

- **샤프지수:** 위험(변동성) 한 단위당 얼마나 벌었나. 높을수록 좋다.
- **MDD(Maximum Drawdown):** 자산이 고점 대비 최대 얼마나 빠졌는가. MDD가 깊으면 샤프지수가 좋아도 실전에서 못 버틴다.
- **Win Rate:** 전체 거래 중 이긴 비율. 50%보다 낮아도 평균 이익이 평균 손실보다 충분히 크면 수익 전략일 수 있으므로, 손익비와 함께 본다.

NOTE

지표 하나만 보고 전략을 판단하면 위험하다. 샤프지수만으로 손실의 깊이를 알 수 없고, MDD만으로 수익이 꾸준한지 알 수 없다. 체크리스트처럼 함께 본다.

위 그림에서는 누적 자산이 오르더라도 그 과정의 낙폭이 상당할 수 있음을 한눈에 확인할 수 있다. 수익 곡선만 제시하지 않고 같은 시간축의 낙폭을 나란히 그려야 백테스트의 경로 위험이 드러난다.

변동성 보정—샤프지수

평균 수익만 보면 전략을 쉽게 오해한다. 하루하루 크게 흔들리며 번 수익과, 작게 흔들리며 번 수익은 다르게 봐야 한다. 그래서 결과 빌드 단계에서는 수익률과 변동성을 함께 본다.



Source: local cached daily OHLCV through 2025-12-29 · Educational reproduction · Not investment advice

그림 1. 로컬에 저장된 SPY 일봉으로 재현한 단순 이동평균 전략의 누적 자산과 낙폭. 신호는 하루 늦춰 적용하고 포지션 변경 때 5bp 비용을 반영했다. 교육용 재현 결과이며 투자 성과를 보장하지 않는다.

첫 회독에서는 공식을 외울 필요가 없다. 핵심은 **수익이 크더라도 변동성과 낙폭이 함께 커지면 좋은 전략으로 보지 않는다**는 점이다. Sharpe를 계산할 때는 같은 총자산 곡선의 연속 수익률을 사용하고 봉 주기에 맞는 연환산 계수를 명시한다. 미국 정규장 5분봉의 $252 * 78$ 가정을 다른 시장이나 일봉에 그대로 적용하면 안 된다. 관측 수가 부족하거나 분산이 0이면 계산 불가로 표시한다.

MDD와 Win Rate

MDD는 시가평가 총자산의 최대 낙폭 비율이고, Win Rate는 양수 손익으로 완료된 왕복 거래의 비율이다. 실습의 fills 개수는 체결 수이므로 왕복 거래 수나 승률로 사용하면 안 된다. 아래 식은 MDD를 **양수인 낙폭의 크기**로 표시한다.

$$\text{MDD} = \text{모든 시점 } t \text{에 대해} \\ (\text{그때까지의 최고 계좌값} - \text{현재 계좌값}) / \text{그때까지의 최고 계좌값} \\ \text{중에서 가장 큰 값}$$

라이브러리나 보고서에 따라 같은 낙폭을 8% 또는 -8%로 표시한다. 전자는 낙폭의 **크기**, 후자는 고정 대비 **수익률 변화**다. 둘은 같은 깊이를 뜻한다. 결과를 비교하기 전에는 부호뿐 아니라 **0.08**과 8% 중 어떤 단위를 쓰는지도 확인한다.

더 정교한 지표 (카탈로그)

같은 데이터를 다른 각도에서 보는 지표들이다. Sortino는 평균 초과수익을 하방편차로, Calmar는 연간 수익을 양수 MDD로 나눈다. Omega는 선택한 기준 수익률 위의 초과분 합을 아래의 부족분 합으로 나눈다. 분모가 0이면 계산 불가로 표시한다. 아래 표는 각 지표가 답하는 질문이다.

지표	한 줄 의미
Sortino	분모에 하방 변동성만 써서 상승 변동성 페널티를 제거
Calmar	연수익률 ÷ MDD —낙폭 대비 수익
Omega	양수 수익의 합 ÷ 음수 수익의 합 절댓값
VaR(95%)	일간 수익률 하위 5% 지점—드물게 발생하는 큰 손실 크기
CVaR(95%)	VaR 아래 5% 구간의 평균 손실
Profit Factor	총 이익 ÷ 총 손실
PnL Gini	종목별 수익 집중도 (3.5)

3.5 PnL Gini —집중도 지표

PnL Gini는 수익 기여가 특정 종목에 얼마나 쏠려 있는지 측정한다. 이 책의 0~1 해석은 종목별 **비율수 기여도**(예: 양수 PnL만 남기거나 절대 PnL을 사용)를 먼저 정한 경우에 해당한다. 0이면 모든 종목이 균등하게 기여하고, 1에 가까우면 한 종목에 거의 전부가 몰린 상태다. 이익과 손실이 섞인 signed PnL을 그대로 넣으면 일반 Gini가 0~1 범위를 벗어나거나 해석이 달라질 수 있으므로, 계산 전에 어떤 기여도를 썼는지 명시한다. 이 프로젝트는 < 0.4를 **초기 경고 기준 예시**로 쓰지만 보편적인 합격선은 아니다. 종목 수와 기여도 정의를 함께 보고 집중 원인을 점검한다.

3.6 디플레이션된 샤프지수(DSR) —다중 검정 보정

백테스트를 많이 돌릴수록 우연히 좋아 보이는 결과가 나올 확률이 높아진다. **디플레이션된 샤프지수 (Deflated Sharpe Ratio, DSR)**는 이 편향을 보정한다.

전략 하나만 테스트해서 좋은 결과가 나왔다면 그 결과는 검토할 가치가 있다. 하지만 전략 500개를 테스트한 뒤 가장 좋은 것만 골랐다면, 그 전략은 그냥 운이 좋았을 가능성이 훨씬 높다. DSR은 “몇 번이나 시도했는가”를 반영해 결과의 신뢰도를 낮춰 보는 장치다.

`n_trials`에는 최종 후보 수만 넣으면 안 된다. 중간에 폐기한 아이디어, 바꿔본 파라미터 조합, 변형해서 다시 돌린 실험까지 모두 포함해야 DSR이 실제 과최적화 위험을 제대로 반영한다.

```
from app.trading.backtest.shared import compute_deflated_score_pvalue

dsr_pval = compute_deflated_score_pvalue(
    score_obs      = score,          # 관측된 샤프지수
    n_obs          = len(bar_ret),   # 봉 개수
    n_trials       = n_strategies_tested, # 시도한 전략 수
    skew           = sk,
    kurtosis_excess = ku,
)
```

반환된 p-value는 미리 정한 기준과 함께 **증거의 강도**로 읽는다. 아래 구간은 설명용 예시이며 자동 채택·기각 규칙이 아니다.

p-value	해석
< 0.005	귀무가설에 반대되는 비교적 강한 증거
0.005 ~ 0.05	추가 검토가 필요한 증거
> 0.05	이번 표본만으로는 증거가 부족함

DSR p-value 하나가 경제적 타당성, 비용 후 OOS 성과, 낙폭 안정성을 대신하지 않는다. `n_trials` 누락이나 수익률 분포 가정이 맞지 않으면 수치 자체도 낙관적일 수 있다.

수식 유도 등 통계적 배경은 Bailey & Lopez de Prado (2014) 논문을 참고한다.

3.7 PortfolioConfig —전략별 권장 설정

전략마다 적합한 포지션 수와 크기가 다르다. 추세 추종처럼 몇 종목에 크게 베팅하는 전략과, 페어 트레이딩처럼 여러 쌍에 분산하는 전략은 설정이 달라진다. 각 신호는 자기 성격에 맞는 추천 설정을 돌

려주고, 시뮬레이터는 `use_signal_config=True`일 때 그 설정을 쓰고 그렇지 않으면 기본값을 쓴다.

`max_positions`는 동시 보유 상한, `per_trade_alloc`은 거래당 자본 비중, `commission`은 체결 비용률, `is_pair_strategy`는 두 다리를 묶어 처리할지, `rebalance_freq`는 재조정 간격이다. 예를 들어 상한 5·거래당 10%면 다섯 신호를 동시에 허용해도 신규 배정 합계는 50%다. 페어의 10%가 두 다리 합계인지 각 다리인지 명시하고, 전략 권장값보다 계좌 전체 위험 한도를 마지막에 우선한다.

3.8 성능 지표 해석

지표 절대값만으론 감이 잘 안 잡힌다. 각 수치를 아래처럼 질문으로 바꿔 놓으면 결과가 좋은지 나쁜지 판단하기 쉬워진다.

지표	질문
샤프지수	수익이 변동성에 비해 과장되어 보이지 않는가
Sortino	손실 방향의 변동성은 감당 가능한가
MDD	최악의 낙폭을 실제 계좌가 버틸 수 있는가
Calmar	낙폭 대비 수익이 충분한가
Omega	수익 분포가 한쪽으로 심하게 치우치지 않았는가
Win Rate	신호가 무작위처럼 보이지 않는가
Profit Factor	총이익이 총손실을 충분히 넘는가
PnL Gini	수익이 한두 종목에 몰려 있지 않은가
DSR p-value	많은 실험 중 우연히 좋은 결과만 고른 것은 아닌가

이 책에서 다루는 전략 결과도 위 질문들로 읽는다. 특정 숫자 하나를 목표로 삼지 말고, 낙폭·거래비용·표본 밖 성과·수익 집중도를 함께 확인해야 한다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. `accounting-lab/backtest.py`를 `--fee-bps 0`과 `--fee-bps 10`으로 실행하라. 최종 자산과 낙폭의 변화를 비교하고 각 체결의 수수료 합계가 자산에서 한 번만 빠지는지 확인하라.
2. `simulate()`의 `targets[i - 1]`을 `targets[i]`로 바꾸면 왜 미래 정보를 사용하게 되는지 설명하라. 목표가 증가 뒤에 정해지는데 같은 봉 시가로 체결한다고 가정할 때의 시간 모순을 짚어라. 이전 신호로 시가에 진입한 포지션의 당일 증가 평가손익은 정상적으로 포함한다.
3. 서로 다른 가격 경로를 가진 두 bars 입력을 작성하라. 최종 자산이 같아도 MDD가 달라지는 사례를 만들고, 자산 곡선과 봉별 손익의 합이 일치하는지 확인하라.

정리

이 장에서는 신호 시점과 체결 시점을 구분하는 회계 예제를 실행하고, 주요 성과 지표의 해석을 살폈다. 이후 전략을 평가할 때에도 같은 시간 순서와 비용 회계 원칙을 적용한다.

이 장에서 반드시 기억할 것:

- **OOS(Out-of-Sample) 분리**가 평가의 출발점이다. 앞 60%로 학습하고, 뒤 40%에서만 수익률을 낸다.
- **시점이 중요하다**: 이전 증가 신호 -> 다음 시가 체결 -> 해당 증가 평가. 신호를 만들 때 아직 모르는 정보를 사용하면 결과가 왜곡된다.
- **수수료는 체결별로 한 번씩** 계산한다. 진입·청산 가격과 수량이 다르면 각 체결 금액을 기준으로 비용을 계산한다.
- **지표 하나만 보면 안 된다**. 샤프지수(수익 대비 변동성), MDD(최악의 낙폭), Win Rate(이긴 비율)을 함께 본다.
- **DSR**은 “많이 시도해서 우연히 좋아 보이는 것”의 위험을 보정하는 보조 지표다. 사전 기준과 함께 읽되, 단일 p-value로 전략을 자동 채택하거나 기각하지 않는다.

다음 장에서는 HMM(Hidden Markov Model)의 기초 이론을 다룬다. HMM이 만들어내는 매수·매도 신호도 결국 이 장의 엔진을 통과해야 가치를 인정받는다—모델이 무엇이든 평가 기준은 같다.

더 읽을거리

- **책:** Ernest Chan, *Algorithmic Trading* (Wiley, 2013) —백테스트 엔진 설계와 거래비용 모델링
- **논문:** “The Deflated Sharpe Ratio” (Bailey & Lopez de Prado, 2014) —다중 검정으로 부풀려진 샤프지수를 보정해 통계적 유의성을 평가
- **라이브러리:** `vectorbt` —고성능 벡터라이즈드 백테스트 엔진, Numba 기반

4장. HMM 기초—시장 레짐을 확률로 추론하기

Part II —HMM 기초 (4~5장) 이 파트에서는 시장을 “하나의 고정된 상태”가 아니라 여러 레짐이 바뀌는 과정으로 본다. 4장은 HMM의 기본 구조, 5장은 미래 데이터를 보지 않는 Forward-only 추론이다. 첫 회독에서는 수식보다 “현재 레짐을 확률로 추정한다”는 감각을 먼저 잡는다.

가격, 거래량, 이동평균, RSI, MACD는 차트에서 직접 읽을 수 있다. 하지만 이 숫자들 뒤에 있는 것—시장이 지금 어떤 상태인지—는 눈에 보이지 않는다. 추세 장, 횡보 장, 고변동성 장 같은 구분이 바로 이 숨겨진 상태(hidden state)다.

경험 많은 트레이더들은 이 상태를 직관으로 판단한다. “지금은 추세 장이니까 RSI를 무시해야 한다”, “지금은 레인지 장이니까 모멘텀 전략이 안 먹힌다”는 식이다. 문제는 이 직관이 사람마다 다르고 일관성이 없다는 것이다. 시스템으로 만들기가 어렵다.

HMM(은닉 마르코프 모델, Hidden Markov Model)은 이 직관을 수학으로 바꾼다. HMM이 답하는 질문은 하나다. “관측 가능한 데이터(수익률, 변동성 등)가 주어졌을 때, 지금 시장이 어떤 레짐에 있을 확률이 얼마인가?” 가격 자체를 맞추기보다 가격을 만드는 구조인 레짐을 먼저 추론한다는 사고방식이다.

처음 읽을 때 수식을 전부 외울 필요는 없다. 이 장에서 잡아야 할 핵심 감각은 세 가지다.

1. HMM은 “가격이 오를까?”를 직접 묻지 않는다. “지금 시장이 어떤 상태인가?”를 먼저 묻는다.
2. 상태는 눈에 보이지 않는다. 그래서 수익률, 변동성, 거래량 같은 관측값으로 확률을 추정한다.
3. 결과는 단 하나의 정답이 아니라 확률 벡터다. 예를 들어 [MR 70%, Trend 20%, Volatile 10%] 형태로 나온다.

HMM은 세 요소(A, B, π)로 정의된다. 이름만 먼저 잡아두자.

- **A** — 상태 간 이동 확률표 (“상태가 얼마나 오래 지속되는가”)
- **B** — 상태별 데이터 분포 (“각 상태에서 어떤 데이터가 나오는가”)
- **π** — 초기 상태 확률 (“처음 시작할 때 어떤 상태라고 볼 것인가”)

이 장의 수식은 “모델을 직접 유도하기 위한 수학”이 아니라 “코드가 왜 그렇게 생겼는지 읽기 위한 지도”다. GaussianHMM(`n_components=3`)을 호출하면 라이브러리가 대부분을 계산해준다. 하지만 어떤 피처를 넣을지, 상태를 몇 개로 둘지, 레짐 번호를 어떻게 다시 붙일지는 사람이 정해야 한다. 이 판단을 내리려면 A, B, π 가 무엇을 뜻하는지는 알아야 한다.

HMM을 머신러닝 모델로 보면

HMM이 낯설게 느껴지는 이유는 이름 때문이다. 하지만 머신러닝의 기본 흐름으로 보면 구조는 단순하다.

1. **입력 피처를 만든다**: 가격을 그대로 넣지 않고 수익률, 변동성, 모멘텀, 거래량 비율로 바꾼다.
2. **과거 구간으로 학습한다**: `fit()`이 과거 피처를 보고 A, B, π 를 맞춘다.
3. **새 봉에서 추론한다**: 학습된 모델이 오늘의 피처를 보고 레짐 확률을 계산한다.
4. **추론 결과를 신호에 연결한다**: MR 확률이 높을 때만 평균회귀 진입을 허용한다.

여기서 GaussianHMM 클래스는 알고리즘을 담은 틀이고, `fit()`이 끝난 객체가 모델이다. `fit()` 전에는 아직 시장을 배운 것이 아니다. `fit()` 후에는 A, B, π 가 데이터에 맞게 정해졌으므로, 새 피처를 넣으면 레짐 확률을 돌려줄 수 있다.

아직 학습 전:
GaussianHMM(`n_components=3`)
상태 3개로 나눌 준비만 된 빈 틀

학습:

빈 틀 + X_train --fit()--> 학습된 HMM 모델

추론:

학습된 HMM 모델 + 오늘 피쳐 --> [MR 70%, Trend 20%, Volatile 10%]

비유하면 GaussianHMM은 아직 눈금을 맞추지 않은 시장 판별기다. fit()은 과거 데이터를 보면서 눈금을 맞추는 과정이다. 눈금이 맞춰진 뒤에야 오늘 데이터를 넣고 “지금은 어떤 시장인가”를 물어볼 수 있다.

즉 HMM은 이 책에서 처음 만나는 “주식용 머신러닝 모델”이다. 다만 지도학습처럼 정답 라벨을 주고 맞추는 방식은 아니다. 데이터 안에 숨어 있는 상태를 스스로 나누는 **비지도 학습(unsupervised learning)**에 가깝다. 우리가 “이 날은 MR, 저 날은 Trend”라고 미리 정답을 붙이지 않아도, HMM은 관측 패턴이 비슷한 날들을 상태별로 묶는다. 학습이 끝난 뒤 그 상태에 MR / Trend / Volatile이라는 의미를 붙이는 일은 사람이 한다.

이 차이를 기억하면 뒤 장의 RL도 더 쉽게 이어진다. HMM은 시장 상태를 추정하는 모델이고, SAC는 그 상태와 다른 피쳐를 받아 포지션 크기를 정하는 모델이다. 둘 다 “데이터 -> 피쳐 -> 학습 -> 추론” 흐름 위에 있지만, 맡은 역할이 다르다.

HMM이 만든 레짐 확률은 이 장에서 끝나지 않는다. 뒤의 강화학습 장에서는 이 확률이 SAC 에이전트의 입력으로 들어간다.

HMM이 찾은 레짐

[MR 70%, Trend 20%, Volatile 10%]

↓
v

RL 상태 벡터의 일부

[z_score, position, pnl, MR_prob, Trend_prob, Volatile_prob, ...]

↓
v

SAC가 포지션 크기 결정

그래서 HMM은 단순한 필터가 아니라, 뒤에서 나올 딥러닝 에이전트에게 “시장 배경”을 알려주는 센서 역할도 한다. HMM이 시장 날씨를 읽고, RL은 그 날씨 속에서 얼마나 빠르게 움직일지 정한다.

같은 신호도 레짐이 다르면 뜻이 달라진다

HMM이 왜 필요한지 가장 쉽게 보는 방법은 같은 Z-score를 두 장면에 놓아보는 것이다.

장면 A

$z\text{-score} = 2.5$

HMM = [MR 80%, Trend 10%, Volatile 10%]

해석:

가격 차이가 크게 벌어졌고, 시장은 평균회귀 장에 가깝다.

-> 벌어진 가격 차이가 다시 줄어들 가능성을 볼 수 있다.

장면 B

$z\text{-score} = 2.5$

HMM = [MR 10%, Trend 70%, Volatile 20%]

해석:

가격 차이는 똑같이 벌어졌지만, 시장은 추세 장에 가깝다.

-> 벌어진 가격 차이가 더 벌어질 수 있으므로 조심해야 한다.

두 장면 모두 Z-score는 2.5다. Z-score만 보면 같은 신호처럼 보인다. 하지만 HMM 레짐을 함께 보면 전혀 다른 상황이다. 평균회귀 장에서는 진입 후보가 될 수 있고, 추세 장에서는 오히려 피해야 할 신호가 될 수 있다.

이 차이가 이 책의 핵심이다. **피쳐 하나만 보고 매매하지 않고, 시장 상태를 먼저 읽은 뒤 신호를 해석한다.** 뒤에서 RL을 붙일 때도 이 생각은 그대로 이어진다. RL은 HMM 레짐을 보고 같은 Z-score에도 다른 포지션 크기를 배울 수 있다.

이 장의 흐름

먼저 HMM이 어떤 질문에 답하는지 잡고, 그다음 세 요소와 코드 흐름, 운영 시 주의점으로 내려간다.

- 4.0 Big Picture —hmm_daily_mr.py 파이프라인
- 4.1 HMM 3요소 (A, B, π)
- 4.2 Baum-Welch —학습 직관
- 4.3 `_DailyHMM.fit()` —실제 구현과 Look-ahead 주의
- 4.4 분산 기반 레짐 정렬
- 4.5 `_DailyHMM.predict_proba()` —레짐 확률 계산
- 4.6 `_update_hmm()` —10봉 주기 재학습
- 4.7 `_check_entry()` —Z-score + HMM 게이트
- 4.8 `_check_exit()` —청산 조건 6가지

- 4.9 PureZscoreDailySignal —기준선 비교
- 4.10 전체 흐름—on_bar() 호출 순서

학습 목표

이 장이 끝나면 다음을 할 수 있다:

- HMM의 3가지 파라미터(A, B, π)가 각각 무엇을 나타내는지 설명할 수 있다
- Baum-Welch 알고리즘이 반복 학습으로 파라미터를 찾는 과정을 직관적으로 설명할 수 있다
- covariance_type="full"을 선택하는 이유와 min_covar=1e-3의 역할을 이해할 수 있다
- 분산을 기준으로 상태를 MR / Trend / Volatile 레짐에 자동 배정하는 코드를 읽을 수 있다
- Z-score 신호와 HMM 게이트가 어떻게 결합되어 진입 신호를 만드는지 설명할 수 있다

첫 회독에서 볼 것

이 장은 HMM이라는 낯선 모델을 처음 다룬다. 처음 읽을 때는 모든 옵션과 상수를 외우지 않아도 된다. 다음 세 가지만 잡으면 충분하다.

- HMM은 가격을 바로 예측하는 모델이 아니라, **지금 시장 상태를 확률로 나누는 모델**이다.
- 이 책에서는 HMM을 매수/매도 신호로 직접 쓰지 않고, **신호를 허용하거나 막는 게이트**로 쓴다.
- Look-ahead Bias를 피하려면 미래 데이터가 섞이지 않게 레짐 확률을 계산해야 한다. 이 부분은 5장에서 더 자세히 다룬다.

min_covar, 재학습 주기, 청산 조건 표 같은 세부 설정은 두 번째 회독에서 봐도 된다. 지금은 “왜 HMM이 필요한가”와 “어디서 위험한가”에 집중하자.

이 장에서 만드는 것

Part 2에서는 **HMM**을 시장 레짐 감지에 적용한다. 레짐(regime)이란 시장이 지금 어떤 성격으로 움직이는지를 가리키는 말이다. 예를 들어 “지금은 조용한 횡보장”, “지금은 방향성 있는 추세장”같은 구분이 레짐이다.

HMM은 이 레짐을 직접 관측할 수 없다고 가정한다. 수익률이나 변동성 같은 숫자만 보면서, 지금 어떤 레짐일지 확률적으로 추론한다. 핵심 원칙은 하나다. **미래 데이터를 쓰지 않고, 지금까지 관측한 데이터만으로 현재 레짐을 판단한다.**

관측 데이터	숨겨진 상태(레짐)	의미
수익률 낮음 / 변동성 낮음	REGIME_MR (0)	평균회귀 적합
수익률 중간 / 변동성 중간	REGIME_TREND (1)	추세 장
수익률 급변 / 변동성 높음	REGIME_VOLATILE (2)	위험 구간

진입 신호는 MR 확률이 일정 수준 이상이고 REGIME_VOLATILE 확률이 낮을 때만 허용한다. REGIME_TREND가 우세해도 MR 확률이 하한을 넘고 변동성 확률이 낮으면 조건부로 통과한다. REGIME_VOLATILE로 전환되면 신규 진입을 막고, 이미 열려 있는 포지션은 조기 청산한다.

TIP

바이브코딩 미션

- **얻을 결과:** HMM 출력 한 줄을 [평균회귀, 추세, 고변동] 확률로 읽는다.
- **첫 회독:** Baum-Welch 유도는 미루고 입력 피쳐, 상태 수, 확률 출력만 본다.
- **용어 발판:** HMM은 보이지 않는 시장 상태를 확률로 추정하는 모델, 피쳐는 입력 숫자, 상태 확률은 각 레짐일 가능성의 묶음이다.
- **AI 요청:** “이 장의 HMM에서 관측 100개·피쳐 4개·상태 3개일 때 입력과 출력 shape를 설명해줘. $A \cdot B \cdot \pi$ 의 역할과 확률 합이 1이어야 하는 축을 짚어줘.”
- **완료 확인:** 세 확률의 합이 1에 가깝고 가장 큰 상태를 말할 수 있으면 성공이다.

4.0 전체 그림—hmm_daily_mr.py 파이프라인

코드를 보기 전에 전체 흐름부터 파악하자. OHLCV(시가·고가·저가·종가·거래량) 데이터가 레짐 확률 벡터를 거쳐 매매 신호로 바뀌는 과정은 4단계다. 1·3·4단계는 매 봉(bar)마다 실행되고, 2단계 학습만 10봉마다 다시 수행해 시장 구조 변화에 적응시킨다.

Step	호출	입력 -> 출력
1	<code>compute_daily_features()</code>	ohlcv 원시 데이터 -> 4차원 피쳐 행렬
2	<code>_DailyHMM.fit()</code>	피쳐 행렬 -> 학습된 GaussianHMM
3	<code>_DailyHMM.predict_proba()</code>	HMM -> 레짐 확률 벡터 [MR, Trend, Volatile]
4	<code>_check_entry()</code> / <code>_check_exit()</code>	확률 벡터 -> 매수 / 공매도 / 청산 신호

여기서 가장 중요한 점을 짚고 가자. **HMM은 매수·매도 신호를 직접 만들지 않는다.**

HMM은“지금 시장이 어떤 상태인가”를 확률로 알려줄 뿐이다. 실제 매수·매도 판단은 Z-score(가격이 평균에서 표준편차의 몇 배만큼 떨어져 있는지를 나타내는 값) 같은 별도 신호가 내린다. HMM은 그 신호를 통과시킬지 막을지 결정하는 **게이트**다.

예를 들어 평균회귀 신호가 발생해도 고변동성 레짐 확률이 높으면 HMM이 진입을 막는다. 신호 생성과 필터링을 이렇게 분리하면, 모델이 틀렸을 때 어느 부분이 문제인지 찾기 쉽다.

4.1 HMM 3요소—A, B, π

전이 행렬 A

전이 행렬은 현재 레짐이 다음 봉에서 어떤 상태로 바뀔지를 나타내는 확률 표다. 상태가 3개면 3×3 행렬이고, 각 행의 확률을 모두 더하면 1이 된다.

예를 들어 $a[0][0] = 0.92$ 라면, 평균회귀 상태였던 시장이 다음 봉에서도 평균회귀에 머물 확률이 92%다. $a[0][2] = 0.03$ 이라면 평균회귀에서 곧바로 고변동성으로 넘어갈 확률은 3%다. 시장 국면은 보통 하루 만에 크게 바뀌지 않으므로, 대각 원소가 0.9 이상으로 높게 나오는 경우가 많다.

이 행렬 덕분에 HMM은 하루 변동 하나에 과민반응하지 않고, 레짐이 어느 정도 지속된다는 시장의 특성을 반영한다.

방출 분포 B

각 레짐에서 어떤 데이터가 나올 법한지를 나타내는 확률 분포다. 쉽게 말해“이 상태라면 이런 수익률·변동성 패턴이 나올 만한가?”를 계산한다.

Gaussian HMM에서는 각 레짐마다 평균(μ)과 공분산(Σ)을 가진 정규분포(Gaussian)를 쓴다. 즉 일반 HMM의 방출 분포 B 가 Gaussian HMM에서는 상태별 μ 와 Σ 로 구체화된다. 고변동성 레짐이라면 수익률이 크게 출렁이고 변동성도 높게 나타나며, 평균회귀 레짐이라면 낮은 변동성과 작은 수익률 변화가 자연스럽다. 새 데이터가 들어올 때 HMM은 각 상태의 분포와 비교해 “지금 어떤 국면일 확률이 높은지”를 갱신한다.

`covariance_type="full"`을 사용하면 피처가 4개(수익률·변동성·모멘텀·거래량 비율)이므로 공분산 행렬이 4×4 완전 행렬이 되어 피처 간 상관관계까지 반영한다. 이 설정이 코드에서 어떻게 쓰이는지는 4.3절에서 다시 확인한다.

초기 상태 분포 π

데이터 시작 시점에 각 레짐일 확률이다. 데이터가 충분하면 결과에 큰 영향을 주지 않으며, `hmmlearn`이 학습 과정에서 자동으로 추정한다. 첫 회독에서는 “있다는 것만 알아두자” 정도면 충분하다.

4.2 Baum-Welch — 학습 직관

HMM의 파라미터(A , μ , Σ , π)를 어떻게 구할까? 문제는 레짐이 숨겨져 있어서 “어떤 날이 어느 상태에 속하는지”를 모른다는 점이다. 그래서 직접 계산할 수가 없다.

이 문제를 푸는 방법이 **EM(Expectation-Maximization)**이다. 아이디어는 단순하다.

1. 파라미터를 임의로 초기화한다.
2. 그 파라미터를 가정하고 “각 날짜가 어느 레짐에 속할 확률”을 계산한다 (E-step).
3. 이 확률을 가중치 삼아 평균과 분산을 다시 계산한다 (M-step).
4. 새 파라미터로 다시 2번으로 돌아간다.
5. 더 이상 개선이 없을 때까지 반복한다.

Baum-Welch는 이 EM 아이디어를 HMM에 맞게 구현한 알고리즘이다. 처음에는 임의에 가깝게 묶였던 그룹이, 반복할수록 데이터의 구조를 반영하는 레짐으로 정리된다. 수렴 기준은 **로그 우도**(모델이 데이터를 얼마나 잘 설명하는지를 나타내는 점수)가 더 이상 오르지 않는 것이다.

수식 차원에서 더 깊이 보고 싶다면 부록과 `hmmlearn` 공식 문서를 참고하라. 첫 회독에서는 “반복으로 점점 좋은 파라미터를 찾는다”는 직관만 잡아도 충분하다.

단, EM은 시작점에 따라 전역 최적해가 아닌 근처의 해로 수렴하는 **로컬 옵티멈(local optimum)** 문제가 있다. 같은 데이터라도 초기값이 다르면 결과가 달라질 수 있다는 뜻이다. 그래서 4.3절에서 `random_state=SEED`로 난수를 고정해 재현성을 확보한다.

또 하나, 알고리즘이 항상 좋은 레짐을 찾아주는 것은 아니다. 입력 피처가 시장 상태를 담고 있지 않으면 HMM은 노이즈를 그럴듯하게 세 그룹으로 나눌 뿐이다. **모델 이름보다 피처 설계가 더 중요한 이유**다. 수익률, 변동성, 모멘텀, 거래량 비율처럼 시장 상태를 설명할 수 있는 입력을 넣어야 한다.

4.3 _DailyHMM.fit() —실제 구현

WARNING

Look-ahead Bias 주의—이 장 전체를 관통하는 가장 큰 함정이다.

`fit(X_all)`처럼 **전체 기간** 데이터로 학습하거나 `predict_proba(X_all)`로 전체 시점의 확률을 한꺼번에 구한 뒤 과거 시점에 사용하면, 모델이 미래를 보고 과거를 맞히는'상황이 된다. 훈련/검증을 나눴다면 학습은 `fit(X_train)`으로만 하고, 실시간 환경에서는 현재 시점 확률만 사용해야 한다.

5장의 `forward_proba_causal()`은 이 문제를 본격적으로 해결하는 도구다. 이 장에서는 일단“현재 봉 이전 데이터만 본다”는 원칙만 지킨다.

`fit()`은 두 가지 일을 한다.

1. 입력 피처를 표준화(StandardScaler로 각 피처를 평균 0, 분산 1로 맞추는 처리)한 뒤 GaussianHMM을 학습한다.
2. 학습이 끝나면 분산이 작은 상태부터 차례로 MR -> Trend -> Volatile 번호를 다시 매긴다.

핵심 설정값은 상수로 분리해 둔다.

상수	값	의미
HMM_N_STATES	3	상태(레짐) 개수
HMM_N_ITER	200	Baum-Welch 최대 반복 횟수
HMM_MIN_DAYS	120	첫 학습에 필요한 최소 봉 수
HMM_REFIT_DAYS	10	재학습 주기(봉 단위)

상수	값	의미
SEED	42	난수 고정 (EM 재현성 확보)

학습 함수의 핵심부는 다음과 같다. 이 코드는 구조 발췌다. 준비 단계에서 학습 구간·피쳐 순서를 고정하고, 수렴과 상태 점유율을 확인한 모델만 추론에 넘긴다. 독립 실행은 뒤의 합성 예제로 흐름을 확인한다.

```
# app/trading/backtest/signals/hmm_daily_mr.py —핵심 발췌

def fit(self, features: np.ndarray) -> bool:
    X = features[~np.any(np.isnan(features), axis=1)]    # (1) NaN 행 제거
    if len(X) < 60:                                     # (2) 60봉 미만이면
        포기
        return False

    scaler = StandardScaler()
    Xs = scaler.fit_transform(X)                         # (3) 훈련 데이터에
    만 fit

    model = GaussianHMM(
        n_components=HMM_N_STATES,
        covariance_type="full",                          # (4) 4×4 공분산
        n_iter=HMM_N_ITER,
        random_state=SEED,
        tol=1e-4,
        min_covar=1e-3,                                  # (5) 분산 하한 —수
        치 안정성
    )
    model.fit(Xs)                                         # (6) Baum-Welch 실행

    # (7) 상태별 총분산 = trace( $\Sigma_k$ ), 오름차순 정렬 (4.4절 참조)
    total_var = [float(np.trace(model.covars_[i])) for i in
    range(HMM_N_STATES)]
    order = np.argsort(total_var)
    self._state_map = {
        int(order[0]): self.REGIME_MR,                   # 가장 낮은 분산
        int(order[1]): self.REGIME_TREND,
        int(order[2]): self.REGIME_VOLATILE,             # 가장 높은 분산
    }
```

```
self.model, self._scaler = model, scaler
return True
```

읽을 때 핵심은 두 줄이다.

- **(3) 스케일러는 훈련 데이터에만 fit_transform해야 한다.** 검증 구간이 함께 들어가면 Look-ahead Bias가 생긴다.
- **(6) Baum-Welch 실행** —여기서 EM 반복이 일어난다. 라이브러리가 알아서 해준다.

앞의 표에서 본 HMM_MIN_DAYS(120)와 (2)의 60이 다른 이유도 짚고 가자. 120은 호출 측이 “이제 첫 학습을 시작해도 되는가”를 판단하는 기준이고, 60은 NaN 행을 제거한 뒤에도 학습을 시도할 만하지 확인하는 함수 내부의 최소 안전선이다.

학습이 실패했을 때 확인할 지점은 두 곳이다. (2)에서 False가 반환되면 데이터가 부족한 것이고, (6)에서 예외가 나면 분산이 너무 작거나 데이터가 한쪽으로 치우친 경우다. 호출 측은 fit 실패 시 직전 모델을 그대로 쓴다.

재학습 주기 고정 주의: 기본값은 HMM_REFIT_DAYS=10이다. 2022년 금리 급등처럼 시장 구조가 단기간에 크게 바뀌면 10봉 주기가 너무 길 수 있다. 변동성 지수(VIX)가 30을 넘거나 레짐 전환이 감지될 때 자동으로 재학습하는 방식을 고려할 수 있다. `### covariance_type="full"`을 쓰는 이유

타입	특징	언제 쓰나
"diag"	피쳐 간 독립 가정, 빠름	피쳐 차원이 높거나 데이터가 적을 때
"full"	피쳐 간 상관관계까지 반영	피쳐가 적고 상관관계가 의미 있을 때

여기서는 피쳐가 4개라 "full"로 학습해도 파라미터 수가 30개 수준으로 크지 않다. 또한 수익률이 높을 때 변동성이 낮아지는 음의 상관관계처럼 피쳐 사이에 의미 있는 관계가 있으므로 "full"을 선택했다.

min_covar=1e-3의 역할

학습 도중 특정 레짐에 샘플이 거의 배정되지 않으면 공분산 값이 0에 가까워져 계산이 불안정해진다. min_covar=1e-3은 공분산이 이 값 아래로 내려가지 못하도록 막는 **안전장치**다.

작은 공분산 하한은 수치적으로 불안정한 상태를 허용하고, 큰 하한은 레짐 간 차이를 흐릴 수 있다. $1e-3$ 은 표준화 피처에서 비교할 시작 후보일 뿐 보편적인 최적값이 아니다. 학습 구간에서 여러 후보와 시드를 비교하고, 특히 공분산·수렴 실패·극소 상태 점유율을 확인한 뒤 별도 검증 구간으로 선택한다.

4.4 분산 기반 레짐 정렬

Baum-Welch 학습이 끝나면 상태 3개가 만들어진다. 그런데 이 상태들에는 숫자 번호만 붙어 있을 뿐, “이 번호가 어떤 시장 국면인지”는 모델이 알려주지 않는다. **학습이 끝난 뒤, 각 상태에 우리가 직접 의미를 붙여야 한다.**

기준으로 삼는 것은 **변동성**이다. 평균회귀 장(MR)은 주가가 평균 근처에서 좁게 오가는 구간이라 변동성이 낮다. 반대로 급변동 장은 변동성이 가장 크다.

수학적으로 표현하면, 각 상태의 공분산 행렬에서 대각 원소를 모두 더한 값(trace, 트레이스)이 그 상태의 총 분산이다. 이 값을 기준으로 정렬하면 된다.

분산 순위	배정 레짐
가장 낮음	MR (평균회귀)
중간	Trend (추세)
가장 높음	Volatile (급변동)

코드에서 핵심은 다음 부분이다.

```
# covars_.shape = (3, 4, 4) — 3개 상태 × 4×4 공분산

total_var = [np.trace(model.covars_[i]) for i in range(3)]
# 예: [0.42, 1.87, 3.61]

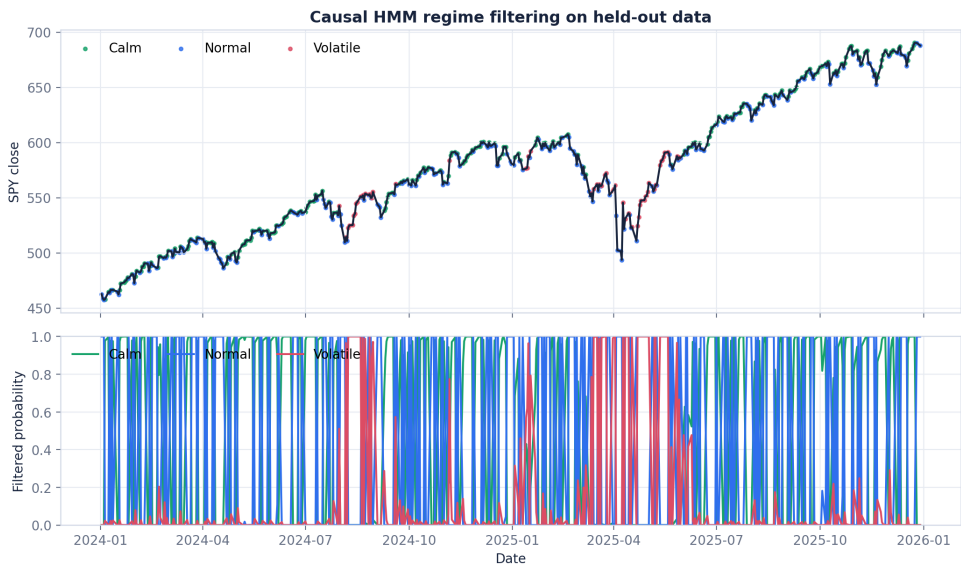
order = np.argsort(total_var)          # 분산 오름차순 인덱스
state_map = {
    int(order[0]): _DailyHMM.REGIME_MR,
    int(order[1]): _DailyHMM.REGIME_TREND,
    int(order[2]): _DailyHMM.REGIME_VOLATILE,
}
```

왜 평균 수익률이 아니라 분산으로 정렬하는가?

직관적으로는“평균 수익률이 가장 높은 상태를 추세, 가장 낮은 상태를 위험”이라고 분류하고 싶을 수 있다. 하지만 이 기준에는 큰 문제가 있다. 평균 수익률은 부호가 양수와 음수를 오가는데, **하락 추세장과 상승 추세장이 평균값으로는 정반대 방향에 자리 잡으면서도 둘 다“추세”라는 본질을 공유**한다. 분산은 부호와 무관하게“흔들림의 크기”를 잡아내므로, 추세장이든 폭락장이든 일관되게 분류할 수 있다. 게다가 매매 의사결정에서 중요한 것은“방향성”이 아니라“이 시장이 평균회귀 전략이 먹힐 만큼 조용한가”이고, 그 답은 분산이 직접 알려준다.

레짐 의미 요약:

레짐	상수값	시장 의미	신호 처리
REGIME_MR	0	좁은 레인지, 반전 기회	진입 허용
REGIME_TREND	1	추세 장	조건부 허용
REGIME_VOLATILE	2	급변동, 위험	진입 차단 + 조기 청산



Source: local cached daily OHLCV through 2025-12-29 · Educational reproduction · Not investment advice

그림 2. 2021~2023년 구간으로 학습한 3상태 HMM을 2024~2025년 SPY 일봉에 순방향 필터링한 예. 위는 상태별 색을 입힌 가격이고, 아래는 각 시점의 필터 확률이다.

그림의 아래쪽 확률은 전체 구간을 한꺼번에 평활화한 값이 아니다. 직전 확률과 현재 관측만으로 갱신한 필터 확률이다. 따라서 급변 구간에서 Volatile 확률이 높아지는 모습을 보면서도 미래 데이터가 현재 상태에 섞이지 않았는지 함께 점검할 수 있다.

4.5 _DailyHMM.predict_proba() —레짐 확률 계산

`predict_proba()`를 호출하면 각 시점마다 3개 레짐의 확률이 벡터 형태로 나온다.

예: 3봉 × 3레짐

[MR 확률, Trend 확률, Volatile 확률]

[0.82, 0.11, 0.07], <- 평균회귀 강함 -> 진입 허용

[0.31, 0.55, 0.14], <- 추세 우세 -> 진입 대기

[0.08, 0.13, 0.79]] <- 변동성 -> 신규 진입 차단

MR 확률이 `MR_THRESHOLD=0.60`을 넘으면 평균회귀 신호가 강하다고 보고 다음 단계인 Z-score 확인으로 넘어간다. 뒤의 `_check_entry()`에서는 실전 게이트의 최소 하한으로 `mr_prob ≥ 0.3`을 쓴다. 0.60은 “강한 MR 후보”, 0.30은 “이보다 낮으면 차단”하는 하한으로 역할이 다르다.

내부 동작은 세 단계다.

1. 입력 피처를 학습 시 만든 스케일러로 표준화한다.
2. `hmmlearn`으로 원본 상태별 확률 행렬을 얻는다.
3. `_state_map`을 이용해 열 순서를 [MR, Trend, Volatile] 순서로 재배치한다.

```
def predict_proba(self, features: np.ndarray) -> Optional[np.ndarray]:
    X = features[~np.any(np.isnan(features), axis=1)]
    if self.model is None or len(X) < 3:
        return None

    Xs = self._scaler.transform(X)                # 스케일러 재사용
    raw = self.model.predict_proba(Xs)             # (T, 3) —원본 상태 순서

    mapped = np.zeros_like(raw)
    for raw_s, logical in self._state_map.items():
        mapped[:, logical] = raw[:, raw_s]         # 열 재배치
    return mapped                                  # 0=MR, 1=Trend, 2=Volatile
```

확인할 출력: 반환 행렬의 각 행은 1로 합산된다. `mapped[-1]`이 가장 최근 봉의 레짐 확률이다.

실패하면 볼 곳: `self.model is None`이면 학습이 아직 안 됐거나 `fit`이 실패한 것이다. 호출 측은 `None`을 받으면 진입 신호를 만들지 않는다.

주의—4.3의 Look-ahead 경고는 여기에도 적용된다: `predict_proba(recent[-30:])` 자체는 최근 30봉까지만 입력으로 받으므로, `mapped[-1]` 만 사용하면 현재 이후의 데이터는 보지 않는다. 다만 윈도우 안의 과거 행까지 성과 계산에 쓰면 그 행들은 윈도우 끝쪽 봉 정보를 본 값이 된다. 5장에서는 이 혼동을 없애기 위해 `step_forward()`로 순방향 확률을 직접 계산한다.

HMM의 3요소, Baum-Welch 알고리즘, `fit/predict` 코드를 살펴봤다. 다음 절부터는 모델이 실전에서 어떻게 갱신되고, 매매 신호가 어떻게 생성되는지 확인한다.

4.6 `_update_hmm()` —10봉 주기 재학습

시장 구조는 시간이 지나면서 변하므로, 고정된 HMM 파라미터로는 최신 패턴을 반영하기 어렵다. 그래서 10봉(영업일 기준 약 2주)마다 모델을 다시 학습하고, 학습이 끝난 뒤에는 최근 30봉으로 현재 레짐 확률을 갱신한다.

핵심은 두 줄이다.

- 학습할 때 -> `features[:bar_idx]`로 **현재 봉 이전 데이터만** 잘라서 넘긴다.
- 예측할 때 -> `features[bar_idx-30:bar_idx]`로 최근 윈도우만 본다.

둘 다 **미래 데이터를 보지 않는다**는 점이 핵심이다.

```
def _update_hmm(self, bar_idx: int) -> None:
    for sym in self._symbols:
        hmm = self._hmm[sym]

        need_fit = (hmm.model is None
                    or bar_idx - hmm._last_fit_idx >= HMM_REFIT_DAYS)

        if need_fit and bar_idx >= HMM_MIN_DAYS:
            feat_train = self._features[sym][:bar_idx]      # 인과적 슬라이스

            if hmm.fit(feat_train):
```

```

hmm._last_fit_idx = bar_idx

if hmm.model is not None:
    recent = self._features[sym][max(0, bar_idx - 30): bar_idx]
    probs = hmm.predict_proba(recent)
    if probs is not None and len(probs) > 0:
        self._regime_probs[sym] = probs[-1]          # 최신 봉의 확
                                                       # 률 벡터

```

중요한 줄: `feat_train = self._features[sym][:bar_idx]` —3장의 인과 원칙(no look-ahead)을 그대로 따른다. `HMM_MIN_DAYS=120`은 최소 120 봉(약 6개월)이 쌓여야 첫 훈련을 시작하도록 막는 안전장치다.

4.7 _check_entry() —Z-score + HMM 게이트

`_check_entry()`는 진입 신호를 최종 승인하는 관문이다. 신호가 발생하려면 두 조건을 동시에 만족해야 한다.

- **Z-score** —가격이 평균에서 얼마나 벗어났는지 (2장에서 다룬 표준화 지표)
- **HMM 게이트** —시장이 평균회귀에 적합한 상태인지

두 조건을 AND로 연결하기 때문에 하나라도 실패하면 신호는 생성되지 않는다.

Z-score 비대칭 임계값

매수와 공매도의 임계값은 같지 않다.

방향	임계값	이유
매수	$z < -1.5$	일반 과매도 진입
공매도	$z > +2.0$	차입 비용 보상 위해 더 보수적

공매도는 주식을 빌리는 비용이 발생하므로, 그 비용을 초과하는 수익이 기대될 때만 진입한다.

HMM 게이트 이중 조건

두 조건 중 하나라도 해당되면 진입을 즉시 차단한다.

1. `mr_prob < 0.3` —평균회귀 레짐 확률이 30% 미만
2. `volatile_prob > 0.4` —변동성 레짐 확률이 40% 초과

특히 변동성 레짐 확률이 높으면 Z-score 조건과 무관하게 즉시 차단한다.

코드에서 두 게이트가 결합되는 핵심부는 다음 발췌만 봐도 충분하다.

```
# HMM 게이트 —둘 중 하나라도 만족하면 차단
if mr_prob < 0.3 or rp[_DailyHMM.REGIME_VOLATILE] > 0.4:
    return None

# Z-score 게이트 —비대칭 임계값
if z < -self._entry_z:      action = SignalAction.BUY      # -1.5
elif z > self._short_entry_z: action = SignalAction.SHORT_SELL # +2.0
else:                      return None

return TradingSignal(
    symbol=symbol,
    action=action,
    confidence=mr_prob * abs(z),
)
```

confidence = mr_prob × abs(z)

여러 종목의 신호가 동시에 나올 때 우선순위를 정하는 점수다. 시장이 평균회귀 상태일 가능성이 높고 가격 이탈이 클수록 점수가 높아진다. PortfolioSimulator는 이 점수가 높은 순서부터 포지션을 연다.

이 코드는 구조 발췌다. 준비 단계에서 학습 구간·피쳐 순서를 고정하고, 수렴과 상태 점유율을 확인한 모델만 추론에 넘긴다. 독립 실행은 뒤의 합성 예제로 흐름을 확인한다.

4.8 _check_exit() —청산 조건 6가지

청산은 보유 중인 포지션을 닫는 것이다. 진입보다 처리할 경우의 수가 많아서 로직이 더 복잡하다.

`_check_exit()`는 매 봉마다 6가지 조건을 위에서 아래로 평가하고, 하나라도 충족되면 즉시 포지션을 닫는다.

#	조건	기본값	의미
1	mr_revert	$\text{abs}(z) \leq 0.3$	평균 회귀 완료—가장 일반적인 이익 실현
2	z_stop	$\text{abs}(z) > 4.0$	Z-score가 계속 벌어짐—MR 실패
3	price_stop	-4%	절대 손실 한도
4	trailing_stop	고점 대비 -2%	수익 확보 후 추가 하락 방어
5	max_hold	15봉	시간 가치 손실 방지
6	volatile_exit	$\text{volatile_prob} > 0.5$	시장 환경 악화—전략 무효화

함수가 돌려주는 문자열이 그대로 거래 로그의 `exit_reason`이 된다. 백테스트가 끝난 뒤 이 분포를 보면 어떤 조건이 주된 손익 요인인지 알 수 있다.

중요한 포인트: **조건 순서가 곧 우선순위다**. `mr_revert`가 가장 먼저 평가되므로, 이미 회귀가 완료된 포지션에는 다른 손절 조건이 적용되지 않는다.

여섯 조건은 매 봉 같은 입력 시점으로 평가하고, 하나라도 해당하면 청산 후보를 만든다. 여러 조건이 동시에 참이면 위험 우선순위로 대표 사유를 정하되 나머지 사유도 기록한다. 이미 청산 주문이 진행 중이면 중복 주문을 만들지 않는다.

4.9 PureZscoreDailySignal —기준선

HMM 게이트를 끈 순수 Z-score 버전이 같은 파일에 들어 있다. `use_hmm=False`만 설정한 얇은 서브클래스로, HMM이 정말로 도움이 되는지 A/B 비교하기 위한 **기준선(baseline)**이다.

게이트의 기여를 확인할 때는 같은 날짜·종목·기본 진입 신호·포지션 상한·비용으로 두 실험을 만든다. A는 모든 기본 신호를 허용하고 B는 HMM 허용 상태에서에서만 진입한다. B가 거래를 줄인 효과와 예측 개선을 구별하도록 총수익·낙폭·거래수·시장 노출 시간을 함께 기록한다. OOS 결과 한 번이 좋아도 우월성을 확정하지 말고 다른 시간 구간에서 다시 비교한다.

4.10 전체 흐름—on_bar() 호출 순서

PortfolioSimulator가 봉마다 호출하는 순서는 4.0의 4단계와 같지만, **청산을 먼저 보고 그다음 진입을 본다**는 우선순위가 추가된다. 같은 봉에서 청산 직후 같은 종목에 재진입하는 일을 막기 위해서다.

흐름을 의사코드로 줄이면 다음과 같다.

```
on_bar(bar_idx):
    update_hmm(bar_idx)                # 1) 10봉 주기 재학습 + 레짐 확률 갱신
    for 보유 중인 종목:                # 2) 청산 먼저 — _check_exit()
        if 청산 조건 만족: 포지션 닫기
    for 미보유 종목:                  # 3) 그다음 신규 진입 — _check_entry()
        if 진입 조건 만족: 포지션 열기
```

이 세 줄은 전체 거래 프로그램이 아니라 게이트의 논리다. 확정된 관측으로 상태 확률을 갱신하고, 허용 상태 확률이 기준을 넘으면 기본 신호를 통과시킨다. 입력이 결측이거나 학습 모델이 준비되지 않았다면 허용하지 않는다. 상태 확률이 $[0.8, 0.2]$ 이고 첫 상태 허용 기준이 0.7이면 통과, 0.5이면 보류하는 식으로 손으로 검사할 수 있다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. covariance_type을 "full", "diag", "spherical"로 각각 바꿔가며 SPY 1년 데이터로 HMM을 학습시켜라. 어느 설정이 가장 안정적으로 레짐을 분리하는지 비교해 보라. (모델 비교 지표인 BIC를 안다면 함께 계산하라.)
2. 학습이 수렴하지 않고 멈출 때(ConvergenceMonitor.converged == False)의 원인 세 가지를 생각해 보고, 각각에 대해 어떤 대응이 가능한지 설명하라.
3. 합성 데이터로 분산 정렬 로직만 따로 검증해 보라. 저/중/고 변동성 구간을 이어 붙인 가짜 피처를 GaussianHMM에 넣고, np.argsort(np.trace(...))로 매겨진 state_map이 의도대로 0(저변동) -> 2(고변동) 순서가 되는지 확인하라. 만약 순서가 뒤섞이면 어떤 피처를 보완해야 할지 적어 보라.

이 장에서 반드시 기억할 것

- HMM은 매수·매도 신호를 직접 만들지 않는다. “**지금 시장이 어떤 상태인가?**”를 확률로 알려줄 뿐이다. 매수·매도 판단은 Z-score가 내리고, HMM은 그 신호를 통과시킬지 막는 **게이트**로 작동한다.
- HMM의 3요소—**A** (전이 행렬), **B** (방출 분포, 각 상태에서 어떤 데이터가 나올지), **π** (초기 상태 확률).
- **Baum-Welch**는 EM 반복으로 파라미터를 점점 좋게 만들어 가는 학습 알고리즘이다. 라이브러리가 알아서 해주므로, 첫 회독에서는 직관만 잡으면 된다. 로컬 옵티멈 문제 때문에 `random_state`로 재현성을 잡는다.
- 학습이 끝난 뒤에는 **분산이 낮은 순서대로 MR / Trend / Volatile**을 배정한다. 변동성이 시장 국면을 구분하는 가장 자연스러운 기준이다(평균 수익률은 부호가 뒤집혀 분류가 깨진다).
- 진입 조건은 세 가지가 동시에 만족해야 한다—`mr_prob ≥ 0.3 AND volatile_prob ≤ 0.4 AND Z-score 임계값 통과`. `MR_THRESHOLD=0.60`은 그중에서도 평균회귀 확률이 강한 후보를 구분하는 기준이다.
- 학습할 때는 **현재 봉 이전 데이터만** 사용한다 (`features[:bar_idx]`). 이 규칙을 어기면 Look-ahead Bias가 생기고, 5장에서 그 문제를 본격적으로 다룬다.

요약 표:

개념	핵심
HMM 3요소	A (전이) · B (방출) · π (초기)
Baum-Welch	파라미터 임의 초기화 → 확률 추정 → 파라미터 갱신 반복
공분산 타입	"full" —피쳐 간 상관관계 반영
안정화	<code>min_covar=1e-3</code> —학습 발산 방지
레짐 정렬	분산 오름차순 → MR(0) / Trend(1) / Volatile(2)
진입 게이트	<code>mr_prob ≥ 0.3 AND volatile_prob ≤ 0.4 AND Z-score</code>
confidence	<code>mr_prob × abs(z)</code> —우선순위 점수
재훈련	10봉 주기, 인과 슬라이싱

다음 장(5장) 예고: `predict_proba()`가 돌려준 중간 행을 과거 시점의 신호로 사용하면, 윈도우 뒤쪽 관측치가 섞여 Look-ahead Bias가 생긴다. 이 장처럼 현재까지 자른 윈도우의 마지막 행만 쓰면

미래 참조는 없지만, 매번 창 전체를 다시 계산하고 과거 행을 잘못 재사용하기 쉽다. 5장에서는 이를 더 명확하고 효율적인 **순방향(forward-only)** 추론으로 바꾼다.

더 읽을거리

- **논문**: Baum, L. E. et al. (1970), “A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains”—원조 Baum-Welch 논문
- **라이브러리**: `hmmlearn` 공식 문서 —GaussianHMM 파라미터 레퍼런스
- **책**: Bishop, Pattern Recognition and Machine Learning Chapter 13 —HMM의 수학적 근거

5장. HMM Forward-Only

—미래 정보 없는 실시간 추론

백테스트에서 좋은 결과를 낸다고 해서 실시간에서도 같은 결과가 나오지는 않는다. 처음 HMM 모델을 만들었을 때 나는 이 차이를 제대로 몰랐다. 백테스트 수치는 나쁘지 않았는데, 실시간 데이터에 붙여보니 신호가 달라졌다. 모델이 갑자기 나빠진 게 아니었다. 백테스트 안에서 이미 미래를 보고 있었던 것이다.

원인은 표준 HMM의 **평활화 결과를 시계열 중간 시점의 실시간 판단처럼 사용한 것**에 있다. `predict_proba(X_1:T)[t]` 처럼 전체 구간을 넣고 $t < T$ 인 행을 꺼내면, 그 확률에는 $t+1$ 이후 관측값이 반영될 수 있다. 연구가 끝난 뒤 과거 국면을 설명할 때는 유용하지만, 당시의 주문 판단으로 재현할 수는 없다. 반대로 시점 t 까지만 잘라 넣은 `predict_proba(X_1:t)[-1]`의 마지막 행은 t 뒤의 데이터를 받지 않는다. 이 장은 이런 슬라이스 규칙에 기대지 않고, 한 봉씩 갱신하는 Forward-only 함수를 명시적으로 구현한다.

이것이 룩어헤드 편향(look-ahead bias)이다. 차트 트레이더가 결과를 이미 알고 나서“나는 여기서 들어갔을 것이다”라고 말하는 것과 같다. 차이가 있다면 이 편향이 사람의 기억이 아니라 라이브러리 함수 안에 숨어 있다는 점이다. 코드가 멀쩡히 돌아가고 그래프도 예쁘게 나오기 때문에 더 발견하기 어렵다.

Forward-only HMM은 이 문제를 직접 해결한다. 미래 구간을 함께 보지 않고, 직전 시점의 레짐 확률과 현재 관측값만으로 다음 레짐 확률을 갱신한다. 사후 평활화보다 분류 점수가 낮게 나올 수 있지만 두 숫자는 같은 조건의 성능이 아니다. 실시간에 재현할 수 없는 높은 점수보다, 실제 주문 시점과 같은 정보 집합으로 얻은 점수가 더 가치 있다. 이 장의 핵심은 바로 그 선택이다.

이 장이 어렵게 느껴지는 이유는“학습”과“실시간 추론”이 코드에서 비슷해 보이기 때문이다. 둘을 분리해서 생각하자. 학습할 때는 과거 훈련 구간 전체를 사용해 HMM의 파라미터를 추정한다. 실제 거래 시점에는 학습된 파라미터를 고정한 채, 새 봉이 들어올 때마다 현재 레짐 확률만 한 단계 앞으로 갱신한다. 이 두 번째 과정이 Forward-only 추론이다.

기준은 단순하다. **시점 t 에서 계산하는 값은 t 이후 데이터를 사용하면 안 된다.** 코드에서 `predict_proba(full_series)`처럼 전체 구간을 한꺼번에 넣는 호출이 보이면 먼저 의심해야 한다.

머신러닝 용어로 표현하면, 이 장은 **훈련과 서빙을 분리하는 장**이다. 훈련(training)은 과거 데이터 묶음으로 모델을 맞추는 오프라인 작업이다. 서빙(serving) 또는 추론(inference)은 학습된 모델을 실제 데이터 흐름에 붙여 한 시점씩 판단하는 온라인 작업이다. 백테스트 코드가 이 둘을 섞으면 성과는 좋아 보이지만, 실제 거래 시스템에서는 같은 숫자가 재현되지 않는다.

이 문장을 더 쉽게 풀면, 학습은“판단 계산기의 눈금을 맞추는 일”이고 추론은“눈금이 맞춰진 계산기에 오늘 숫자를 넣어 답을 받는 일”이다. HMM에서는 A , μ , Σ 같은 숫자가 그 눈금이다. 학습 구간에서 이 숫자들을 정하고 나면, OOS 구간에서는 그 숫자를 고정한 채 오늘 피쳐만 넣는다.

학습: 과거 데이터로 눈금을 맞추다
추론: 맞춰진 눈금으로 오늘을 읽는다
검증: 그렇게 읽은 결과가 실제 수익으로 이어졌는지 본다

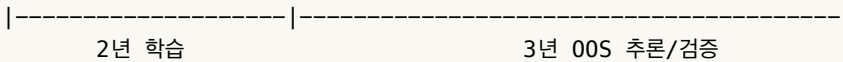
구분	학습(training)	추론(inference)
사용하는 데이터	과거 훈련 구간 전체	현재 시점까지 관측된 데이터
바뀌는 것	모델 파라미터(A , μ , Σ 등)	레짐 확률 α
실행 주기	일정 기간마다 재학습	새 봉마다 실행

구분	학습(training)	추론(inference)
대표 함수	<code>fit()</code>	<code>step_forward()</code>

결측치 처리도 이 경계를 따라야 한다. 훈련 구간에서 NaN 행을 제거했다면, 실시간 추론에서도 NaN 이 들어온 봉은 같은 방식으로 거절하거나 보수적으로 처리해야 한다. 학습 때와 추론 때 전처리 규칙 이 다르면 모델은 자신이 배운 것과 다른 입력을 보게 된다.

데이터를 5년치 가지고 있다고 가정하면 흐름은 이렇게 된다.

5년 데이터



1) 학습 구간

- 피처를 만든다.
- NaN 행을 제거한다.
- `scaler.fit(X_train)`, `hmm.fit(X_train)`을 실행한다.
- 마지막 학습 봉의 α 를 저장한다.

2) OOS 구간

- 하루치 피처가 새로 들어온 것처럼 한 줄씩 처리한다.
- `scaler`는 다시 `fit`하지 않고 `transform`만 한다.
- HMM도 다시 `fit`하지 않고 `step_forward()`로 레짐 확률만 갱신한다.
- 이 구간에서 나온 신호와 수익률만 검증 성과로 본다.

이 그림에서 확인할 것: 학습 구간에서만 `fit`하고, OOS에서는 스케일러와 HMM 파라미터를 고정한 채 한 줄씩 추론한다. 두 구간의 역할이 섞이지 않는지가 핵심이다.

초보자는 “학습 데이터는 교재, OOS 데이터는 모의고사”라고 생각하면 된다. 교재를 보고 공부하는 것은 허용된다. 모의고사 답안지를 먼저 보고 공부한 뒤 같은 모의고사로 점수를 채면 안 된다. 백테스트에서 OOS를 분리하는 이유가 바로 이것이다.

학습이 끝난 모델을 실제로 쓰려면 저장과 로드가 필요하다. HMM에서는 모델 자체뿐 아니라 스케일러와 마지막 α 도 함께 보관해야 한다. 그래야 추론을 중간에서 이어갈 수 있다.

학습 완료 후 저장할 것

- | | |
|-----------------|---|
| 1. HMM 파라미터 | A, μ, Σ, π |
| 2. 스케일러 | 평균과 표준편차 |
| 3. 상태 매핑 | raw state $\rightarrow$ MR / Trend / Volatile |
| 4. 마지막 α | 다음 봉 추론을 시작할 현재 상태 확률 |

코드 흐름은 아래처럼 생각하면 된다. 실제 저장 형식은 pickle, joblib, MLflow artifact 등으로 바뀔 수 있지만 개념은 같다.

```
# 학습 직후
hmm.fit(X_train)
hmm.forward_proba_causal(X_train)
save_model("hmm_model.pkl", hmm)

# 추론/서빙 시작 시
hmm = load_model("hmm_model.pkl")

# 새 봉마다
x_today = compute_today_features(...)
regime_proba = hmm.step_forward(x_today)
```

여기서 `load_model()` 뒤에 다시 `fit()` 을 호출하면 안 된다. 추론 시점의 목적은 모델을 다시 공부시키는 것이 아니라, 저장된 판단 계산기에 오늘 피처를 넣어 현재 레짐 확률을 받는 것이다.

Forward-only를 한 장면으로 보기

Forward-only가 추상적으로 느껴지면 상태 확률 α 가 하루씩 넘어가는 장면만 보면 된다. α 는“현재 시장이 각 레짐일 확률”이다.

학습 마지막 날
 $\alpha = [\text{MR } 0.60, \text{Trend } 0.25, \text{Volatile } 0.15]$
 이 값을 모델 파일과 함께 저장한다.

OOS 첫날
 오늘 피처 `x_today`가 들어온다.

저장된 α 와 전이행렬 A , 오늘 관측 가능도를 이용해 새 α 를 만든다.
 새 $\alpha = [\text{MR } 0.64, \text{Trend } 0.22, \text{Volatile } 0.14]$

OOS 둘째 날
 어제 계산한 α 에서 다시 시작한다.
 새 $\alpha = [\text{MR } 0.58, \text{Trend } 0.30, \text{Volatile } 0.12]$

핵심은 매일 이전 α 에서 이어 간다는 점이다. 전체 OOS 구간을 한꺼번에 넣고 “가장 그럴듯한 전체 경로”를 찾지 않는다. 실제 거래에서는 내일 데이터를 모르기 때문이다.

금지:
 OOS 전체 데이터 $\rightarrow$ `predict_proba()` $\rightarrow$ 모든 날짜 레짐 한 번에 계산

허용:
 어제 α + 오늘 피쳐 $\rightarrow$ `step_forward()` $\rightarrow$ 오늘 레짐 확률

이 방식은 조금 답답해 보일 수 있다. 하지만 실전 시스템이 실제로 할 수 있는 계산만 하므로, 백테스트와 운영 결과가 같은 논리로 이어진다. 이 책이 Forward-only를 강조하는 이유가 여기에 있다.

HMM 한 줄 복습 (4장에서 처음 본 분이라면)

HMM은 보이지 않는 **상태**(예: “추세장/평균회귀장/위기장”)가 시간에 따라 바뀐다고 가정하는 모델이다. 세 가지 요소가 있다. - **상태 전이 확률** A : 어제 상태 j 였다면 오늘 상태 k 가 될 확률 A_{jk} . - **방출 확률** $B_k(\mathbf{x})$: 상태 k 일 때 관측값 $\mathbf{x}$ (수익률, 변동성 등)가 나올 확률. - **초기 분포** π : 맨 처음 시점에 각 상태에 있을 확률.

우리가 보는 것은 가격뿐, 상태는 볼 수 없다. 그래서 **관측값으로 상태를 거꾸로 추정**한다.

표기 안내: 굵은 소문자($\alpha, \mathbf{e}_t, \mathbf{x}_t$)는 벡터, 대문자(A)는 행렬이다. 파이썬 코드의 @는 행렬곱, *는 같은 자리끼리의 원소별 곱(element-wise)이다.

이 장의 흐름

1. **전체 그림** (§5.0) —hmm_core.py의 4단계 파이프라인
2. **왜 필요한가** (§5.1) —predict_proba가 미래를 보는 이유
3. **핵심 개념과 수식** (§5.2~5.3) —Forward 재귀와 α 초기화
4. **온라인 추론** (§5.4) —새 봉마다 한 스텝씩 갱신

5. 실전 부품 (\$5.5~5.10) —속도 최적화, 7-state, 승수, 안정화, walk-forward

\$5.0~5.4가 핵심 개념·구현이고, \$5.5~5.10은 실전에서 함께 쓰는 부품이다.

학습 목표

이 장이 끝나면 다음을 할 수 있다.

- 전체 평가 구간에 대한 `predict_proba()`의 중간 행을 과거 신호로 쓰면 왜 미래 데이터가 섞이는지 한 줄로 설명할 수 있다.
- Forward 재귀식 $\alpha_t(k)$ 가 무엇을 뜻하는지, 정규화가 왜 필요한지 안다.
- `forward_proba_causal()`로 초기화하고 `step_forward()`로 한 스텝씩 갱신하는 흐름을 머릿속에 그릴 수 있다.
- 7-state 레짐 확률이 `regime_mult`(포지션 승수)로 어떻게 하나로 합쳐지는지 이해한다.

두 가지 Forward 구현

이 코드베이스에는 레짐을 추론하는 방식 두 가지가 공존한다.

방식	사용처	인과성
<code>predict_proba(recent_Bombdaily_mr.py)</code>		근사적 (최근 30봉 안에서 평활화)
<code>forward_proba_causal</code> <code>hmm_core.py</code> <code>+ step_forward()</code>		엄밀한 Online Forward α

여기서 '인과성'은 미래 데이터를 사용하지 않는다는 뜻이다. 4장의 일봉 MR 시그널은 첫 번째 방식을 쓰고, 이 장의 5분봉 전략은 두 번째 경로를 따른다.

TIP

바이브코딩 미션

- **얻을 결과:** 전체 구간 추론과 하루씩 진행하는 Forward-only 추론의 차이를 확인한다.
- **첫 회독:** 재귀식 암기보다 “시점 t 에서 t 이후 데이터를 읽지 않는다”는 조건을 본다.

- **용어 발판:** Forward-only는 현재까지의 데이터만 쓰는 추론, Look-ahead Bias는 미래 정보를 미리 본 오류, 재귀는 직전 결과로 다음 값을 계산하는 방식이다.
- **AI 요청:** “step_forward()가 미래 행을 참조하지 않는지 호출 경로로 검토해줘. 근거가 되는 파일과 함수만 인용하고 수정은 하지 마.”
- **완료 확인:** 현재 확률 계산에 사용되는 데이터의 마지막 시점을 지목할 수 있으면 성공이다.

5.0 전체 그림—hmm_core.py 파이프라인

hmm_core.py는 네 단계로 동작한다. 앞 단계의 결과가 다음 단계의 입력으로 이어진다.

Step	단계	입력 -> 출력
1	학습— <code>BWHMM.fit()</code>	훈련 피쳐 -> 파라미터 (A, μ, Σ) 학습, 분산 기반 state 정렬 (0=DeepMR ...6=Crisis)
2	초기화— <code>for—ward_proba_causal()</code> 저장 (<code>_alpha</code>)	훈련 피쳐 전체 -> Forward α 재귀 -> 마지막 붕의 α
3	온라인 추론— <code>step_forward()</code>	새 피쳐 벡터 1개 -> <code>_alpha</code> 갱신 -> 레짐 확률
4	포트폴리오 승수— <code>GlobalRegime—Layer.step()</code>	확률 $\times$ scale 가중합 -> <code>regime_mult</code> (0.1 ~ 2.0)

쉽게 풀면 이렇다.

Step 1 (학습): 과거 데이터로 모델 파라미터를 결정한다. 각 시장 상태의 평균(μ)·공분산(Σ)과 상태 사이의 전이 확률(A)을 계산한다.

Step 2 (초기화): 학습 데이터를 처음부터 끝까지 다시 한 번 훑어, 마지막 시점의 상태 확률 벡터 α 를 저장한다. 이후 실시간 추론이 이 지점부터 이어진다.

Step 3 (온라인 추론): 새 붕이 하나 들어올 때마다 α 를 한 칸씩 갱신한다. 과거 데이터를 다시 처리하지 않으므로 빠르고, 미래 데이터를 참조하지 않으므로 룩어헤드 편향이 없다.

Step 4 (포트폴리오 승수): 현재 상태 확률을 0.1~2.0 범위의 단일 숫자(regime_mult)로 변환해 포지션 크기 조절에 사용한다.

5.1 predict_proba가 미래 정보를 섞는 경우

먼저 왜 표준 `predict_proba`를 그대로 쓰면 안 되는지 짚자.

IMPORTANT

일반적인 HMM 라이브러리(hmmlearn)의 predict_proba는 Forward-Backward 알고리즘으로 각 행의 사후확률을 계산한다. 전체 배열 $X_{1:T}$ 를 넣으면 중간 행 $t < T$ 의 결과에는 $t + 1$ 이후 관측치가 반영된다. 따라서 이 중간 행들을 과거 시점의 매매 신호로 쓰면 안된다. 반면 현재까지 자른 배열의 **마지막 행**에는 그 뒤 관측치가 없으므로 미래 참조가 아니다. 이 장의 목표는 이 예외에 기대지 않고, 매 봉마다 같은 규칙이 분명하게 적용되는 온라인 Forward 재귀를 만드는 것이다.

구체적인 예를 들어 보자. 월요일부터 금요일까지의 데이터를 한꺼번에 넣고 수요일의 레짐을 판단한다고 하자. 알고리즘은 목요일과 금요일에 급락이 있었다는 사실을 이미 알고 있다. 그래서 해당 시점을 “위험 레짐”으로 분류한다. 백테스트 결과는 매우 정확해 보이지만, 실제 수요일 장중에는 그 이후 이틀치 데이터가 존재하지 않는다.

수식으로 보면 predict_proba가 계산하는 값은 이렇다.

$$\gamma_t(k) = \alpha_t(k) * \beta_t(k) / P(X)$$

여기서 α_t 는 **과거**만 보고 만든 항, β_t 는 **미래** 전체를 본 항이다. 두 항을 곱하는 순간 미래가 끼여든다. 그림으로 표현하면 시점 t 에서 사용하는 데이터 범위가 두 방식에서 이렇게 갈린다.

시간축: ... t-3 t-2 t-1 t t+1 t+2 t+3 ...

predict_proba(X_1:T)[t], t<T:

_____ [t] _____ (양쪽 다 본다)

<- 과거 -> <- 미래 ->

↑ 룩어헤드!

Forward-only: $\text{---} [t]$ (과거 + 현재만)
 $\leftarrow$ 과거 $\rightarrow$

세 가지 호출 방식을 인과성이 약한 쪽부터 순서대로 비교하면 다음과 같다.

```
# 금지 —전체 구간 predict_proba: t시점에서 미래 T까지 참조
proba = model.predict_proba(X_all)

# 제한적 사용 — 4장 hmm_daily_mr.py: 최근 30봉만 잘라 마지막 행만 사용
recent = features[max(0, bar_idx - 30): bar_idx] # bar_idx 미포함
proba_recent = hmm.predict_proba(recent)
# proba_recent[-1]만 읽는다. 창 안의 과거 행은 성과 계산에 쓰지 않는다.

# 엄밀한 Forward-only —이 장의 hmm_core.py: 직전  $\alpha$  + 현재 관측만
alpha = bw hmm.step_forward(feats_row)
```

세 방식을 차례로 보면 “어디까지의 데이터를 보고 있는가”가 분명해진다. 이 장에서는 가장 아래 방식인 `step_forward()`를 쓴다.

5.2 Forward 알고리즘 재귀식

Forward 알고리즘은 “지금 시점까지의 관측값이 나올 확률”을 시점마다 순서대로 누적한다. 그 값을 $\alpha_t(k)$ 로 표기한다. t 는 시점, k 는 레짐 번호다.

핵심 재귀식은 다음과 같다.

$$\alpha_t(k) = \left[\sum_j \alpha_{t-1}(j) A_{jk} \right] \cdot B_k(\mathbf{x}_t)$$

말로 풀면 이렇다.

- 대괄호 안: “직전 시점의 각 레짐에서 레짐 k 로 넘어올 확률의 합”
- 곱하는 항 $B_k(\mathbf{x}_t)$: “레짐 k 일 때 지금 관측값이 나올 확률”

즉 (과거 상태 확률 $\times$ 전이 확률) $\times$ (지금 관측이 그 상태에서 나올 확률), 이 한 줄이 전부다. 첫 시점에는 직전 상태가 없으므로 초기 분포 π 에서 출발한다.

한 가지 실용적 주의—정규화: 붕이 많아지면 작은 확률을 계속 곱하기 때문에 값이 0으로 사라지는 수 치 언더플로(underflow)가 생긴다. 그래서 매 단계 α 를 자기 합으로 나누어 항상 합이 1이 되도록 맞춘다. 결과적으로 α 는 “각 레짐에 속할 확률 분포”로 깔끔하게 읽을 수 있다.

손계산 예시—2-state, 3 step: 직접 숫자를 넣어 α 가 어떻게 갱신되는지 따라가 보자. 초기 분포는 $\pi = [0.5, 0.5]$ 이고, 전이 행렬은 $A = [[0.9, 0.1], [0.2, 0.8]]$ 이다. 각 시점의 방출 확률 e_t 도 함께 가정한다.

시점 t	e_t (상태0, 상태1)	전이 적용 ($\alpha \times A$)	방출 곱한 뒤	정규화 후 α_{t+1}
1 (초기)	(0.6, 0.2)	$\pi = (0.5, 0.5)$	(0.30, 0.10)	(0.75, 0.25)
2	(0.4, 0.7)	(0.725, 0.275)	(0.290, 0.193)	(0.601, 0.399)
3	(0.3, 0.8)	(0.621, 0.379)	(0.186, 0.303)	(0.381, 0.619)

두 번째 붕부터 상태1의 방출 확률이 커지기 시작하면서, 처음에 0.25였던 상태1 확률이 세 붕 만에 0.619까지 올라온다. 미래는 한 번도 보지 않았는데도 관측값이 누적되면서 분포가 자연스럽게 이동한다. 이것이 Forward 재귀의 본질이다.

5.3 _BWHMM.forward_proba_causal() —학습 후 α 초기화

이 함수는 학습 데이터를 처음부터 끝까지 순서대로 훑으며 α 를 누적하고, 마지막 붕의 α 를 `self._alpha`에 저장한다. 실시간 추론은 이 값에서 출발한다.

전체 흐름은 5.2의 수식 그대로다. 핵심만 발췌하면 다음과 같다.

```
# app/trading/backtest/signals/hmm_core.py —핵심 발췌

def forward_proba_causal(self, features: np.ndarray) -> None:
    Xs = self._scaler.transform(features)

    # 방출 확률 e_t
    log_e = self.model._compute_log_likelihood(Xs)
```

```

log_e -= log_e.max(axis=1, keepdims=True)
e      = np.exp(log_e)

#  $\alpha_1 = \pi \odot e_1$ 
alpha = self.model.startprob_ * e[0]
alpha = alpha / alpha.sum()

# Forward 재귀
for t in range(1, len(Xs)):
    alpha = (alpha @ self.model.transmat_) * e[t]
    alpha = alpha / alpha.sum()

self._alpha = alpha # 마지막 붕의  $\alpha \rightarrow$  step_forward()의 출발점

```

수식과 코드는 이름만 다를 뿐 같은 것을 가리킨다. startprob_가 π , transmat_가 A , $e[t]$ 가 $B_k(\mathbf{x}_t)$ 에 대응한다. 핵심인 $\alpha = (\alpha @ \text{self.model.transmat_}) * e[t]$ 는 행렬 곱(@)으로 전이 확률을 적용하고 원소별 곱(*)으로 방출 확률을 반영한 뒤, 다음 줄에서 합으로 나누어 정규화한다. 이렇게 저장한 _alpha가 다음 절의 step_forward() 출발점이다.

확률 갱신에서는 관측의 유한성, 공분산의 유효성, 정규화 합을 검사한다. 합이 0에 가까우면 로그 공간의 log-sum-exp로 계산하는 것이 우선이다. 그래도 실패하면 해당 신호를 무효로 표시한다. 균등 확률을 수치적 폴백으로 쓰더라도 신뢰도 회복으로 해석해 주문을 허용해서는 안 된다. 초기화 직후 상태 확률의 합이 1인지 확인한다.

5.4 _BWHMM.step_forward() —한 스텝 온라인 추론

실시간 트레이딩에서는 새 붕이 생길 때마다 이 함수를 호출한다. 붕마다 전체 데이터를 다시 계산하면 너무 느리므로, step_forward()는 **한 스텝만** 계산한다. 직전 붕에서 저장해 둔 _alpha에서 한 칸 앞으로 가는 셈이다.

호출 예시는 다음과 같다.

```

>>> hmm.step_forward(x_new)
# 실행 시간: ~0.2ms
# 반환값: array([0.76, 0.18, 0.06]) <- MR/Trend/Volatile 확률

```

함수 안에서는 세 단계가 일어난다.

1. 새 관측값을 학습 때와 동일한 평균·표준편차로 스케일링한다.
2. 각 상태에 대해 방출 확률 e 를 계산한다.
3. 5.2의 재귀식으로 α 를 한 스텝 갱신하고 정규화한다.

코드는 5.3과 거의 똑같다. 차이는“한 봉만”처리한다는 것뿐이다.

```
# 핵심 발췌 —한 스텝 갱신
def step_forward(self, feat_row: np.ndarray) -> np.ndarray | None:
    Xs = ((feat_row - self._sc_mean) / self._sc_scale).reshape(1, -1)

    diff = Xs[0] - self.model.means_
    log_e = -0.5 * (diff * diff * self.model._covars_inv).sum(axis=1) \
            + self.model._covars_lognorm
    log_e -= log_e.max()
    e = np.exp(log_e)

    alpha = (self._alpha @ self.model.transmat_) * e
    self._alpha = alpha / alpha.sum()

    return self._to_logical(self._alpha) # raw -> 논리 인덱스
```

_to_logical()의 역할: HMM 학습은 매번 같은 순서로 상태를 정렬하지 않는다. 그래서 학습 직후 분산을 기준으로“raw 인덱스 -> 논리 인덱스”매핑(0=DeepMR, ..., 6=Crisis)을 만들어 둔다. 이 함수는 그 맵에 따라 α 원소의 위치를 바꿔준다.

함수가 None을 돌려주는 경우는 (a) 학습이 안 됐을 때, (b) 입력에 NaN이 있을 때, (c) 예외가 발생했을 때다. 운영에서 None이나 NaN이 신호 계산에 그대로 흘러가면 포지션이 조용히 0이 될 수 있으므로, 연속 실패 횟수를 세어 임계값을 넘으면 알림이 가도록 만들어야 한다(자세한 운영 규칙은 별도 운영 챕터에서 다룬다).

훈련과 OOS 구간을 잇는 흐름

훈련 구간(학습용)과 OOS(Out-of-Sample, 검증) 구간에서 α 재귀가 어떻게 이어지는지 보자. 훈련이 끝난 시점의 α 를 저장해 두고, OOS 구간에서 그 값을 이어받아 계속 계산한다.

구간	호출	동작
훈련 첫 봉	forward_proba_causal()	$\alpha = \pi \times e$

구간	호출	동작
훈련 중간	동일 (반복)	$\alpha = (\alpha @ A) \times e$
훈련 마지막	동일	_alpha 저장
OOS 첫 봉	step_forward(new_feat)	$\alpha = (_alpha @ A) \times e$
OOS 이후	step_forward(new_feat)	$\alpha = (\alpha @ A) \times e$

이 구조에서는 매 시점에서 **그 시점까지의 데이터만** 사용한다. 그래서 룩어헤드 편향이 생기지 않는다.

여기부터는 **실전에서 함께 쓰는 부품이다**. 핵심 개념(5.0~5.4)을 처음 읽었다면 이 절들은 가볍게 훑고 넘어가도 좋다. 실제 시스템을 만들 때 다시 돌아와 자세히 봐도 된다.

5.5 _precompute_cov_fast() —속도 최적화

7-state × 5분봉 환경에서 step_forward()는 자주 호출된다. 그때마다 공분산 Σ 의 역수와 정규화 상수를 다시 계산하면 느리다. _precompute_cov_fast()는 fit() 직후 한 번만 호출돼, 역공분산과 로그 정규화 상수를 **미리 캐시**해 둔다. 이 코드베이스는 대각 공분산을 쓰므로 원소별 역수만 구하면 되어, hmmlearn 내부 호출보다 약 3~5배 빠르다.

fit() 뒤에는 상태별 공분산의 Cholesky 분해와 로그 행렬식을 한 번 계산해 둔다. 관측마다 가우시안 로그우도 $-0.5 * (d * \log(2\pi) + \log(\det(\Sigma)) + (x - \mu)^T \Sigma^{-1} (x - \mu))$ 를 계산할 때 이 결과를 재사용한다. 역행렬을 명시적으로 만들기보다 삼각 연립방정식을 풀면 수치적으로 유리하다. 재학습으로 Σ 가 바뀌면 캐시도 반드시 새로 만든다.

5.6 7-State Regime Structure —_REGIME_A_SCALE

hmm_daily_mr.py는 3-state를, hmm_core.py는 7-state를 쓴다. 7개 상태는 분산 기준으로 정렬돼 있다. 0번이 가장 변동성이 낮은 **DeepMR**(극저변동 평균회귀), 6번이 가장 변동성이 큰 **Crisis**(위기) 상태다. 그 사이에 추세 강도가 다른 상태들이 들어간다.

각 상태에는 포지션 크기를 조절할 **스케일(scale)** 값이 곱해진다. 직관은 두 가지뿐이다.

- **변동성이 매우 낮을 때(DeepMR):** 스프레드 수익이 작고 수수료 비중이 커지므로 레버리지를 줄인다.
- **변동성이 매우 높을 때(HighVol/Crisis):** HMM 신호 자체의 신뢰도가 낮고 손실 위험이 크므로 포지션을 크게 줄인다.

상태별 배율은 모델이 학습한 확률과 별도로 정하는 위험 정책이다. 예시로 평상시 1.0, 높은 변동성 0.5, 위기 0.1처럼 둘 수 있지만 정답표는 아니다. 확률 가중 배율은 `sum(prob[state]*scale[state])`로 계산하고 전체 포지션 상한을 마지막에 적용한다. 상태 ID는 재학습마다 바뀔 수 있으므로 의미를 맞춘 뒤 배율을 연결한다.

5.7 GlobalRegimeLayer.step() —단일 승수로 줄이기

이 메서드가 하는 일은 한 가지다. 새 봉이 들어오면 `step_forward()`로 7-state 레짐 확률 α 를 받고, 거기에 `_REGIME_A_SCALE` 값을 곱해 단일 숫자 `regime_mult`로 줄인다. 이 숫자가 9장 Kelly Sizing의 포지션 승수로 그대로 들어간다.

```
# 핵심 발췌
def step(self, bar_idx: int) -> float:
    if not self._hmm.is_fitted:
        return 1.0 # 학습 전엔 중립

    feat_row = self._feats[bar_idx]
    alpha = self._hmm.step_forward(feat_row) # (7,) 레짐 확률
    if alpha is None:
        return 1.0
    self._last_alpha = alpha

    #  $\alpha_k \times scale_k$  의 가중합
    n = len(alpha)
    mult = sum(float(alpha[i]) * _regime_mult_from_rank(i, n) for i in range(n))

    self._update_dominant_state(alpha)
    return float(np.clip(mult, 0.0, 2.0))
```

핵심은 $\text{mult} = \sum(\alpha_i \times \text{scale}_i)$ 한 줄이다. 단일 상태를 고르지 않고 모든 상태의 확률을 섞어 쓰기 때문에 레짐 전환 경계에서 스케일이 급변하지 않고, 트레이딩 비용도 줄어든다.

regime_mult는 0.1 ~ 2.0 사이의 실수다. 1.0이면 기준 포지션, 0.5면 절반으로 줄여 보유, 1.5면 1.5배로 키워 보유한다는 뜻이다.

5.8 dominant_state — 3봉 확인 안정화

HMM은 매 봉마다 확률을 새로 계산하기 때문에, 시장이 잠시 흔들리면 확률도 덩달아 흔들린다. 이 떨림을 그대로 신호로 쓰면 불필요한 매매가 늘어난다.

dominant_state는 이 떨림을 막는 장치다. 매 봉마다 가장 확률이 높은 상태를 보되, **3봉 연속으로 같은 레짐이 확인될 때만** 상태를 갱신한다. 한두 봉만 다르다고 해서 곧바로 바꾸지 않는다. 게다가 가장 높은 확률조차 일정 임계값(코드에서는 약 0.32) 아래로 떨어지면 레짐이 불분명하다고 보고 현 상태를 그대로 유지한다.

지배 상태 전환은 후보 확률이 현재 상태보다 충분히 높은지와 그 조건이 여러 봉 연속 유지되는지를 함께 본다. 예를 들어 차이가 0.15 이상인 조건이 3봉 유지될 때만 바꾸는 정책을 비교할 수 있다. 조건이 깨지면 연속 카운터를 초기화한다. 이 숫자는 설명용이며 최종 값은 과거 학습·검증 구간에서 선택하고 OOS에서 재조정하지 않는다.

5.9 Walk-Forward 사용 예시

NOTE

재훈련 주기가 너무 짧으면(예: 3일) 과적합 위험이 크고, 너무 길면(예: 120일) 시장 변화를 반영하기 어렵다. 실전에서는 **10일 주기**가 균형점이다.

지금까지 다룬 부품을 묶어 실제 walk-forward 루프를 만들어 보자. 흐름은 단순하다: 데이터 준비 -> 학습 + α 초기화 -> OOS 구간에서 봉마다 한 스텝씩.

```
from app.trading.backtest.signals.hmm_core import GlobalRegimeLayer,
REGIME_LABELS
```

```

# 5분봉 데이터 준비
layer = GlobalRegimeLayer(n_states=7)
layer.prepare(spy_df, macro_dfs={"GLD": gld_df, "TLT": tlt_df})

n_train = int(len(spy_df) * 0.6)      # 훈련 비율 60%

# [1] cold refit +  $\alpha$  초기화 (5.3 단계까지 한 번에)
layer.refit_and_init(n_train)

# [2] OOS 구간에서 봉마다 1-step Forward
results = []
for bar_idx in range(n_train, len(spy_df)):
    mult = layer.step(bar_idx)        # 내부적으로 step_forward() 호출
    results.append({
        "bar_idx": bar_idx,
        "regime_mult": mult,
        "dominant": REGIME_LABELS[layer.dominant_state],
        "trans_risk": layer.transition_risk,
    })

```

확인할 출력: regime_mult의 평균이 1.0 근처라면 정상이다. 0.3 이하로 쏠려 있다면 HighVol/Crisis 비중이 비정상적으로 크다는 신호다.

refit_and_init() 안에서는 세 가지 일이 일어난다.

1. trailing 60세션 창($60 \times 78 = 4680$ 봉)을 잘라낸다.
2. 새 _BWHMM을 만들고 fit()을 호출한다 (warm-start 대신 cold 재초기화).
3. forward_proba_causal()로 α 를 초기화한다.

관측이 NaN이거나 필요한 창 길이에 못 미치면 패턴 판단은 False가 아니라 우선“판단 불가”로 기록하는 편이 원인을 보존한다. 주문 허용 여부는 이 상태를 보수적으로 거부한다. 같은 실패를 정상적인 패턴 없음과 구별해 로그와 모니터에 남긴다.

5.10 transition_risk —레짐 불확실성 측정

마지막 부품은“지금 레짐이 바뀔 것 같은가?”를 측정하는 지표다. 현재 확률 분포 α 가 한 레짐에 집중돼 있으면 0에 가깝고, 여러 레짐에 분산돼 있으면 1에 가까운 값이 나오도록 정의한다(엔트로피 기반, 0~1 사이).

- $\text{transition_risk} \approx 0$: 확률이 한 레짐에 집중 $\rightarrow$ 안정
- $\text{transition_risk} \approx 1$: 확률이 여러 레짐에 분산 $\rightarrow$ 전환 임박

전이 위험을 이해할 간단한 정의는 다음 상태에서 현재 지배 상태를 벗어날 확률이다. 현재 지배 상태를 k , 필터 확률을 α , 전이행렬을 A 라 하면 다음 확률은 $p_{\text{next}} = \alpha A$, 위험은 $1 - p_{\text{next}}[k]$ 다. 예를 들어 다음 확률에서 k 가 0.8이면 이 정의의 위험은 0.2다. 전략별 운영 지표가 이 정의와 같다는 뜻은 아니며, 사용할 정의와 신규 진입 제한 기준을 명시해야 한다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. $\text{forward_proba_causal}(X)$ 와 $\text{predict_proba}(X)$ 두 함수를 같은 500개 관측치에 적용하고, 레짐 확률 차이를 시각화하라. 어느 시점부터 차이가 크게 벌어지는가?
2. 5.2의 손계산 예시를 확장해 5스텝까지 직접 계산하라. 4·5번째 붓의 방출 확률은 임의로 정하되, 각 단계마다 정규화 전후 값을 모두 적어 비교하라.
3. Walk-forward 훈련 윈도우를 250일에서 125일로 줄였을 때 성과가 어떻게 달라지는지 분석하라.

이 장에서 반드시 기억할 것

이 장에서 꼭 기억할 것은 다음 세 가지다.

1. 표준 **predict_proba** 는 미래를 본다. 백테스트에서는 멀쩡해 보이지만 실시간에서는 쓸 수 없다.
2. **Forward-only**는 “**직전 α + 현재 관측**”만 쓴다. 그래서 실시간 환경과 같은 조건으로 추론한다.
3. **학습(fit) $\rightarrow \alpha$ 초기화(forward_proba_causal) $\rightarrow$ 한 스텝 갱신(step_forward)** 세 단계가 전체 흐름의 뼈대다.

장 첫머리의 사례로 돌아가 보자. 그 백테스트가 실시간에서 무너졌던 이유는 모델이 나빠서가 아니라, 수요일의 레짐을 판단하면서 목·금요일 데이터를 이미 보고 있었기 때문이다. **predict_proba**

를 `step_forward`로 바꿔 같은 데이터로 다시 돌리면 백테스트 수치 자체는 줄어들지만, **실시간으로 옮겼을 때 같은 값이 나온다**. 이 일관성이 Forward-only가 주는 진짜 가치다.

구현 체크리스트는 다음과 같다.

단계	메서드	목적
학습	<code>_BWHMM.fit(train_feats)</code>	파라미터 추정과 state 정렬
속도	<code>_precompute_cov_fast(model)</code>	역공분산 사전 계산
초기화	<code>forward_proba_causal(train_feats)</code>	9를 훈련 종료 시점 상태로 설정
추론	<code>step_forward(feats_row)</code>	1봉씩 α 갱신, 레짐 확률 반환
합산	<code>GlobalRegimeLayer.step(bar_idx)</code>	$\alpha \times \text{scale}$ 가중합 $\rightarrow$ <code>regime_mult</code>
안정화	3-bar confirm	<code>dominant_state</code> 급변 방지
불확실성	<code>transition_risk</code>	엔트로피 기반 전환 위험도

핵심 수식 한 줄: 새 봉이 들어올 때마다 α 를 다음과 같이 한 단계 앞으로 갱신한다.

```
raw_alpha = (previous_alpha * A) * e_t
new_alpha = raw_alpha / sum(raw_alpha)
```

그리고 이 확률들을 가중합해 단일 승수를 얻는다.

```
regime_mult = sum(alpha_k * scale_k for each regime k)
```

다음 장(6장): 가격 데이터에는 크고 작은 노이즈가 섞여 있다. 6장은 이 노이즈를 실시간으로 걸러내는 1D 칼만 필터(Kalman Filter)를 다룬다.

더 읽을거리

- **논문:** Rabiner, L. R. (1989), “A tutorial on Hidden Markov Models”—Forward-Backward 알고리즘의 원전

- **논문:** Olivier Cappe (2011), “Online EM Algorithm for Hidden Markov Models”—온라인 HMM 학습의 이론적 토대
- **소스 코드:** hmmlearn의 Forward-Backward 구현—버전마다 비공개 함수명이 달라질 수 있으므로, 이 책의 `forward_proba_causal()`처럼 공개 파라미터로 재귀를 구현하는 편이 안전하다.

6장. 칼만 필터—노이즈 제거와 혁신 Z-스코어

Part III —노이즈와 다중 레짐 (6~8장) 이 파트는 시스템의 핵심이다. 6장은 가격과 피처의 잡음을 줄이고, 7장은 여러 시장을 함께 보며, 8장은 거시 레짐과 개별 자산 신호를 두 층으로 나눈다. 첫 회독에서는 각 모델의 수식보다 “왜 한 종목만 보지 않는가”를 이해하는 데 집중한다.

2장 Kalman vs 6장 Kalman —역할 구분

	2장 ScalarKalmanFilter1D	6장 KalmanFilter4D
입력	단일 피처 (수익률 1개)	4개 피처 (수익률·변동성·방향성·거래량)
주 역할	피처 노이즈 제거	피처 노이즈 제거 + Innovation Z-score 생성

출력

평활화된 피쳐 (HMM 직접 입력) 평활화된 피쳐 + novelty_gate용
Z-score

2장의 Kalman이“기초 청소 도구”라면 6장의 Kalman은“청소 + 이상 탐지 기능 추가”다.

RSI, MACD, 볼린저 밴드 같은 기술적 지표는 관측된 가격을 그대로 계산 재료로 쓴다. 이 지표들이 종가를“진짜 시장 가치”라고 가정하는 것은 아니지만, 실제 종가에 섞인 매수-매도 스프레드의 틈, 장 마감 직전 주문 불균형, 시장 미시구조 잡음도 계산에 함께 들어간다. 따라서 짧은 구간에서는 일시적 흔들림이 신호처럼 보일 수 있다.

특히 레짐(시장 상태, 예: 상승장->하락장)이 바뀌는 시점에서 단순 이동평균은 곤란해진다. 반응을 빠르게 하면 노이즈에 흔들리고, 느리게 하면 전환을 늦게 포착한다.

칼만 필터는 과거 가격을 단순히 평균하지 않는다. 가격을“노이즈가 섞인 관측값”으로 보고, 새 관측이 들어올 때마다 모델이 가정한 잠재 상태의 추정치를 갱신한다. 금융에서 이 잠재 상태가 객관적인 공정 가치라고 보장되지는 않는다. 선택한 상태 방정식과 Q/R 설정 아래 만든 평활 추정값이다.

핵심 정의는 이렇다. **칼만 필터는 선형·가우시안 모델과 올바른 노이즈 가정 아래, 노이즈 있는 관측값에서 숨겨진 상태를 순차 추정하는 도구다.** 여기서“최적”이라는 표현은 이 가정이 맞을 때의 최소 평균 제곱오차 의미이지, 어떤 금융 데이터에서도 최선이라는 뜻은 아니다.

칼만 필터를“고급 이동평균”으로만 보면 핵심을 놓친다. 이동평균은 단순히 최근 N개를 평균한다. 칼만 필터는 두 가지를 비교한다. “내 예측이 얼마나 불확실한가”와“새 관측값이 얼마나 믿을 만한가”다. 새 관측이 신뢰할 만하면 빠르게 따라가고, 노이즈가 크다고 판단하면 천천히 움직인다.

상황	이동평균	칼만 필터
갑작스러운 가격 점프	윈도우 길이만큼 늦게 반응	설정에 따라 빠르게 반응 가능
일시적 노이즈	평균에 섞여 지표를 흔들	노이즈 크면 일부만 반영
불확실성 표현	없음	추정 분산(P)로 추적

이 장에서는 칼만 필터를 두 곳에 적용한다. HMM 입력 피쳐를 다듬는 곳과, 평균회귀 신호용 Z-score를 만드는 곳이다.

이 책 전체의 머신러닝 흐름에서 칼만 필터는 **모델이 읽기 전에 숫자를 덜 흔들리게 만드는 정리 도구**다. HMM이나 RL은 입력 숫자를 그대로 믿고 판단한다. 입력이 노이즈로 흔들리면 모델은 시장 상태가 바뀐 것으로 착각할 수 있다.

```

원시 가격 / 원시 피쳐
노이즈, 틸, 일시적 이상치가 섞임
    |
    v
Kalman Filter
모델 가정에 따라 일시적 흔들림을 덜 반영해 평활화
    |
    v
HMM / Z-score / RL 입력
모델이 덜 흔들리는 숫자를 읽음 (효과는 OOS에서 확인)

```

초보자는 칼만 필터를 “가격을 예측하는 모델”로 이해하면 헛갈린다. 이 장에서의 칼만 필터는 내일 가격을 맞히는 장치가 아니라, **오늘 관측값을 얼마나 믿을지 결정하는 장치**다. 새 가격이 튀었을 때 “진짜 변화인가, 순간 노이즈인가”를 숫자로 판단한다.

새 관측값이 들어왔을 때 칼만 필터의 질문:

1. 내가 방금 전까지 믿고 있던 값은 얼마나 확실한가?
2. 새 관측값은 얼마나 믿을 만한가?
3. 그럼 새 값을 얼마나 반영할까?

이렇게 정리된 숫자는 HMM 레짐과 Z-score의 불필요한 흔들림을 줄일 수 있다. 반대로 Q/R을 잘못 잡으면 실제 전환을 늦게 따라가므로, 원자료 기준선보다 나은지는 OOS에서 확인한다.

이 장의 흐름

- 전체 그림—칼만 필터의 두 가지 역할
- 로컬 레벨 모델—직관적인 수학
- KalmanFilter1D —HMM 피쳐 평활화
- 4차원 피쳐별 칼만 파라미터
- `_kalman_innovations()` —가격에서 Innovation 뽑기
- 인과적 Z-score와 반감기
- 평균회귀 시그널과 백테스트

학습 목표

이 장이 끝나면 다음을 할 수 있다:

- 칼만 필터의 예측-업데이트 사이클을 큰 그림으로 설명할 수 있다
- KalmanFilter1D로 HMM 피쳐 4개를 처리하는 흐름을 안다
- Innovation(예측과 실제의 차이)이 무엇이고 왜 평균회귀 신호로 쓰는지 안다
- Innovation Z-score 기반 시그널(kalman_self_mr)의 진입/청산 로직을 이해한다
- 반감기(halflife)가 보유 기간을 결정하는 역할을 한다는 것을 안다

TIP

바이브코딩 미션

- **얻을 결과:** 원래 값과 칼만 평활값 다섯 개를 나란히 출력한다.
- **첫 회독:** 행렬식보다 Q가 크면 빠르게 반응, R이 크면 관측을 덜 신뢰한다는 감각을 잡는다.
- **용어 발판:** 평활화는 흔들림을 줄이는 처리, Q는 상태 변화의 불확실성, R은 관측값 노이즈의 불확실성이다.
- **AI 요청:** “기존 Kalman 클래스를 사용해 임의의 잡음 데이터 20개를 평활화하는 최소 예제를 찾아 실행해줘. 새 구현을 만들지 말고 원본 API를 사용해줘.”
- **완료 확인:** 출력이 덜 흔들리면서도 방향을 완전히 잃지 않으면 성공이다.

6.0 전체 그림—칼만 필터의 두 가지 역할

이 코드베이스에서 칼만 필터는 두 가지 용도로 쓴다. 알고리즘은 같지만 목적이 다르다.

첫 번째는 **노이즈 제거**다. 원시 피쳐 데이터에는 잡음이 섞여 있는데, 이를 그대로 쓰면 HMM이 잘못된 레짐을 판단할 수 있다. 칼만 필터로 잡음을 걸러내면 HMM이 더 안정적인 입력을 받는다.

두 번째는 **평균회귀 신호 생성**이다. 가격이 칼만 필터의 예측값에서 얼마나 벗어났는지 측정해 매수·매도 신호로 쓴다.

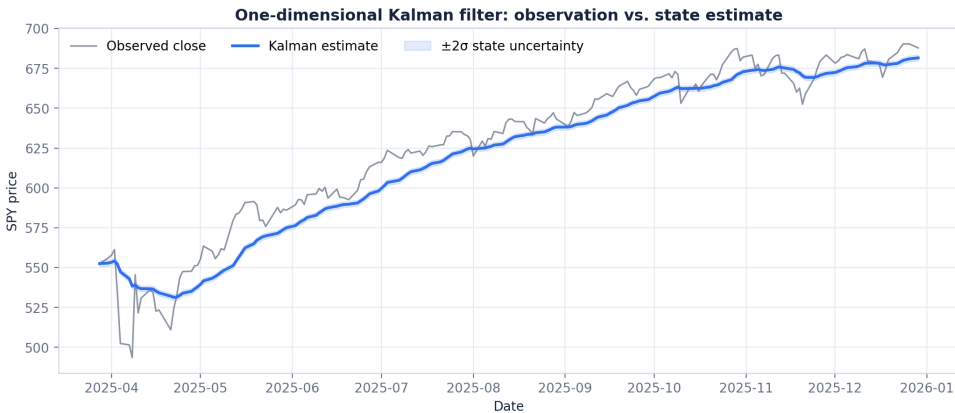
역할 1: HMM 입력 피쳐 평활화 (features.py)

입력은 raw 피쳐 4개(수익률, 5일 변동성, 20일 모멘텀, 거래량비율). 각 차원에 KalmanFilter1D(Q, R)을 독립적으로 적용해, 평활화된 피쳐를 HMM에 넘긴다.

역할 2: 가격 Innovation으로 Z-score 구성 (kalman_self_mr.py)

여기서 “Innovation”은 이 책의 부호 규칙으로 “**실제 관측값 - 갱신 전 예측값**”이다. 순서를 고정해야 양수·음수 신호를 거꾸로 해석하지 않는다. 이 차이가 크면 가격이 예측에서 많이 벗어난 것이고, 이를 Z-score로 정규화하면 진입·청산을 검토할 후보 신호가 된다.

입력은 log(price) 시계열. `_kalman_innovations()`로 시점별 예측 오차(Innovation)를 뽑고, `_build_z_series(window=60)`으로 그 시계열의 롤링 Z-score를 만든다. 이 Z-score가 진입·청산 신호의 핵심이다.



Source: local cached daily OHLCV through 2025-12-29 · Educational reproduction · Not investment advice

그림 3. SPY 종가 관측값과 1차원 칼만 필터의 상태 추정값 비교. 회색 선의 단기 잡음은 줄이면서 가격 변화의 큰 흐름은 따라간다. 음영은 상태 추정 불확실성의 $\pm 2\sigma$ 범위다.

원시 관측값과 필터 결과를 같은 축에 놓으면 Q와 R의 역할이 분명해진다. 필터 선이 지나치게 늦게 따라오면 관측을 덜 믿고 있는 것이고, 원시 가격처럼 요동치면 관측 잡음을 너무 작게 잡은 것이다.

6.1 로컬 레벨 모델—직관적인 수학

식을 보기 전에 두 핵심 파라미터의 의미를 먼저 짚자. 이 두 값만 잡으면 칼만 필터는 거의 다 이해한 셈이다.

NOTE

- **Q(프로세스 노이즈):** 숨겨진 상태가 실제로 얼마나 변하는지. 값이 크면 최신 관측값을 더 신뢰한다(반응이 빠르다).

- **R(관측 노이즈)**: 측정 오차의 크기. 값이 크면 이전 추정치를 더 신뢰한다(반응이 느리다).

$Q=0.01$, $R=1.0$ 은 개념 확인용 출발점일 뿐이다. Q 와 R 은 분산 단위라 입력 피처의 스케일에 의존한다. 가격, 수익률, 표준화 피처에 같은 숫자를 그대로 옮기지 않는다. 먼저 원자료와 필터 출력의 지연·평활 정도, 이노베이션의 안정성을 보고 범위를 정한 뒤 전략 성과는 별도 OOS에서 확인한다. 한 번의 백테스트 점수를 직접 최대화해 Q/R 을 고르면 과적합되기 쉽다.

KalmanFilter1D는 **로컬 레벨(Local Level) 모델**이다. 핵심 가정은 하나다. “숨겨진 실제 상태는 이전 값에서 조금씩만 움직인다.” 가격이나 수익률이 한 순간에 전혀 다른 값으로 튀지 않는다고 본다. 식으로 쓰면 두 줄이다.

- **상태 방정식**: 숨겨진 진짜 값 x_t 는 이전 값에 작은 랜덤 노이즈만 더해진다.

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q)$$

- **관측 방정식**: 우리가 보는 값 y_t 는 진짜 값에 측정 오차가 더해진 것.

$$y_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0, R)$$

필터는 매 시점마다 두 단계를 반복한다.

1단계: 예측. 시간이 흐른 만큼 추정의 불확실성 P 가 Q 만큼 커진다.

2단계: 업데이트. 새 관측값이 들어오면 **칼만 이득 K** 를 계산해 예측값을 관측값 쪽으로 당겨 갱신한다.

$$K_t = P_{\text{pred}} / (P_{\text{pred}} + R)$$

$$x_{\text{updated}} = x_{\text{pred}} + K_t * (y_t - x_{\text{pred}})$$

괄호 안의 $(y_t - \hat{x}_{t|t-1})$ 이 **Innovation(혁신)**이다. “예측이 얼마나 틀렸나”를 나타내고, 이 오차를 얼마나 반영할지를 K_t 가 결정한다.

K의 의미를 직관으로 정리하면: - $K \rightarrow 0$: 관측 노이즈 R이 크다 $\rightarrow$ 새 관측값을 거의 무시하고 이전 추정치를 유지(부드러운 추정). - $K \rightarrow 1$: 예측 불확실성 P_{pred} 가 관측 노이즈 R보다 크다 $\rightarrow$ 새 관측값을 많이 반영(빠른 반응). Q가 크면 시간이 지나며 P_{pred} 를 키우는 한 원인이 된다.

실전에서는 Q와 R을 따로 외우기보다 비율 **Q/R**로 이해하면 편하다. Q/R이 크면 “시장이 실제로 변하고 있다”고 빨리 인정하고, 작으면 “지금 움직임은 노이즈일 수 있다”고 보고 천천히 반응한다. 너무 빠르면 노이즈를 신호로 착각하고, 너무 느리면 레짐 전환을 놓친다.

온도계를 떠올리면 쉽다. 방 온도가 22도라고 믿고 있었는데 새 온도계가 30도를 가리킨다. 온도계가 자주 고장 나는 물건이라면 30도를 그대로 믿지 않는다. 반대로 창문이 열려 방 온도가 실제로 빨리 변하는 상황이라면 새 측정값을 더 믿는다. 칼만 필터의 Q/R도 이 판단을 숫자로 바꾼 것이다. Q/R이 크면 “방 온도가 정말 바뀌었을 수 있다”, Q/R이 작으면 “온도계가 튼 것일 수 있다”에 가깝다.

작은 숫자로 보면 더 분명하다. 예측값이 100이고 새 관측값이 110일 때, K가 0.2라면 새 추정값은 $100 + 0.2 \times 10 = 102$ 다. K가 0.8이라면 $100 + 0.8 \times 10 = 108$ 이다. 같은 관측값 110을 봐도 K가 작으면 조금만 움직이고, K가 크면 빠르게 따라간다.

정상상태는 관측이 비슷한 품질로 반복될 때 P와 K가 거의 변하지 않는 상태다. 같은 Q·R로 위 식을 반복해 $\text{abs}(P_{\text{new}} - P_{\text{old}})$ 가 충분히 작아지는지 확인할 수 있다. 이는 데이터가 더 이상 변하지 않는다는 뜻이 아니라 새 관측을 반영하는 비율이 안정된다는 뜻이다.

6.2 KalmanFilter1D —HMM 피쳐 평활화

KalmanFilter1D는 값이 하나인 시계열을 다루는 칼만 필터다. 붕이 닫힐 때마다 한 번씩 `update(z)`를 호출하면 평활화된 추정치를 돌려준다.

다음 코드는 6.1절에서 본 예측-업데이트 두 단계를 그대로 옮긴 것이다.

```
# app/trading/backtest/features.py

class KalmanFilter1D:
    """단변량 1-D 칼만 필터 (피쳐 평활화용)."""

    def __init__(self, Q: float = 1e-4, R: float = 1e-2):
        self.Q, self.R = Q, R
        self.x = 0.0 # 상태 추정치
        self.P = 1.0 # 추정치 분산
```

```
def update(self, z: float) -> float:
    self.P += self.Q          # 1단계: 예측 (분산 증가)
    K = self.P / (self.P + self.R) # 칼만 이득
    self.x += K * (z - self.x)    # 2단계: 상태 갱신
    self.P *= 1 - K              # 분산 축소
    return self.x
```

확인할 출력: `update(z)`를 반복 호출하면서 관측값과 추정값을 나란히 본다. 초기 `P`가 크면 첫 관측을 빠르게 따라갈 수 있고, 이후 `P`는 `Q`와 `R`이 정한 정상상태 값 근처로 간다. “갈수록 무조건 더 부드러워진다”는 뜻은 아니다.

점검 팁: 양수인 유한 `Q·R`을 쓰는 이 로컬 레벨 구현에서 `P`가 계속 무한히 커진다면 단순 튜닝 문제가 아니라 `NaN·음수 분산·업데이트 누락` 같은 구현 오류부터 확인한다. `R`을 키우면 오히려 정상상태 `P`가 커질 수 있으므로 “`P`가 크다”는 이유만으로 `R`을 올리지 않는다.

6.3 4차원 피처별 칼만 파라미터

피처마다 변화 속도가 다르다. 수익률처럼 급변하는 피처가 있고, 모멘텀처럼 천천히 변하는 피처도 있다. 그래서 **각 피처 차원에 (Q, R) 쌍을 따로 설정한다**.

차원	피처	Q	R	반응 속도	이유
0	log_return	1e-4	5e-3	빠름	수익률은 급변—지연 없이 반영
1	5d_vol	5e-5	1e-3	중간	변동성은 군집 효과가 있음
2	20d_momentum	5e-5	1e-3	중간	모멘텀은 서서히 전환
3	log_vol_ratio	1e-4	5e-3	빠름	거래량 스파이크 즉시 반영

코드에서는 이 표를 프로젝트의 초기값으로 둔다(`KALMAN_PARAMS_4D`). 피처 계산 함수(`compute_daily_features`)는 위 순서대로 네 차원을 만든다(로그 수익률, 5일 실현변동성, 20일 모멘텀, 거래량 비율). 데이터 단위나 정규화가 바뀌면 `Q`와 `R`도 다시 보정해야 한다.

그런 다음 각 차원에 **독립된 `KalmanFilter1D`**를 하나씩 두고 시간 순서대로 `update()`를 돌린다. 여기서 핵심은 **차원마다 새 필터 인스턴스를 만든다**는 점이다. 같은 필터를 모든 차원에 돌려쓰면 상태(`x`, `P`)가 섞여 의미가 사라진다.

NaN 관측은 갱신을 건너뛰고, 첫 유효 관측 전에는 결과를 만들지 않는다. 혁신 Z-score는 $\text{innovation}/\sqrt{P_{\text{pred}}+R}$ 로 계산한다. 분모가 0·음수·비유한 값이면 입력을 거부한다. 워밍업 동안에는 잔차 분포가 초기 P에 민감하므로 거래 판단에서 제외하고, 그 구간 길이는 실험 조건에 기록한다.

6.4 _kalman_innovations() —가격에서 Innovation 뽑기

이제 두 번째 역할로 넘어간다. 같은 칼만 필터를 **로그 가격 시계열**에 적용하고, 매 시점의 예측 오차 (Innovation)를 시계열로 뽑아낸다.

이 함수가 맡는 일: 매 시점의 로그 가격에 대해 한 단계 예측을 만들고, Innovation인 “**실제 관측 - 예측**”차이를 기록한다. 이 책의 부호 정의에서는 값이 양수면 가격이 칼만 예측 레벨보다 위, 음수면 아래다. 이를 곧바로 과매수·과매도로 확정하지 않고 Z-score와 회귀 검사를 더 거친다.

```
# app/trading/backtest/signals/kalman_self_mr.py

def _kalman_innovations(log_prices, delta=1e-4, obs_noise=1.0):
    T = len(log_prices)
    innovations = np.full(T, np.nan)

    first = np.argmax(~np.isnan(log_prices))
    x, P = log_prices[first], 1.0

    for t in range(first + 1, T):
        y = log_prices[t]
        if np.isnan(y):
            continue
        # 예측 (F=1, 드리프트 없음 ->  $\hat{x}_{t|t-1} = x_{t-1}$ )
        innov = y - x
        innovations[t] = innov
        # 업데이트
        P_pred = P + delta
        K = P_pred / (P_pred + obs_noise)
        x = x + K * innov
        P = (1.0 - K) * P_pred
```

```
return innovations
```

중요한 줄: $innov = y - x$. 갱신 전의 x 를 사용한다는 게 핵심이다. 즉 t 시점 이노베이션은 “ $t-1$ 까지의 정보만으로 만든 예측”과 실제 t 시점 관측값의 차이이다. 이렇게 해야 미래 정보가 섞이지 않는다(이른바 look-ahead 누수가 없다).

Innovation과 평균회귀 가설: 필터와 잡음 가정이 적절하면 Innovation은 0 주변의 예측 오차로 기대한다. 그러나 크게 벗어난 값이 매번 되돌아온다는 보장은 없다. 이 전략은 큰 Innovation이 일시적 이탈이라는 가설을 세우고, Z-score·반감기·OOS 검증으로 그 가설이 해당 자산과 기간에서 유지되는지 확인한다.

6.5 _build_z_series() —인과적 Z-score

2장 단순 rolling Z-score와 비교: rolling mean/std는 윈도우 크기가 고정이라 변동성 변화에 느리게 적응한다. 칼만은 잡음 모델에 맞춰 적응 속도를 자동으로 조정한다.

Z-score를 만들 때도 미래 데이터가 들어가면 안 된다. 그래서 롤링 평균·표준편차를 한 칸 뒤로 미뤄(`shift(1)`) **현재 봉 자신을 통계에서 제외**한다.

```
def _build_z_series(spreads, window=60):
    s = pd.Series(spreads)
    roll_mu = s.rolling(window).mean().shift(1)           # <- shift(1)이
    핵심
    roll_sd = s.rolling(window).std(ddof=0).shift(1)
    z = ((s - roll_mu) / roll_sd).values
    z[~np.isfinite(z)] = np.nan
    return z
```

shift(1)의 의미: 시점 t 의 Z-score 계산에 쓰는 평균·표준편차는 $t-1$ 까지의 데이터로 만든다. 현재 봉의 이노베이션은 통계에서 빠지므로 인과적(look-ahead-free) Z-score가 된다.

6.6 _calc_halflife() —AR(1) 반감기 추정

평균회귀 전략에서 한 가지 더 알아둬야 하는 개념이 **반감기(half-life)**다. 스프레드와 평균 사이의 거리가 절반으로 줄어드는 데 몇 봉이 걸리는지를 뜻한다. 반감기가 짧을수록 회귀가 빠르고, 길수록 평균회귀가 약하다.

추정 방법은 간단하다. 스프레드의 변화량을 “평균에서 얼마나 벗어났는가”로 회귀해서 회귀 기울기 ϕ 를 얻은 뒤, 반감기는 다음 식으로 구한다.

```
half_life ≈ -ln(2) / phi
단, phi < 0일 때 평균회귀로 해석한다.
```

반감기는 $\Delta s_t = a + b \cdot s_{(t-1)} + \varepsilon_t$ 를 과거 창에서 회귀해 추정한다. $\phi=1+b$ 가 0과 1 사이일 때 이산 AR(1)의 반감기는 $\log(0.5)/\log(\phi)$ 봉이다. b 가 0에 가까운 음수일 때 $-\log(2)/b$ 를 근사로 쓸 수 있다. ϕ 가 1 이상이면 이 모델에서 평균회귀 반감기가 없고, ϕ 가 음수면 진동하는 과정이므로 단조 감쇠 공식으로 판단하지 않는다.

핵심만 알면 되는 점: $-\phi \geq 0$ 이면 이 회귀에서 평균회귀 근거가 없다. 진입 필터가 60봉 미만만 허용할 때 80봉 같은 센티널을 쓰면 차단되지만, 설정이 바뀌면 잘못 통과할 수 있다. 더 견고한 구현은 `inf`를 반환하고 `np.isfinite()`로 명시적으로 탈락시킨다. `-hl > hl_max (=60) -> 회귀가 너무 느림 -> 진입 차단` - `eff_hold = min(max_hold, int(hl_hold_mult * hl)) -> 반감기 기반 동적 보유 기간`

6.7 KalmanSelfMRSIGNAL —진입과 청산 로직

앞서 만든 이노베이션, Z-score, 반감기를 하나로 묶은 시그널 클래스다. 초기화 단계에서 각 종목의 이노베이션과 Z-score 시계열을 미리 계산해 둔다.

```
@register_signal("kalman_self_mr")
class KalmanSelfMRSIGNAL(SignalGenerator):
    def initialize(self, data):
        for sym in self._symbols:
            log_p = np.log(np.maximum(self._close[sym], 1e-10))
            innov = _kalman_innovations(log_p, self._kalman_delta,
            self._kalman_obs)
```

```
self._spreads[sym] = innov
self._z_series[sym] = _build_z_series(innov, self._z_window)
```

핵심 파라미터 기본값 정리:

파라미터	기본값	역할
z_entry	3.0	진입 Z-score 임계값
z_exit	0.3	청산 Z-score 임계값
hard_stop_z	4.5	Z-score 극단 손절
hl_max	60.0봉	최대 허용 반감기
price_stop	-8.0%	가격 기반 손절
max_hold	30봉	최대 보유 기간
kalman_delta	1e-4	Q (상태 노이즈)
kalman_obs	1.0	R (관측 노이즈)

진입 4중 필터

진입은 네 조건을 **동시에** 만족해야 한다. 하나라도 빠지면 포지션을 열지 않는다. 거름망이 많을수록 잘못된 진입이 줄어든다.

순서	조건	의미
1	$\text{abs}(z) \geq z_{\text{entry}} \text{ (3.0)}$	충분히 벗어났는가
2	$\text{abs}(z_{\text{now}}) < \text{abs}(z_{\text{prev}})$	더 벌어지지 않고 수렴 시작인가
3	$\text{halflife} \leq \text{hl_max} \text{ (60)}$	평균회귀가 충분히 빠른가
4	방향 결정	$z < 0 \rightarrow \text{LONG}, z > 0 \rightarrow \text{SHORT}$

스코어는 $\text{abs}(z) \times (1/\text{halflife})$ 로 두어, “많이 벗어났으면서 빨리 돌아올”종목을 우선한다.

청산 6중 체크

열린 포지션은 다음 여섯 가지 중 하나만 걸려도 닫는다.

조건	임계값	의미
price_stop	$PnL < -8\%$	절대 손실 한도
z_hard_stop	$abs(z) > 4.5$	Z-score 극단 이탈
hl_degrade	halflife > 100봉	평균회귀 특성 소멸
z_converge	$abs(z) < 0.3$	목표 달성
z_cross	z가 반대 부호로 교차	오버슈팅 후 발산
max_hold	$days \geq hl_hold_mult \times hl$	동적 보유 기간 초과

진입 조건은 빠빠하게, 청산 조건은 다양하게 두는 것이 평균회귀 전략의 일반적인 형태다. 손실은 빨리 자르고 이익은 목표치에 닿으면 챙기는 구조다.

6.8 kalman_self_mr vs hmm_daily_mr 비교

TIP

같은 기간에 단순 EWMA Z-score와 칼만 Z-score를 함께 돌려보면 차이가 뚜렷하다. 2020년 3월 코로나 충격이나 2022년 금리 급등처럼 시장이 갑자기 바뀌는 구간에서 칼만이 더 빨리 반응한다. EWMA는 감쇠 계수를 고정해 쓰지만, 칼만은 신호와 잡음의 비율을 매 시점마다 다시 계산해 현재 상황에 맞게 조정하기 때문이다.

두 전략 모두 일봉 Z-score로 평균회귀를 포착하지만, 그 방식은 다르다.

항목	hmm_daily_mr	kalman_self_mr
Z-score 기반	가격 이동평균 (20봉)	칼만 Innovation (로그 가격)
레짐 필터	HMM 3-state	없음 (halflife 필터)

항목	hmm_daily_mr	kalman_self_mr
Z 진입 임계	1.5 / 2.0	3.0 (더 보수적)
Z 청산 임계	0.3	0.3
보유 기간	고정 15봉	동적 (half-life 기반)
최대 포지션	5개	60개 (다종목 동시)
할당 비율	15%	3%

kalman_self_mr은 종목 하나에 거는 비중을 3%로 낮추는 대신, 최대 60개 종목을 동시에 보유한다. 한 종목에 집중하지 않고 넓게 분산하는 전략이다.

6.9 백테스트 실행 예시

앞서 다룬 컴포넌트를 모두 엮어 실제 백테스트를 실행하는 모습이다.

```
from app.trading.backtest.signals.kalman_self_mr import
KalmanSelfMRSIGNAL
from app.trading.backtest.portfolio import PortfolioSimulator
from app.trading.backtest.data import download_bars

# S&P 500 유동성 상위 50개 종목, 일봉 2년
symbols = ["AAPL", "MSFT", "NVDA", "GOOGL", "META", ...]
data = download_bars(symbols, days=730, interval="1d")

sig = KalmanSelfMRSIGNAL(z_entry=3.0, hl_max=60.0,
                        kalman_delta=1e-4, kalman_obs=1.0)
sig.initialize(data)

result = PortfolioSimulator(max_positions=60,
per_trade_alloc=0.03).run(sig)
result.print_summary()
```

확인할 출력: print_summary()는 누적 수익률, 샤프 비율, 최대 낙폭, 진입 횟수, 평균 보유 기간을 출력한다. Z 진입 임계(3.0)를 1.5로 낮춰서 다시 돌려 보면 진입 빈도는 늘어나지만 승률은 떨어질 수 있다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. $Q=0.01/R=10$ 과 $Q=1.0/R=0.01$ 두 설정으로 SPY-QQQ 스프레드의 Z-score를 계산하고, 2020년 3월 시점의 차이를 플롯으로 비교하라.
2. `KalmanFilter1D.update(z)`에 NaN 관측값 처리 로직을 추가하라. NaN이 연속 5개 이상이면 예측 상태를 그대로 유지하도록 구현하라.
3. 칼만 Z-score와 단순 20일 rolling Z-score를 같은 페어에 적용해, 어느 쪽이 false positive가 적은지 비교하라.

이 장에서 반드시 기억할 것

- 칼만 필터는 단순 이동평균이 아니다. 예측의 불확실성과 새 관측값의 신뢰도를 비교해 추정치를 갱신한다.
- Q와 R의 비율(Q/R)이 반응 속도를 결정한다. Q/R이 크면 빠르게, 작으면 느리게 반응한다.
- Innovation = 실제 관측값 - 예측값. 이 값이 0에서 크게 벗어났다가 돌아오는 패턴을 평균회귀 신호로 쓴다.
- 인과적 Z-score는 `shift(1)`로 현재 봉을 통계에서 빼서 미래 정보 누수를 막는다.
- 반감기가 짧을수록 평균회귀가 빠르고, 너무 길면(60봉 초과) 진입을 막는다.

개념	핵심
로컬 레벨 모델	$F=1$, 상태=가격 레벨, Innovation=예측 오차
칼만 이득 K	Q/R 비율로 결정—높을수록 빠르게 반응, 낮을수록 평활화
4D 피쳐 평활화	각 차원에 독립된 <code>KalmanFilter1D</code> , <code>KALMAN_PARAMS_4D</code>
Innovation Z-score	<code>shift(1)</code> 인과적 Z-score로 과매도/과매수 감지
반감기 halflife	AR(1) OLS로 추정; 60봉 초과 시 진입 금지

개념	핵심
동적 보유 기간	$\text{eff_hold} = \text{hl_hold_mult} \times \text{halflife}$

다음 장(7장): Cross-asset Joint HMM을 다룬다. 자산 하나가 아니라 여러 자산의 움직임이 얼마나 함께 변하는지(공분산 구조)를 동시에 모델링한다. 이렇게 하면 시장 레짐을 더 안정적으로 포착할 수 있다.

더 읽을거리

- **책:** Grewal & Andrews, Kalman Filtering: Theory and Practice —파라미터 튜닝과 수치 안정성을 깊이 파고든다
- **논문:** “Pairs Trading with Kalman Filtering” (Pole, 2007) —칼만 필터를 스프레드 추적에 처음 적용한 논문

7장. 크로스 애셋 Joint HMM —여러 시장을 함께 보는 레짐 탐지

개인 투자자 대부분은 차트 한 장을 들여다본다. 그 종목의 추세선, 이동평균, 거래량을 확인하고 다음 방향을 예측한다. 하지만 이 방식에는 빈틈이 있다. 그 종목이 속한 시장 전체가 지금 어떤 상태인지, 즉 투자자들이 위험을 기꺼이 감수하는 상태인지 아니면 안전 자산으로 피신하고 있는지를 보지 않으면 중요한 경고 신호를 놓칠 수 있다.

기관 트레이더들은 단일 종목만 분리해서 보지 않고 여러 시장이 동시에 무엇을 말하는지도 살핀다. 예를 들어 시장이 위험 회피 상태(Risk-Off)로 바뀔 때 채권 시장과 신용 스프레드(회사채와 국채의 금리 차이로, 커지면 기업 부도 우려가 커진다는 뜻)가 주식과 다른 시점에 반응할 수 있다. 채권과 변동성 지표는 주식만으로 보이지 않는 보완 정보를 줄 수 있지만, 언제나 먼저 움직이는 것은 아니다. 선행 여부와 실제 탐지 개선은 기간별 OOS에서 확인해야 한다.

2020년 3월 급락 전후에는 국채 금리 하락, VIX 상승, 신용 스프레드 확대가 주식과 함께 위험 신호를 보였다. 다만 이 한 사례가 모든 위기에서 같은 선행 순서를 보장하지는 않는다. 주식 가격만 보는 모델은 이런 다른 시장의 정보를 입력으로 받지 못한다.

이 장의 출발점은 단순하다. 시장 레짐을 제대로 파악하려면 한 시장만 보지 말고, 여러 시장이 동시에 보내는 신호를 함께 봐야 한다.

Joint HMM은 이 아이디어를 구현하는 방법이다. “Joint(공동)”은 “여러 자산과 파생 피처를 함께”라는 뜻이다. SPY에 HMM 하나, TLT에 HMM 하나를 따로 만드는 대신, SPY·TLT·HYG 같은 자산 수익률과 SPY 변동성 같은 파생 피처를 하나의 모델에 동시에 입력한다. 이렇게 하면 레짐을 판단할 때 모든 크로소마켓 정보를 한꺼번에 활용한다. 단일 자산 HMM이 “SPY가 이미 움직였다”를 보는 모델이라면, Joint HMM은 “채권, 신용, 변동성이 지금 함께 무엇을 말하고 있는가”를 보는 모델이다.

단, 자산과 피처마다 값의 단위와 크기가 다르다. SPY 일별 수익률이 보통 몇 퍼센트 범위에서 오르내리는 데 비해, 변동성 피처는 훨씬 넓은 범위로 변한다. 이 값들을 원시 그대로 하나의 모델에 넣으면 크기가 큰 피처가 모델 전체를 지배한다. 그래서 각 자산과 피처를 자기 과거 분포 기준으로 표준화한다 (롤링 z-score). “이 값이 최근 내 역사에서 얼마나 높거나 낮은가”로 바꿔 표현하는 방식이다.

이 장에서 만드는 시스템의 구조는 세 단계다. **Macro HMM**은 “지금 시장 전체가 위험 선호인가, 위험 회피인가”를 판단한다. **Micro HMM**은 “이 개별 자산을 지금 살 만한가”를 판단한다. **Kelly**(승률과 손익비로 적정 베팅 비율을 계산하는 공식)는 그 판단을 실제 포지션 크기로 바꾼다. 읽는 순서는 시장 전체 방향 → 개별 자산 신호 → 포지션 크기다.

자산 수가 늘고 PCA(차원 축소 기법) 같은 도구가 등장해서 복잡해 보일 수 있다. 하지만 위 세 단계 구조를 머릿속에 유지하면 각 세부 구현도 훨씬 따라가기 쉽다.

초보자는 이 장을 “더 어려운 HMM”으로 보기보다, **시장 전체를 더 넓은 화면으로 보는 HMM**으로 이해하면 된다. 4장의 HMM이 한 종목의 체온만 재는 체온계였다면, 7장의 Joint HMM은 체온, 혈압, 맥박, 산소포화도를 함께 보는 건강 검진에 가깝다.

단일 자산 HMM

SPY만 보고 시장 상태를 추정

|

v

다른 시장의 정보를 직접 반영하지 못함

Joint HMM

SPY + TLT + HYG + GLD + SPY 변동성 등을 함께 봄

|

v

채권, 신용, 변동성의 보완 정보까지 반영

이 장의 핵심은 피처가 많아졌다는 사실이 아니다. **서로 다른 시장이 보내는 경고를 하나의 관측 벡터로 묶어 HMM이 읽게 한다**는 점이다. HMM 입장에서는 여전히 같은 흐름이다.

데이터 -> 피처 벡터 -> HMM 학습 -> 레짐 확률 추론 -> 포지션 크기 조정

달라진 것은 피처 벡터의 폭이다. 4장에서는 수익률, 변동성, 모멘텀 같은 몇 개 숫자를 봤다면, 7장에서는 여러 자산의 움직임을 한꺼번에 요약한 20개 숫자를 본다.

이 장의 흐름

- 전체 Pipeline —6단계
- 크로스애셋이 필요한 이유
- bar 단위 노이즈 제거 (Kalman 필터)
- 20차원 세션 관측 벡터 만들기
- Layer 1 —Macro HMM (일봉)
- Layer 2 —Micro HMM (세션)
- macro_bias를 Kelly 포지션 사이징에 적용

학습 목표

이 장이 끝나면 다음을 할 수 있다.

- 단일 종목 HMM 대신 여러 자산을 함께 입력으로 쓰는 이유를 설명할 수 있다.
- Kalman 필터로 수익률 노이즈를 제거하는 이유를 안다.
- 18개 자산·파생 피처 + 2개 마이크로스트럭처 지표(가격이 형성되는 미세 구조를 나타내는 지표) = 20차원 관측 벡터의 구성 원리를 안다.
- Layer 1(Macro)과 Layer 2(Micro)의 역할 분담을 설명할 수 있다.
- macro_bias가 Kelly 포지션 크기에 곱해져 시장 방향과 개별 신호를 결합하는 과정을 이해한다.

이 장의 코드: run_rentec_joint_hmm_v7.py

이 장에서는 크로스애셋 관측값과 두 계층 HMM이 왜 단일 자산 HMM보다 안정적인 레짐 판단을 만들 수 있는지에 집중한다.

TIP**바이브코딩 미션**

- **연을 결과:** 여러 자산이 하나의 관측 벡터로 합쳐질 때 열 이름과 shape를 확인한다.
- **첫 회독:** 자산 18개의 세부 선정 근거보다 서로 다른 시장 경고를 함께 읽는 이유를 본다.
- **용어 발판:** 크로스애셋은 여러 자산군을 함께 보는 것, Joint HMM은 그 입력을 한 모델에서 읽는 구조, shape는 입력의 행·열 크기다.
- **AI 요청:** “Joint HMM 입력을 만드는 함수에서 자산·피처 순서를 추적해 표로 정리해줘. 누락값 처리와 최종 shape도 실제 코드로 확인해줘.”
- **완료 확인:** 한 행이 어느 날짜의 어떤 시장 정보를 담는지 설명하면 성공이다.

7.0 전체 파이프라인—6단계

4장에서 본 단일 자산 HMM은 SPY 가격만 입력으로 받았다. 그래서 채권 급반등이나 달러 강세, 변동성 확대 같은 신호를 놓쳤다. 이 장에서는 SPY 하나 대신 SPY·TLT·UUP·HYG·SPY 변동성 등 여러 시장 신호를 함께 학습하는 Joint HMM으로 확장한다.

NOTE

SPY + TLT + UUP + HYG + SPY 변동성을 함께 쓰는 목적은 단일 자산만 볼 때 놓치는 시장 맥락을 보완하는 것이다. 이 책에 독자가 재현 가능한 비교 로그를 첨부하지 않았으므로 조기 감지 성능을 확정된 결과로 제시하지 않는다. 동일한 기간·비용의 OOS에서 단일 자산 기준선과 비교해야 한다.

전체 흐름을 표로 먼저 확인하자.

단계	내용
1. 자산 다운로드	관측 벡터의 재료가 될 18개 자산·파생 피처의 원본 시계열 확보

단계	내용
2. Kalman 필터	각 자산의 bar 수익률 노이즈 제거
3. 세션 집계	78개 5분봉을 한 세션으로 묶고, 마이크로스트럭처 지표 2개를 더해 20차원 관측 벡터 구성
4. Macro HMM	일봉 단위로 시장 전체 방향(macro_bias) 판단
5. Micro HMM	세션 단위로 각 자산의 상승 확률(p_up) 계산
6. Kelly sizing	p_up과 macro_bias를 곱해 포지션 크기 결정

HMM이 왜 두 개인가?

- **Macro HMM**은 시장 전체의 방향을 판단한다. “지금 시장이 위험을 피할 때인가, 감수할 때인가?”
- **Micro HMM**은 개별 자산의 진입·청산 시점을 판단한다. “Macro가 정한 조건 안에서 이 자산을 지금 살 만한가?”

이 두 층을 분리해야 시스템이 왜 그 결정을 내렸는지 나중에 추적할 수 있다. 예를 들어, Micro HMM이 AAPL 상승 확률을 높게 보더라도 Macro HMM이 Risk-Off를 가리키면 포지션 크기는 줄어든다. 반대로 Macro가 Risk-On이고 Micro도 상승을 가리키면 포지션이 커진다. 요약하면 “좋은 개별 신호라도 시장 전체가 위험하면 작게 들어간다”는 운용 원칙을 코드로 구현한 구조다.

7.1 크로스애셋이 필요한 이유

WARNING

단일 자산 HMM은 주식 가격만 보고 레짐을 판단한다. 하지만 현실에서는 변동성이 급등할 때 다음 봉의 주식 수익률 분포가 달라질 수 있다. 다른 자산과 변동성 피처가 추가 정보를 주는지는 검증할 가설이다. 크로스애셋 피처를 넣은 모델은 반드시 단일 자산 기준선과 같은 기간의 OOS에서 비교한다.

단일 종목 HMM의 한계는 입력 범위다. 주가가 움직인 정보만 보므로 채권·신용·변동성 시장이 보내는 별도 정보를 활용하지 못한다.

크로스애셋 접근은 채권, 금, 신용 스프레드, 변동성 피처를 함께 사용한다. 이런 피처가 특정 구간에서 먼저 반응했다면 탐지 지연을 줄일 수 있고, 동시에 움직였다면 레짐 판단의 추가 근거가 될 수 있다. 반대로 관계가 바뀌면 잡음만 늘 수 있다.

구분	단일 종목 HMM	크로스애셋 Joint HMM
관측	SPY 수익률만	SPY + TLT + GLD + HYG + UUP + ...
레짐 신호	주가 자체의 변화만 반영	다른 시장의 보완 정보도 반영하며, 조기 감지 여부는 OOS로 검증

어떤 자산이 다른 자산보다 먼저 움직이는지는 시차 상관으로 확인할 수 있다(선행-후행 관계, lead-lag). 이 프로젝트의 2018~2024년 일봉 추정에서는 3일 시차에서 상관계수의 절댓값이 가장 컸다. UUP와 GLD는 약 0.238, QQQ와 HYG는 약 -0.208이었다. 이는 어느 기간에나 유지되는 법칙이 아니라, 학습 구간에서 관측한 후보 관계다. 운용 전에는 최신 구간으로 시차와 상관계수를 다시 측정해야 한다.

7.2 bar 단위 노이즈 제거—Kalman 필터

원시 5분봉 수익률은 노이즈가 크다. 한두 개의 이상치 때문에 모델이 잘못된 레짐을 판단할 수 있다. 그래서 각 자산의 bar 수익률을 6장에서 다룬 Kalman 필터로 한 번 걸러서 부드럽게 만든 뒤 모델에 넣는다.

복습하자면, Kalman 필터는 “지금까지 본 관측들”을 가중평균해 진짜 신호를 추정하는 도구다. 이동평균과 비슷하지만, 새 관측이 들어올 때마다 자동으로 가중치를 조정한다는 점이 다르다.

이 장의 필터에는 한 가지 특징이 있다. 이전 추정값을 살짝 0 쪽으로 끌어당긴다는 점이다. 가격 레벨(예: SPY가 450달러)을 추적할 때는 “다음 값도 비슷할 것”이라고 가정한다. 하지만 수익률은 평균 0 근처에서 진동한다. 그래서 “특별한 신호가 없으면 수익률은 0으로 회귀한다”는 가정을 약하게 반영한다.

이렇게 노이즈를 걸러낸 bar 수익률이 다음 절에서 만들 20차원 세션 관측 벡터의 재료가 된다. 또 한 이 필터는 **causal(인과적)**이다. 미래값을 보지 않고 과거와 현재만으로 갱신한다. 백테스트에서 look-ahead bias(미래 정보 누설)가 생기지 않는다.

칼만 필터의 step은 이전 추정과 공분산을 F로 예측하고, 새 관측과의 차이를 이득 K만큼 반영하는 한 번의 갱신이다. F는 상태 전이, Q는 상태 변화의 불확실성, R은 관측 잡음이다. fit_series는 이

갱신을 시간순으로 반복하는 절차로 이해하면 된다. 6장의 1차원 식에서 $F=1$ 인 경우부터 계산하고, 여러 자산이면 행렬 차원과 관측 시각 정렬을 추가로 검사한다.

7.3 20차원 세션 관측 벡터

하루 거래 시간은 5분 단위로 나뉜다. 5분봉이 하루에 78개 모이면 하나의 **세션**이 된다. 한 세션마다 자산별 수익률·변동성을 계산해 20개의 숫자로 압축한다. 이 묶음이 **관측 벡터**다. HMM은 이 벡터 하나씩을 입력으로 받아 레짐을 판단한다.

20개 차원은 세 그룹으로 나뉜다.

그룹	포함되는 것	의미
A —핵심 시장 신호 (10개)	SPY, QQQ-SPY, TLT, SPY 변동성, GLD, USO, UUP, EEM, HYG, XLE	시장 방향·안전자산·매크로
B —섹터·국제 (8개)	IWM, XLF, VNQ, EFA, XLV, XLI, XLP, UNG	섹터별 상대 강세/약세
C —마이크로 (2개)	SPY 시가-종가 수익률, SPY-QQQ 세션 내 상관	세션 내 미세 구조

20차원이라는 말이 부담스럽다면, “질문 20개짜리 체크리스트”라고 생각하면 된다. 모델은 차트 그림을 보지 않는다. 대신 하루가 끝날 때 아래 같은 질문에 숫자로 답한 표를 읽는다.

오늘 주식은 올랐는가?
채권은 안전자산처럼 움직였는가?
금은 강했는가?
SPY 변동성은 불안해졌는가?
신용 위험은 커졌는가?
섹터별로 돈이 어디로 움직였는가?
장중에는 매수세가 강했는가?

이 질문들의 답을 20개 숫자로 압축한 것이 관측 벡터다. 따라서 “20차원”은 어려운 수학 용어라기보다, 시장을 여러 각도에서 본 체크리스트의 길이다.

왜 이 18개 자산·파생 피쳐인가? 세 가지 기준으로 골랐다.

1. **레짐 보완성**: 채권(TLT, HYG), 금(GLD), SPY 변동성은 주식과 다른 시점·방향으로 움직여 추가 정보를 줄 수 있다.
2. **상관 다양성**: 비슷하게 움직이는 자산을 여러 개 넣어도 새 정보가 없다. SPY·QQQ·IWM은 시가총액 레짐, TLT는 금리 레짐, GLD·SPY 변동성은 공포 레짐을 각각 대표한다.
3. **데이터 가용성**: 충분히 긴 과거 데이터가 필요하다.

자산 구성을 바꿀 때는 두 가지를 먼저 확인하자. 추가할 자산이 기존 자산들과 상관계수가 너무 높지 않은가, 그리고 충분한 과거 데이터가 있는가.

자산 클래스별 Macro 민감도

자산군은 equity, em, commodity, credit, fx, safe_haven, volatility처럼 분류할 수 있다. Risk-Off에서 주식 노출을 줄이고 방어 자산 노출을 유지하는 것은 설계 가설이며 금·채권도 항상 방어적이지는 않다. 자산별 최종 배율은 기본 비중 × 레짐 배율로 계산하고 총위험 상한을 적용한다. 배율표의 각 행에는 자산군·레짐·배율·검증 기간을 함께 기록해야 비교가 가능하다.

7.4 Layer 1 —Macro HMM (일봉)

Layer 1은 **일봉** 데이터로 시장의 큰 흐름을 판단한다. 입력은 SPY·QQQ·TLT·GLD·USO·UUP·EEM·HYG 8개 자산의 5일 롤링 수익률에 SPY 변동성과 SPY-TLT 상관을 더한 10차원 벡터다. HMM은 시장을 3개 레짐(Risk-Off / Neutral / Risk-On)으로 나눈다.

매일 새 관측이 들어오면 Forward 알고리즘으로 “지금 어느 레짐일 확률”을 갱신한다. 결과는 예를 들어 [Risk-Off 0.7, Neutral 0.2, Risk-On 0.1] 같은 분포다.

macro_bias —분포를 한 숫자로 압축

세 레짐의 분포를 그대로 쓰면 Kelly에 적용하기 번거롭다. 그래서 한 숫자로 줄인다. 각 레짐에서 예상되는 SPY 5일 수익률을 현재 레짐 분포로 가중평균한 뒤, 표준편차로 나눠 -1에서 +1 사이로 만든다.

- macro_bias = -1에 가까우면 Risk-Off (방어)
- macro_bias = +1에 가까우면 Risk-On (공격)

레짐	SPY 5일 수익률	TLT 5일 수익률	SPY 변동성
Risk-Off	음수 (↓)	양수 (↑)	높음
Neutral	중립	중립	낮음

레짐	SPY 5일 수익률	TLT 5일 수익률	SPY 변동성
Risk-On	양수 (↑)	음수 (↓)	낮음

Risk-On에서는 주식이 오르고 채권이 내리며 변동성이 낮다. Risk-Off는 그 반대다. Neutral은 둘 다 큰 움직임 없는 조용한 상태다.

왜 레짐 지속성을 강제하나?

레짐의 지속성을 유도하려면 전이행렬의 대각 원소에 상대적으로 큰 사전 의사 관측치를 준다. 예컨대 3상태에서 대각 8·나머지 1이면 학습 전 같은 상태에 머무를 선호가 강해진다. 이 값은 설명용이며, 너무 크면 실제 전환을 늦게 잡는다. 여러 시드의 수렴·상태 점유율·OOS 지연을 비교하고 실패한 적합 결과를 그대로 채택하지 않는다.

7.5 Layer 2 —Micro HMM (세션)

Layer 2는 **세션 단위**로 단기 레짐을 감지한다. 4개 레짐으로 나누고, 입력은 7.3에서 만든 20차원 관측 벡터다.

PCA로 먼저 차원을 줄인다.

20차원을 그대로 쓰면 두 가지 문제가 생긴다.

1. **데이터가 부족하다.** 학습 윈도우 360 세션에 비해 모델 파라미터가 너무 많아진다.
2. **정보가 중복된다.** SPY-QQQ 상관은 약 0.9다. 두 차원이 거의 같은 정보를 두 번 전달한다.

PCA(주성분 분석)는 이 두 문제를 한 번에 해결한다. 20차원을 서로 독립적인 15차원으로 압축한다. 그 결과를 HMM에 입력한다.

여기서 PCA_DIM=15는 압축 차원, N_ENSEMBLE=3은 서로 다른 적합 결과 수, TRAIN_SESSIONS=360은 과거 학습 세션 수, REFIT_SESSIONS=40은 재학습 간격의 설정 예다. 관측·피쳐 수보다 큰 PCA 차원을 요구하면 안 된다. PCA와 스케일링은 학습 구간에서만 적합하고 다음 재학습 전까지 함께 고정한다. 이 값들은 최적성이 입증된 공통 설정이 아니다.

7.6 Walk-forward 구조

모델을 한 번 학습하고 끝내면 시장 구조가 바뀔 때 따라갈 수 없다. 그래서 일정 주기로 재학습한다. 이 방식이 **walk-forward(전진 검증)**다.

0 ————— 360 | 361 — 400 | 401 — 440 | ...
 └─ 훈 련 360 ─┘ 00S 40 다시 학습 + 00S 40

매 재학습마다 직전 360 세션으로 Micro HMM을 새로 학습한다. 자산 변동성 분포가 시간에 따라 바뀌기 때문에 표준화 도구(StandardScaler)와 PCA도 같이 새로 학습한다. 이렇게 하면 시장 구조 변화에 모델이 계속 적응한다.

7.7 macro_bias -> Kelly 변조

이제 두 레이어의 결과를 하나로 합친다. Layer 2의 p_up(상승 확률)에서 출발해, Layer 1의 macro_bias로 크기를 조정한다.

```
def kelly_fraction(p_up, macro_bias, asset_class):
    edge = 2 * p_up - 1.0 # Kelly의 기본 우위
    class_amp = MACRO_CLASS_AMP.get(asset_class, 0.5)
    adjustment = macro_bias * class_amp
    kelly_adj = edge * (1.0 - adjustment) # macro로 변조
    return float(np.clip(kelly_adj * 0.5, KELLY_MIN, KELLY_MAX))
```

숫자로 따라가보자. Micro가 p_up=0.58(약한 상승 신호)을 주고, Macro가 macro_bias=-0.7(Risk-Off)을 가리키고, 자산이 주식(equity)인 경우를 생각하자.

- 우위(edge): $2 \times 0.58 - 1 = 0.16$
- Macro 조정: Risk-Off + equity -> 포지션을 줄여야 한다
- 최종 포지션: 약 0.108 (원래 우위 0.16의 약 2/3 수준)

한 줄로 정리하면: **“Micro가 상승 신호를 줘도 Macro가 Risk-Off면 주식 포지션을 줄인다.”** 같은 신호여도 시장 환경에 따라 들어가는 크기가 달라진다.

7.8 P(up) 계산—상승 확률은 어떻게 만들어지나

Layer 2가 출력하는 p_{up} 은 어떻게 만들어지나? 각 레짐별로 “이 자산의 다음 수익률이 0보다 클 확률”을 계산한 뒤, 현재 레짐 분포로 가중평균한다. 순서는 항상 [Risk-Off, Neutral, Risk-On]으로 적는다. 예를 들어 각 레짐의 상승 확률이 Risk-Off 30%, Neutral 50%, Risk-On 70%이고, 현재 분포가 [Risk-Off=0.1, Neutral=0.2, Risk-On=0.7]이라면 $p_{up} = 0.1 \times 0.3 + 0.2 \times 0.5 + 0.7 \times 0.7 = 0.62$ 가 된다.

매매 임계값은 단순하다. $p_{up} > 0.53$ 이면 롱, $p_{up} < 0.47$ 이면 숏, 그 사이는 중립.

7.9 결과를 만든 구조적 요인

아래 표는 성과 개선에 기여한 구조적 요인을 정리한 것이다. 숫자 자체보다 어떤 설계가 어떤 문제를 줄였는지 보는 것이 중요하다.

요소	효과
20차원 크로스애셋 관측	시장 맥락 보완; 조기 감지 효과는 OOS 비교 필요
Macro + Micro 두 레이어	일봉 방향성 편향 × 세션 상승 확률 결합
레짐 지속성 강제	노이즈로 인한 과도한 전환 방지
Walk-forward 재학습	시장 구조 변화에 지속 적응
PCA 15차원	공선성 제거로 학습 안정화
Half-Kelly × macro_bias	9장에서 다루는 포지션 사이징 방식. 자산 클래스별로 비대칭 포지션을 조정

코드로 보기—합성 데이터로 Macro HMM 학습 흐름 확인

실제 시장 자료를 준비하기 전, 아래 합성 데이터로 “관측 벡터 → HMM 학습 → 레짐 추론”을 확인한다. 시장 자료를 쓰려면 자산별 가격을 같은 시각에 맞추고 수익률·변동성을 계산한 뒤, 학습 구간에 맞춘 스케일러로 변환한다. 아래 실습은 다운로드나 운영 저장소 없이 흐름을 이해하기 위한 예다.

이 코드가 보여주는 것: 20차원 관측 벡터 500개를 만들고 -> 60%로 Macro HMM을 학습한 뒤 -> Forward 알고리즘으로 레짐을 추론하기까지의 최소 흐름.

```
import numpy as np
from hmmlearn.hmm import GaussianHMM

SEED, T = 42, 500
N_MACRO_STATES = 3
rng = np.random.default_rng(SEED)

# 20차원 관측 벡터 500개 (합성)
X = rng.normal(0, 1, size=(T, 20))

# 60% 훈련, 40% out-of-sample
train_len = int(T * 0.6)
macro_hmm = GaussianHMM(
    n_components=N_MACRO_STATES,
    covariance_type="full",
    n_iter=200,
    random_state=SEED,
)
macro_hmm.fit(X[:train_len])

# OOS를 한 시점씩 공개하고 마지막 행만 읽는다.
# 마지막 행 뒤에는 관측치가 없으므로 미래 정보가 섞이지 않는다.
alpha_oos = []
for t in range(train_len, T):
    alpha_t = macro_hmm.predict_proba(X[:t + 1])[-1]
    alpha_oos.append(alpha_t)

alpha_oos = np.asarray(alpha_oos)
regime_oos = alpha_oos.argmax(axis=1)

print(f"관측 벡터 shape: {X.shape}")
print(f"학습 완료: {N_MACRO_STATES} 상태")
print(f"OOS 레짐 분포: {np.bincount(regime_oos)} (OOS {T - train_len}봉)")
```

확인할 출력: - 관측 벡터 shape: (500, 20) —시점 × 차원이 맞는지 - OOS 레짐 분포: [..., ..., ...] —세 레짐이 모두 0이 아닌지 확인한다. 한 레짐만 나왔다고 곧바로 학습 실패로 단정하지 말고, OOS 구간이 실제로 단일 국면이었는지도 함께 확인한다.

실패하면 볼 곳: - 한 레짐만 출력되면 상태별 평균·분산과 OOS 가격 흐름을 먼저 확인한다. 단순히 `random_state`를 바꿔 좋은 결과가 나올 때까지 반복하면 선택 편향이 생긴다. - 위 코드는 원리를 보여주려고 매 시점의 전체 과거를 다시 계산하므로 느리다. 실제 시스템은 5장의 `step_forward()`로 직전 α 만 갱신한다.

시장 관측 벡터 X 는 행이 시간, 열이 자산별 수익률·변동성 등의 피처다. 열 순서를 학습 때 확정하고 추론 때 그대로 유지한다. 한 자산의 휴장일을 미래 가격으로 채우지 않고, 자료가 부족한 시각은 제외하거나 결측 정책에 따라 무효로 표시한다. 합성 예제의 결과는 투자 성과를 의미하지 않는다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. Macro HMM이 Risk-Off를 가리키고, Micro HMM이 SPY 상승 확률을 0.58로 예측할 때, 최종 포지션 크기는 어떻게 결정되는가? 7.7절의 식을 따라 직접 계산해 보자.
2. 현재 매크로 자산에 금(GLD)을 추가하면 레짐 감지가 어떻게 달라질지 가설을 세우고, 자산 선택 기준(7.3절)에 비춰 적절성을 평가하라.
3. Walk-forward 재학습 주기를 짧게 하면 어떤 장단점이 있는지 생각해 보자.

정리

이 장에서 반드시 기억할 것:

- 여러 자산과 파생 피처는 단일 자산에 없는 보완 정보를 줄 수 있다. 다만 피처를 늘린다고 레짐 탐지가 자동으로 정확해지는 것은 아니며, 단일 자산 기준선과 기간별 OOS 비교가 필요하다.
- 시스템은 두 층으로 나뉜다. **Macro HMM**은 시장 전체 방향(`macro_bias`)을, **Micro HMM**은 개별 자산의 상승 확률(`p_up`)을 판단한다.
- 두 결과를 곱해서 Kelly 포지션 크기를 정한다. 좋은 개별 신호여도 시장 전체가 위험하면 작게 들어간다.
- 모델은 한 번 학습하고 끝내지 않고 walk-forward로 계속 재학습한다.

레이어	입력	출력	역할
Macro HMM (일봉, 3상태)	주요 자산 10차원	macro_bias $\in [-1, +1]$	거시 방향성 편향
Micro HMM (세션, 4상태)	18개 자산·파생 피처 + 마이크로 (20차원) → PCA 15차원	자산별 p_up	개별 자산 방향 확률
Kelly sizing	p_up × macro_bias × 자산 클래스	포지션 크기 (%)	리스크 조정

다음 장(8장): Two-layer HMM 심화를 다룬다. 앙상블 서브모델 구조, 그리고 레짐 전환이 임박했을 때 확률을 계산하는 방법을 살펴본다.

더 읽을거리

- **논문:** Ang & Bekaert (2002), “International Asset Allocation with Regime Shifts”—교차자산 레짐 모델의 효시
- **논문:** Hamilton (1989), “A New Approach to the Economic Analysis of Nonstationary Time Series”—Markov Switching 모델의 토대
- **책:** Ilmanen, Expected Returns (Wiley, 2011) —자산 클래스 간 상관관계와 레짐 연구

8장. 2계층 HMM 앙상블— Layer A × Layer B

시장은 항상 같은 방식으로 움직이지 않는다. 주가가 한 방향으로 계속 오르거나 내리는 **추세 장**이 있고, 일정 범위 안에서 오르락내리락하는 **횡보 장**이 있다. RSI 같은 지표는 이 두 상황에서 다르게 작동한다. 돌파 신호도 시장 변동성이 클 때와 작을 때 결과가 달라진다. 베테랑 트레이더는 차트를 보며 “지금은 추세를 따라야 할 때”인지 “지금은 반등을 기다려야 할 때”인지 감각적으로 안다. 하지만 그 감각은 사람마다 다르고 일정하지 않아서 코드로 옮기기 어렵다.

대부분의 개인 투자자는 시장 상황(레짐, regime)에 상관없이 같은 신호를 쓴다. 추세가 강한 장에서 잘 맞는 모멘텀 신호를 횡보 장에도 그대로 쓰고, 변동성이 큰 구간에서도 평소와 같은 기준으로 진입한다. 신호 자체가 나쁜 게 아니다. 그 신호가 잘 맞는 상황과 맞지 않는 상황을 구분하지 않는 것이 문제다.

레짐이라는 말이 낯설다면 날씨를 떠올리면 된다. 같은 옷이 봄과 한겨울에 적절하지 않듯, 같은 매매 신호가 모든 시장 국면에서 좋을 수는 없다. 시장에도 계절처럼 국면이 있고, 우리는 그 국면부터 먼저 구분해야 한다.

4·5·7장 복습 박스 - 4장에서 단일 자산 HMM을 만들어“숨겨진 상태”라는 개념을 익혔다. - 5장에서 Forward 재귀로 매 봉마다 상태 확률을 갱신하는 `step_forward()`를 구현했다. - 7장에서 여러 자산을 하나의 모델로 묶는 Joint HMM으로 확장했다.

이 장은 그 위에 **서브모델 8개**를 쌓아 다양한 알파 소스를 하나의 Ensemble로 묶는다. HMM이 “관측 데이터로 숨겨진 상태를 추정하는 모델”이라는 점만 기억하면 따라올 수 있다.

2계층 HMM 앙상블은 이 문제를 구조적으로 해결한다. **Layer A(매크로 HMM)**는 여러 자산의 움직임을 함께 보고 현재 시장 상태를 분류한다. **Layer B**에는 8개의 전문가 서브모델이 있으며, 각 서브모델은 특정 시장 상태에서 가장 잘 작동하도록 설계되어 있다. Layer A가“지금은 평균회귀에 유리한 상태”라고 판단하면 평균회귀 전문가의 신호가 활성화된다. “추세 레짐”이라고 판단하면 모멘텀 전문가의 비중이 높아진다.

이 구조는 머신러닝의 Mixture of Experts(여러 전문가 모델 중에서 상황에 맞는 것을 골라 쓰는 방식) 아이디어와 같다. 핵심은 만능 신호 하나에 의존하지 않는 것이다. 지금 상황에 맞는 전략을 선택하고, 맞지 않는 전략은 비중을 낮춘다.

초보자는 2계층 구조를 회사 조직처럼 생각하면 쉽다. Layer A는 시장 전체 분위기를 보는 팀장이고, Layer B는 각자 전문 분야가 다른 실무자들이다. 팀장이“지금은 위기 국면이니 공격적인 신호는 줄여라”라고 말하면, 실무자의 개별 신호가 좋아도 최종 비중은 작아진다.

```

Layer A: 시장 전체 판단
오늘은 Risk-On인가, Risk-Off인가, 추세인가, 위기인가?
    |
    v
Layer B: 개별 신호 판단
VWAP, Kalman, 거래량, 모멘텀 서브모델들이 방향 의견을 냄
    |
    v
결합
방향 의견 e_sym × 시장 배율 regime_mult = 최종 신호
  
```

이렇게 나누는 이유는 한 모델에게 너무 많은 일을 시키지 않기 위해서다. 일봉 매크로 분위기와 5분봉 진입 타이밍은 시간 스케일이 다르다. 둘을 한 모델에 넣으면 모델은“큰 장세를 배워야 하는지, 짧은 진입 타이밍을 배워야 하는지”를 동시에 해결해야 한다. 두 층으로 나누면 각 층이 맡은 질문이 명확해진다.

Layer A의 질문:

지금은 공격해도 되는 시장인가?

Layer B의 질문:

그 시장 안에서 지금 이 봉은 어느 방향인가?

최종 결합의 질문:

그럼 실제 신호는 얼마나 강해야 하는가?

이 장의 흐름

먼저 왜 두 레이어가 필요한지를 잡고, 그다음 각 레이어의 내부 구조와 합치는 방법으로 내려간다.

- **8.1 왜 두 레이어가 필요한가** —시간 스케일이 다른 두 신호를 왜 분리하는가
- **8.2 효과부터 확인** —이 구조가 실제로 성과를 내는지
- **8.3 Layer A** —매크로 레짐을 추적하는 7-state HMM
- **8.4 8개 서브모델 개요** —각 모델이 어떤 시장에서 작동하는지
- **8.5 SubModelSpec** —서브모델을 선언형으로 다루는 데이터 구조
- **8.6 HMMForwardPredictor** —서브모델의 학습과 추론
- **8.7 W2 Wasserstein 투표** —아웃라이어를 누르고 다수결을 강화하는 방식
- **8.8~8.10 결합과 검증** —최종 신호 합성, 운영 원칙, 요약

학습 목표

이 장이 끝나면 다음을 할 수 있다.

- 시스템이 두 층으로 나뉜다는 것을 이해한다. Layer A는 시장 전체 상태(매크로 레짐)를 분류하고, Layer B는 그 결과를 받아 8개의 작은 예측 모델을 조합한다. 각 층의 역할과 설계 이유를 설명할 수 있다.
- 8개 서브모델이 어떤 시장 상황을 노리는지 큰 그림에서 구분할 수 있다.
- W2 Wasserstein 가중의 설계 의도와 단순 평균 대비 효과를 검증할 조건을 설명할 수 있다.
- 레짐 게이트(active_regimes)가 무엇이고 왜 필요한지 설명할 수 있다.

- `RenaissanceEnsemble5m.step()` 이 돌려주는 `e_sym`이 무엇을 의미하는지 파악하고, 백 테스트 코드에서 올바르게 받아 쓸 수 있다.

TIP

바이브코딩 미션

- **얻을 결과:** Layer A와 Layer B의 입력·출력을 두 줄로 구분한다.
- **첫 회독:** 8개 서브모델의 내부보다 거시 배율 $\times$ 개별 신호 결합만 이해한다.
- **용어 발판:** 레이어는 역할별 처리 단계, 거시 배율은 시장 상황에 따른 크기 조정값, 개별 신호는 자산별 방향 제안이다.
- **AI 요청:** “두 레이어가 만나는 함수를 찾아 입력과 반환값을 표로 정리해줘. 한 날짜의 가상 숫자로 최종 신호 계산을 손으로 보여줘.”
- **완료 확인:** 어느 레이어가 크기를, 어느 레이어가 방향을 정하는지 말하면 성공이다.

8.1 왜 두 레이어가 필요한가

단일 HMM으로 5분봉을 학습하면 성격이 다른 두 종류의 신호를 한 모델에 넣어야 한다. 일봉 수준의 매크로 환경은 며칠에서 몇 주에 걸쳐 천천히 바뀐다. 반면 5분봉 수준의 진입 타이밍은 몇 개 봉 안에서 바뀐다.

두 레이어가 처음에는 복잡해 보인다. 하지만 각 레이어의 역할은 단순하다.

- **Layer A**는 “오늘 시장 분위기가 어떤가”를 판단한다.
- **Layer B**는 “그 분위기 안에서 지금 5분봉 신호를 거래할 만한가”를 판단한다.

두 판단을 한 모델에 섞으면 모델이 어떤 패턴을 배워야 할지 방향을 잃는다. 두 레이어는 모델 수를 늘리기 위한 장식이 아니다. 시간 스케일이 다른 두 판단을 분리하기 위한 구조다.

시간 스케일	현상	필요한 모델
일(day) 단위	매크로 레짐: 추 세/평균회귀 (MR)/위기	Layer A — 7-state HMM

시간 스케일	현상	필요한 모델
분봉(bar) 단위	단기 방향, 평균회귀, VWAP 이탈, 거래량 돌파 등 8개 신호	Layer B —8 서브모델 Ensemble

최종 신호는 $e_sym \times regime_mult$ 다. 즉 Layer B가 방향을 정하고, Layer A가 그 방향 신호의 크기를 조절한다. 이 한 줄이 이 장의 결론이다. 이후 절들은 모두 e_sym 과 $regime_mult$ 를 어떻게 만드느지를 설명한다.

입력 봉 주기와 레짐의 변화 속도는 다르다. 기본 Layer A도 5분봉 시장 맥락 피처를 사용한다. 피처 집계 창과 상태 지속성으로 느린 맥락을 표현하며 일봉 전용 학습을 뜻하지 않는다. Layer B의 약 10 세션 분봉 학습 창과 Layer A의 재학습 간격은 각각 실행 설정에서 확인한다.

8.2 왜 두 레이어 + W2 투표인가

세부로 들어가기 전에, 이 구조가 왜 필요한지부터 본다. 단일 모델만 쓰면 특정 레짐에 과하게 맞춰지기 쉽다. 두 레이어와 W2 투표를 넣는 이유는 모델 하나의 우연한 강점에 기대지 않고, 여러 신호가 같은 방향을 가리킬 때만 더 확신을 주기 위해서다.

버전	변경 사항	확인할 변화
기준 버전	7-state, 최근 PnL 기반 가중만	단일 가중 방식의 한계 확인
W2 투표 도입	Wasserstein + k-means 다수결	레짐 전환 구간에서 신호 안정성 확인

여기서 최근 PnL 기반 가중은 최근 구간에서 PnL이 높았던 서브모델에 더 큰 가중치를 주는 단순 방식이다. 이 책에는 독자가 재현할 수 있는 해당 실험 수치와 로그를 첨부하지 않았다. 따라서 성능 개선의 확정 근거가 아니라 비교 실험의 설계로 읽는다.

8.7에서 이 차이가 어디서 나오는지 직접 확인한다. 지금은 레짐 전환에서 단순 평균과 다른 결정을 하도록 설계했다는 점을 기억하자. 더 좋은 성과인지 여부는 동일한 OOS·비용 조건에서 따로 검증한다.

8.3 Layer A: GlobalRegimeLayer

구분	기본 GlobalRegimeLayer
입력 봉	5분봉으로 정렬된 시장 맥락 피쳐
피쳐 집계	1세션 집계 피쳐 포함; 학습과 추론에서 같은 단위 사용
학습 창	refit_and_init(n_train)에 전달하는 과거 피쳐 창
추론	새 5분봉마다 과거에서 이어진 Forward 상태 갱신
재학습	호출하는 walk-forward 설정에서 결정; 입력 봉 주기와 구분

실제 `set_data()/refit_and_init()` 계약을 확인한 설명이다. 같은 코드베이스의 일봉 게이트나 멀티스케일 대체 레이어를 기본 Layer A와 혼동하지 않는다. `n_train`과 재학습 간격은 실행 설정에서 확인한다.

Layer A는 `hmm_v5_core.py`의 `GlobalRegimeLayer` 클래스로 구현된다. 이 모델은 7개의 시장 상태를 구분하는 HMM이다.

입력은 5분봉이지만, 목적은 개별 종목의 단기 등락을 보는 것이 아니다. 변동성, 거래량, 자산 간 상관처럼 시장 전체를 설명하는 거시적 피쳐를 추적해, 지금 시장이 어떤 분위기인지를 파악한다.

7개 레짐 각각에는 신호 배율(`regime_mult`)이 미리 정해져 있다.

상태	이름	배율	설명
0	DeepMR	0.55	극초저변동 평균회귀 -> 신호 감쇄
1	QuietTrend	1.00	저변동 드리프트 (기본 배율)
2	TrendMod	1.10	완만한 추세
3	TrendStr	1.40	강한 추세
4	TrendBreak	1.70	돌파/모멘텀—신호 최대 증폭
5	HighVol	0.35	고변동 노이즈 -> 신호 감쇄
6	Crisis	0.10	위기/크래시 -> 거의 flat

예를 들어 Layer B가 $e_{\text{sym}} = +1.0$ (강한 롱)을 냈는데 Layer A가 TrendBreak(state 4)이라고 판단하면, 최종 신호는 $1.0 \times 1.70 = 1.70$ 이 된다. 같은 e_{sym} 이라도 Crisis(state 6)에서는 $1.0 \times 0.10 = 0.10$ 으로 거의 비워진다.

세 그룹으로 묶으면 의도가 보인다. DeepMR은 약하게 감쇄하고, HighVol·Crisis는 강하게 감쇄한다.

- **신호 감쇄 그룹 (0, 5, 6):** 평균회귀·고변동·위기 구간. 방향성 신호를 억제해 포지션 크기를 줄인다.
- **추세/드리프트 그룹 (1, 2, 3):** 저변동 드리프트부터 강한 추세까지의 구간. 기본~강한 배율 적용.
- **돌파 그룹 (4):** 박스권을 뚫고 모멘텀이 붙는 국면. 배율을 최대 1.70까지 키운다.

이 매핑 값은 코드 상수 `_REGIME_A_SCALE` 딕셔너리에 들어 있다.¹

8.3.1 레짐 안정화—3바 연속 확인

5분봉이 새로 들어올 때마다 각 state의 확률을 갱신한다. 이때 확률이 가장 높은 state를 곧바로 레짐으로 쓰면 봉마다 레짐이 바뀔 수 있다. 이런 잦은 전환에는 실제 급변과 추정 노이즈가 모두 섞일 수 있으므로, 무조건 노이즈로 지우지 말고 지속 시간과 전환 비용을 함께 본다.

이 프로젝트는 초기 정책으로 **3바 연속 확인**을 쓴다. 새 state가 3개 봉(=15분)에 걸쳐 연속으로 임계 확률 이상을 유지해야 레짐을 바꾼다. 전환 횟수를 줄이는 대신 실제 급변 인식도 최대 3바 늦어질 수 있다. 3은 보편 기준이 아니므로 1·2·3·5바를 OOS에서 비교하고, 전환 지연과 과매매를 함께 측정한다.

8.4 Layer B: 8개 서브모델 개요

이제 Layer B로 내려간다. 클래스 구조를 보기 전에, **8개 서브모델이 각각 무엇을 보는지** 먼저 정리하자. 서브모델은 두 묶음으로 나뉜다. 4개는 HMM 기반이고, 4개는 칼만 필터·이동평균·거래량 같은 비-HMM 방식이다. 여기서 “비-HMM”은 HMM 계열 학습 절차를 거치지 않고 신호를 만든다는 뜻이다. 칼만 필터도 상태공간 모델이지만, 여기서는 가격 평활화 도구로만 쓴다. HMM이 놓치는 신호를 비-HMM 모델이 독립적으로 보완한다.

¹Layer A의 state 수가 반드시 7일 필요는 없다. 5-state로 실험할 때처럼 state 개수가 달라져도 배율 표를 다시 짜지 않아도 되도록, 순위 기반으로 자동 매핑하는 헬퍼(`_regime_mult_from_rank()`)가 들어 있다.

8.4.1 HMM 서브모델 (대표 2개)

HMM 서브모델 4개는 M3_vwap, M5_RetKalman, M6_VolKalman, M7_MRFast다. 대표로 두 개를 먼저 보자.

모델	피쳐 철학	활성 레짐	역할
M3_vwap	VWAP 이탈 + BB 위치	전 레짐	범용 평균회귀 + 급락 솟
M5_RetKalman	5/10/20 봉 수익률 + Kalman 속도	QuietTrend~TrendBreak (1~4)	추세 방향성 포착

M6_VolKalman과 M7_MRFast도 같은 명세 구조를 따른다. 전자는 변동성 관련 관측의 평활화, 후자는 짧은 평균회귀 창에 초점을 두는 역할 구분이다. 모델마다 피쳐 열·과거 창·활성 레짐·출력 단위를 적어 두면 이름이나 전체 클래스 없이도 어떤 정보를 합치는지 읽을 수 있다. 동일 정보를 중복한 모델을 더한다고 분산 효과가 자동으로 생기지는 않는다.

레짐 게이트 설계 원리: 모든 모델을 항상 켜두지 않는다. 레짐마다 유효한 신호가 다르기 때문이다.

- QuietTrend~TrendBreak 레짐(1~4)에서는 변동성 전용 모델(M6)을 끈다. 가격 방향과 무관한 신호는 이 구간에서 노이즈만 늘린다.
- M5의 `active_regimes`는 $1_{4(\text{QuietTrend} \sim \text{TrendBreak})}$ 로 제한되어 있어 Crisis(6)에서는 자동으로 꺼진다. Layer A 배율(0.10)로도 신호가 거의 비워지지만, 게이트가 한 번 더 차단해 추세 신호가 살아남는 일을 막는다.

M6는 가격 방향이 아니라 변동성 구조만 보는 서브모델이다. 그래서 방향성이 선명한 추세·돌파 구간에서는 꺼두고, 흔들림 자체가 중요한 구간에서만 참고한다.

8.4.2 비-HMM 서브모델 (대표 2개)

M8_Kalman, M10_VolBreak, M11_EMAMom, M12_TODRet5b 네 개는 HMM이 아닌 다른 방식(칼만 필터·이동평균·거래량 규칙 등)으로 신호를 만드는 모델이다. 현재 운영 중인 모델은 HMM 계열 4개와 비-HMM 계열 4개, 총 8개다. 대표로 두 개만 보자.

`model_type`은 `SubModelSpec`이 어떤 신호 생성 경로를 쓸지 지정하는 식별자다.

모델	model_type	투표 기준
M8_Kalman	kalman	칼만 필터로 평활화된 수익률의 부호 -> 장기 추세 방향
M10_VolBreak	volbreak	거래량이 평소보다 크게 실린 봉에서만 방향 투표

나머지 비-HMM 모델(M11_EMAMom, M12_TODRet5b)은 부록 표에 정리되어 있다.

M1~M5의 기여도는 같은 시간 분할·비용·데이터로 하나씩 제거하는 ablation으로 확인해야 한다. 이 책에 독자가 재현할 수 있는 해당 결과를 첨부하지 않았으므로 특정 모델의 기여 순위를 검증된 결론으로 제시하지 않는다.

8.5 SubModelSpec —서브모델을 데이터로 선언하기

서브모델의 종류와 역할을 봤다면, 이제 이 8개가 코드에서 어떻게 다뤄지는지가 궁금할 차례다. Layer B의 서브모델은 클래스가 아니라 **데이터로 선언한다**. SubModelSpec 데이터클래스가 서브모델 하나의 모든 설정을 담는다. 새 서브모델을 추가하려면 클래스를 새로 짜는 게 아니라 SubModelSpec 객체 하나를 리스트에 추가하기만 하면 된다.

본문에서는 핵심 필드 3개만 본다.

필드	의미
name	모델 식별자 (예: "M3_vwap")
feat_cols	이 모델이 사용할 피처 컬럼 목록
active_regimes	Layer A 레짐 게이트—None이면 전 레짐에서 활성

n_states는 잠재 상태 수, model_type은 모델 종류, mu_clip_bp는 상태 평균 수익률의 절댓값 상한을 bp 단위로 나타낸다. 예를 들어 상한 20bp이면 평균을 -0.002~0.002로 제한한다. active_regimes는 어떤 상위 레짐에서 해당 모델의 출력을 사용할지 정한다. 활성 모델이 하나도 없으면 예측을 0으로 확신하는 대신 거래 보류 상태를 반환한다.

여기서부터는 구현 세부다. 1회독 독자는 8.6과 8.7을 한 번 훑은 뒤 8.8로 건너뛰어도 된다. 8.8의 결합 흐름과 8.10의 요약만 보면 이 장의 큰 그림은 완성된다. 세부는 두 번째 회독에서 채우면 충분하다.

8.6 HMMForwardPredictor: Layer B HMM Predictor

각 HMM 서브모델은 HMMForwardPredictor 클래스로 감싸진다. 이 클래스가 하는 일은 두 가지다.

1. **fit()**: 과거 데이터로 HMM을 학습하고, 각 상태의 기대 수익률을 계산한다.
2. **step()**: 새 봉이 들어올 때마다 현재의 기대 수익률을 갱신해 반환한다.

8.6.1 fit() —학습 단계가 하는 일

후보 상태 수는 $BIC \ k \cdot \log(n) - 2 \cdot \log L$ 로 비교할 수 있다. k 는 자유 파라미터 수, n 은 학습 관측 수, $\log L$ 은 같은 자료에 대한 로그우도다. 낮을수록 복잡도 대비 적합이 낮지만 금융 성과를 보장하지 않는다. 공분산 형태에 따라 k 가 달라지므로 같은 모형 정의로 계산한다. 상태 평균의 클리핑은 적은 표본의 극단 추정치가 주문 크기를 지배하지 않도록 제한하는 별도 정책이다.

확인할 출력: `fit()`이 `True`를 반환하면 학습 성공이다. `False`라면 입력 데이터가 부족한 경우다.

8.6.2 step() —새 봉마다 기대 수익률 갱신

학습이 끝나면 매 5분봉마다 `step(x_bar)`를 호출한다. 5장의 단일 HMM `step_forward()`처럼 새 관측값으로 상태 확률을 갱신하지만, 여기서는 그 확률로 상태별 평균 수익률을 가중합해 현재 기대 수익률까지 계산한다. HMM 계열 4개가 같은 방식으로 `step()`을 호출하고, 비-HMM 4개는 8.4.2의 경로로 별도 신호를 만든다.

실패하면 볼 곳: 입력 피처에 `NaN`이 섞여 있으면 확률 갱신이 깨질 수 있다. 피처 전처리에서 `NaN`을 거르는 게 우선이다.

8.7 W2 Wasserstein Voting —아웃라이어 억제

8개 서브모델의 예측을 단순 평균하면 문제가 생긴다. **이상치 서브모델 하나가 전체 신호를 오염시킬 수 있다.**

미니 예시로 감을 잡자. 8개 모델 중 7개가 약한 LONG(예: +0.2)을 내고, 1개가 강한 SHORT(-1.0)를 낸다고 하자. 단순 평균을 내면 $(7 \times 0.2 + 1 \times (-1.0)) / 8 = +0.05$ 로, 다수 의견이 거의 사라진다. 외톨이 하나가 전체 신호를 약하게 만든 것이다. W2 투표는 이 외톨이의 가중치를 거의 0으로 누른 뒤 다수 진영을 더 키워, 결과를 다수 의견인 약한 LONG 쪽으로 끌어준다.

Wasserstein 거리(W2)는 두 확률 분포가 얼마나 다른지 재는 척도 중 하나다. 두 분포의 값이 거의 겹치지 않을 때 KL-divergence는 매우 큰 값(이론상 무한대)이 되어 사용하기 어렵지만, W2는 두 분포 사이의“이동 비용”을 재는 방식이라 이런 극단적 경우에도 유한한 거리값을 얻을 수 있다.

W2 투표는 두 단계로 작동한다.

Phase 1 —외톨이 누르기. 각 서브모델은 가우시안 형태의 예측 분포(평균, 분산)를 출력한다. 모든 모델 쌍의 W2 거리를 계산해“나머지 모델들과 평균적으로 얼마나 떨어져 있는지”로 외톨이 정도를 잰다. 외톨이일수록 가중치를 거의 0으로 누르고, 다수와 가까운 모델은 정상 가중치를 유지한다.

Phase 2 —다수결 클러스터 부스트. Phase 1만으로는 LONG과 SHORT로 의견이 반반 갈릴 때 신호가 0 근처로 수렴한다. Phase 2는 서브모델을 두 묶음(LONG 진영 / SHORT 진영)으로 나눈 뒤, 다수결 진영의 가중치를 키우고 소수 진영을 누른다. 여기서 다수결 기준은 **개수가 아니라 신호 크기의 총합**이다. LONG 모델이 3개지만 신호가 약하고 SHORT 모델이 2개지만 신호가 강하면, SHORT 진영이 다수결로 선택된다.

분포 거리 d 를 유사도 가중치로 바꾸는 설명용 식은 $w_i = \exp(-d_i/T) / \sum_j \exp(-d_j/T)$ 이며 T 는 양수다. 가까운 분포일수록 가중치가 커지지만, “분포가 비슷함”이“수익 예측이 좋음”을 뜻하지는 않는다. 성과 기반 가중치와 섞는다면 두 성분을 각각 정규화하고 혼합 비율을 사전에 고정해 별도로 검증한다.²

²실제 코드에는 최근 PnL 기반 가중과 W2 가중을 섞을 수 있는 블렌딩 비율이 준비되어 있다. 이 장에서는 W2 가중이 왜 필요한지와 어떻게 결합되는지를 개념 중심으로 설명한다.

8.8 RenaissanceEnsemble5m.step() — 두 레이어 결합

step()은 8개 서브모델의 출력을 모아 최종 매매 신호를 만든다. 호출 결과 중 가장 중요한 값은 첫 번째인 e_sym이다. e_sym은 살아남은 서브모델들의 W2 가중합으로 만든 **Layer B의 방향 신호**다. 함께 돌려주는 나머지 값들(서브모델별 신호 디셔너리, 롱/숏 투표 수 등)은 디버깅과 안전장치 판정에 쓰인다. vote_ratio는 롱 투표 수와 전체 투표 수의 비율로, 두 진영이 비슷하면 포지션을 비우는 데 사용한다.

한 추론 단계는 (1) 확정 봉 검사 (2) 상위 레짐 갱신 (3) 활성 서브모델 선택 (4) 각 모델의 과거 정보 기반 예측 (5) 지난 예측과 이미 확정된 결과로 가중치 갱신 (6) 가중합과 위험 한도 적용 순서다. 현재 예측의 미래 실현 수익을 같은 단계의 가중치에 쓰면 누수다. 가중치 갱신 간격은 봉수 또는 세션수 중 단위를 정해 실험 설정에 기록한다.

8.1의 결론—최종 신호 = e_sym × regime_mult —이 코드로는 다음과 같이 생긴다.

다음 코드는 매 5분봉마다 Layer B의 방향 신호와 Layer A의 레짐 배율을 곱해 최종 포지션을 만든다. 의견이 갈리면 포지션을 비운다(flat).

```
for bar_idx in range(trainBars, len(df_5m)):
    x_row = feat_df.iloc[bar_idx]
    prev_ret = log_returns[bar_idx - 1] if bar_idx > 0 else None

    e_sym, sub, v_long, v_short, n_voted = ensemble.step(
        x_row, actual_ret=prev_ret,
        regime_state=layer_a.dominant_state,
    )

    # Layer A × Layer B 결합
    regime_mult = _REGIME_A_SCALE.get(layer_a.dominant_state, 1.0)
    signal = e_sym * regime_mult

    # 의견이 갈리면 포지션을 비운다 (안전장치)
    vote_ratio = v_long / n_voted if n_voted > 0 else 0.5
    if vote_ratio < VOTE_THRESH and vote_ratio > (1 - VOTE_THRESH):
        signal = 0.0

    positions[bar_idx] = signal
```

e_sym은 방향 신호, regime_mult는 크기 조절, vote_ratio는 안전장치다. 셋이 곱해져서 최종 포지션이 된다.

8.9 설계 원칙 요약

설정 상수 두 개로 시스템의 시간 척도가 정해진다.

- **재학습 주기 (REFIT_EVERY=156)**: 약 2세션마다 모델을 새로 학습한다. 너무 길면 레짐 전환을 놓치고, 너무 짧으면 학습 창이 너무 작아 BIC가 제대로 작동하지 않는다. 2세션이 그 타협점이다.
- **최소 보유 봉 (MIN_HOLD_BARS=78)**: 한 포지션을 최소 1세션(=하루) 유지한다. 미국 정규장 6.5시간은 5분봉 78개에 해당하므로 1세션 = 78봉이다. 매매를 자주 할수록 수수료·슬리피지가 누적되어 수익을 잠식하기 때문이다.

거래 비용은 의외로 빨리 누적된다. 예를 들어 편도 거래비용을 8bp(=0.08%)로 잡고 연간 왕복 100회 매매한다고 하자. 왕복 1회의 비용은 편도 2번이므로 0.16%이고, 100회 누적하면 매매대금 대비 약 16%가 비용으로 빠져나간다. 실제 손익에 반영되는 정도는 평균 포지션 크기에 따라 달라지지만, 매매 빈도를 줄여야 하는 이유를 보여주는 거친 추정치다.

코드로 보기—2단계 HMM 최소 구현

아래는 앞 절의 두 레이어 결합을 압축한 구조 예다. 완성된 학습 프로그램으로 복사 실행하는 코드가 아니다. 학습 산출물에는 스케일러·피쳐 순서·모델·상태 의미·학습 종료 시각을 함께 저장하고, 추론에서는 그 묶음을 한 버전으로 읽는다. 어느 하나라도 불일치하면 신규 신호를 보류한다.

이 코드가 맡는 일: Layer A의 우세 레짐으로 Layer B 서브모델을 거르고, 살아남은 모델의 $P(up)$ 을 가중 평균한 뒤 Kelly 크기로 바꾼다.

```
import numpy as np

def combine_two_layers(global_alpha, submodels, t, kelly_half=0.5):
    dom_state = int(global_alpha[t].argmax()) # Layer A 우세 레짐

    p_ups, weights = [], []
    for model in submodels:
```

```

if dom_state not in model.active_regimes:
    continue # 레짐 게이트
p_ups.append(model.predict_up(t)) # Layer B 방향 확률
weights.append(model.rolling_score) # 모델 신뢰도

if not p_ups:
    return 0.0, 0.0

w = np.array(weights) / np.sum(weights)
p_up = float(np.dot(w, p_ups))

edge = 2 * p_up - 1.0
kelly = float(np.clip(max(edge, 0.0) * kelly_half, 0.10, 1.0))
return p_up, kelly

```

확인할 출력: p_up은 0~1 사이, kelly는 0.10~1.0 사이여야 한다. p_up이 0.5 근처면 방향 우위가 거의 없고, 0.55 이상이면 상승 쪽 우위가 있다는 뜻이다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. 서브모델 하나를 비활성화하고 전체 성과가 어떻게 바뀌는지 측정하라. 이런 분석을 **ablation study(제거 실험)**라 부른다. 구성 요소를 하나씩 빼면서 전체 성능에 미치는 영향을 수치로 확인하는 방법이다.
2. 투표 가중치 계산에 쓰이는 Wasserstein W2 거리를 KL-divergence로 교체해보라. 본문에서 언급한 발산이 어떤 입력에서 실제로 나타나는지 재현하고, 그때 가중치가 어떻게 변하는지 관찰하라.
3. HMMForwardPredictor.step(x) 를 처음 호출하는 100봉 구간을 warm-up 구간이라 한다. 이 구간에서는 각 상태의 확률이 아직 안정되지 않아 예측이 흔들릴 수 있다. warm-up 구간에서 포지션 크기를 자동으로 줄이는 로직을 추가해보라.

8.10 이 장에서 반드시 기억할 것

이번 장에서 8개 서브모델을 모아 하나의 앙상블(Ensemble)을 완성했다. 가장 중요한 세 가지만 다시 정리한다.

1. **시간 스케일이 다른 두 판단을 분리한다.** Layer A는 일 단위 시장 분위기(매크로 레짐)를 읽고, Layer B는 분봉 단위 매매 방향을 정한다. 한 모델에 두 판단을 섞으면 학습이 흔들린다.
2. **레짐 게이트로 맞지 않는 신호를 끈다.** 추세 레짐에서 평균회귀 모델을, 위기 레짐에서 모멘텀 모델을 비활성화한다. 이 한 번의 필터링만으로 신호 품질이 크게 올라간다.
3. **단순 평균 대신 Wasserstein 투표를 쓴다.** 외톨이 모델은 가중치를 누르고, 다수 의견은 키운다. 모델 하나가 틀린 신호를 내도 전체로 영향이 퍼지지 않는다.

요약 표는 다음과 같다.

레이어	클래스	역할
Layer A	GlobalRegimeLayer	매크로 레짐 -> 신호 크기 배율
Layer B	RenaissanceEnsemble5m	8개 서브모델 -> 방향 신호 e_sym
결합	e_sym × regime_mult	최종 포지션 신호

이 장을 책 전체 파이프라인에 끼워 넣으면 다음 위치다.

```

데이터 / 피쳐
  |
  v
Layer A + Layer B HMM Ensemble
시장 분위기와 단기 방향을 함께 판단
  |
  v
Kelly / Risk
신호를 포지션 크기와 안전 한도로 변환
  |
  v
Broker / Monitoring
주문 실행과 운영 감시
  
```

즉 8장은 딥러닝 장으로 가기 전, 머신러닝 모델 여러 개를 조합해 신호를 더 안정적으로 만드는 장이다. 뒤에서 SAC를 붙일 때도 이 구조가 그대로 도움이 된다. RL은 원시 가격만 보는 것이 아니라, 이렇게 정리된 레짐과 신호를 상태 벡터로 받아 행동을 배운다.

한 줄 요약

`signal = e_sym × regime_mult`

이 장의 모든 내용은 이 한 줄을 어떻게 만드는지에 관한 것이다.

다음 장 안내: 9장에서는 이렇게 만든 `e_sym × regime_mult` 신호를 실제 주문 크기로 바꾸는 **Half-Kelly 포지션 사이징**을 살펴본다. 10장에서는 단일 자산에 대해 여러 HMM을 forward 방식으로 합치는 가벼운 앙상블 구조를 정리한다. 8장의 2계층 구조와는 결합 방식이 다르다. 11장부터는 단일 종목 레짐 전략과 다른 축인 통계적 페어 차익거래로 넘어간다.

더 읽을거리

- **키워드:** “Wasserstein distance time series”—W2 거리를 시계열 분석에 적용한 연구를 찾을 때 유용한 검색어
- **논문:** Jacobs, Jordan, Nowlan & Hinton (1991), “Adaptive Mixtures of Local Experts”—전문가 모델을 결합하는 Mixture of Experts의 고전적 출발점
- **코드:** `hmmlearn`—기본 HMM 구현 참고용. 계층적 HMM을 더 깊게 보려면 `pyhsmm` 계열 구현을 함께 참고할 수 있다.

9장. 켈리 절반 포지션 사이징

Part IV —신호를 포지션으로 바꾸기 (9~10장) 앞 파트까지는 “시장 상태를 어떻게 읽을까”를 다뤘다. 이제는 그 판단을 실제 포지션 크기와 방향 신호로 바꾼다. 9장은 얼마나 크게 들어갈지, 10장은 여러 HMM 신호를 어떻게 합칠지 다룬다. 첫 회독에서는 공식을 외우기보다 “좋은 신호도 너무 크게 베팅하면 망가진다”는 원칙을 잡는다.

대부분의 개인 투자자는 포지션 크기를 거의 고민하지 않는다. “이거에 10% 넣어야지”, “100주 사야지”처럼 둥근 숫자로 결정하는 경우가 많다. 그 숫자는 계산 결과라기보다 감에 가깝다. 전략, 신호, 진입, 청산은 많은 주의를 받지만 포지션 사이징(한 번에 얼마를 투자할지 정하는 일)은 뒷전으로 밀린다. 퀀트 접근법에서는 다르다. 포지션 사이징은 실제 수익이 실현되는 곳이고, 손실이 누적되는 곳이기도 하다. 같은 신호라도 2% 비중으로 들어가면 건널 수 있는 변동이 된다. 30% 비중으로 들어가면 계좌 전체를 흔드는 사건이 된다. 신호의 품질과 포지션 크기는 따로 볼 수 없다.

손실의 수학은 대칭이 아니다. 10% 손실을 회복하려면 약 11% 수익이 필요하다. 50% 손실은 100%, 75% 손실은 300%가 있어야 본전이다. 더 깊이 내려갈수록 회복이 구조적으로 어려워진다. 시장 탓

이 아니라 산술 때문이다. 자신의 엣지(우위)보다 크게 포지션을 잡으면, 수익성 있는 전략도 결국 손실로 끝난다.

켈리 기준(Kelly Criterion)은 이 문제를 수학적으로 접근한다. 승률과 손익비를 바탕으로, 확률 모형이 맞고 같은 기회가 반복된다는 가정 아래 장기 로그 자산 성장을 최대화하는 베팅 비율을 계산한다. 켈리 절반(Half-Kelly)은 그 비율의 50%만 쓰는 보수적 변형이다. 이 책의 시스템은 여기서 한 걸음 더 나아가 6팩터 사이징 모델을 쓴다. 다만 뒤의 구현은 교과서적 켈리를 그대로 계산하는 함수가 아니라, Half-Kelly의 직관에서 출발해 시장 상태·변동성·매크로 조건으로 조정한 **프로젝트용 사이징 규칙**이다.

이 장에서 가장 중요한 점은 켈리를 “많이 벌기 위한 공식”으로 이해하지 않는 것이다. 켈리는 오히려 과도한 베팅을 막기 위한 장치다. $p_{up}=0.53$ 처럼 엣지가 작으면 아무리 모델이 복잡해도 기본 포지션은 작아야 한다. 확률 추정치 조금만 틀려도 엣지가 사라질 수 있기 때문이다.

처음 적용할 때는 켈리 결과를 주문 크기로 바로 쓰기보다, “내 모델이 말하는 엣지가 얼마나 작은가”를 확인하는 진단값으로 먼저 보는 편이 좋다. 실전 포지션은 Half-Kelly, 최대 포지션 한도, 종목별 손실 한도, 포트폴리오 전체 노출 한도를 모두 통과한 뒤에만 결정해야 한다. 켈리는 사이징의 출발점이지 마지막 결론이 아니다.

이 책 전체의 파이프라인에서 켈리 사이징은 **모델의 판단을 돈의 크기로 번역하는 단계**다. HMM과 양상블이 방향과 확률을 말하고, 켈리는 그 확률 우위를 얼마나 크게 베팅할지 계산한다.

```

모델 출력
p_up = 0.53
상승 쪽 우위가 조금 있음
    |
    v
Kelly / Half-Kelly
작은 우위이므로 작은 포지션만 허용
    |
    v
리스크 관리
최대 비중, 손절, 포트폴리오 한도 재확인
  
```

중요한 점은 $p_{up} = 0.53$ 이 “53% 확신”처럼 들리지만 실제로는 아주 작은 우위라는 것이다. 100번 중 53번 맞고 47번 틀릴 수 있다는 뜻이다. 이런 우위를 크게 베팅하면 몇 번의 연속 손실만으로 계좌가 흔들린다.

예측 모델의 역할:
조금 유리한 방향을 찾는다.

켈리의 역할:
그 유리함이 작으면 포지션도 작게 만든다.

리스크 관리의 역할:
그래도 너무 크면 한 번 더 줄이거나 막는다.

그래서 켈리는 공격 버튼이 아니라 번역기이자 브레이크다. “좋은 신호니까 크게 들어가자”가 아니라, “이 정도 우위면 이 정도 크기까지만 허용하자”에 가깝다.

이 장의 흐름

먼저 켈리 공식 자체를 이해하고, 그다음 본 시스템이 켈리에 어떤 보정 인자를 곱하는지 하나씩 본다.

1. Kelly 공식의 직관과 Half-Kelly를 쓰는 이유 (9.1)
2. 전체 사이징 함수 `calculate_kelly_size()`의 구조 (9.2)
3. 6개 보정 인자의 역할 (9.3 ~ 9.7)
4. 단순한 5분봉용 변형 `fractional_kelly()`와 두 구현의 비교 (9.8 ~ 9.9)
5. `KELLY_MIN`을 0이 아닌 0.10으로 둔 설계 의도 (9.10)

학습 목표

이 장이 끝나면 다음을 할 수 있다:

- Kelly Criterion 공식과 Half-Kelly를 쓰는 이유를 설명한다
- `calculate_kelly_size()`가 어떤 흐름으로 포지션 크기를 결정하는지 따라간다
- 6개 보정 인자가 각각 “무엇을 막거나 키우는지” 구분한다
- 5분봉 단순 구현 `fractional_kelly()`의 핵심 아이디어를 적용한다

TIP**바이브코딩 미션**

- **얻을 결과:** 같은 신호에 Full Kelly와 Half Kelly를 적용해 크기를 비교한다.
- **첫 회독:** 공식 증명보다 좋은 신호도 과도한 베팅 때문에 망가질 수 있다는 원칙을 본다.
- **용어 발판:** Kelly는 우위를 베팅 크기로 바꾸는 공식, 승률은 이긴 거래 비율, 손익비는 평균 이익과 평균 손실의 비율이다.
- **AI 요청:** “이 장의 Kelly 식에 승률 0.5·0.6·0.7과 손익비 1을 넣어 비교해줘. 반 Kelly와 포지션 상한 10%를 적용한 결과도 보여줘.”
- **완료 확인:** 입력이 불확실하거나 변동성이 커질 때 크기가 줄어들면 성공이다.

9.1 Kelly 기준의 기초

Kelly 공식은 1956년 벨 연구소에서 나왔다. J. L. Kelly는 정보 이론을 활용해 “같은 종류의 베팅을 반복할 때 자본을 장기적으로 가장 크게 늘리는 베팅 비율”을 계산했다. 이후 수학자 에드워드 소프가 도박과 주식에 이 공식을 적용했고, 여러 퀀트 운용사도 비슷한 방식을 쓴 것으로 알려졌다.

Kelly 공식의 핵심을 먼저 짚는다. **단기 승률을 높이는 공식이 아니다.** 같은 거래를 수백 번 반복할 때 자본이 장기적으로 가장 크게 성장하도록 한 번에 얼마를 투자할지 정해주는 공식이다.

IMPORTANT

풀 Kelly는 대체로 너무 공격적이다. Kelly가 이론적으로 최적이 되려면 엷지를 정확히 알아야 하지만, 실전에서는 불가능하다. 엷지를 실제보다 크게 잡으면 풀 Kelly는 큰 손실로 이어진다. 이 책에서는 Half-Kelly 이하를 쓴다.

이기거나 지는 두 가지 결과만 있는 이진 베팅에서, 이길 확률을 p , 이겼을 때 수익과 졌을 때 손실의 비율을 b 라 하면 최적 베팅 비율은 다음과 같다.

$$f^* = (p * b - (1 - p)) / b$$

아래 단순 예시에서는 이겼을 때와 졌을 때의 크기가 같다고 가정해 $b = 1$ 로 놓는다. 실제 주식 전략의 평균 이익과 평균 손실은 같다고 가정할 수 없으므로, 실전 계산에서는 비율을 반영한 손익비를 별도로 추정해야 한다.

```
f* = 2p - 1
edge = p_up - 0.5
f* = 2 * edge
```

예를 들어 $p_{up} = 0.53$ 이면 $edge = 0.03$ 이고 풀 Kelly는 6%, Half-Kelly는 3%다.

$p_{up} = 0.53$ 은 “이 거래가 53% 확실하다”는 뜻이 아니다. 비슷한 상황이 100번 반복될 때 평균적으로 53번 상승했다는 통계적 추정이다. 나머지 47번은 틀린다. 그래서 포지션 크기는 확신의 크기로 정하지 않는다. 작은 확률 우위를 장기간 안정적으로 반복할 수 있도록 보수적으로 잡아야 한다.

Half-Kelly를 쓰는 이유: 실제로는 p 와 손익비를 정확히 알 수 없다. Full Kelly는 가정한 분포와 옛지가 맞을 때 기대 로그성장을 최대화하지만, 추정 오차가 크면 과도한 비중이 된다. Fractional Kelly는 성장을 일부 포기하고 비중 민감도를 낮추는 정책이다. Half-Kelly가 모든 데이터에서 MDD를 일정 폭 줄이거나 성장률을 보장하지는 않으므로, 비용·추정 오차·OOS 낙폭을 함께 비교한다. 위의 $b = 1$ 단순 예시에서만 Half-Kelly 비율이 $edge$ 와 같고, 이후 보정 인자는 프로젝트의 크기 배율 규칙에 적용된다.

9.2 calculate_kelly_size() 구조

v7.py의 사이징 함수는 Half-Kelly가 계산한 기본 $edge$ 에 6개의 보정 인자를 순서대로 곱해 최종 주문 크기를 결정한다. 인자 하나 하나는 다음 절에서 다루고, 여기서는 “어떤 순서로, 무엇을 곱하는가”의 구조만 잡는다.

아래는 계산 순서를 보여주는 골격이다. 승률 p 와 평균 이익/평균 손실 비율 b 가 유효할 때 $f = p - (1 - p)/b$ 를 계산하고, 음수면 신규 롱 배정을 0으로 둔다. 반 Kelly는 $f/2$ 이며 마지막에 계좌 포지션 상한을 적용한다. b 가 0이거나 표본이 부족하면 계산을 보류한다.

```
# run_rentec_joint_hmm_v7.py (요약 발췌)
KELLY_MIN, KELLY_MAX = 0.10, 1.0
THRESHOLD = 0.53          # p_up > THRESHOLD -> long entry

def calculate_kelly_size(
    p_up, direction,
    vol_mult=1.0, macro_gate=1.0, novelty_gate=1.0,
    er=0.0, macro_bias=0.0, macro_class_amp=0.0,
) -> float:
```

```

# (1) edge: direction에 따른 p_up 우위
edge = (p_up - 0.5) if direction > 0 else ((1.0 - p_up) - 0.5)
if edge <= 0:
    return 0.0

# (2) Half-Kelly baseline (edge 그대로)
f = edge

# (3) 보장 인자들을 곱해서 줄이거나 키움
f *= vol_mult * macro_gate * novelty_gate
# (ER amplifier와 macro_bias 조정은 생략 - 9.6, 9.7에서 설명)

# 정규화 후 클리핑
edge_max = max(THRESHOLD - 0.5, 1e-6)
return float(np.clip(f / edge_max, KELLY_MIN, KELLY_MAX))

```

읽을 때 봐야 할 곳은 두 군데다. 첫째, `if edge <= 0: return 0.0` —엣지가 없으면 함수는 바로 0을 돌려준다. 둘째, 마지막 `np.clip` —모든 보정이 끝난 뒤 `KELLY_MIN`과 `KELLY_MAX` 사이로 잘린다. 그 사이에서 일어나는 일이 6개 인자의 곱셈이다.

여기서 반환값은 교과서 공식이 말하는 “총자산 중 정확히 몇 퍼센트를 베팅하라”는 비율이 아니라, 전략의 기준 포지션에 곱할 **0~1 사이 크기 배율**이다. 예를 들어 반환값 1.0은 풀 Kelly를 뜻하지 않고 “이 전략에 미리 정한 기준 크기를 100% 사용”한다는 뜻이다. 기준 포지션 자체는 포트폴리오 한도와 리스크 예산에서 별도로 정한다.

각 인자가 갖는 범위와 역할은 다음과 같다.

인자	범위	역할
edge	(0, 0.5]	p_up 기반 Half-Kelly 기본값
vol_mult	[0.5, 1.0]	시장이 불안할수록 줄임
macro_gate	[0.55, 1.0]	시장 상태가 불확실할수록 줄임
novelty_gate	[0.55, 1.0]	본 적 없는 시장이면 줄임
er_mult	[0.5, 1.5]	신호 방향과 일치할 때 키움
macro_class_amp	[0.3, 2.0]	자산 클래스별 매크로 반응

인자 조합 효과 주의: 6개 인자가 동시에 낮은 값을 가지면 포지션이 예상보다 훨씬 작아진다. `KELLY_MIN=0.10`은 하한선 역할을 해서 포지션이 완전히 0에 가깝게 내려가는 상황을 막는다.

9.3 vol_mult — 변동성이 커질 때 포지션 줄이기

시장이 불안정해지면 포지션 크기를 자동으로 줄여야 한다. `hmm_vol_mult()`가 이 일을 맡는다. 핵심 아이디어는 **과거 가격의 표준편차 대신 HMM이 추정해 둔 레짐별 분산을 쓴다**는 것이다.

두 방식의 반응 속도가 왜 다른지가 이 절의 요점이다.

- **롤링 윈도우 변동성:** 과거 20~60개 봉의 가격 움직임을 평균한다. 새 급락은 다음 계산부터 바로 들어가지만, 이전 관측값들과 함께 평균되므로 변화가 완만할 수 있다.
- **HMM 레짐 분산:** HMM이 각 레짐마다 추정해 둔 변동성을 레짐 확률로 가중한다. `alpha_t`가 고변동 레짐으로 빠르게 이동하면 롤링 지표보다 민감하게 반응할 수 있지만, 상태 전이 확률과 피처에 따라 HMM도 늦게 반응할 수 있다.

`alpha_t`가 고변동 레짐 쪽으로 이동하면 다음 순서로 계산이 진행된다.

1. 레짐 가중 변동성(`sigma_hmm`)이 평균 변동성보다 커진다.
2. 변동성 비율이 1.0을 초과한다.
3. `vol_mult`가 1.0 미만으로 떨어진다.
4. 포지션 크기가 자동으로 줄어든다.

변동성 비율이 1.2~3일 때 배율을 $1.0 \cdot 0.75 \cdot 0.50$ 으로 두는 설명용 식은 `clip(1-0.25*(ratio-1), 0.25, 1)`이다. 이는 원리를 이해할 보간 예이며 운영 최적값을 뜻하지 않는다. `ratio`의 분자·분모는 같은 단위여야 하고, 기준 변동성이 0이거나 자료가 부족하면 포지션을 늘리지 않는다.

9.4 macro_gate — 시장 상태가 헛갈리면 줄이기

HMM은 현재 시장이 각 레짐에 속할 확률을 `alpha_t` 배열로 출력한다. 예를 들어 레짐이 4개라면 `alpha_t = [0.9, 0.05, 0.03, 0.02]` 같은 형태다. 숫자가 한쪽에 몰려 있으면 “지금 레짐이 뭔지 확실하다”는 뜻이고, 균등하게 퍼져 있으면 “지금 어느 레짐인지 모르겠다”는 뜻이다.

macro_gate는 이 확실성을 0.55~1.0 사이의 숫자로 바꿔, 신호를 얼마나 신뢰할지 결정한다. 확실한 레짐이면 신호를 그대로 사용하고, 레짐이 뒤섞여 있으면 신호를 줄인다. 확실성을 측정하는 데는 정보 이론의 섀넌 엔트로피(Shannon entropy)를 쓴다.

레짐 4개를 예로 들면 다음과 같다.

- 확률이 한 레짐에 100% 몰림 -> 게이트 = 1.0 (그대로 통과)
- 네 레짐에 25%씩 분산 -> 게이트 = 0.55 (최소치까지 축소)
- 전환 중(50/30/10/10) -> 게이트 $\approx$ 0.59 (살짝만 열림)

MACRO_GATE_MIN = 0.55로 뒤서 완전히 불확실한 상황에서도 신호의 55%는 유지한다. 0(완전 차단)을 쓰지 않는 이유는, 레짐이 바뀌는 순간 모든 포지션이 한꺼번에 청산되면 슬리피지와 비용이 급격히 커지기 때문이다.

9.5 novelty_gate —본 적 없는 시장이면 줄이기

HMM은 학습된 패턴을 기준으로 현재 시장을 분류한다. 그런데 타리프 쇼크 직후처럼 한 번도 본 적 없는 패턴이 들어오면, 어떤 레짐에도 잘 들어맞지 않아 전체 적합도 점수가 낮아진다. hmm_novelty_gate() 는 이 상황을 감지해 자동으로 포지션을 줄인다.

핵심 아이디어는 “지금 관측이 평소 대비 얼마나 낮선가”를 보는 것이다. 절댓값이 아니라 **최근 60세션의 분포 안에서 백분위 순위**로 본다. 평소보다 훨씬 낮설면 게이트가 작동해 포지션을 최소 0.55까지 줄인다.

과거 실행 예시—2020년 3월 COVID 구간: 특정 데이터 스냅샷과 설정으로 실행했을 때 2020년 3월 16~23일(8거래일)의 novelty_gate 평균은 0.61이었고 포지션은 약 40% 축소됐다. 같은 실행에서 S&P 500 구간 수익률이 약 -24%일 때 전략 드로우다운은 약 -11%였다. 이는 메커니즘을 확인하는 한 번의 역사적 사례이지, NOVELTY_GATE_MIN=0.55가 다른 위기에서도 같은 방어 성과나 반등 참여를 보장한다는 뜻은 아니다. 데이터 버전·비용·비교 기간을 고정한 OOS 재현이 필요하다.

이 게이트는 칼만 필터와 하는 일이 다르다.

- **칼만 필터:** 단일 종목의 bar 단위 가격 노이즈를 제거한다.
- **HMM novelty gate:** 7장에서 구성한 13차원 크로스자산 관측값이 학습된 레짐과 얼마나 일치하는지 측정한다.

새로운 패턴에 대한 경계는 최근 자료가 쌓이면서 달라질 수 있지만 “60세션이면 회복”을 보장하지 않는다. 예를 들어 학습 자료의 거리 분포에서 99백분위수를 초과한 관측은 진입을 보류하고, 재학습 후에도 별도 검증을 통과해야 해제한다. 경보 기준을 OOS 손실에 맞춰 즉시 완화하면 검증 구간을 학습에 사용한 샘플이 된다.

9.6 ER Amplifier —신호가 일치할 때 키우기

지금까지의 세 인자(vol_mult, macro_gate, novelty_gate)는 모두 **포지션을 줄이는 방향**으로 작동했다. ER amplifier는 반대로 **신호가 강하고 방향이 맞을 때 포지션을 키우는** 인자다.

ER(Expected Return, 기대수익률)은 HMM이 추정한 다음 세션의 기대 수익률이다. 각 레짐별 평균 수익을 현재 레짐 확률로 가중평균해 구한다.

규칙은 간단하다.

- 기대수익(er)이 포지션 방향(direction)과 같은 부호 -> 최대 1.5배까지 증폭
- 기대수익이 포지션 방향과 반대 부호 -> 0.5배로 반감

세 번째 경우가 핵심이다. p_up은 롱을 가리키는데 ER은 음수라면 두 신호가 충돌한 것이다. 이때는 포지션을 절반으로 줄인다. 모델 안에서 신호끼리 견제하는 안전장치 역할이다.

9.7 자산 클래스별 매크로 편향 조정

2-Layer Macro HMM이 산출하는 macro_bias는 -1에서 +1 사이의 값이다. -1이면 완전한 Risk-Off(위험 회피), +1이면 완전한 Risk-On(위험 선호)을 뜻한다. 이 값은 모든 자산에 똑같이 적용되지 않는다. 자산 클래스마다 반응 방식이 다르기 때문에, macro_class_amp라는 계수를 곱해 각 자산에 맞게 조정한다.

클래스	macro_class_amp	Risk-Off 국면에서의 효과
equity (주식)	+0.5	주식 롱을 강하게 축소
safe_haven (GLD, TLT)	-0.3	안전자산 롱을 확대 (부호 반전)
commodity	+0.3	원자재 노출 완만하게 축소

클래스	macro_class_amp	Risk-Off 국면에서의 효과
credit (HY)	+0.3	하이일드 노출 축소
fx	+0.2	약한 FX 민감도
vix-linked (VIXY)	-0.6	VIXY 롱을 확대 (부호 반전, 상품 한도 별도 적용)
em (신흥국)	+0.4	EM 주식 노출 축소

양수 부호의 자산은 매크로 편향과 같은 방향으로 움직이고, 음수 부호의 자산(safe_haven, vix)은 반대 방향으로 움직인다.

예시로 확인해 보자.

- **예시 1 —Risk-Off 국면에서 주식 롱:** 포지션 약 25% 축소.
- **예시 2 —Risk-Off 국면에서 VIXY 롱:** 포지션 약 30% 확대.

원리는 간단하다. 이 규칙에서는 Risk-Off 국면에 주식 롱을 줄이고 VIXY 롱을 늘리도록 설계했다. 다만 VIXY는 현물 VIX가 아니라 단기 VIX 선물에 노출된 상품이라 롤 비용과 장기 가치 하락 위험이 있다. 따라서 포지션 확대가 자동으로 포트폴리오 방어를 보장하지 않으며, 별도의 상품 한도와 스트레스 테스트가 필요하다. 정확한 공식과 클리핑 범위는 `calculate_kelly_size()` 본문에서 확인한다.

9.8 fractional_kelly() —5분봉용 단순 버전

5분봉 Ensemble 시스템에서는 6개의 게이트가 필요 없다. 단일 종목, 단기 신호이기 때문에 더 단순한 quarter-Kelly로 충분하다. `fractional_kelly()` 는 이산 베팅의 p , b 추정 대신 연속 신호의 평균(μ)과 분산(σ^2)을 그대로 사용한다.

```
# hmm_v5_core.py (excerpt)
def fractional_kelly(mu, sigma2, kelly_frac=0.25, max_size=1.0) ->
float:
    """f* = kelly_frac × |μ| / σ²
    분산이 크면(불확실하면) 작게, 신호가 강하면 크게.
    """
    if sigma2 <= 1e-12 or np.isnan(mu) or np.isnan(sigma2):
```

```

    return float(max_size)
k = kelly_frac * abs(mu) / (sigma2 + 1e-12)
return float(np.clip(k, 0.0, max_size))

```

핵심은 분자에 $|\mu|$ (신호의 크기), 분모에 σ^2 (불확실성)가 들어가는 것이다. 신호가 같아도 변동성이 두 배가 되면 포지션은 1/4로 줄어든다.

백테스트에서는 신호의 부호로 방향을, `fractional_kelly()` 출력으로 크기를 정한다.

9.9 두 가지 Kelly 구현 비교

항목	<code>calculate_kelly_size()</code> (v7.py)	<code>fractional_kelly()</code> (v5_core.py)
시스템	일봉/세션 크로스셋	5분봉 단일 종목
기본 공식	$\text{edge} = p_{\text{up}} - 0.5$	$f* = \text{kelly_frac} \times \mu / \sigma^2$
Kelly 비율	Half (0.5×)	Quarter (0.25×)
변동성 보정	HMM 레짐 분산	78봉 롤링 표준편차
추가 게이트	macro_gate, novelty_gate, ER amp	없음 (단순)
클리핑	[0.10, 1.0]	[0.0, 1.0]

`calculate_kelly_size()`는 여러 자산 간의 흐름, 시장 전체 분위기, 자산 클래스별 특성을 모두 반영한다. 고려하는 변수가 많은 만큼 구조도 복잡하다. `fractional_kelly()`는 단일 종목 단기 거래용으로 만들어졌고, 구조가 훨씬 단순하다. 복잡한 조건을 동시에 다룰 필요가 없는 상황에서는 단순한 쪽이 오히려 안정적으로 작동한다.

9.10 KELLY_MIN = 0.10으로 설정한 이유

신뢰도가 낮아도 최소 포지션은 유지한다. 포지션을 완전히 비우면 좋은 기회가 왔을 때 재진입 타이밍을 잡기 어렵기 때문이다.

```
KELLY_MIN = 0.10    # 최소 포지션 10%
KELLY_MAX = 1.0     # 최대 포지션 100%
```

KELLY_MIN = 0.10은 “신호가 있으면 최소 10%는 베팅한다”는 의미다. Kelly edge가 거의 0에 가까워도 함수는 0이 아닌 0.1을 반환한다.

왜 하필 10%인가? 이 시스템에는 포지션 크기를 줄이는 보정 인자가 여러 개 있다. 이 인자들이 모두 최솟값에 동시에 도달해도 포지션이 0이 되지 않도록 10%를 하한으로 정했다. 만약 0으로 낮추면, 시장 상황이 나쁜 구간에서 포지션이 완전히 사라져 레짐 전환 직후 반등 구간에 진입 기회를 놓친다.

단, $\text{edge} \leq 0 (p_{\text{up}} \leq 0.5)$ 일 때는 KELLY_MIN보다 먼저 0을 반환한다. “엣지 자체가 없는 경우”와 “엣지는 있는데 보정 때문에 줄어드는 경우”는 다르게 처리한다.

시나리오로 보기

머릿속에서 함수를 한번 굴러보자. 아래 네 시나리오를 넣으면 다음과 같은 경향이 나온다.

시나리오	p_up	vol_mult	macro_gate	novelty_gate	결과
강한 bull 레짐	0.72	0.9	0.95	1.0	큰 포지션
중립 레짐	0.56	0.7	0.70	0.9	KELLY_MIN 부근
고변동성 레짐	0.63	0.5	0.60	0.8	작은 포지션
이상치 감지	0.68	0.8	0.85	0.2	KELLY_MIN 까지 억제

마지막 줄이 `novelty_gate`의 역할을 잘 보여준다. `p_up`이 0.68로 꽤 강한 신호여도, 관측이 모든 레짐에서 낮설어 `novelty_gate = 0.2`가 되면 포지션은 최소치까지 내려간다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. $b = 1, p = 0.55$ 로 Full Kelly, Half Kelly, `fractional_kelly(factor=0.25)` 각각의 포지션 크기를 계산하라. 셋 중 어느 쪽이 가장 변동이 작을지 생각해보라.
2. `macro_gate`의 임계값을 낮추면 어떤 거래가 더 차단되거나 더 통과될지 예상해 보라.
3. `KELLY_MIN = 0.10`을 0으로 바꾸면, 어떤 시장 상황에서 전략이 포지션을 완전히 닫게 될지 시나리오를 그려보라.

정리

이 장에서 반드시 기억할 것은 다음 네 가지다.

- **포지션 사이징은 신호만큼 중요하다.** 같은 전략도 베팅 크기에 따라 결과가 완전히 달라진다.
- **풀 Kelly는 위험하다.** 추정 오차가 있는 현실에서는 Half-Kelly가 더 오래 살아남는다.
- **Half-Kelly에서 베팅 비율은 결국 edge 자체다.** 모든 보정 인자는 이 edge에 곱해진다.
- **6개의 보정 인자는“줄이는 인자”가 대부분이다.** 시장이 불안할수록, 레짐이 헛갈릴수록, 낯선 관측이 들어올수록 포지션은 자동으로 작아진다. `KELLY_MIN=0.10`은 그 모든 게이트가 닫혀도 완전히 0이 되지 않게 막는 하한이다.

Half Kelly 포지션 파이프라인을 한 줄로 요약하면:

`p_up -> edge = p_up - 0.5 -> 보정 인자 곱하기 -> [0.10, 1.0] 사이로 자르기`

다음 장(10장): HMM Forward Ensemble을 다룬다. `run_hmm_forward_ensemble_v2.py`와 `v3.py`의 구조를 분석하고, Walk-forward 방식으로 여러 서브모델을 비교하는 파이프라인을 설명한다.

더 읽을거리

- **논문:** Kelly, J. L. (1956), “A New Interpretation of Information Rate”—원조 Kelly 공식 논문
- **책:** Thorp, Beat the Dealer, The Mathematics of Gambling —실전 Kelly 적용 사례
- **책:** Lopez de Prado, Machine Learning for Asset Managers Chapter 6 —포지션 사이징과 신호 강도 연계

10장. HMM 포워드 앙상블

—V1 -> V3 진화

복잡한 시스템의 첫 버전은 대부분 부족하다. 퀀트 시스템도 다르지 않다. 중요한 것은 어디서 왜 실패했는지 파악하고, 다음 버전에서 무엇을 고칠지 정하는 일이다.

V1 앙상블은 기본 구조는 작동했지만 실전 운용을 맡기기에는 약했다. V2에서는 Wasserstein-distance 보팅을 추가했고, V3에서는 여러 시간 단위 신호를 더했다. 중요한 점은 숫자 자체가 아니라, 버전이 올라갈 때마다 어떤 실패를 줄이려 했는지다.

이 장은 그 반복 과정을 버전별로 남긴 기록이다. 결과 숫자만 보지 말고, 어떤 실패 가설이 다음 구조 변경으로 이어졌는지를 함께 보자. AI는 각 사이클을 빠르게 돌리는 데 도움이 됐다. 예측 결과가 어디서 가장 많이 틀리는지를 AI에 물으면, 레짐(시장 상태) 전환 구간에서 서브모델들의 의견이 가장 많이 갈린다는 진단이 돌아왔다. 그 피드백이 V3 개선 방향의 출발점이 됐다.

왜 앙상블인가

단일 HMM은 시장의 한 측면만 본다. 추세를 보는 모델은 횡보장에서 약하고, 변동성을 보는 모델은 추세장에서 신호가 흐릿하다. 어떤 종류의 시장이 올지 미리 알 수 없으니, 서로 다른 정보를 보는 여러 모델을 만들어 그 의견을 합치는 편이 안전하다.

비유하자면 한 명의 전문가에게 모든 판단을 맡기는 대신, 분야가 다른 여러 전문가에게 의견을 듣고 평균을 내는 것과 같다. 평소에는 의견이 비슷하지만 결정적인 순간(레짐 전환) 한두 명이 다른 신호를 보낸다. 그 때 다수결 또는 가중 평균이 도움이 된다.

이 책에서 이 장의 위치는 분명하다. 9장에서 포지션 크기 결정 원리를 봤고, 10장에서는 그 크기를 정하기 전에 필요한 **신호 생성**을 다룬다. 11장부터는 종목 페어를 발굴해 이 신호 체계를 확장한다.

9장 복습: Kelly Sizing은 Ensemble이 계산한 기대 수익률 신호($E[r]$)를 받아서 포지션 크기를 결정한다. 이 장에서는 그 신호를 어떻게 만드는지 살펴본다.

이 책 전체에서 앙상블은 **여러 모델의 의견을 한 숫자로 정리하는 회의실**이다. HMM 모델 하나가 모든 시장을 잘 읽을 수는 없다. 어떤 모델은 추세에 민감하고, 어떤 모델은 변동성 변화에 민감하고, 어떤 모델은 섹터 흐름을 더 잘 본다. 앙상블은 이 의견들을 모아 최종 기대수익률 $E[r]$ 을 만든다.

```

모델 A: 내일 +0.15% 가능성
모델 B: 내일 -0.05% 가능성
모델 C: 내일 +0.20% 가능성
    |
    v
앙상블
성과가 좋았던 모델에 조금 더 무게를 주고 합침
    |
    v
최종 신호  $E[r]$ 
예: +0.10%
    |
    v
Kelly / Risk
포지션 크기와 주문 가능 여부 결정
  
```

초보자는 앙상블을 “모델을 많이 넣으면 무조건 좋아지는 방법”으로 보면 안 된다. 모델이 많아지면 의견도 많아지고 노이즈도 많아진다. 그래서 이 장의 핵심은 모델 개수를 늘리는 것이 아니라, **서로 다른 실패를 하는 모델들을 어떻게 조합할지**다.

좋은 앙상블:

서로 다른 정보를 보는 모델들이
비슷한 방향을 말할 때 더 신뢰한다.

나쁜 앙상블:

비슷한 모델을 많이 복사해서
같은 착각을 여러 번 세는 구조가 된다.

이 장의 흐름

이 장은 큰 그림에서 시작해 한 단계씩 안으로 들어간다.

- 먼저 V1/V2/V3의 차이를 한눈에 본다 (10.1)
- 세 버전이 공유하는 핵심 계산—Forward α 로 $E[r]$ 만들기 (10.2)
- 버전마다 어떤 종류의 정보를 더했는가 (10.3)
- Optuna 임계값 재최적화가 빠른 이유 (10.4)
- 성과 비교와 적응형 임계값 (10.5, 10.6)

학습 목표

이 장을 마치면 다음을 할 수 있다.

- V1/V2/V3의 핵심 차이를 설명하고, 각 버전이 해결한 문제를 짚을 수 있다
- Forward α 를 써서 다음 날 기대 수익률 $E[r_{t+1}]$ 을 어떻게 만드는지 그림으로 그릴 수 있다
- Optuna 임계값 재최적화가 왜 빠르지 한 문장으로 설명할 수 있다
- 동적 가중치와 멀티 타임프레임이 앙상블에 어떤 역할을 하는지 이해한다

첫 회독에서 볼 것

이 장에는 서브모델, Forward α , 전이행렬, Optuna, 동적 가중치처럼 어려운 단어가 많이 나온다. 처음 읽을 때는 다음 흐름만 잡으면 된다.

1. 여러 모델이 각자 “내일 수익률이 어느 쪽일 것 같다”는 의견을 낸다.
2. 성과가 좋았던 모델의 의견을 조금 더 크게 반영한다.
3. 최종적으로 $E[r]$ 이라는 숫자 하나를 만든다.
4. 다음 장들은 이 숫자를 포지션 크기와 실제 매매 조건으로 바꾼다.

수식 표기와 Softmax 세부 계산은 두 번째 회독에서 봐도 된다. 첫 회독의 목표는 “**앙상블은 여러 모델의 의견을 한 숫자로 합치는 장치**”라고 이해하는 것이다.

TIP

바이브코딩 미션

- **언을 결과**: 서브모델별 신호와 최종 가중합 한 줄을 확인한다.
- **첫 회독**: V1~V3의 모든 차이보다 여러 모델이 같은 실수를 동시에 하는지 본다.
- **용어 발판**: 앙상블은 여러 모델의 의견을 합치는 방식, 가중치는 각 의견의 반영 비율, 가중합은 비율을 곱해 더한 결과다.
- **AI 요청**: “앙상블 결합 함수에서 모델별 값, 가중치, 최종값을 로그로 확인할 가장 작은 방법을 제안해줘. 먼저 읽기 전용으로 호출 경로를 설명해줘.”
- **완료 확인**: 가중치 합과 최종 출력 계산을 재현하면 성공이다.

10.1 아키텍처 비교—V1, V2, V3

항목	V1	V2	V3
종목	SPY 1개	5개 (UNIVERSE)	10개
서브모델 수	5개 (M1~M5)	7개 (M1~M7)	15개 (M1~M9, 멀티TF)
가중치	고정 (equal)	고정 (equal)	동적 (롤링 성과 기반 softmax)
비-HMM 모델	없음	없음	Kalman(M8) + MeanRev(M9)
Optuna	없음	있음 (re-thresholding)	있음
병렬 처리	없음	없음	ThreadPoolExecutor(4 스레드)

V1->V2->V3는 세 가지 축으로 진화했다. 서브모델 종류를 늘렸고, 가중치를 상황에 따라 움직이게 만들었고, 종목 수를 늘렸다.

V2의 핵심 변화는 Optuna 도입이다. Optuna는 모델의 최적 임계값을 자동으로 찾아주는 라이브러리다. V3에서는 신호를 여러 시간 단위로 나누어 보는 멀티 타임프레임 방식을 추가했다.

이 순서에는 이유가 있다. 종목을 먼저 늘리면 노이즈(의미 없는 신호)도 같이 늘어난다. 모델 구조와 임계값이 안정적으로 동작하는지 확인한 뒤에 종목 수를 늘리는 편이 안전하다.

10.2 V1/V2 공통 코어—Forward α 로 $E[r]$ 계산하기

V1과 V2는 핵심 계산을 공유한다. 버전이 달라져도 이 부분은 바뀌지 않는다.

흐름을 한 문장으로 요약하면 이렇다. **지금까지 본 데이터로 현재 레짐 확률을 갱신하고, 그 확률을 한 칸 앞으로 밀어 내일 수익률 기대값을 만든다.**

레짐(regime)은 현재 시장 상태를 나눈 범주다. ‘상승 추세’, ‘횡보’, ‘급락’같은 구분이 여기에 해당한다. 그때그때 갱신되는 확률 벡터를 **Forward α** 라고 부른다. 예를 들어 레짐이 3개일 때 오늘의 Forward α 가 $[0.7, 0.2, 0.1]$ 이라면 “오늘은 70% 확률로 상승 추세, 20% 횡보, 10% 급락 상태로 보인다”는 뜻이다.

여기서 두 가지 재료가 더 필요하다.

- **전이행렬(transition matrix) A**: 레짐이 다음 스텝에서 어디로 옮겨갈 확률을 모은 표다. 예를 들어 $A[\text{Bull}, \text{Bear}] = 0.05$ 라면 “오늘 Bull 상태일 때 내일 Bear로 바뀔 확률은 5%”라는 뜻이다. 이 표는 HMM 학습 단계에서 과거 데이터로부터 Baum-Welch 알고리즘으로 자동 추정된다(자세한 건 8장 참고).
- **레짐별 평균 수익률 μ_{ret}** : 학습 시 각 레짐에서 관측된 수익률의 평균이다. 학습이 끝난 뒤 `m.means_`에 고정되어 저장되며, 백테스트 중에는 절대 갱신하지 않는다(왜 그런지는 10.2.1).

작은 숫자로 보면 더 쉽다. 오늘 레짐 확률이 $[\text{Bull } 70\%, \text{Neutral } 20\%, \text{Bear } 10\%]$ 라고 하자. 전이행렬을 적용하면 내일 확률이 $[\text{Bull } 65\%, \text{Neutral } 25\%, \text{Bear } 10\%]$ 처럼 조금 움직인다. 각 레짐의 평균 수익률이 $[+0.2\%, 0.0\%, -0.3\%]$ 라면, 내일 기대 수익률은 대략 $0.65 \times 0.2\% + 0.25 \times 0.0\% - 0.10 \times 0.3\% = +0.10\%$ 가 된다. 이 $+0.10\%$ 가 바로 $E[r]$ 이다.

핵심 계산은 두 줄로 줄어든다.

- **Step 4 (예측)**: $\text{next_proba} = \alpha_t \times A$ —오늘 레짐 확률을 한 칸 미래로 민다.
- **Step 5 (E{return})**: $E[r_{t+1}] = \text{next_proba} \cdot \mu_{\text{ret}}$ —그 확률에 레짐별 평균 수익률을 곱해 합한다.

이 두 줄이 모든 버전의 심장이다. 다른 단계(초기화, 관측 확률 갱신, 앙상블 평균, 임계값으로 $\pm 1/0$ 신호 만들기)는 이 두 줄을 떠받치는 보조 역할이다. 전체 의사코드(Step 1~7)는 부록 A에 정리했다.

출력 예시:

2024-01-03 -> e_ret = 0.0034 (Bull 레짐 0.87)

2024-01-04 -> e_ret = -0.0012 (전환 국면)

2024-01-05 -> e_ret = 0.0021 (Bull 복귀)

매일 새 봉(bar, 하루치 시세)이 들어올 때마다 위의 두 줄을 돌려 다음 날 e_ret 한 숫자를 얻는다고 기억하면 된다.

10.2.1 인과성(causality) 함정 피하기

레짐별 기대 수익을 계산할 때 자주 발생하는 함정이 있다. **미래 데이터가 과거 계산에 흘러들어가는 문제**다. 이것 인과성 위반이라고 부른다. 실전에서는 미래를 볼 수 없으니, 이 오류가 있으면 백테스트 결과가 실제보다 훨씬 좋게 나온다.

기억해야 할 규칙은 한 줄이다.

- **사용:** 학습이 끝난 후 고정된 `m.means_` (레짐별 평균이 학습 시점에 동결됨)
- **금지:** `m.predict()` —Viterbi 평활화로 미래까지 보고 상태를 정함

아래는 실제 코드에서 이 규칙이 적용되는 핵심 한 줄이다.

```
# run_hmm_forward_ensemble_v2.py 발췌
srm = sc.inverse_transform(m.means_[:, idx] # causal
# m.predict(Xs) # look-ahead bias
```

실패하면 볼 것: 백테스트 성과가 비현실적으로 높게 나온다면(예: 3 이상) 이 부분에 미래 데이터가 섞이고 있을 가능성이 크다.

10.3 버전마다 어떤 정보를 더했는가

서브모델을 추가할 때는 입력과 역할을 먼저 분리한다. 수익률 모델은 방향, 변동성 모델은 위험 크기, 평균회귀 모델은 과거 평균과의 이탈을 본다. 각 출력은 같은 보유 기간의 기대 수익률처럼 공통 단위로 변환한 뒤 합친다. 표준화 점수와 수익률을 그대로 평균하면 숫자의 크기만으로 가중치가 결정된다.

V2에서 추가—섹터·주문 흐름

- **M6_Sector**: 섹터 로테이션을 본다. 경기 흐름에 따라 돈이 몰리는 업종이 바뀌는 현상을 잡는다. 시장 전체 대비 종목의 초과 수익, 같은 섹터 대비 종목의 초과 수익, 섹터 ETF들의 전반적인 강세 여부 등을 묶어 3개 레짐(섹터 강세/약세/중립)을 학습한다.
- **M7_OrderFlow**: 오더플로우(매수자와 매도자 중 누가 거래를 주도했는가)를 본다. 호가창이 없어도 일봉의 OHLCV(시가·고가·저가·종가·거래량)만으로 추정할 수 있다. 핵심 아이디어는 “장중 저점 대비 종가의 위치가 높을수록 매수세가 강했다”이다.

V3에서 추가—멀티 타임프레임 & 비-HMM

V3는 두 가지를 더했다.

멀티 타임프레임. 기존 서브모델 일부(M1·M2·M3·M5)를 단기(1d), 중기(5d), 장기(21d) 세 시간 단위로 복제했다. 단기 레짐은 하루 단위로 빠르게 바뀌는 신호를, 장기 레짐은 한 달 이상 이어지는 추세를 잡는다. 두 타임프레임이 같은 방향을 가리키면 신호가 더 믿을 만하다고 판단한다.

같은 모델을 단순히 더 많이 복제하는 것과는 다르다. 1d와 21d는 서로 다른 종류의 정보(빠른 흔들림 vs. 천천히 가는 추세)를 본다. 정보가 겹치지 않을수록 동적 가중치(아래)가 “지금 어느 시간 단위가 더 잘 맞고 있는지”에 맞춰 비중을 옮길 수 있어 시너지가 크다.

비-HMM 두 모델. HMM이 아닌 다른 통계 도구를 끌어왔다.

- **M8_Kalman**: 가격에 섞인 잡음을 건어낸 부드러운 “추세 추정값”을 만들어, 그 값을 그대로 다음 날의 기대 수익률로 쓴다. (도구 이름이 Kalman 필터다.)
- **M9_MeanRev**: 가격이 한쪽으로 가도 곧 평균으로 되돌아오려는 성질을 본다. 평균 대비 너무 멀리 벗어나 있고 그 성질이 강하게 잡힐 때만, 반대 방향 신호를 낸다.

멀티 타임프레임은 이렇게 읽으면 된다.

상황	해석
1d, 5d, 21d가 모두 상승	단기와 장기가 같은 방향이라 신호 신뢰도가 높다
1d만 상승, 21d는 하락	단기 반등일 수 있으므로 가중치를 낮춘다
1d는 하락, 21d는 상승	장기 추세 속 일시 조정일 수 있다

칼만 필터의 수식 유도, 분산비·z-score 같은 통계 도구의 자세한 정의, 그리고 노이즈 분산 같은 세부 하이퍼파라미터는 부록에 정리했다.

V3 동적 가중치—성과 좋은 서브모델에 더 큰 비중

V2까지는 모든 서브모델의 가중치가 같았다. V3는 서브모델별 성과를 매일 추적하고, **최근 성과가 좋은 서브모델에 더 큰 가중치를 준다**.

흐름은 입력 -> 계산 -> 출력 한 줄로 보면 더 간단하다.

입력	계산	출력
각 서브모델의 최근 일별 (예측 방향 × 실제 수익률)	롤링 성과를 구하고 Softmax로 정규화	합이 1이 되는 가중치 벡터 w_k

성과 점수 s 를 가중치로 바꾸는 예는 $w_i = \exp((s_i - \max(s))/T) / \sum_j \exp((s_j - \max(s))/T)$ 다. T 가 클수록 균등해지고 작을수록 한 모델에 집중한다. T 는 양수여야 한다. 점수는 이미 결과가 확정된 과거 예측만으로 계산하고, 최소 관측 수 미달 모델은 제외하거나 사전 정의한 균등 가중치를 쓴다. 예를 들어 점수가 모두 같으면 모델 3개의 가중치는 각각 1/3이다.

다음 스텝의 앙상블 신호는 $E[r]_{ens} = \sum_k w_k \cdot E[r]_k$ 로 합산한다.

예를 들어 어제 가중치가 $[0.30, 0.25, 0.20, 0.15, 0.10]$ 였는데 첫 번째 모델이 계속 잘 맞았다면, 오늘은 $[0.40, 0.20, 0.20, 0.10, 0.10]$ 처럼 바뀔 수 있다. 정확한 계산식보다 중요한 것은 **성과가 좋은 모델의 목소리가 커진다는 점**이다.

10.4 V2 핵심 아이디어—Optuna 임계값 재최적화

V2에서 가장 중요한 변화는 하나다. **Walk-Forward**를 처음부터 다시 돌리지 않고, 임계값만 새로 조정한다.

여기서 임계값이란“예상 수익이 이 수준을 넘으면 매수, 이 수준 아래로 내려가면 매도”처럼 신호를 거르는 기준값이다.

왜 빠른가? HMM을 매번 다시 학습·추론하는 일은 시간이 많이 걸린다. 그런데 이미 계산해 둔 $E[\text{return}]$ 시계열이 있다면, 임계값만 바뀌가며 실험하는 일은 훨씬 빠르게 반복할 수 있다. 모델은 그대로 두고 기준선만 옮겨보는 셈이다.

대략적인 계산량 차이는 이렇다.

- **Walk-Forward 재실행**: 데이터 전체에서 모델을 반복 학습 -> 매우 느림
- **Re-thresholding**: 저장해 둔 $E[r]$ 에 임계값만 적용 -> 훨씬 빠름

여기서 한 가지 주의할 점이 있다. 모델 학습에 쓰지 않았더라도 임계값을 찾는 데 사용한 순간 그 구간은 더 이상 최종 OOS 테스트가 아니다. 이 책은 모델 학습 뒤의 시간 구간을 다시 앞 40% **튜닝 검증 구간**과 뒤 60% **건드리지 않은 테스트 구간**으로 나눈다. 뒤 60% 결과를 본 뒤 다시 임계값을 고치면 그 구간도 오염되므로, 다음 Walk-forward 창이나 별도 최종 홀드아웃에서 다시 확인한다.

이 재활용이 가능하려면 walk-forward 단계에서 매일의 e_ret 을 미리 저장해 두어야 한다.

10.5 V2/V3 성과 비교 요약

아래는 세 버전의 역할 차이다. OOS(Out-of-Sample)는 모델 훈련에 쓰지 않은 데이터로 측정한 구간을 말한다.

버전	서브모델	유니버스	특이사항
V1	5개 HMM	SPY 1개	기본 구조 검증
V2	7개 HMM	5개 종목	Optuna re-threshold 효과 확인
V3	15개 (HMM+이종)	10개 종목	동적 성과 기반 가중치 효과 확인

실전 투자에는 복잡한 V3가 항상 답은 아니다. V2는 구성요소가 적어 추적과 장애 진단이 쉽지만, 단순하다는 이유만으로 성과나 안정성이 더 좋다고 단정할 수는 없다. 같은 OOS 구간·비용에서 V2와 V3를 비교한다.

10.6 적응형 임계값— $\sigma \times \text{scale}$

임계값은 고정하지 않고, 최근 $E[r]$ 분포에 맞춰 동적으로 조정한다. 핵심 아이디어는 한 줄이다. **임계값을 최근 $E[r]$ 의 표준편차에 비례시킨다.**

그러면 다음과 같이 동작한다.

- **시장이 조용할 때:** 표준편차가 작으니 임계값도 낮다. 작은 신호에도 진입한다.
- **시장이 혼란스러울 때:** 표준편차가 크니 임계값도 높다. 강한 신호일 때만 진입한다.

비례 상수(thresh_scale)는 사람이 정하지 않고 Optuna가 자동으로 찾는다. 이 자기 보정(self-calibrating) 구조 덕분에 시장 레짐이 바뀌어도 거래 빈도가 크게 흔들리지 않는다. 실제 임계값 계산 코드와 신호 생성 한 줄은 아래“따라 해보기”에서 직접 다뤄본다.

따라 해보기—Optuna로 임계값 최적화 (선택)

이 절은 첫 회독에서 건너뛰어도 좋다. 한 번 읽고 흐름을 잡은 뒤, 두 번째 회독 때 손으로 따라가 보는 것을 권한다.

아래는 미리 계산된 e_ret 에 대해 임계값을 탐색하는 합성 예다. 세 모델의 예측이 $0.01 \cdot 0 \sim 0.01$ 이고 가중치가 $0.5 \cdot 0.3 \cdot 0.2$ 라면 합친 예측은 0.003 이다. 임계값 0.002 이면 후보 신호가 되고, 비용과 위험 검사를 거쳐야 거래로 이어진다. 이 예제는 서브모델을 학습하는 전체 운영 파이프라인이 아니다.

아래 코드는 thresh_scale을 $0.2 \sim 2.0$ 범위에서 탐색하면서 OOS 성과를 최대화하는 값을 찾는다. 핵심은 두 줄이다. 임계값을 e_ret 의 표준편차 $\times$ scale로 정하고, 그 임계값으로 $\pm 1/\sigma$ 신호를 만든다.

```
import numpy as np
import optuna

def optimize_threshold(e_ret, actual_ret, n_trials=50):
```

```
def objective(trial):
    scale = trial.suggest_float("scale", 0.2, 2.0)
    threshold = float(e_ret.std()) * scale          # <- 적응형 임계값
    signals = np.where(e_ret > threshold, 1,
                       np.where(e_ret < -threshold, -1, 0)) # <- 신호 만들기
    pnl = signals * actual_ret
    if (signals != 0).sum() < 5:
        return -999.0
    return evaluate_objective(pnl)

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=n_trials)
return study.best_params["scale"], study.best_value
```

확인할 출력: thresh_scale은 보통 중간 범위에서 선택되고, 합성 데이터에서는 성과가 0 근처에서 진동한다. 실제 데이터에서는 숫자 자체보다 IS/OOS 차이가 과하게 벌어지지 않는지를 본다.

실패하면 볼 것: -999.0이 계속 나오면 임계값이 너무 커서 신호가 거의 발생하지 않는 상태다. e_ret의 분포(np.std, 히스토그램)부터 확인하라.

연습 문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. 두 서브모델 M1, M2가 반대 방향 신호를 낸다고 하자. 예를 들어 어떤 날 M1의 $E[r] = +0.003$, M2의 $E[r] = -0.002$ 이고 두 모델 가중치가 같다($w = 0.5$). 앙상블 합계 $E[r]_{\text{ens}}$ 는 얼마이고, 임계값이 ± 0.001 일 때 최종 신호(+1/-1/0)는 무엇이 되는가?
2. 새 서브모델 M10_Earnings(실적 발표 전후 3일 이벤트 패턴)를 추가하려면 어떤 피처가 필요하고, 어떤 데이터 소스를 써야 할지 적어보라.
3. (사고 실험) 서브모델 8개 중 두 개를 꺼야 한다면, 어떤 종류의 서브모델을 끄는 것이 성과에 큰 타격을 줄지 추측해 보라. 힌트: 다른 서브모델과 신호 종류가 겹치지 않는 모델은 끄면 손실이 크고, 비슷한 신호를 보는 모델은 하나 꺼도 영향이 작다.

정리

이 장에서는 **여러 신호를 하나로 합치는 앙상블 방식**을 살펴봤다. 단일 HMM으로는 잡기 어려운 섹터 로테이션 신호나 주문 흐름 신호를, 별도의 서브모델로 만들어 하나의 예측 신호로 묶은 것이 핵심이다.

버전	핵심 기여	남은 문제
V1	Forward-only α 기반 $E[r]$ 파이프라인	SPY 단일, 고정 가중치
V2	M6_Sector, M7_OrderFlow + Optuna re-thresholding	동적 가중치 없음
V3	멀티 TF + M8_Kalman + M9_MeanRev + 성과 기반 softmax	—

이 장에서 반드시 기억할 것

- 인과성을 지키자:** 레짐별 평균 수익률은 학습이 끝난 `m.means_`만 쓴다. `m.predict()`는 미래를 보기 때문에 백테스트를 망친다.
- 한 스텝 앞 예측:** (현재 레짐 확률 $\times$ 전이행렬) $\cdot$ 레짐별 평균 수익률—이 한 줄이 모든 버전의 핵심이다.
- 임계값은 고정하지 않는다:** 최근 $E[r]$ 의 표준편차에 비례시키고, Optuna로 그 비율(scale)을 자동 조정한다. 그래야 시장 상황이 바뀌어도 거래 빈도가 안정적이다.

다음 장(11장): 통계적 차익거래(Stat Arb) 페어를 고르는 방법을 다룬다. Engle-Granger 공적분 검정으로 함께 움직이는 종목 쌍을 찾고, Kalman 필터로 두 종목 사이의 스프레드와 헤지 비율을 동적으로 계산한다.

더 읽을거리

- 논문:** Sharpe (1994), “The Sharpe Ratio”—샤프 비율의 의미와 한계를 정리한 글
- 논문:** “Stacked Generalization” (Wolpert, 1992)—Ensemble 학습의 이론적 기반
- 책:** Grinold & Kahn, Active Portfolio Management—Ensemble 알파를 포트폴리오에 녹이는 방법

11장. 통계적 차익거래—페어 선정

Part V —페어 트레이딩 기본형 (11~12장) 이 파트에서는 “한 종목이 오를까”가 아니라 “두 종목의 관계가 다시 좁혀질까”를 본다. 11장은 거래할 만한 페어를 고르고, 12장은 진입과 청산 신호를 만든다. 첫 회독에서는 공적분 검정의 세부보다 “상관관계만으로는 부족하다”는 점을 기억하면 된다.

차트를 보고 거래하는 트레이더는 보통 이런 질문을 한다. “이 종목이 오를까, 내릴까?” 오를 것 같으면 사고, 내릴 것 같으면 판다. 일반 투자자 대부분이 쓰는 방식이다.

통계적 차익거래(statistical arbitrage)는 다른 질문을 한다. “이 두 자산 사이에 안정적인 관계가 있는가?” 그리고 “지금 그 관계가 일시적으로 어긋났다가 다시 돌아올 가능성이 있는가?” 방향보다 관계에 집중하는 것이다. 예를 들어, 코카콜라(KO)가 오를지 펄스(PEP)가 내릴지를 맞히는 게 아니라, 두 종목의 가격 차이가 평소보다 크게 벌어졌다면 다시 좁혀질 것이라는 데 베팅한다.

이 방식을 **시장 중립 전략**이라고 부른다. 시장 전체가 오르든 내리든 상관없이, 두 자산 사이의 관계가 수렴하는 것에만 베팅하기 때문이다. 손실이 나는 상황은 주로 두 자산의 관계 자체가 깨질 때다. 그래서 우리가 찾는 엣지(우위)도 방향 예측보다 측정하기 쉽다.

개인 투자자가 페어 트레이딩을 시도할 때 자주 저지르는 실수가 있다. “코카콜라와 펩시는 같이 움직이니가 좋은 페어겠지”라는 생각이다. 직관처럼 들리지만 검증이 아니다. 두 종목이 함께 오르는 것처럼 보여도, 그냥 시장 전체가 오르는 것일 수 있다. **상관관계(correlation)**는 두 자산이 비슷한 방향으로 움직이는 정도이고, **공적분(cointegration)**은 두 자산 사이에 장기적으로 안정적인 관계가 존재한다는 의미다. 둘은 다른 개념이고, 우리에게 필요한 건 후자다.

정량적으로 페어를 고르려면 세 가지가 필요하다. 수많은 페어에 공적분 검정을 돌리고, 우연히 통과한 것들을 걸러내고, 마지막으로 “왜 이 두 회사가 함께 움직여야 하는가?”라는 경제적 논리로 한 번 더 거른다. 단순히 p-value가 낮다고 좋은 페어가 아니다. 1,000개를 검정하면 아무 관계가 없어도 우연히 약 50개가 통과한다(다중 비교 문제).

이 책 전체의 머신러닝 흐름에서 11장은 **거래 후보를 만드는 장**이다. HMM이 시장 상태를 읽고, RL이 포지션 크기를 정하려면 먼저 “무엇을 거래할지”가 있어야 한다. 페어 선정은 그 후보 목록을 만든다.

```

시장 전체 종목
수백 개 후보
    |
    v
섹터 필터
경제적으로 관계 있을 만한 종목만 남김
    |
    v
공적분 검정
스프레드가 평균으로 돌아오는 성질이 있는지 확인
    |
    v
반감기 / 품질 필터
실제로 거래할 만큼 빨리 돌아오는지 확인
    |
    v
거래 후보 페어
12장에서 진입/청산 신호를 만든다
  
```

이 흐름에서 확인할 것: 통계 검정 하나로 페어를 확정하지 않는다. 경제적 관계, 다중 비교 보정, 회귀 속도까지 차례로 통과한 후보만 12장의 신호 생성 단계로 넘긴다.

초보자는 페어 선정을 “좋아 보이는 두 종목 찾기”로 생각하면 안 된다. 더 정확히는 **관계가 깨졌을 때 손실이 커질 후보를 미리 버리는 작업**이다. 좋은 페어는 단순히 같이 움직이는 두 종목이 아니라, 벌어진 차이가 다시 좁혀질 통계적·경제적 이유가 있는 두 종목이다.

상관관계:

둘이 같은 방향으로 움직이는가?

공적분:

둘 사이의 차이가 너무 벌어지면 다시 돌아오는가?

페어 트레이딩에 필요한 것은 공적분이다.

이 장의 흐름

페어 트레이딩의 직관을 먼저 잡고, 그다음 실제로 페어를 고르는 기준으로 내려간다.

- 왜 섹터 안에서 짝을 고를까?
- 페어 선정 파이프라인
- 반감기(half-life) 계산
- Kalman 필터로 동적 헤지 비율 구하기
- 페어 품질 지표

학습 목표

이 장이 끝나면 다음을 할 수 있다.

- 같은 섹터 안에서 종목 쌍을 골라 공적분 검정으로 거를 수 있다.
- 스프레드의 **반감기(half-life)**가 무엇이고, 왜 페어 품질의 지표가 되는지 설명할 수 있다.
- **Kalman 필터**가 페어 선정에서 어떤 역할을 하는지(고정 비율 대신 시간에 따라 변하는 헤지 비율) 설명할 수 있다.
- `_RLConfig`의 핵심 파라미터(`z_entry`, `corr_min`, `hl_max`)의 의미를 안다.

TIP

바이브코딩 미션

- **연을 결과:** 후보 종목에서 테스트용 페어 한 쌍과 선정 근거를 출력한다.
- **첫 회독:** 검정 수식보다 상관관계와 공적분이 다른 질문이라는 점을 잡는다.

- **용어 발판:** 상관관계는 같이 움직인 정도, 공적분은 벌어진 관계가 장기적으로 돌아오는 성질, p-value는 관계가 없다는 가정 아래 지금만큼 극단적인 결과가 나올 가능성을 나타내는 보조값이다.
- **AI 요청:** “페어 선정 파이프라인을 작은 종목 목록으로 실행할 진입점을 찾아줘. 결과에 상관계수, 공적분 p-value, 반감기를 함께 표시해줘.”
- **완료 확인:** 단순히 같이 오르는 종목이 아니라 관계가 되돌아오는 후보를 구분하면 성공이다.

11.1 왜 섹터 안에서 짝을 고를까?

Stat Arb(통계적 차익거래)는 두 종목이 비슷한 외부 요인의 영향을 받는다고 가정한다. 같은 업종의 두 회사는 금리 변화나 원자재 가격에 비슷하게 반응한다.

시장 전체에서 아무 두 종목이나 짝을 지으면 쌍이 수천 개 나온다. 통계 검정을 수천 번 반복하면 실제 관계가 없어도 우연히 $p < 0.05$ 를 넘기는 쌍이 약 5%씩 섞인다. 이를 **다중 검정 문제**라 한다.

섹터 안에서만 후보를 추리면 두 가지 효과가 있다.

- 검정 횟수가 줄어 가짜 양성도 줄어든다.
- 같은 섹터 기업은 경제적으로 연결되어 있어, “왜 함께 움직이는가?”를 설명할 수 있다.

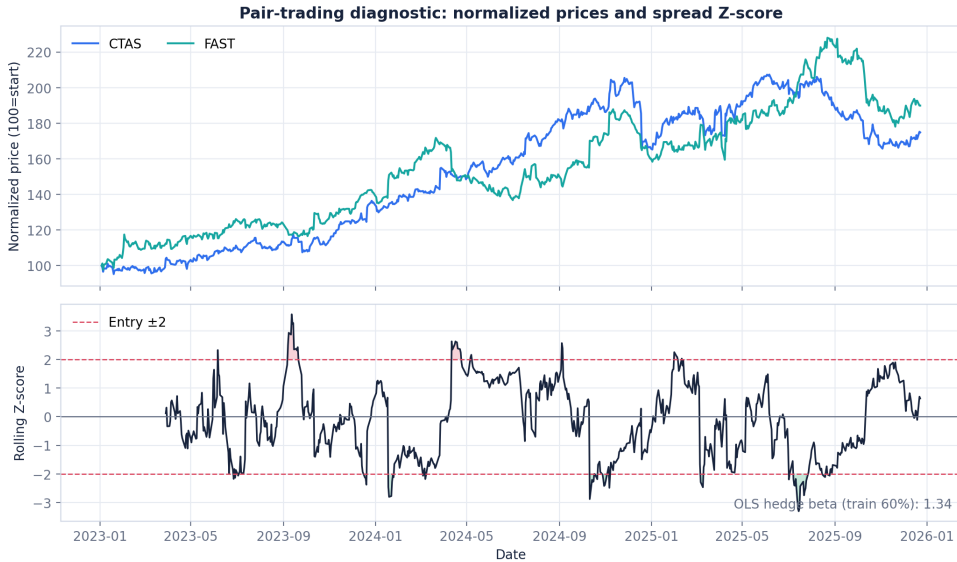
예를 들면 다음과 같다.

- XLK(기술 섹터) 안의 AAPL-MSFT: 대형 기술주로서 금리·경기·시장 심리를 일부 공유
- XLF(금융 섹터) 안의 JPM-BAC: 금리 변동에 비슷한 반응

공적분이란? 두 시계열이 각각은 불규칙하게 움직이지만 그 차이는 일정 범위 안에서 유지되는 관계다. 차이가 일시적으로 벌어지면 다시 좁혀지는 성질이라, 그 순간에 거래할 수 있다.

구현에서는 전체 종목을 한꺼번에 비교하지 않고, 미리 정의한 `SECTOR_SYMBOLS` 딕셔너리로 섹터 11개로 나눠 같은 섹터 안에서만 짝을 만든다. 코드 위치는 `app/trading/strategy/signals/mean_reversion_signal.py`이다.

위쪽 가격선이 비슷하게 움직인다는 사실만으로 페어가 완성되지는 않는다. 아래쪽 스프레드가 평균 주변으로 되돌아오는지, ± 2 같은 경계를 넘은 뒤 복귀하는지, 그 관계가 검증 구간에서도 유지되는지를 함께 확인해야 한다.



Source: local cached daily OHLCV through 2025-12-23 · Educational reproduction · Not investment advice

그림 4. 로컬에 저장된 CTAS와 FAST 일봉을 100으로 정규화한 가격과, 첫 60% 구간에서 추정된 헤지 비율로 계산한 스프레드 Z-score. ± 2 선은 설명용 진입 후보 경계다.

11.2 페어 선정 파이프라인

수백 개 후보 페어를 단계적으로 걸러낸다. 각 단계는 서로 다른 기준으로 불필요한 후보를 제거한다. 상관관계 필터는 움직임이 너무 다른 페어를 버리고, 공적분 검정은 장기적 관계 유무를 확인하고, 반감기 필터는 스프레드가 실전에서 쓸 수 있을 만큼 빨리 평균에 돌아오는지 본다.

Step	단계	기준 / 산출물
1	전체 유니버스	S&P 500
2	섹터별 분류	SECTOR_SYMBOLS
3	가격 데이터 로드	3년 일봉
4	상관관계 필터	$\text{corr} \geq 0.4$
5	공적분 검정	$p < 0.05$
6	OLS β 계산	정적 헤지 비율
7	반감기 계산	$hl \leq 80$ 일

Step	단계	기준 / 산출물
8	최종 페어 목록	(sym_a, sym_b, corr, pvalue, beta, halflife)

역사적 맥락 D.E. Shaw, PDT Partners, 르네상스는 1990년대부터 상대가치·통계적 차익거래를 대규모로 운용했다. LTCM은 단순한 단일 섹터 페어 펀드가 아니라 레버리지를 크게 사용한 광범위한 상대가치 전략이었다. 1998년 위기는 유동성이 마르면 여러 수렴 거래가 동시에 더 벌어질 수 있음을 보여준다. 같은 섹터로 후보를 제한해도 이런 모델 붕괴·상관 급등 위험은 사라지지 않는다.

WARNING

p-value 하나만 보지 말라. 공적분 검정은 회귀 방향($A \rightarrow B$ vs $B \rightarrow A$)에 따라 p-value가 달라진다. 실전에서는 반감기, 잔차 정상성 같은 보조 지표도 함께 본다. 더 엄밀한 교차 검증(Johansen 검정 등)은 19장에서 다룬다.

통계 배경이 없어도 괜찮다: 이 챕터에서 쓰는 검정(공적분, ADF)의 핵심 질문은 하나다—“관계가 없다는 귀무가설 아래, 지금만큼 극단적인 결과가 얼마나 드문가?” p-value가 작으면 평균 회귀를 뒷받침하는 증거가 강해지지만, 미래에 반드시 돌아온다는 보장은 아니다. 미리 정한 기준, 경제적 이유, 관계 안정성, 다중 검정 보정, OOS 결과를 함께 본다.

공적분 검정—한 페어를 판정하기

statsmodels의 `coint()` 함수를 쓰면 한 줄로 끝난다. 내부적으로 (1) OLS 회귀로 두 가격의 비율을 구하고, (2) 그 잔차(=스프레드)가 평균회귀하는지 ADF 검정으로 확인한다. 우리가 쓰는 출력은 두 번째 반환값인 p-value뿐이다.

```
from statsmodels.tsa.stattools import coint

_, pvalue, _ = coint(price_a.values, price_b.values)
```

```

if pvalue >= self.coimt_pvalue:    # default = 0.05
    continue                      # 공적분 아님 -> 건너뛴

```

p-value가 미리 정한 0.05 미만이면 **1차 후보**로 남긴다. 이는 관계가 참일 확률이 95%라는 뜻이 아니며, 최종 선정에는 다중 검정 보정과 반감기·경제적 근거·OOS 검증이 더 필요하다.

전체 파이프라인—섹터별로 모든 페어 검정

후보 페어는 같은 섹터 등 경제적 관계가 있는 조합에서 만들고, 학습 구간의 날짜를 맞춘 뒤 결측 비율·표본 수·공적분 검정·반감기를 순서대로 검사한다. 여러 조합을 검사했다면 개별 p-value가 아니라 다중 검정 보정값도 본다. 아래 코드는 이 선별 순서를 나타내며, 선정 뒤 OOS에서 다시 성과를 평가해야 한다.

```

import numpy as np
import statsmodels.api as sm
from statsmodels.stats.multitest import multipletests
from statsmodels.tsa.stattools import coimt

def find_cointegrated_pairs(self, symbols) -> list:
    tested = []
    for sector, sector_symbols in SECTOR_SYMBOLS.items():
        prices = self._load_prices(sector_symbols, period="3y")
        corr_matrix = prices.pct_change().dropna().corr()

        for i, sym_a in enumerate(corr_matrix.columns):
            for sym_b in corr_matrix.columns[i+1:]:
                # 1) 상관관계 ≥ 0.4
                corr = corr_matrix.loc[sym_a, sym_b]
                if corr < self.min_correlation:
                    continue

                pair = prices[[sym_a, sym_b]].dropna()
                price_a = pair[sym_a]
                price_b = pair[sym_b]

                # 2) 검정한 모든 페어의 raw p-value를 보관
                _, pvalue, _ = coimt(price_a.values, price_b.values)
                X = sm.add_constant(price_b.to_numpy())

```

```

fit = sm.OLS(price_a.to_numpy(), X).fit()
intercept, beta = fit.params
spread = price_a - (intercept + beta * price_b)
tested.append({
    "symbols": (sym_a, sym_b), "corr": corr,
    "pvalue": pvalue, "beta": beta,
    "halflife": calc_halflife(spread.to_numpy()),
})

if not tested:
    return []

# 3) Benjamini-Hochberg 방식으로 다중 검정의 FDR 제어
reject, qvalues, _, _ = multipletests(
    [x["pvalue"] for x in tested], alpha=self.coimt_pvalue,
    method="fdr_bh"
)
selected = []
for row, keep, qvalue in zip(tested, reject, qvalues):
    hl = row["halflife"]
    if keep and np.isfinite(hl) and hl <= self.max_halflife:
        selected.append(**row, "qvalue": qvalue)
return sorted(selected, key=lambda x: (x["qvalue"], x["halflife"]))

```

코드에서 눈여겨볼 부분은 정렬보다 **검정한 p-value를 모두 모은 뒤 한 번에 보정하는 순서**다. q-value가 작다는 이유만으로 앞쪽 페어가 자동으로 최선이 되는 것은 아니다. 그다음 반감기, 거래비용, 경제적 관계, 여러 OOS 구간의 안정성을 비교해야 한다.

11.3 반감기(halflife) 계산

반감기란 스프레드가 평균값 쪽으로 절반만큼 돌아오는 데 걸리는 시간(일)이다. 공적분 관계가 있어도 반감기가 너무 길면 실전에서 쓰기 어렵다. 반감기가 200일이라면 포지션을 수백 일 동안 묶어야 하므로, 비용과 리스크가 그만큼 커진다.

수학적으로는 “스프레드가 평균에서 멀어진 만큼, 다음 시점에 얼마나 당겨지는가?”를 회귀로 추정한다. 이 당기는 강도(ϕ)에서 반감기가 나온다. ϕ 가 음수일수록 강하게 당겨지므로 반감기가 짧다.

```
# app/trading/rl/features.py 발췌
def calc_half-life(spread, min_obs=20):
    y = np.diff(spread)          #  $\Delta s_t$ 
    x = spread[:-1] - np.mean(spread)  # 평균 중심화
    phi = np.sum(x * y) / np.sum(x * x)  # OLS 기울기
    if phi >= 0:
        return float("inf")        # 평균회귀 증거 없음 -> 명시적 탈락
    return float(max(-np.log(2) / phi, 1.0))
```

확인할 출력은 일 단위 숫자다. 예를 들어 $\phi = -0.03$ 이면 반감기는 약 23일이다. 즉 모델상 이탈 폭의 기대값이 약 23일 뒤 절반으로 줄어든다. ϕ 가 0 이상이면 ∞ 를 반환해 `np.isfinite()` 검사에서 명시적으로 탈락시킨다. 상한값을 반환하면 $hl > 80$ 같은 조건에서 경계값이 잘못 통과할 수 있다.

11.4 Kalman 필터로 동적 헤지 비율 구하기

OLS로 한 번 구한 헤지 비율 β 는 시간이 지나도 그대로다. 하지만 두 회사의 사업 구조나 시장 환경이 바뀌면 두 가격을 묶는 비율도 변한다. **Kalman 필터**는 매 시점마다 β 를 새로 갱신해 주는 도구다. 새 가격이 들어올 때마다 “지금까지의 β 추정치”와 “오늘 본 새 정보”를 가중 평균해서 β 를 업데이트한다고 이해하면 된다.

핵심 아이디어는 다음과 같다.

- **상태**: 현재의 헤지 비율 β 와 상수 c
- **예측**: 매일 β 가 조금씩 흔들릴 수 있다고 가정한다.
- **갱신**: 오늘 본 실제 가격과 예측 가격의 차이를 보고 β 를 살짝 조정한다.

시변 회귀는 관측 $y_t = a_t + \beta_t x_t + \text{noise}$ 와 상태 $[a_t, \beta_t]$ 로 이해할 수 있다. 상태 변화 분산 Q 가 클수록 계수가 빨리 움직이고, 관측 분산 R 이 클수록 새 가격에 덜 반응한다. 6장의 예측·갱신 원칙을 벡터에 적용하며, 헤지 수량을 정할 때는 β 의 부호·상한과 추정 불확실성을 함께 확인한다.

- `kalman_delta = 1e-4`: β 변화 속도. 작을수록 β 가 안정적으로 움직이지만 변화에 둔감하다.
- `kalman_obs_noise = 1.0`: 관측 노이즈. 작을수록 새 가격을 더 신뢰한다.

매일 갱신된 β 로 그날의 스프레드를 만든다.

```
spread = log_price_a - beta_arr * log_price_b
```

이 스프레드 시계열이 다음 장(12장)의 거래 신호의 입력이 된다.

11.5 페어 품질 지표

페어를 한 번 고른 뒤에도 시간이 지나면서 공적분이 깨질 수 있다. 그래서 진입 직전에 페어 품질을 한번 더 확인한다.

확인하는 것은 두 가지다.

1. 최근 250일 상관관계가 `corr_min(=0.4)` 이상인가?
2. 최근 250바 스프레드의 반감기가 `hl_max(=60일)` 이하인가?

두 검사를 통과하면 그 페어로 진입하고, 보유 기간 상한도 반감기에 맞춰 동적으로 정한다.

```
# stat_pair_rl.py —핵심 검사 두 줄
if kdata["corr"] < cfg.corr_min:                # 1) 상관 ≥ 0.4
    return False
if calc_half-life(recent_spreads) > cfg.hl_max:  # 2) 반감기 ≤ 60일
    return False
dyn_max_hold = min(cfg.max_hold_days, int(cfg.hl_hold_mult * hl))
```

`kdata`는 미리 계산된 결과 캐시(Kalman 스프레드, 동적 β , 250일 롤링 상관관계)다. 진입할 때마다 다시 계산하지 않아도 되어 빠르다.

11.6 페어 선정 파라미터 한눈에 보기

처음에는 의미만 훑고, 필요할 때 표로 돌아오면 된다.

파라미터	기본값	역할
<code>z_window</code>	60	z-score 계산 창 (영업일)

파라미터	기본값	역할
z_entry	1.8	진입 임계 z-score
z_exit	0.3	청산 임계 z-score
hard_stop_z	4.5	하드 스톱 z-score
kalman_delta	1e-4	Kalman β 변화 속도
kalman_obs_noise	1.0	Kalman 관측 노이즈
max_hold_days	30	최대 보유 기간 (일)
corr_min	0.4	진입 최소 상관관계
warmup_days	120	Kalman 예열 기간
hl_max	60	진입 시 최대 반감기 (일)
hl_hold_mult	3.0	$\text{max_hold} = 3 \times \text{halflife}$

$z_{\text{entry}} = 1.8$ 은 백테스트 탐색에서 선택된 값이다. 다만 특정 기간에서 최적이었던 값이 항상 좋은 기본값이라는 뜻은 아니다. 실제로는 거래 비용, 종목 universe, 시장 구간을 바꿔도 지나치게 흔들리지 않는지 함께 확인해야 한다.

11.7 페어 선별 기준 요약

기준	값	역할
섹터 제한	같은 섹터 내	다중 검정 문제 완화
상관관계	≥ 0.4	공동 요인 존재 확인
공적분 검정	raw p-value 사전 기준 + FDR 보정	평균회귀의 1차 통계 증거
반감기	≤ 60 일 (진입부)	보유 기간 합리성 확보
Kalman δ	1e-4	동적 β 변화 속도의 예시 초기값

페어를 고를 때는 **수보다 품질**이다. 공적분이 강한 페어 수십 개가, 기준을 느슨하게 통과한 페어 수백 개보다 낫다.

다음 장(12장): 이렇게 고른 페어를 실제 거래 신호로 바꾼다. Kalman 스프레드의 z-score로 진입 시점을 잡고, 레짐 필터와 RL 게이트로 신호를 한 번 더 거른다.

따라 해 보기—합성 페어로 파이프라인 돌려 보기

지금까지 본 세 가지(상관관계, 공적분, 반감기)를 합쳐서 한 페어를 판정하는 최소 실행 코드다. 합성 데이터로 돌려 봐서 통과/실패가 어떻게 나오는지 감을 잡는 용도다. 직접 실행해 보길 권한다.

```
import numpy as np
from scipy import stats
from statsmodels.tsa.stattools import coint

MIN_CORR, MAX_PVALUE, MAX_HALFLIFE = 0.40, 0.05, 60

def estimate_halflife(spread):
    y = spread[1:] - spread[:-1]
    x = spread[:-1] - spread[:-1].mean()
    slope, *_ = stats.linregress(x, y)
    if slope >= 0:
        return float("inf")
    return float(-np.log(2) / slope)

def screen_pair(prices_a, prices_b):
    log_a, log_b = np.log(prices_a), np.log(prices_b)
    # 1) 공통 움직임의 1차 필터: 로그수익률 상관관계
    ret_corr = np.corrcoef(np.diff(log_a), np.diff(log_b))[0, 1]
    if ret_corr < MIN_CORR:
        return None
    # 2) OLS  $\beta$  + 스프레드
    slope, intercept, *_ = stats.linregress(log_b, log_a)
    spread = log_a - slope * log_b - intercept
    # 3) 잔차 추정을 반영한 Engle-Granger 공적분 검정
    _, eg_pvalue, _ = coint(log_a, log_b)
    if eg_pvalue >= MAX_PVALUE:
        return None
    # 4) 반감기
    hl = estimate_halflife(spread)
    if hl <= 0 or hl > MAX_HALFLIFE:
```

```

    return None
    return {"return_corr": float(ret_corr),
           "eg_p": float(eg_pvalue),
           "hl": hl}

if __name__ == "__main__":
    rng = np.random.default_rng(42)
    common = np.cumsum(rng.normal(0, 1, 300))
    a = np.exp(common * 0.9 + rng.normal(0, 0.2, 300)) * 100
    b = np.exp(common * 1.1 + rng.normal(0, 0.2, 300)) * 150
    print(screen_pair(a, b))

```

출력은 {"return_corr": ..., "eg_p": ..., "hl": ...} 형태의 딕셔너리 한 개다. None이 나오면 어느 단계에서 떨어졌는지 위 조건을 차례로 확인하면 된다. 이 예제는 한 페어의 흐름만 보여준다. 실 시장 데이터로 여러 페어를 시험할 때는 날짜 정렬·거래비용·다중 검정 보정까지 추가해야 하며, 가격 배열만 바꾼다고 실전 검증이 끝나는 것은 아니다.

연습 문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. SPY-QQQ 페어에 공적분 검정을 돌려 보고, p-value가 진입 임계값(0.05)과 가까이 나올 때 반감기와 상관관계 같은 보조 지표로 어떻게 판단할지 정리해 보자.
2. 위 estimate_halflife 코드를 응용해, 반감기가 3일 미만이거나 60일 초과인 페어를 자동으로 걸러 내는 필터를 만들어 보자. 너무 짧은 반감기는 왜 위험할지도 생각해 보자.
3. 페어 선정을 분기마다 다시 돌릴 때 선정 페어가 바뀌면 기존 포지션을 어떻게 청산할지 생각해 보자. 교체할 때 발생하는 거래비용도 함께 고려한다.

정리

이 장에서 반드시 기억할 것은 다음 세 가지다.

기준	이유
같은 섹터	두 회사가 왜 함께 움직이는지 설명할 수 있어야 한다
공적분 $p\text{-value} < 0.05$	가격 차이가 결국 평균으로 돌아온다는 통계적 근거
반감기 제한	너무 느리게 돌아오는 페어는 실전에서 못 쓴다

좋은 페어는 단순히 같이 움직이는 두 종목이 아니다. 같은 섹터 안에 있고, 공적분 $p\text{-value}$ 가 낮으며, 스프레드가 거래 가능한 속도로 평균에 돌아와야 한다. **Kalman 필터**는 이 비율이 시간에 따라 변할 때 따라가 주는 도구이고, 자세한 내부 수식은 다음 장과 19장에서 더 다룬다.

다음 장에서는 이렇게 고른 페어를 실제 진입·청산 신호로 바꾼다.

더 읽을거리

- **논문**: Engle & Granger (1987), “Co-Integration and Error Correction”—공적분 검정 원조 논문 (노벨상)
- **책**: Vidyamurthy, Pairs Trading (Wiley, 2004) —통계적 차익거래 전략의 실전 구현
- **논문**: Gatev, Goetzmann & Rouwenhorst (2006), “Pairs Trading: Performance of a Relative Value Arbitrage Rule”—학술적 Backtest

이론적 기반 심화: 이 챕터에서 다룬 공적분 검정과 Kalman 헤지 비율의 수학적 배경은 19~21장에서 깊이 다룬다. Johansen 교차 검증, 레짐 조건부 페어 필터링 같은 고급 주제도 그곳에서 살펴본다.

12장. 통계적 차익거래—신호 생성

공적분 페어를 찾았다면 다음 질문은 단순하다. 언제 들어가고, 언제 나오며, 관계가 깨진 건 어떻게 알 것인가. 이 세 가지가 실전 구현의 전부다.

가장 단순한 규칙은 이렇다. 스프레드의 Z-score(이탈 정도를 표준화한 값. 정의는 11장 공적분 절 참고)가 2.0을 넘으면 진입하고, 0으로 돌아오면 청산한다. 처음 배울 때는 이 정도로 충분해 보인다. 하지만 실전에서는 부족하다. 시장은 스프레드가 왜 벌어졌는지 알려주지 않기 때문이다. 단순한 Z-score는 “일시적으로 벌어진 것”과 “구조가 영영 바뀐 것”을 구분하지 못한다.

실패 방식은 분명하다. 스프레드가 2에서 4로, 다시 6으로 계속 벌어질 수 있다. 일시적인 과열이라면 기다리면 된다. 하지만 페어 중 한 회사의 실적이 관계 자체를 바꿔버렸다면 이야기가 다르다. 단순한 시스템은 새 극단을 “더 좋은 진입 가격”으로 착각하고 포지션을 늘린다. 페어 트레이딩판 몰타기다.

청산도 마찬가지다. 교과서는 “스프레드가 0으로 돌아오면 청산”이라고 한다. 하지만 이 규칙은 시장 상황, 즉 레짐(regime, 시장의 국면. 11장에서 HMM으로 도입함)을 고려하지 않는다. 변동성이 큰 구간에서는 스프레드가 0 근처에서 크게 흔들리며 깔끔하게 교차하지 않는다. 고정된 규칙은 불필요한 매매와 수수료만 늘린다.

진입보다 청산이 먼저다

이 장을 읽을 때 한 가지 원칙을 먼저 새겨두면 좋다. **진입 규칙이 기회를 만들지만, 손익을 결정하는 것은 청산 규칙이다.** 진입은 틀려도 다음 기회가 오지만, 청산이 늦으면 한 번의 거래가 계정을 깎는다. 책의 절 순서는 “진입 -> 청산”이지만, 머릿속에 그릴 때는 “어디서 빠질 것인가”를 먼저 정한 뒤 “어디서 들어갈 것인가”를 따져야 한다. 좋은 신호는 많이 맞는 신호가 아니라, 틀렸을 때 손실이 커지기 전에 물러나는 신호다.

이 장의 해법은 레짐 조건부 신호다. HMM이 분류한 시장 상태에 맞춰 Z-score 임계값과 청산 규칙을 조정한다. 안정적인 평균회귀 레짐에서는 진입 기준을 낮춰 기회를 더 잡고, 추세가 강한 레짐에서는 진입을 줄인다.

이 신호를 “내일 어느 종목이 오를지 맞는 장치”로 생각하면 안 된다. “지금 이 페어에 들어가도 안전한가”, “관계가 깨졌다면 어디서 멈출 것인가”를 코드로 판단하는 규칙 묶음이다.

11장이 “무엇을 거래할까”를 정했다면, 12장은 **언제 들어가고 언제 나올까**를 정한다. 즉 페어 후보를 실제 매매 신호로 바꾸는 장이다.

```

11장 출력
거래 후보 페어: KO / PEP
    |
    v
12장 입력
현재 스프레드, Z-score, 반감기, 상관관계, 거래량
    |
    v
신호 생성
진입 가능 / 관망 / 청산
    |
    v
13~14장 RL
신호를 보고 포지션 크기를 조절
  
```

초보자는 Z-score를 “크면 들어간다”로 외우면 위험하다. Z-score는 신호의 출발점일 뿐이다. 이 장은 그 숫자 주변에 안전 필터를 붙인다.

```

Z-score가 크다
    |
    └─ 페어 관계가 아직 유지되는가?
  
```

- └ 회귀가 시작됐는가?
- └ 거래량과 유동성이 괜찮은가?
- └ 너무 오래 들고 있지는 않은가?
- └ 손실 한도를 넘지는 않았는가?

그래서 12장은 진입 장이라기보다 **진입과 청산을 함께 설계하는 장**이다. 특히 RL과 결합할 때도 이 신호는 그대로 중요하다. RL은 아무 페어에나 마음대로 들어가는 것이 아니라, 이 장의 필터를 통과한 상황에서 포지션 크기를 조절한다.

이 장의 흐름

- 진입 자격 (5단계 필터)
- Entry Score (entry_score) —점수 곱셈식
- 청산 조건 (우선순위 6단계)
- BW-HMM 스프레드 레짐 보정
- 진입·청산 흐름 요약
- 코드로 보기
- RL과 결합하는 방법 (참고)

학습 목표

이 장이 끝나면 다음을 할 수 있다.

- z-score 기반 진입/청산 로직의 흐름과 청산 조건의 우선순위를 설명할 수 있다
- entry_score를 구성하는 6개 인자가 각각 어떤 역할을 하는지 말할 수 있다
- BW-HMM 레짐 보정이 진입 기준에 어떤 영향을 주는지 설명할 수 있다
- 왜 청산 조건이 진입 조건만큼, 혹은 그 이상으로 중요한지 설명할 수 있다

TIP

바이브코딩 미션

- **얻을 결과:** Z-score 한 개가 진입·보유·청산 중 무엇을 뜻하는지 확인한다.

- **첫 회독:** 점수 구성요소 전체보다 청산 조건이 먼저 평가되는 이유를 본다.
- **용어 발판:** Z-score는 평균에서 떨어진 거리, 진입은 포지션을 여는 것, 청산은 포지션을 닫는 것이다.
- **AI 요청:** “기존 신호 함수에 Z-score 시퀀스 `[0, 1, 2.1, 1.4, 0.3]`을 넣었을 때 상태 변화를 추적해줘. 실제 조건과 임계값은 코드에서 확인해줘.”
- **완료 확인:** 진입보다 청산이 먼저 검사되는 흐름을 설명하면 성공이다.

12.1 진입 자격 (Entry Prerequisites)

진입 점수를 계산하기 전에, 먼저 후보 자체를 거를 다섯 가지 필터가 있다. 점수가 아무리 높아도 이 다섯 조건 중 하나라도 통과하지 못하면 진입하지 않는다. 점수는 “얼마나 매력적인가”를 재는 것이고, 자격은 “들어갈 수도 되는 후보인가”를 재는 것이다.

단계	필터	조건	직관
1	반감기(half-life)	$hl \leq 60$ 일	회귀가 너무 느린 페어 제외
2	상관관계	$corr \geq 0.4$	같이 안 움직이는 페어 제외
3	Z-score 임계값	$ z_{now} \geq z_{entry}$ (BW-HMM 보정 포함)	충분히 이탈했을 때만 진입
4	수렴 방향	$ z_{now} < z_{prev} $	아직 더 벌어지는 중이면 진입 안 함
5	거래량 미시구조	$vol_mult \geq 0.90$	유동성이 비정상적으로 얇으면 제외

이 중 **4번 “수렴 방향” 조건이 핵심**이다. 스프레드가 아직 더 벌어지는 중이면 들어가지 않고, 피크를 지나 회귀가 시작된 시점에만 진입한다. 이 한 줄 덕분에 “더 벌어진 곳에 또 들어가는” 물타기를 막을 수 있다.

3번의 `z_entry`는 고정값이 아니라 BW-HMM 레짐에 따라 동적으로 조정된다(12.4에서 설명). 구현 코드는 `stat_pair_rl.py`의 `_get_effective_z_entry()`에서 확인한다.

12.2 진입 점수 (entry_score)

WARNING

Z-score 임계값(z_entry)을 너무 낮게 잡으면 거래가 너무 잦아져 수수료가 누적된다. 기본값은 2.0이다. 페어와 레짐에 따라 1.5~2.5 사이에서 Optuna로 최적화한다(-> 15장).

12.1의 다섯 자격을 모두 통과한 후보에 대해 진입 점수를 계산한다. 점수는 6개 인자의 곱이다. 이 점수는 “얼마나 벌 수 있는가”가 아니라 “지금 들어가도 안전한가”를 검사하는 품질 점수다.

```
entry_score = abs(z_now) * (1.0 / max(hl, 1.0)) * corr * ivr_mult *
conv_boost * vol_mult
```

각 인자의 의미는 다음과 같다.

인자	의미	범위	중요도
<code>\ z_now\ </code>	현재 z-score 절대값. 이 탈이 얼마나 극단적인지 표시하며, 클수록 관계 붕 괴 위험도 함께 커짐	0~5+	5/5
<code>1/hl</code>	반감기(half-life) 역수. 짧을수록 빠르게 회귀	0~1	4/5
<code>corr</code>	최근 250일 상관관계. 공 적분 품질	0~1	3/5
<code>ivr_mult</code>	실현변동성 비율 보정. “지금 시장이 평소처럼 잠 잠한가, 비정상적으로 출 렁이는가”를 단기/장기 변 동성 비(IVR)로 재서 점 수에 곱함. 잠잠하면 신호 를 신뢰해 그대로 두고, 출렁이면 점수를 깎는다	0.5~1.0	3/5
<code>conv_boost</code>	수렴 모멘텀 가점	1.0~1.2	2/5

인자	의미	범위	중요도
vol_mult	거래량 미시구조 확증	0.5~1.0	2/5

왜 곱셈인가

덧셈으로 계산하면 Z-score 하나가 매우 클 때 다른 인자의 약점을 덮어버린다. 곱셈은 반대다. 한 조건이 약하면 전체 점수도 같이 낮아진다. Z-score가 아무리 커도 상관관계가 낮으면 점수는 줄어든다. 페어 트레이딩에서 가장 위험한 상황은 “스프레드가 많이 벌어졌다”는 이유만으로 이미 구조가 깨진 페어에 들어가는 것이다. 큰 절대 Z-score는 **이탈의 크기**일 뿐 평균으로 돌아올 확률이 아니다. 곱셈 점수도 안전을 보장하지 않으므로, 공격분 안정성·손절·비용·OOS 검증을 별도로 통과해야 한다. 각 인자의 가중 방식과 임계값은 검증할 전략 가설이다.

숫자 예시 한 줄

가 상 의 페 어 가 $z=-2.8$, $hl=25$, $corr=0.65$, $ivr_mult=1.0$, $conv_boost=1.1$, $vol_mult=0.95$ 라고 하자.

$$score = 2.8 \times (1/25) \times 0.65 \times 1.0 \times 1.1 \times 0.95 \approx 0.076$$

같은 z , 같은 반감기에 $corr$ 만 0.4로 떨어지면 점수는 약 0.047로 줄어든다. 곱셈이라 한 인자가 약해지면 전체 점수가 같이 내려간다는 것을 수치로 확인할 수 있다.

수렴 모멘텀(conv_boost)

최근 세 봉의 $abs(z)$ 가 $2.8 \rightarrow 2.5 \rightarrow 2.1$ 처럼 감소하면 회귀 방향으로 움직였다고 해석할 수 있다. 설명용 가점은 세 값이 모두 유한하고 엄격히 감소할 때 1.2, 아니면 1.0으로 정할 수 있다. 이는 회귀 완료 확률이 아니라 신호 점수의 배율이다. 점수가 높아도 진입 기준·비용·포지션 한도를 통과해야 한다.

12.3 청산 조건 (우선순위 순서)

IMPORTANT

청산 규칙이 없으면 수렴하지 않는 페어에서 손실이 계속 쌓인다. `max_holding_days`(최대 보유일 초과 시 강제 청산)는 실전에서 반드시 필요한 안전장치다. 공적분 관계가 무너지면 Z-score는 평균으로 돌아오지 않는다. 따라서 시간 기반 손절도 함께 사용해야 한다.

포지션을 닫을 때는 아래 6개 조건을 **순서대로** 확인한다. 앞 조건이 충족되면 뒤 조건은 검사하지 않는다.

우선순위	조건	파라미터	역할
(1)	<code>price_stop</code>	<code>price_stop_pct = -8.0%</code>	절대 손실 제한
(2)	<code>pair_degrade_hl</code>	<code>hl > 100</code>	공적분 붕괴 감지
(2)	<code>pair_degrade_corr</code>	<code>corr < 0.3</code>	상관관계 붕괴
(3)	<code>z_hard_stop</code>	<code>hard_stop_z = 4.5</code>	극단 이탈 시 손실 최소화
(4)	<code>rl_exit</code>	<code>rl_exit_threshold_hybrid = -0.6</code>	RL 액션이 매우 부정적일 때(RL 모드 한정)
(5)	<code>z_converge</code>	<code>z_exit = 0.3</code>	정상 수준으로 수렴
(6)	<code>max_hold</code>	<code>min(30, 3×hl)</code>	시간 기반 강제 청산

각 조건을 한 줄로 풀면 다음과 같다.

- **price_stop**: 손실이 -8%를 넘으면 무조건 청산. 이유를 따지기 전에 손실부터 막는다.
- **pair_degrade_hl / _corr**: 반감기 100일 초과 또는 상관관계 0.3 미만 -> 페어 자체가 깨졌다고 판단.
- **z_hard_stop**: Z-score가 4.5 초과 -> 극단적으로 벌어진 상태. 더 기다리지 않고 청산.
- **z_converge**: Z-score가 0.3 이하 -> 목표 달성. 정상 수렴 청산.
- **max_hold**: 위 조건이 모두 안 맞아도 `min(30일, 3×hl)`이 지나면 강제 청산.

이 순서가 곧 우선순위다. 손실 차단(price_stop)이 가장 먼저고, 시간 강제(max_hold)가 가장 나중이다. “정상 청산 신호를 기다리다가 손실이 -10%, -15%로 커지는”사고를 막기 위해서다.

어느 조건이 먼저 걸리는지—두 시나리오

시나리오 A —손절이 먼저 걸리는 케이스. 페어 A에 $z=-2.5$ 에서 진입했다. 다음 봉에서 한 쪽 회사의 실적 발표 충격으로 스프레드가 더 벌어져 손실이 -9%에 도달했다. Z-score는 아직 -3.2로 청산 기준 ($\text{abs}(z)<0.3$)과는 거리가 멀다. 우선순위 (1) price_stop이 먼저 발동하고, (5) z_converge까지 가지 않는다. 손실 한도가 없었다면 손실이 더 깊어지기 전에 빠져나오지 못했을 것이다.

시나리오 B —z 수렴이 먼저 걸리는 케이스. 페어 B에 $z=-2.7$ 에서 진입했다. 일주일 동안 가격이 천천히 움직여 손익은 +1.5%, Z-score는 $-2.7 \rightarrow -1.8 \rightarrow -0.9 \rightarrow -0.2$ 로 깔끔하게 수렴했다. (1) (2) (3) (4)는 모두 충족되지 않고, (5) $\text{z_converge}(\text{abs}(z)<0.3)$ 에서 청산된다. (6) max_hold(시간 강제)까지 갈 필요가 없는, 정상 청산 시나리오다.

실제 구현(if-elif 사슬)은 stat_pair_rl.py에서 확인한다.

12.4 BW-HMM 스프레드 레짐 보정

스프레드 자체에 HMM을 적용해 평균회귀(MR) 레짐과 추세 레짐을 구분한다. 같은 Z-score여도 어느 레짐인지에 따라 의미가 다르다. MR 레짐의 큰 Z는 기회지만, 추세 레짐의 큰 Z는 관계가 깨지는 신호일 수 있다.

따라서 MR 레짐에서는 z_{entry} 를 낮춰 진입을 쉽게 하고, 추세 레짐에서는 높여 진입을 억제한다.

파라미터	기본값	효과
spread_bw_mr_boost	0.15	MR 레짐: $z_{\text{entry}} \times 0.85$ (진입 완화)
spread_bw_trend_penalty	0.25	Trend 레짐: $z_{\text{entry}} \times 1.25$ (진입 억제)

예를 들어 $z_{\text{entry}}=2.0$ 이 MR 레짐에서는 1.7로 낮아지고, 추세 레짐에서는 2.5로 올라간다. 추세 레짐에서는 추가로 청산 임계값과 보유 기간도 조여지는데(spread_bw_trend_exit,

`spread_bw_trend_hold_mult`), 이 두 파라미터는 부록과 `_RLConfig` 정의를 참고한다. 보정 함수 본체는 `_get_effective_z_entry()` 다.

12.5 진입·청산 흐름 요약

봉(bar)이 하나 완성될 때마다 아래 두 단계가 순서대로 실행된다. 먼저 열린 포지션을 닫을지 확인하고, 그다음 새 포지션 후보를 고른다.

단계	세부 항목	조건 / 처리
EXIT (기존 포지션 청산)	1. <code>price_stop</code>	손익 -8%
EXIT	2. <code>pair_degrade</code>	<code>hl>100</code> 또는 <code>corr<0.3</code>
EXIT	3. <code>z_hard_stop</code>	<code>abs(z)>4.5</code>
EXIT	4. RL exit	<code>rl_full / rl_hybrid</code> , <code>min_hold=5</code> 봉 이후
EXIT	5. <code>z_converge</code>	<code>abs(z)<0.3</code> 또는 <code>z</code> 부호 전환
EXIT	6. <code>max_hold</code>	<code>min(30봉, 3×hl)</code>
ENTRY (새 포지션 진입)	자격 5단계	<code>hl≤60</code> , <code>corr≥0.4</code> , <code>abs(z)≥z_entry</code> (보정됨), <code>z</code> 감소 추세, <code>vol_mult≥0.90</code>
ENTRY	점수 계산	$\text{abs}(z) \times 1/\text{hl} \times \text{corr} \times \text{ivr_mult} \times \text{conv_boost} \times \text{vol_mult}$
ENTRY	최종 선택	점수 내림차순 -> 상위 <code>max_positions=30</code> 개 진입

청산 체크는 위에서 아래로 적용된다. 앞 조건에 걸리면 그 즉시 청산하고 나머지는 보지 않는다. 진입은 자격을 모두 통과한 후보를 점수로 줄 세워 상위 종목만 선택한다.

12.6 코드로 보기—진입 점수 계산과 진입 자격 판정

지금까지 본 진입 로직을 짧은 함수 두 개로 실행해 보자. 실제 `stat_pair_rl.py`는 훨씬 길지만, 핵심 의사결정은 이 두 함수에 다 들어 있다. 단계 번호는 12.1의 자격 표와 동일하다.

```
import numpy as np

Z_ENTRY, HL_MAX, CORR_MIN, VOL_MIN = 2.0, 60, 0.40, 0.90

def compute_entry_score(z, hl, corr, ivr=1.0, conv=1.0, vol=1.0):
    """ $abs(z) \times 1/hl \times corr \times IVR \times$  수렴부스트  $\times$  거래량 (값이 클수록 좋은 후
    보)"""
    return abs(z) * (1.0 / max(hl, 1.0)) * corr * ivr * conv * vol

def check_entry(z, z_prev, hl, corr, vol_mult, z_entry_adj=Z_ENTRY):
    """12.1의 진입 자격 5단계 체크. (go, reason) 반환."""
    # 1. 반감기
    if hl > HL_MAX:
        return False, f"hl={hl}>{HL_MAX}"
    # 2. 상관관계
    if corr < CORR_MIN:
        return False, f"corr={corr:.2f}<{CORR_MIN}"
    # 3. Z-score 임계값 (BW-HMM으로 보정된 z_entry_adj 사용)
    if abs(z) < z_entry_adj:
        return False, f"abs(z)<{z_entry_adj}"
    # 4. 수렴 방향
    if abs(z) >= abs(z_prev):
        return False, "z still rising"
    # 5. 거래량 미시구조
    if vol_mult < VOL_MIN:
        return False, f"vol_mult={vol_mult:.2f}<{VOL_MIN}"
    return True, "ok"

if __name__ == "__main__":
    cases = [
        ("정상 진입", dict(z=-2.8, z_prev=-3.1, hl=25, corr=0.65,
        vol_mult=0.95)),
        ("HL 과다", dict(z=-2.5, z_prev=-2.9, hl=80, corr=0.60,
        vol_mult=0.95)),
        ("상관관계 약", dict(z=-2.6, z_prev=-3.0, hl=30, corr=0.30,
        vol_mult=0.95)),
        ("Z 미달", dict(z=-1.5, z_prev=-1.8, hl=30, corr=0.70,
        vol_mult=0.95)),
```

```

        ("Z 상승중", dict(z=-3.0, z_prev=-2.5, hl=30, corr=0.65,
vol_mult=0.95)),
        ("거래량 부족", dict(z=-2.8, z_prev=-3.1, hl=25, corr=0.65,
vol_mult=0.80)),
    ]
    for label, c in cases:
        go, reason = check_entry(**c)
        score = compute_entry_score(c["z"], c["hl"], c["corr"],
vol=c["vol_mult"]) if go else 0.0
        print(f'{label:<10} go={go!s:<5} score={score:.4f}
reason={reason}')

```

확인할 출력: “정상 진입”만 go=True로 통과하고, 나머지 5개는 각기 다른 이유(reason)로 거부돼야 한다. 거부 사유가 어떤 단계에서 막혔는지 그대로 보여준다. 모든 케이스가 통과한다면 임계값이 너무 느슨하거나 필터가 빠진 것이다.

12.7 RL과 결합하는 방법 (다음 장 예고)

지금까지는 통계 규칙만으로 진입/청산을 결정하는 베이스라인(rl_stat_only)을 살펴봤다. 실제 stat_pair_rl.py는 강화학습(RL)과 결합하는 네 가지 모드를 더 지원한다. 모드를 나눈 이유는 단순하다. 통계 규칙과 RL을 한꺼번에 바꾸면 어느 쪽이 성능에 영향을 줬는지 알 수 없기 때문이다.

모드	진입	청산	설명
rl_stat_only	통계 (z-score)	통계	순수 Stat Arb 베이스라인 (이 장에서 다룬 것)
rl_veto	통계 -> RL 거부권	통계	RL이 위험한 진입 차단
rl_hybrid	통계 -> RL 조정	RL	통계 진입 + RL 청산
rl_entry	RL	통계	RL 진입 + 통계 청산
rl_full	RL	RL	완전 RL 제어

기본값은 rl_veto다. 통계 규칙이 먼저 진입 후보를 고르고, RL은 그중 위험해 보이는 것만 거부한다. 거부 임계값(rl_veto_threshold = -0.9)은 일부러 낮게 잡아 RL이 어지간한 이유로는 통

계 베이스라인을 흔들지 못하게 한다. RL 액션의 의미, 상태 벡터 구조, 보상 함수 같은 세부는 13장에서 다룬다.

처음 구현한다면 `rl_stat_only`로 기준 성능을 먼저 확인한 뒤 `rl_veto`를 켜고 비교하라. 그래야 변화의 원인을 알 수 있다.

이 장에서 미룬 것: RL veto 임계값의 튜닝, 28차원 RL 상태 벡터 구조, 보상 설계는 13장(RL 환경 설계)에서 이어서 다룬다.

정리

이 장에서 반드시 기억할 것은 세 가지다.

1. **진입 조건보다 청산 조건이 더 중요하다.** 좋은 신호는 많이 맞히는 신호가 아니라, 틀렸을 때 빨리 물러나는 신호다. 청산 6단계의 우선순위(손실 차단 -> 페어 붕괴 -> 극단 이탈 -> RL -> 정상 수렴 -> 시간)를 머릿속에 그릴 수 있어야 한다.
2. **entry_score는 곱셈이다.** 6개 인자 중 하나만 약해도 점수가 떨어진다. 이미 깨진 페어에 Z만 보고 들어가는 것을 막는 방어 장치다.
3. **레짐에 따라 같은 Z의 의미가 다르다.** 평균회귀 레짐의 큰 Z는 기회, 추세 레짐의 큰 Z는 위험 신호다. BW-HMM 보정은 이 차이를 진입 기준에 반영한다.

아래는 핵심 파라미터 한눈 정리다.

구성요소	파라미터	역할
z_entry	2.0 (sweep 시 1.5~2.5)	MR 진입 트리거
z_exit	0.3	수렴 청산
hard_stop_z	4.5	극단 손실 차단
price_stop_pct	-8%	절대 손실 한도
BW-HMM 보정	mr_boost=0.15, trend_penalty=0.25	스프레드 레짐별 z_entry 조정
conv_boost	max 1.2	이미 회귀 중인 페어 우대
max_hold	min(30, 3×hl)	적응형 보유 기간

다음 장(13장): RL 환경 설계. StatPairREnv의 상태 공간(28차원), 행동 공간(연속), 보상 함수 설계, 그리고 `rl_veto` 임계값의 의미를 다룬다.

연습문제

첫 회독에서는 1번만 하거나 바이브코딩 미션의 완료 확인으로 대체해도 된다. 나머지는 두 번째 회독용이다.

1. `z_entry` 임계값을 1.5에서 2.5까지 0.25 간격으로 grid search하라. 각 설정별 성과, 연간 거래 횟수, 평균 수익/손실을 표로 정리하라.
2. `conv_boost`(수렴 모멘텀 보정) 값을 0과 최댓값으로 각각 고정한 뒤 1년 PnL 곡선을 비교하라. 이 파라미터 효과가 가장 두드러지는 시장 국면은 무엇인가?
3. 2019~2023년 백테스트 로그에서 6가지 청산 조건 중 가장 자주 발동한 조건을 확인하라. 조건별 발동 비율을 출력하는 코드를 추가하라.

더 읽을거리

- **논문:** Avellaneda & Lee (2010), “Statistical Arbitrage in the U.S. Equities Market”—Z-score 기반 신호 설계
- **책:** Chan, Quantitative Trading —통계적 차익거래 구현과 리스크 관리
- **논문:** “Momentum and Mean Reversion in Strategic Asset Allocation”—수렴 모멘텀 보정의 이론적 배경